```
diff --git a/Core/Inc/main.h b/Core/Inc/main.h
index 59bd689..45c5516 100644
--- a/Core/Inc/main.h
+++ b/Core/Inc/main.h
@@ -57,10 +57,6 @@ void Error_Handler(void);
 /* USER CODE END EFP */

 /* Private defines -----------------------------------------------------------*/
-#define RTC_IN_Pin GPIO_PIN_14
-#define RTC_IN_GPIO_Port GPIOC
-#define RTC_OUT_Pin GPIO_PIN_15
-#define RTC_OUT_GPIO_Port GPIOC
 #define LORA_RESET_Pin GPIO_PIN_4
 #define LORA_RESET_GPIO_Port GPIOC
 #define LORA_NSS_Pin GPIO_PIN_5
diff --git a/Core/Inc/rtc.h b/Core/Inc/rtc.h
deleted file mode 100644
index a012e98..0000000
--- a/Core/Inc/rtc.h
+++ /dev/null
@@ -1,52 +0,0 @@
-/* USER CODE BEGIN Header */
-/**
-  ******************************************************************************
-  * @file    rtc.h
-  * @brief   This file contains all the function prototypes for
-  *          the rtc.c file
-  ******************************************************************************
-  * @attention
-  *
-  * Copyright (c) 2026 STMicroelectronics.
-  * All rights reserved.
-  *
-  * This software is licensed under terms that can be found in the LICENSE file
-  * in the root directory of this software component.
-  * If no LICENSE file comes with this software, it is provided AS-IS.
-  *
-  ******************************************************************************
-  */
-/* USER CODE END Header */
-/* Define to prevent recursive inclusion -------------------------------------*/
-#ifndef __RTC_H__
-#define __RTC_H__
-
-#ifdef __cplusplus
-extern "C" {
-#endif
-
```

```
-/* Includes ------------------------------------------------------------
------*/
-#include "main.h"
-
-/* USER CODE BEGIN Includes */
-
-/* USER CODE END Includes */
-
-extern RTC_HandleTypeDef hrtc;
-
-/* USER CODE BEGIN Private defines */
-
-/* USER CODE END Private defines */
-
-void MX_RTC_Init(void);
-
-/* USER CODE BEGIN Prototypes */
-
-/* USER CODE END Prototypes */
-
-#ifdef __cplusplus
-}
-#endif
-
-#endif /* __RTC_H__ */
-
diff --git a/Core/Inc/stm32f4xx_hal_conf.h
b/Core/Inc/stm32f4xx_hal_conf.h
index c8dd315..2277e5a 100644
--- a/Core/Inc/stm32f4xx_hal_conf.h
+++ b/Core/Inc/stm32f4xx_hal_conf.h
@@ -58,7 +58,7 @@
 /* #define HAL_IWDG_MODULE_ENABLED */
 /* #define HAL_LTDC_MODULE_ENABLED */
 /* #define HAL_RNG_MODULE_ENABLED */
-#define HAL_RTC_MODULE_ENABLED
+/* #define HAL_RTC_MODULE_ENABLED */
 /* #define HAL_SAI_MODULE_ENABLED */
 /* #define HAL_SD_MODULE_ENABLED */
 /* #define HAL_MMC_MODULE_ENABLED */
diff --git a/Core/Inc/stm32f4xx_it.h b/Core/Inc/stm32f4xx_it.h
index c250ea7..46ac4cb 100644
--- a/Core/Inc/stm32f4xx_it.h
+++ b/Core/Inc/stm32f4xx_it.h
@@ -52,7 +52,6 @@ void MemManage_Handler(void);
 void BusFault_Handler(void);
 void UsageFault_Handler(void);
 void DebugMon_Handler(void);
-void RTC_WKUP_IRQHandler(void);
 void EXTI0_IRQHandler(void);
 void EXTI1_IRQHandler(void);
 void TIM2_IRQHandler(void);
diff --git a/Core/Src/freertos.c b/Core/Src/freertos.c
index 30b9683..016323d 100644
--- a/Core/Src/freertos.c
+++ b/Core/Src/freertos.c
@@ -34,9 +34,6 @@ extern TIM_HandleTypeDef htim1;
 extern TIM_HandleTypeDef htim3;
```

```
 extern UART_HandleTypeDef huart1;
 extern SPI_HandleTypeDef hspi2;
-extern RTC_HandleTypeDef hrtc;
-extern void SystemClock_Config();
-

 // Shared Hardware State Variables from main.c
 extern uint32_t lastBlinkTime;
@@ -95,7 +92,6 @@ typedef enum {
     STATE_INIT,
     STATE_TX,
     STATE_RX1,
-    STATE_RX2,
     STATE_SLEEP
 } LoraState_t;

@@ -351,7 +347,7 @@ void StartLoRaTask(void *argument)
                // Align operational modes for Transmission
                Lora_WriteReg(REG_DIO_MAPPING_1, 0x40); // Map DIO0 to
TxDone
                Lora_WriteReg(0x33, 0x27);              // Normal IQ for
TX
-               Lora_WriteReg(0x3B, 0x1D);
            // RegInvertIq2 standard default <-- ADDED FOR SX1276 RESET
+           //   Lora_WriteReg(0x3B, 0x1D);
            // RegInvertIq2 standard default <-- ADDED FOR SX1276 RESET

                // Assemble the exact framework
                idx = 0;
@@ -421,227 +417,95 @@ void StartLoRaTask(void *argument)
                break;


-                           case STATE_TX:
-                               // 1. Synchronously wait for the
physical transmission to clear the airwaves (PE0 / EXTI0)
-                               ulTaskNotifyTake(pdTRUE, portMAX_DELAY);
-
-                               // 2. Acknowledge and drop the hardware
radio TX flag
-                               Lora_WriteReg(0x12, 0x08);
-
-                               // 3. Save current counter immediately
to maintain timeline accuracy
-
Save_Frame_Counter_Wear_Leveled(frame_counter);
-                               frame_counter++;
-
-                               HAL_UART_Transmit(&huart1,
(uint8_t*)"[LoRa] -> State: TX Done. Scheduling low-power 980ms
wait...\r\n", 61, 100);
-
-        // 4. Shift modem parameters over to Receive mappings while in
Standby
-    Lora_WriteReg(REG_DIO_MAPPING_1, 0x00); // Re-map DIO0 to watch for
RxDone
-    Lora_WriteReg(0x33, 0x66);              // Invert IQ for gateway
synchronization
```

```
-    Lora_WriteReg(0x3B, 0x19);              // Invert IQ 2 (SX1276
critical patch)
-
-    // 5. FIX: Hard-write 0x1E instead of read-modifying to prevent
Spreading Factor corruption.
-    // 0x73 = Spreading Factor 7 (0x70) | No continuous mode (0x00) | No
Rx CRC (0x00) | SymbTimeout MSB (0x03)
-    Lora_WriteReg(0x1E, 0x73);
-    Lora_WriteReg(0x1F, 0xFF);              // Max out the lower 8 bits
of SymbTimeout to 255
-
-    // 6. FIX: Reset the internal FIFO pointer to the RX base address
before listening
-    uint8_t rx_base_addr = Lora_ReadReg(0x0F); // Read RegFifoRxBaseAddr
-    Lora_WriteReg(0x0D, rx_base_addr);         // Reset RegFifoAddrPtr
to base point
-
-    // 7. THE 980ms FIRST SLEEP PHASE: Wait for the RTC alarm clock to
ring
-    if (HAL_RTCEx_SetWakeUpTimer_IT(&hrtc, 2007,
RTC_WAKEUPCLOCK_RTCCLK_DIV16) != HAL_OK)
-    {
-        Error_Handler();
-    }
-
-
-                                vTaskSuspendAll();
-
HAL_PWR_EnterSTOPMode(PWR_LOWPOWERREGULATOR_ON, PWR_STOPENTRY_WFI);
-
-                                //
==================================================================
-                                // WAKEUP EVENT 1: The 980ms RTC Alarm
expires! The MCU resumes here.
-                                //
==================================================================
-                                SystemClock_Config();
-                                HAL_ResumeTick();
-                                xTaskResumeAll();
-
-                                HAL_RTCEx_DeactivateWakeUpTimer(&hrtc);
-
-    HAL_UART_Transmit(&huart1, (uint8_t*)"[LoRa] -> RTC Alert! Opening
RX1 window precisely on time...\r\n", 61, 100);
-
-    // 7. Strobe the radio into active single-receive mode
-    Lora_WriteReg(REG_OP_MODE, MODE_LONG_RANGE_MODE | MODE_RX_SINGLE);
-    HAL_Delay(2);
-
-    // --- NEW DIAGNOSTIC READBACK BLOCK ---
-    uint8_t verified_mode = Lora_ReadReg(0x01); // Read REG_OP_MODE
-    uint8_t verified_flags = Lora_ReadReg(0x12); // Read REG_IRQ_FLAGS
-
-    char debug_msg[128];
-    int debug_len = snprintf(debug_msg, sizeof(debug_msg),
-                             "[LoRa Diagnostic] Mode Reg: 0x%02X | IRQ
Flags: 0x%02X\r\n",
-                             verified_mode, verified_flags);
```

```
-    HAL_UART_Transmit(&huart1, (uint8_t*)debug_msg, debug_len, 100);
-    // -------------------------------------
-
-    // Move state pointer forward to the receive state
-    current_state = STATE_RX1;
-    break;
-
-
-
-
-
-                        case STATE_RX1:
-                            // 1. THE SECOND SLEEP PHASE: Safely lock the
MCU core down *inside* the state itself
-                //        vTaskSuspendAll();
-                        //
HAL_PWR_EnterSTOPMode(PWR_LOWPOWERREGULATOR_ON, PWR_STOPENTRY_WFI);
-
-                            //
=====================================================================
-                            // WAKEUP EVENT 2: Radio physically asserts
PE0 (RxDone) or PE1 (RxTimeout)!
-                            //
=====================================================================
-                        //    SystemClock_Config(); // Restore full CPU
operational speeds
-                //        HAL_ResumeTick();
-                        //    xTaskResumeAll();
-
-                            // 2. Consume the pending FreeRTOS task
notification variables cleanly
-                            ulTaskNotifyTake(pdTRUE, portMAX_DELAY);
-
-                            // 3. Capture and reset radio flags
immediately to release the physical hardware lines
-                            irqFlags = Lora_ReadReg(0x12);
-                            Lora_WriteReg(0x12, 0xFF);
-
-                            // 4. Evaluate matching metrics (PayloadReady
is High, PayloadCrcError is Low)
-                            if ((irqFlags & 0x40) && !(irqFlags & 0x20)) {
-                                HAL_UART_Transmit(&huart1,
(uint8_t*)"[LoRa] -> State: RX1 SUCCESS! Valid ACK packet caught.\r\n",
56, 100);
-
-                                uint8_t payload_length =
Lora_ReadReg(0x13); // RegRxNbBytes
-
-                                if (payload_length > 0 && payload_length
<= 256)
-                                {
-                                    uint8_t current_fifo_addr =
Lora_ReadReg(0x10); // RegFifoRxCurrentAddr
-                                    Lora_WriteReg(0x0D,
current_fifo_addr);         // RegFifoAddrPtr
-
-                                    for (uint8_t i = 0; i <
payload_length; i++)
```

```
-                                   {
-                                       lora_rx_packet_buffer[i] =
Lora_ReadReg(0x00); // RegFifo
-                                   }
-
-                                   char hex_print_buf[128];
-                                   int len = snprintf(hex_print_buf,
sizeof(hex_print_buf), "[LoRa] Extracted %d bytes: ", payload_length);
-                                   HAL_UART_Transmit(&huart1,
(uint8_t*)hex_print_buf, len, 100);
-
-                                   for (uint8_t i = 0; i <
payload_length; i++)
-                                   {
-                                       len = snprintf(hex_print_buf,
sizeof(hex_print_buf), "%02X ", lora_rx_packet_buffer[i]);
-                                       HAL_UART_Transmit(&huart1,
(uint8_t*)hex_print_buf, len, 100);
-                                   }
-                                   HAL_UART_Transmit(&huart1,
(uint8_t*)"\r\n", 2, 100);
-                               }
-               } else if (irqFlags & 0x20) {
-                   HAL_UART_Transmit(&huart1, (uint8_t*)"[LoRa] -> State:
RX1 Packet caught but failed CRC.\r\n", 52, 100);
-
-                   // Handle CRC failure by routing to sleep normally
-                   Lora_WriteReg(0x01, 0x80 | 0x01);     // Force Standby
mode
-                   uint8_t rx_base = Lora_ReadReg(0x0F); // Read
RegFifoRxBaseAddr
-                   Lora_WriteReg(0x0D, rx_base);         // Reset
RegFifoAddrPtr to base point
-                   current_state = STATE_SLEEP;
-               } else {
-                   // --- ROUTE TIMEOUT TO RX2 INSTEAD OF SLEEP ---
-                   HAL_UART_Transmit(&huart1, (uint8_t*)"[LoRa] -> RX1
Timeout. Pivoting to RX2 (869.525MHz, SF9)...\r\n", 59, 100);
-
-                   // Move to Standby briefly to change config registers
safely
-                   Lora_WriteReg(0x01, 0x80 | 0x01);
-
-                   // Reprogram to mandatory EU868 RX2 fixed frequency
-                   Lora_SetFrequency(869525000);
-
-                   // Set Spreading Factor 9 (SF9) for RX2 window
-                   Lora_WriteReg(0x1E, 0x93);
-                   Lora_WriteReg(0x1F, 0xFF);
-
-                   // Reset FIFO memory pointers to base RX block
-                   uint8_t rx_base_addr = Lora_ReadReg(0x0F);
-                   Lora_WriteReg(0x0D, rx_base_addr);
-
-                   // Strobe the radio receiver into active single-
receive mode
-                   Lora_WriteReg(REG_OP_MODE, MODE_LONG_RANGE_MODE |
MODE_RX_SINGLE);
```

```
-                   HAL_Delay(2); // Let fractional-N synthesizer lock
onto 869.525MHz
-
-                   // Update state machine pointer to look at the new
block next loop
-                   current_state = STATE_RX2;
-               }
+           case STATE_TX:
+               // 1. Synchronously wait for the physical transmission to
clear the airwaves
+               ulTaskNotifyTake(pdTRUE, portMAX_DELAY);
+
+               // 2. Acknowledge and drop the hardware flag
+               Lora_WriteReg(0x12, 0x08);

-               break; // Master break cleanly exits STATE_RX1
+               // 3. Save current counter immediately to maintain
timeline accuracy
+               Save_Frame_Counter_Wear_Leveled(frame_counter);
+               frame_counter++;

+               HAL_UART_Transmit(&huart1, (uint8_t*)"[LoRa] -> State: TX
Done. Starting 1s countdown to RX1...\r\n", 58, 100);



-           case STATE_RX2:
-               // 1. THE LOW-POWER WAITING PHASE: Suspend task ticks and
sleep the STM32F4 core
-               vTaskSuspendAll();
-               HAL_PWR_EnterSTOPMode(PWR_LOWPOWERREGULATOR_ON,
PWR_STOPENTRY_WFI);
+               // 5. Shift modem parameters over to Receive mappings
+               Lora_WriteReg(REG_DIO_MAPPING_1, 0x00); // Re-map DIO0 to
watch for RxDone
+               Lora_WriteReg(0x33, 0x66);              // Invert IQ for
gateway synchronization
+         //      Lora_WriteReg(0x3B, 0x19);

-               //
==========================================================================
-               // PHYSICAL WAKEUP: PE0 (RxDone) or PE1 (RxTimeout)
asserts for RX2!
-               //
==========================================================================
-               SystemClock_Config(); // Instantly restore 168MHz HSE +
PLL clock trees
-               HAL_ResumeTick();
-               xTaskResumeAll();
-
-               // 2. Safely consume the pending FreeRTOS task
notification flag
-               ulTaskNotifyTake(pdTRUE, portMAX_DELAY);

-               // 3. Read and clear active radio flags to drop physical
PE0/PE1 lines
+               // --- FIXED: MAX OUT THE HARDWARE TIMEOUT NET ---
```

```
+              // Force the upper 2 bits of SymbTimeout to 1 (Register
0x1E bits 1:0)
+              Lora_WriteReg(0x1E, Lora_ReadReg(0x1E) | 0x03);
+              // Max out the lower 8 bits of SymbTimeout to 255
(Register 0x1F)
+              Lora_WriteReg(0x1F, 0xFF);
+              // Total timeout is now 1023 symbols (~1047ms), preventing
premature closing.
+
+
+              // Move pointer and open the window
+              current_state = STATE_RX1;
+
+              // 4. Strict 1-second delay execution
+              osDelay(pdMS_TO_TICKS(950UL));
+
+              Lora_WriteReg(REG_OP_MODE, MODE_LONG_RANGE_MODE |
MODE_RX_SINGLE);
+              break;
+
+
+           case STATE_RX1:
+              // 1. Safety net wait to handle automated silent timeout
windows gracefully
+              BaseType_t rx_notified = xTaskNotifyWait(0x00, ULONG_MAX,
NULL, pdMS_TO_TICKS(200));
+
+              // 2. Capture and reset flags to guarantee safe loop re-
entry paths
               irqFlags = Lora_ReadReg(0x12);
               Lora_WriteReg(0x12, 0xFF);

-              // 4. Evaluate RX2 packet metrics (PayloadReady is High,
PayloadCrcError is Low)
-              if ((irqFlags & 0x40) && !(irqFlags & 0x20))
-              {
-                  HAL_UART_Transmit(&huart1, (uint8_t*)"[LoRa] -> State:
RX2 SUCCESS! Valid Downlink Caught.\r\n", 55, 100);
+              // 3. Evaluate matching metrics
+              if (rx_notified == pdTRUE && (irqFlags & 0x40) &&
!(irqFlags & 0x20)) {
+                  HAL_UART_Transmit(&huart1, (uint8_t*)"[LoRa] -> State:
RX1 SUCCESS! Valid ACK packet caught.\r\n", 56, 100);

+                  // Read length securely
                   uint8_t payload_length = Lora_ReadReg(0x13); //
RegRxNbBytes

-                  if (payload_length > 0 && payload_length <= 256)
-                  {
-                      uint8_t current_fifo_addr = Lora_ReadReg(0x10);
-                      Lora_WriteReg(0x0D, current_fifo_addr);
+                  // HARD BOUNDARY CHECK: Prevent memory overruns
+                  if (payload_length > 0 && payload_length <= 256) {
+                      uint8_t current_fifo_addr = Lora_ReadReg(0x10); //
RegFifoRxCurrentAddr
+                      Lora_WriteReg(0x0D, current_fifo_addr);         //
RegFifoAddrPtr
```

```
+                     // Stream safely into the global/static buffer
                      for (uint8_t i = 0; i < payload_length; i++) {
-                         lora_rx_packet_buffer[i] = Lora_ReadReg(0x00);
+                         lora_rx_packet_buffer[i] = Lora_ReadReg(0x00);
// RegFifo
                      }

+                     // Print out the raw hexadecimal values over UART
                      char hex_print_buf[128];
-                     int len = snprintf(hex_print_buf,
sizeof(hex_print_buf), "[LoRa RX2] Extracted %d bytes: ",
payload_length);
+                     int len = snprintf(hex_print_buf,
sizeof(hex_print_buf), "[LoRa] Extracted %d bytes: ", payload_length);
                      HAL_UART_Transmit(&huart1,
(uint8_t*)hex_print_buf, len, 100);

+                     // Safely display the data bytes
                      for (uint8_t i = 0; i < payload_length; i++) {
+                         // Bounded print to prevent string buffer
overflow
                          len = snprintf(hex_print_buf,
sizeof(hex_print_buf), "%02X ", lora_rx_packet_buffer[i]);
                          HAL_UART_Transmit(&huart1,
(uint8_t*)hex_print_buf, len, 100);
                      }
                      HAL_UART_Transmit(&huart1, (uint8_t*)"\r\n", 2,
100);
                  }
-             }
-             // 5. Handle standard RX2 window timeout or frame
corruption closures
-             else
-             {
-                 HAL_UART_Transmit(&huart1, (uint8_t*)"[LoRa] -> State:
RX2 Closed (Timeout/No Response).\r\n", 52, 100);
+             } else if (irqFlags & 0x20) {
+                 HAL_UART_Transmit(&huart1, (uint8_t*)"[LoRa] -> State:
RX1 Packet caught but failed CRC.\r\n", 52, 100);
+             } else {
+                 HAL_UART_Transmit(&huart1, (uint8_t*)"[LoRa] -> State:
RX1 Closed (Timeout/No Response).\r\n", 52, 100);
              }

-             // 6. HARDWARE CLEANUP: Put the radio back to standby mode
-             Lora_WriteReg(0x01, 0x80 | 0x01);
-
-             // 7. MANDATORY CRITICAL PROTOCOL FIX: Re-read the active
dynamic channel frequency
-             // and update the radio register right now. If you omit
this, your next transmission
-             // will accidentally broadcast on 869.525MHz, resulting in
network drops.
-             active_tx_frequency = eu868_channels[channel_index];
-             Lora_SetFrequency(active_tx_frequency);
-
```

```
-              // Restore Spreading Factor 7 default settings for your
upcoming transmission sequences
-              Lora_WriteReg(0x1E, 0x73);
+              // 4. CLEAN THE HARDWARE POINTERS SAFELY
+              Lora_WriteReg(0x01, 0x80 | 0x01);     // Force Standby
mode
+              uint8_t rx_base = Lora_ReadReg(0x0F); // Read
RegFifoRxBaseAddr
+              Lora_WriteReg(0x0D, rx_base);         // Reset
RegFifoAddrPtr to base point

-              // 8. Safely advance to your 30-second low-power cooldown
state
+              // 5. Advance to system sleep securely
               current_state = STATE_SLEEP;
               break;

@@ -671,6 +535,7 @@ void StartLoRaTask(void *argument)
   /* USER CODE END StartLoRaTask */
 }

+
 /* Private application code ------------------------------------------
------*/
 /* USER CODE BEGIN Application */

@@ -694,35 +559,30 @@ uint8_t Lora_ReadReg(uint8_t addr) {
 }

 // This function automatically fires the exact millisecond a packet
leaves the antenna
-// This function automatically fires the exact millisecond a packet
leaves the antenna or a timeout occurs
 void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
 {
-    if (GPIO_Pin == LORA_DIO0_Pin) // Check if the interrupt came from
PE0 (RxDone / TxDone)
-    {
-        BaseType_t xHigherPriorityTaskWoken = pdFALSE;
-
-        // ONLY notify the waiting FreeRTOS task. Do no SPI, do no UART!
-        if (LoRaTaskHandle != NULL) {
-            vTaskNotifyGiveFromISR(LoRaTaskHandle,
&xHigherPriorityTaskWoken);
-        }
+  if (GPIO_Pin == LORA_DIO0_Pin) // Check if the interrupt came from PE0
+  {
+     BaseType_t xHigherPriorityTaskWoken = pdFALSE;

-        // Force FreeRTOS to instantly switch to the LoRa task if it has
high priority
-        portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
+    // ONLY notify the waiting FreeRTOS task. Do no SPI, do no UART!
+    if (LoRaTaskHandle != NULL) {
+        vTaskNotifyGiveFromISR(LoRaTaskHandle,
&xHigherPriorityTaskWoken);
     }
-    // --- ADD THIS NEW BLOCK TO CATCH THE TIMEOUT ---
```

```
-    else if (GPIO_Pin == GPIO_PIN_1) // Check if the interrupt came from
PE1 (RxTimeout)
-    {
-        BaseType_t xHigherPriorityTaskWoken = pdFALSE;
-
-        if (LoRaTaskHandle != NULL) {
-            vTaskNotifyGiveFromISR(LoRaTaskHandle,
&xHigherPriorityTaskWoken);
-        }

-        portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
-    }
+    // Force FreeRTOS to instantly switch to the LoRa task if it has
high priority
+    portYIELD_FROM_ISR(xHigherPriorityTaskWoken);
+
+
+    // Clear the radio's internal IRQ flags so it can transmit again
next time
+    // Register 0x12 is RegIrqFlags. Writing 0x08 clears the TxDone
flag.
+   // Lora_WriteReg(0x12, 0x08);
+
+    // Print immediate confirmation to your serial monitor
+   // HAL_UART_Transmit(&huart1, (uint8_t*)"[LoRa] -> TXDone Interrupt
Received! Packet is in the air.\r\n", 60, 10);
+  }
 }

-
 /**
  * @brief  Reads back the contents of the radio's TX FIFO to verify
exactly
  *         what bytes are staged for transmission.
diff --git a/Core/Src/gpio.c b/Core/Src/gpio.c
index e01b934..90966ac 100644
--- a/Core/Src/gpio.c
+++ b/Core/Src/gpio.c
@@ -46,8 +46,8 @@ void MX_GPIO_Init(void)

   /* GPIO Ports Clock Enable */
   __HAL_RCC_GPIOE_CLK_ENABLE();
-  __HAL_RCC_GPIOC_CLK_ENABLE();
   __HAL_RCC_GPIOH_CLK_ENABLE();
+  __HAL_RCC_GPIOC_CLK_ENABLE();
   __HAL_RCC_GPIOA_CLK_ENABLE();
   __HAL_RCC_GPIOB_CLK_ENABLE();

diff --git a/Core/Src/main.c b/Core/Src/main.c
index 5f70e6a..bed2223 100644
--- a/Core/Src/main.c
+++ b/Core/Src/main.c
@@ -19,7 +19,6 @@
 /* Includes ------------------------------------------------------------
------*/
 #include "main.h"
 #include "cmsis_os.h"
-#include "rtc.h"
```

```
 #include "spi.h"
 #include "tim.h"
 #include "usart.h"
@@ -122,7 +121,6 @@ int main(void)
   MX_TIM1_Init();
   MX_SPI2_Init();
   MX_TIM2_Init();
-  MX_RTC_Init();
   /* USER CODE BEGIN 2 */
 // Pristine, safe startup scanner without any task management blocks
 printf("\r\n--- [Full-Sector 7 Scanner Active] ---\r\n");
@@ -203,9 +201,8 @@ void SystemClock_Config(void)
   /** Initializes the RCC Oscillators according to the specified
parameters
   * in the RCC_OscInitTypeDef structure.
   */
-  RCC_OscInitStruct.OscillatorType =
RCC_OSCILLATORTYPE_LSI|RCC_OSCILLATORTYPE_HSE;
+  RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
   RCC_OscInitStruct.HSEState = RCC_HSE_ON;
-  RCC_OscInitStruct.LSIState = RCC_LSI_ON;
   RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
   RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
   RCC_OscInitStruct.PLL.PLLM = 8;
diff --git a/Core/Src/rtc.c b/Core/Src/rtc.c
deleted file mode 100644
index dc4901d..0000000
--- a/Core/Src/rtc.c
+++ /dev/null
@@ -1,148 +0,0 @@
-/* USER CODE BEGIN Header */
-/**
-
******************************************************************************
*****
-  * @file    rtc.c
-  * @brief   This file provides code for the configuration
-  *          of the RTC instances.
-
******************************************************************************
*****
-  * @attention
-  *
-  * Copyright (c) 2026 STMicroelectronics.
-  * All rights reserved.
-  *
-  * This software is licensed under terms that can be found in the
LICENSE file
-  * in the root directory of this software component.
-  * If no LICENSE file comes with this software, it is provided AS-IS.
-  *
-
******************************************************************************
*****
-  */
-/* USER CODE END Header */
-/* Includes ------------------------------------------------------------
------*/
```

```
-#include "rtc.h"
-
-/* USER CODE BEGIN 0 */
-
-/* USER CODE END 0 */
-
-RTC_HandleTypeDef hrtc;
-
-/* RTC init function */
-void MX_RTC_Init(void)
-{
-
-  /* USER CODE BEGIN RTC_Init 0 */
-
-  /* USER CODE END RTC_Init 0 */
-
-  RTC_TimeTypeDef sTime = {0};
-  RTC_DateTypeDef sDate = {0};
-
-  /* USER CODE BEGIN RTC_Init 1 */
-
-  /* USER CODE END RTC_Init 1 */
-
-  /** Initialize RTC Only
-  */
-  hrtc.Instance = RTC;
-  hrtc.Init.HourFormat = RTC_HOURFORMAT_24;
-  hrtc.Init.AsynchPrediv = 127;
-  hrtc.Init.SynchPrediv = 255;
-  hrtc.Init.OutPut = RTC_OUTPUT_DISABLE;
-  hrtc.Init.OutPutPolarity = RTC_OUTPUT_POLARITY_HIGH;
-  hrtc.Init.OutPutType = RTC_OUTPUT_TYPE_OPENDRAIN;
-  if (HAL_RTC_Init(&hrtc) != HAL_OK)
-  {
-    Error_Handler();
-  }
-
-  /* USER CODE BEGIN Check_RTC_BKUP */
-
-  /* USER CODE END Check_RTC_BKUP */
-
-  /** Initialize RTC and set the Time and Date
-  */
-  sTime.Hours = 0x0;
-  sTime.Minutes = 0x0;
-  sTime.Seconds = 0x0;
-  sTime.DayLightSaving = RTC_DAYLIGHTSAVING_NONE;
-  sTime.StoreOperation = RTC_STOREOPERATION_RESET;
-  if (HAL_RTC_SetTime(&hrtc, &sTime, RTC_FORMAT_BCD) != HAL_OK)
-  {
-    Error_Handler();
-  }
-  sDate.WeekDay = RTC_WEEKDAY_MONDAY;
-  sDate.Month = RTC_MONTH_JANUARY;
-  sDate.Date = 0x1;
-  sDate.Year = 0x0;
-
-  if (HAL_RTC_SetDate(&hrtc, &sDate, RTC_FORMAT_BCD) != HAL_OK)
```

```
-  {
-    Error_Handler();
-  }
-
-  /** Enable the WakeUp
-  */
-  if (HAL_RTCEx_SetWakeUpTimer_IT(&hrtc, 0,
RTC_WAKEUPCLOCK_RTCCLK_DIV16) != HAL_OK)
-  {
-    Error_Handler();
-  }
-  /* USER CODE BEGIN RTC_Init 2 */
-
-  /* USER CODE END RTC_Init 2 */
-
-}
-
-void HAL_RTC_MspInit(RTC_HandleTypeDef* rtcHandle)
-{
-
-  RCC_PeriphCLKInitTypeDef PeriphClkInitStruct = {0};
-  if(rtcHandle->Instance==RTC)
-  {
-  /* USER CODE BEGIN RTC_MspInit 0 */
-
-  /* USER CODE END RTC_MspInit 0 */
-
-  /** Initializes the peripherals clock
-  */
-    PeriphClkInitStruct.PeriphClockSelection = RCC_PERIPHCLK_RTC;
-    PeriphClkInitStruct.RTCClockSelection = RCC_RTCCLKSOURCE_LSI;
-    if (HAL_RCCEx_PeriphCLKConfig(&PeriphClkInitStruct) != HAL_OK)
-    {
-      Error_Handler();
-    }
-
-    /* RTC clock enable */
-    __HAL_RCC_RTC_ENABLE();
-
-    /* RTC interrupt Init */
-    HAL_NVIC_SetPriority(RTC_WKUP_IRQn, 5, 0);
-    HAL_NVIC_EnableIRQ(RTC_WKUP_IRQn);
-  /* USER CODE BEGIN RTC_MspInit 1 */
-
-  /* USER CODE END RTC_MspInit 1 */
-  }
-}
-
-void HAL_RTC_MspDeInit(RTC_HandleTypeDef* rtcHandle)
-{
-
-  if(rtcHandle->Instance==RTC)
-  {
-  /* USER CODE BEGIN RTC_MspDeInit 0 */
-
-  /* USER CODE END RTC_MspDeInit 0 */
-    /* Peripheral clock disable */
-    __HAL_RCC_RTC_DISABLE();
```

```
-
-    /* RTC interrupt Deinit */
-    HAL_NVIC_DisableIRQ(RTC_WKUP_IRQn);
-  /* USER CODE BEGIN RTC_MspDeInit 1 */
-
-  /* USER CODE END RTC_MspDeInit 1 */
-  }
-}
-
-/* USER CODE BEGIN 1 */
-
-/* USER CODE END 1 */
-
diff --git a/Core/Src/stm32f4xx_it.c b/Core/Src/stm32f4xx_it.c
index f1f624e..89bfbdf 100644
--- a/Core/Src/stm32f4xx_it.c
+++ b/Core/Src/stm32f4xx_it.c
@@ -55,7 +55,6 @@
 /* USER CODE END 0 */

 /* External variables ---------------------------------------------------
------*/
-extern RTC_HandleTypeDef hrtc;
 extern TIM_HandleTypeDef htim2;
 extern TIM_HandleTypeDef htim6;

@@ -161,20 +160,6 @@ void DebugMon_Handler(void)
 /* please refer to the startup file (startup_stm32f4xx.s).
*/

/************************************************************************
******/

-/**
-  * @brief This function handles RTC wake-up interrupt through EXTI line
22.
-  */
-void RTC_WKUP_IRQHandler(void)
-{
-  /* USER CODE BEGIN RTC_WKUP_IRQn 0 */
-
-  /* USER CODE END RTC_WKUP_IRQn 0 */
-  HAL_RTCEx_WakeUpTimerIRQHandler(&hrtc);
-  /* USER CODE BEGIN RTC_WKUP_IRQn 1 */
-
-  /* USER CODE END RTC_WKUP_IRQn 1 */
-}
-
 /**
   * @brief This function handles EXTI line0 interrupt.
   */
diff --git a/Drivers/STM32F4xx_HAL_Driver/Inc/stm32f4xx_hal_rtc.h
b/Drivers/STM32F4xx_HAL_Driver/Inc/stm32f4xx_hal_rtc.h
deleted file mode 100644
index d787b4b..0000000
--- a/Drivers/STM32F4xx_HAL_Driver/Inc/stm32f4xx_hal_rtc.h
+++ /dev/null
@@ -1,922 +0,0 @@
```

```
-/**
-
******************************************************************************
*****
-  * @file    stm32f4xx_hal_rtc.h
-  * @author  MCD Application Team
-  * @brief   Header file of RTC HAL module.
-
******************************************************************************
*****
-  * @attention
-  *
-  * Copyright (c) 2016 STMicroelectronics.
-  * All rights reserved.
-  *
-  * This software is licensed under terms that can be found in the
LICENSE file
-  * in the root directory of this software component.
-  * If no LICENSE file comes with this software, it is provided AS-IS.
-  *
-
******************************************************************************
*****
-  */
-
-/* Define to prevent recursive inclusion -------------------------------
------*/
-#ifndef STM32F4xx_HAL_RTC_H
-#define STM32F4xx_HAL_RTC_H
-
-#ifdef __cplusplus
-extern "C" {
-#endif
-
-/* Includes ------------------------------------------------------------
------*/
-
-#include "stm32f4xx_hal_def.h"
-
-/** @addtogroup STM32F4xx_HAL_Driver
-  * @{
-  */
-
-/** @addtogroup RTC
-  * @{
-  */
-
-/* Exported types ------------------------------------------------------
------*/
-
-/** @defgroup RTC_Exported_Types RTC Exported Types
-  * @{
-  */
-
-/**
-  * @brief  HAL State structures definition
-  */
-typedef enum
```

```
-{
-  HAL_RTC_STATE_RESET             = 0x00U,  /*!< RTC not yet initialized
or disabled */
-  HAL_RTC_STATE_READY             = 0x01U,  /*!< RTC initialized and
ready for use   */
-  HAL_RTC_STATE_BUSY              = 0x02U,  /*!< RTC process is ongoing
*/
-  HAL_RTC_STATE_TIMEOUT           = 0x03U,  /*!< RTC timeout state
*/
-  HAL_RTC_STATE_ERROR             = 0x04U   /*!< RTC error state
*/
-} HAL_RTCStateTypeDef;
-
-/**
-  * @brief  RTC Configuration Structure definition
-  */
-typedef struct
-{
-  uint32_t HourFormat;      /*!< Specifies the RTC Hour Format.
-                                 This parameter can be a value of @ref
RTC_Hour_Formats */
-
-  uint32_t AsynchPrediv;    /*!< Specifies the RTC Asynchronous
Predivider value.
-                                 This parameter must be a number between
Min_Data = 0x00 and Max_Data = 0x7F */
-
-  uint32_t SynchPrediv;     /*!< Specifies the RTC Synchronous
Predivider value.
-                                 This parameter must be a number between
Min_Data = 0x0000 and Max_Data = 0x7FFF */
-
-  uint32_t OutPut;          /*!< Specifies which signal will be routed
to the RTC output.
-                                 This parameter can be a value of @ref
RTC_Output_selection_Definitions */
-
-  uint32_t OutPutPolarity;  /*!< Specifies the polarity of the output
signal.
-                                 This parameter can be a value of @ref
RTC_Output_Polarity_Definitions */
-
-  uint32_t OutPutType;      /*!< Specifies the RTC Output Pin mode.
-                                 This parameter can be a value of @ref
RTC_Output_Type_ALARM_OUT */
-} RTC_InitTypeDef;
-
-/**
-  * @brief  RTC Time structure definition
-  */
-typedef struct
-{
-  uint8_t Hours;            /*!< Specifies the RTC Time Hour.
-                                 This parameter must be a number between
Min_Data = 0 and Max_Data = 12 if the RTC_HourFormat_12 is selected
-                                 This parameter must be a number between
Min_Data = 0 and Max_Data = 23 if the RTC_HourFormat_24 is selected */
-
```

```
-  uint8_t Minutes;          /*!< Specifies the RTC Time Minutes.
-                                This parameter must be a number between
Min_Data = 0 and Max_Data = 59 */
-
-  uint8_t Seconds;          /*!< Specifies the RTC Time Seconds.
-                                This parameter must be a number between
Min_Data = 0 and Max_Data = 59 */
-
-  uint8_t TimeFormat;       /*!< Specifies the RTC AM/PM Time.
-                                This parameter can be a value of @ref
RTC_AM_PM_Definitions */
-
-  uint32_t SubSeconds;      /*!< Specifies the RTC_SSR RTC Sub Second
register content.
-                                This parameter corresponds to a time
unit range between [0-1] Second
-                                with [1 Sec / SecondFraction +1]
granularity */
-
-  uint32_t SecondFraction;  /*!< Specifies the range or granularity of
Sub Second register content
-                                corresponding to Synchronous prescaler
factor value (PREDIV_S)
-                                This parameter corresponds to a time
unit range between [0-1] Second
-                                with [1 Sec / SecondFraction +1]
granularity.
-                                This field will be used only by
HAL_RTC_GetTime function */
-
-  uint32_t DayLightSaving;  /*!< This interface is deprecated. To manage
Daylight
-                                Saving Time, please use HAL_RTC_DST_xxx
functions */
-
-  uint32_t StoreOperation;  /*!< This interface is deprecated. To manage
Daylight
-                                Saving Time, please use HAL_RTC_DST_xxx
functions */
-} RTC_TimeTypeDef;
-
-/**
-  * @brief  RTC Date structure definition
-  */
-typedef struct
-{
-  uint8_t WeekDay;  /*!< Specifies the RTC Date WeekDay.
-                         This parameter can be a value of @ref
RTC_WeekDay_Definitions */
-
-  uint8_t Month;    /*!< Specifies the RTC Date Month (in BCD format).
-                         This parameter can be a value of @ref
RTC_Month_Date_Definitions */
-
-  uint8_t Date;     /*!< Specifies the RTC Date.
-                         This parameter must be a number between
Min_Data = 1 and Max_Data = 31 */
-
```

```
-  uint8_t Year;     /*!< Specifies the RTC Date Year.
-                        This parameter must be a number between
Min_Data = 0 and Max_Data = 99 */
-
-} RTC_DateTypeDef;
-
-/**
-  * @brief  RTC Alarm structure definition
-  */
-typedef struct
-{
-  RTC_TimeTypeDef AlarmTime;     /*!< Specifies the RTC Alarm Time
members */
-
-  uint32_t AlarmMask;            /*!< Specifies the RTC Alarm Masks.
-                                      This parameter can be a value of
@ref RTC_AlarmMask_Definitions */
-
-  uint32_t AlarmSubSecondMask;   /*!< Specifies the RTC Alarm SubSeconds
Masks.
-                                      This parameter can be a value of
@ref RTC_Alarm_Sub_Seconds_Masks_Definitions */
-
-  uint32_t AlarmDateWeekDaySel;  /*!< Specifies the RTC Alarm is on Date
or WeekDay.
-                                      This parameter can be a value of
@ref RTC_AlarmDateWeekDay_Definitions */
-
-  uint8_t AlarmDateWeekDay;      /*!< Specifies the RTC Alarm
Date/WeekDay.
-                                      If the Alarm Date is selected,
this parameter must be set to a value in the 1-31 range.
-                                      If the Alarm WeekDay is selected,
this parameter can be a value of @ref RTC_WeekDay_Definitions */
-
-  uint32_t Alarm;                /*!< Specifies the alarm .
-                                      This parameter can be a value of
@ref RTC_Alarms_Definitions */
-} RTC_AlarmTypeDef;
-
-/**
-  * @brief  RTC Handle Structure definition
-  */
-#if (USE_HAL_RTC_REGISTER_CALLBACKS == 1)
-typedef struct __RTC_HandleTypeDef
-#else
-typedef struct
-#endif /* USE_HAL_RTC_REGISTER_CALLBACKS */
-{
-  RTC_TypeDef                 *Instance;  /*!< Register base address
*/
-
-  RTC_InitTypeDef             Init;       /*!< RTC required parameters
*/
-
-  HAL_LockTypeDef             Lock;       /*!< RTC locking object
*/
-
```

```
-  __IO HAL_RTCStateTypeDef    State;      /*!< Time communication state
*/
-
-#if (USE_HAL_RTC_REGISTER_CALLBACKS == 1)
-  void (* AlarmAEventCallback)      (struct __RTC_HandleTypeDef *hrtc);
/*!< RTC Alarm A Event callback        */
-
-  void (* AlarmBEventCallback)      (struct __RTC_HandleTypeDef *hrtc);
/*!< RTC Alarm B Event callback        */
-
-  void (* TimeStampEventCallback)   (struct __RTC_HandleTypeDef *hrtc);
/*!< RTC Timestamp Event callback      */
-
-  void (* WakeUpTimerEventCallback) (struct __RTC_HandleTypeDef *hrtc);
/*!< RTC WakeUpTimer Event callback    */
-
-  void (* Tamper1EventCallback)     (struct __RTC_HandleTypeDef *hrtc);
/*!< RTC Tamper 1 Event callback       */
-
-#if defined(RTC_TAMPER2_SUPPORT)
-  void (* Tamper2EventCallback)     (struct __RTC_HandleTypeDef *hrtc);
/*!< RTC Tamper 2 Event callback       */
-#endif /* RTC_TAMPER2_SUPPORT */
-
-  void (* MspInitCallback)          (struct __RTC_HandleTypeDef *hrtc);
/*!< RTC Msp Init callback             */
-
-  void (* MspDeInitCallback)        (struct __RTC_HandleTypeDef *hrtc);
/*!< RTC Msp DeInit callback           */
-
-#endif /* USE_HAL_RTC_REGISTER_CALLBACKS */
-
-} RTC_HandleTypeDef;
-
-#if (USE_HAL_RTC_REGISTER_CALLBACKS == 1)
-/**
-  * @brief  HAL RTC Callback ID enumeration definition
-  */
-typedef enum
-{
-  HAL_RTC_ALARM_A_EVENT_CB_ID          = 0x00U,    /*!< RTC Alarm A
Event Callback ID       */
-  HAL_RTC_ALARM_B_EVENT_CB_ID          = 0x01U,    /*!< RTC Alarm B
Event Callback ID       */
-  HAL_RTC_TIMESTAMP_EVENT_CB_ID        = 0x02U,    /*!< RTC Timestamp
Event Callback ID     */
-  HAL_RTC_WAKEUPTIMER_EVENT_CB_ID      = 0x03U,    /*!< RTC Wakeup
Timer Event Callback ID  */
-  HAL_RTC_TAMPER1_EVENT_CB_ID          = 0x04U,    /*!< RTC Tamper 1
Callback ID            */
-#if defined(RTC_TAMPER2_SUPPORT)
-  HAL_RTC_TAMPER2_EVENT_CB_ID          = 0x05U,    /*!< RTC Tamper 2
Callback ID            */
-#endif /* RTC_TAMPER2_SUPPORT */
-  HAL_RTC_MSPINIT_CB_ID                = 0x0EU,    /*!< RTC Msp Init
callback ID            */
-  HAL_RTC_MSPDEINIT_CB_ID              = 0x0FU     /*!< RTC Msp DeInit
callback ID          */
```

```
-} HAL_RTC_CallbackIDTypeDef;
-
-/**
-  * @brief  HAL RTC Callback pointer definition
-  */
-typedef  void (*pRTC_CallbackTypeDef)(RTC_HandleTypeDef *hrtc);  /*!<
pointer to an RTC callback function */
-#endif /* USE_HAL_RTC_REGISTER_CALLBACKS */
-
-/**
-  * @}
-  */
-
-/* Exported constants --------------------------------------------------
------*/
-
-/** @defgroup RTC_Exported_Constants RTC Exported Constants
-  * @{
-  */
-
-/** @defgroup RTC_Hour_Formats RTC Hour Formats
-  * @{
-  */
-#define RTC_HOURFORMAT_24              0x00000000U
-#define RTC_HOURFORMAT_12              RTC_CR_FMT
-/**
-  * @}
-  */
-
-/** @defgroup RTC_Output_selection_Definitions RTC Output Selection
Definitions
-  * @{
-  */
-#define RTC_OUTPUT_DISABLE             0x00000000U
-#define RTC_OUTPUT_ALARMA              RTC_CR_OSEL_0
-#define RTC_OUTPUT_ALARMB              RTC_CR_OSEL_1
-#define RTC_OUTPUT_WAKEUP              RTC_CR_OSEL
-/**
-  * @}
-  */
-
-/** @defgroup RTC_Output_Polarity_Definitions RTC Output Polarity
Definitions
-  * @{
-  */
-#define RTC_OUTPUT_POLARITY_HIGH       0x00000000U
-#define RTC_OUTPUT_POLARITY_LOW        RTC_CR_POL
-/**
-  * @}
-  */
-
-/** @defgroup RTC_Output_Type_ALARM_OUT RTC Output Type ALARM OUT
-  * @{
-  */
-#define RTC_OUTPUT_TYPE_OPENDRAIN      0x00000000U
-#define RTC_OUTPUT_TYPE_PUSHPULL       RTC_TAFCR_ALARMOUTTYPE
-/**
-  * @}
```

```
-  */
-
-/** @defgroup RTC_AM_PM_Definitions RTC AM PM Definitions
-  * @{
-  */
-#define RTC_HOURFORMAT12_AM             ((uint8_t)0x00)
-#define RTC_HOURFORMAT12_PM             ((uint8_t)0x01)
-/**
-  * @}
-  */
-
-/** @defgroup RTC_DayLightSaving_Definitions RTC DayLight Saving
Definitions
-  * @{
-  */
-#define RTC_DAYLIGHTSAVING_SUB1H        RTC_CR_SUB1H
-#define RTC_DAYLIGHTSAVING_ADD1H        RTC_CR_ADD1H
-#define RTC_DAYLIGHTSAVING_NONE         0x00000000U
-/**
-  * @}
-  */
-
-/** @defgroup RTC_StoreOperation_Definitions RTC Store Operation
Definitions
-  * @{
-  */
-#define RTC_STOREOPERATION_RESET         0x00000000U
-#define RTC_STOREOPERATION_SET           RTC_CR_BKP
-/**
-  * @}
-  */
-
-/** @defgroup RTC_Input_parameter_format_definitions RTC Input Parameter
Format Definitions
-  * @{
-  */
-#define RTC_FORMAT_BIN                   0x00000000U
-#define RTC_FORMAT_BCD                   0x00000001U
-/**
-  * @}
-  */
-
-/** @defgroup RTC_Month_Date_Definitions RTC Month Date Definitions (in
BCD format)
-  * @{
-  */
-#define RTC_MONTH_JANUARY               ((uint8_t)0x01)
-#define RTC_MONTH_FEBRUARY              ((uint8_t)0x02)
-#define RTC_MONTH_MARCH                 ((uint8_t)0x03)
-#define RTC_MONTH_APRIL                 ((uint8_t)0x04)
-#define RTC_MONTH_MAY                   ((uint8_t)0x05)
-#define RTC_MONTH_JUNE                  ((uint8_t)0x06)
-#define RTC_MONTH_JULY                  ((uint8_t)0x07)
-#define RTC_MONTH_AUGUST                ((uint8_t)0x08)
-#define RTC_MONTH_SEPTEMBER             ((uint8_t)0x09)
-#define RTC_MONTH_OCTOBER               ((uint8_t)0x10)
-#define RTC_MONTH_NOVEMBER              ((uint8_t)0x11)
-#define RTC_MONTH_DECEMBER              ((uint8_t)0x12)
```

```
-/**
-  * @}
-  */
-
-/** @defgroup RTC_WeekDay_Definitions RTC WeekDay Definitions
-  * @{
-  */
-#define RTC_WEEKDAY_MONDAY             ((uint8_t)0x01)
-#define RTC_WEEKDAY_TUESDAY            ((uint8_t)0x02)
-#define RTC_WEEKDAY_WEDNESDAY          ((uint8_t)0x03)
-#define RTC_WEEKDAY_THURSDAY           ((uint8_t)0x04)
-#define RTC_WEEKDAY_FRIDAY             ((uint8_t)0x05)
-#define RTC_WEEKDAY_SATURDAY           ((uint8_t)0x06)
-#define RTC_WEEKDAY_SUNDAY             ((uint8_t)0x07)
-/**
-  * @}
-  */
-
-/** @defgroup RTC_AlarmDateWeekDay_Definitions RTC Alarm Date WeekDay
Definitions
-  * @{
-  */
-#define RTC_ALARMDATEWEEKDAYSEL_DATE      0x00000000U
-#define RTC_ALARMDATEWEEKDAYSEL_WEEKDAY   RTC_ALRMAR_WDSEL
-/**
-  * @}
-  */
-
-/** @defgroup RTC_AlarmMask_Definitions RTC Alarm Mask Definitions
-  * @{
-  */
-#define RTC_ALARMMASK_NONE                0x00000000U
-#define RTC_ALARMMASK_DATEWEEKDAY         RTC_ALRMAR_MSK4
-#define RTC_ALARMMASK_HOURS               RTC_ALRMAR_MSK3
-#define RTC_ALARMMASK_MINUTES             RTC_ALRMAR_MSK2
-#define RTC_ALARMMASK_SECONDS             RTC_ALRMAR_MSK1
-#define RTC_ALARMMASK_ALL                 (RTC_ALARMMASK_DATEWEEKDAY | \
-                                           RTC_ALARMMASK_HOURS       | \
-                                           RTC_ALARMMASK_MINUTES     | \
-                                           RTC_ALARMMASK_SECONDS)
-/**
-  * @}
-  */
-
-/** @defgroup RTC_Alarms_Definitions RTC Alarms Definitions
-  * @{
-  */
-#define RTC_ALARM_A                       RTC_CR_ALRAE
-#define RTC_ALARM_B                       RTC_CR_ALRBE
-/**
-  * @}
-  */
-
-/** @defgroup RTC_Alarm_Sub_Seconds_Masks_Definitions RTC Alarm Sub
Seconds Masks Definitions
-  * @{
-  */
```

```
-/*!< All Alarm SS fields are masked. There is no comparison on sub
seconds for Alarm */
-#define RTC_ALARMSUBSECONDMASK_ALL         0x00000000U
-/*!< SS[14:1] are don't care in Alarm comparison. Only SS[0] is
compared.     */
-#define RTC_ALARMSUBSECONDMASK_SS14_1      RTC_ALRMASSR_MASKSS_0
-/*!< SS[14:2] are don't care in Alarm comparison. Only SS[1:0] are
compared.  */
-#define RTC_ALARMSUBSECONDMASK_SS14_2      RTC_ALRMASSR_MASKSS_1
-/*!< SS[14:3] are don't care in Alarm comparison. Only SS[2:0] are
compared.  */
-#define RTC_ALARMSUBSECONDMASK_SS14_3      (RTC_ALRMASSR_MASKSS_0 |
RTC_ALRMASSR_MASKSS_1)
-/*!< SS[14:4] are don't care in Alarm comparison. Only SS[3:0] are
compared.  */
-#define RTC_ALARMSUBSECONDMASK_SS14_4      RTC_ALRMASSR_MASKSS_2
-/*!< SS[14:5] are don't care in Alarm comparison. Only SS[4:0] are
compared.  */
-#define RTC_ALARMSUBSECONDMASK_SS14_5      (RTC_ALRMASSR_MASKSS_0 |
RTC_ALRMASSR_MASKSS_2)
-/*!< SS[14:6] are don't care in Alarm comparison. Only SS[5:0] are
compared.  */
-#define RTC_ALARMSUBSECONDMASK_SS14_6      (RTC_ALRMASSR_MASKSS_1 |
RTC_ALRMASSR_MASKSS_2)
-/*!< SS[14:7] are don't care in Alarm comparison. Only SS[6:0] are
compared.  */
-#define RTC_ALARMSUBSECONDMASK_SS14_7      (RTC_ALRMASSR_MASKSS_0 |
RTC_ALRMASSR_MASKSS_1 | RTC_ALRMASSR_MASKSS_2)
-/*!< SS[14:8] are don't care in Alarm comparison. Only SS[7:0] are
compared.  */
-#define RTC_ALARMSUBSECONDMASK_SS14_8      RTC_ALRMASSR_MASKSS_3
-/*!< SS[14:9] are don't care in Alarm comparison. Only SS[8:0] are
compared.  */
-#define RTC_ALARMSUBSECONDMASK_SS14_9      (RTC_ALRMASSR_MASKSS_0 |
RTC_ALRMASSR_MASKSS_3)
-/*!< SS[14:10] are don't care in Alarm comparison. Only SS[9:0] are
compared. */
-#define RTC_ALARMSUBSECONDMASK_SS14_10     (RTC_ALRMASSR_MASKSS_1 |
RTC_ALRMASSR_MASKSS_3)
-/*!< SS[14:11] are don't care in Alarm comparison. Only SS[10:0] are
compared. */
-#define RTC_ALARMSUBSECONDMASK_SS14_11     (RTC_ALRMASSR_MASKSS_0 |
RTC_ALRMASSR_MASKSS_1 | RTC_ALRMASSR_MASKSS_3)
-/*!< SS[14:12] are don't care in Alarm comparison. Only SS[11:0] are
compared. */
-#define RTC_ALARMSUBSECONDMASK_SS14_12     (RTC_ALRMASSR_MASKSS_2 |
RTC_ALRMASSR_MASKSS_3)
-/*!< SS[14:13] are don't care in Alarm comparison. Only SS[12:0] are
compared. */
-#define RTC_ALARMSUBSECONDMASK_SS14_13     (RTC_ALRMASSR_MASKSS_0 |
RTC_ALRMASSR_MASKSS_2 | RTC_ALRMASSR_MASKSS_3)
-/*!< SS[14] is don't care in Alarm comparison. Only SS[13:0] are
compared. */
-#define RTC_ALARMSUBSECONDMASK_SS14        (RTC_ALRMASSR_MASKSS_1 |
RTC_ALRMASSR_MASKSS_2 | RTC_ALRMASSR_MASKSS_3)
-/*!< SS[14:0] are compared and must match to activate alarm. */
-#define RTC_ALARMSUBSECONDMASK_NONE        RTC_ALRMASSR_MASKSS
-/**
```

```
-  * @}
-  */
-
-/** @defgroup RTC_Interrupts_Definitions RTC Interrupts Definitions
-  * @{
-  */
-#define RTC_IT_TS                          RTC_CR_TSIE         /*!<
Enable Timestamp Interrupt                */
-#define RTC_IT_WUT                         RTC_CR_WUTIE        /*!<
Enable Wakeup timer Interrupt             */
-#define RTC_IT_ALRB                        RTC_CR_ALRBIE       /*!<
Enable Alarm B Interrupt                  */
-#define RTC_IT_ALRA                        RTC_CR_ALRAIE       /*!<
Enable Alarm A Interrupt                  */
-/**
-  * @}
-  */
-
-/** @defgroup RTC_Flags_Definitions RTC Flags Definitions
-  * @{
-  */
-#define RTC_FLAG_RECALPF                   RTC_ISR_RECALPF     /*!<
Recalibration pending flag                */
-#if defined(RTC_TAMPER2_SUPPORT)
-#define RTC_FLAG_TAMP2F                    RTC_ISR_TAMP2F      /*!<
Tamper 2 event flag                       */
-#endif /* RTC_TAMPER2_SUPPORT */
-#define RTC_FLAG_TAMP1F                    RTC_ISR_TAMP1F      /*!<
Tamper 1 event flag                       */
-#define RTC_FLAG_TSOVF                     RTC_ISR_TSOVF       /*!<
Timestamp overflow flag                   */
-#define RTC_FLAG_TSF                       RTC_ISR_TSF         /*!<
Timestamp event flag                      */
-#define RTC_FLAG_WUTF                      RTC_ISR_WUTF        /*!<
Wakeup timer event flag                   */
-#define RTC_FLAG_ALRBF                     RTC_ISR_ALRBF       /*!< Alarm
B event flag                         */
-#define RTC_FLAG_ALRAF                     RTC_ISR_ALRAF       /*!< Alarm
A event flag                         */
-#define RTC_FLAG_INITF                     RTC_ISR_INITF       /*!< RTC
in initialization mode flag           */
-#define RTC_FLAG_RSF                       RTC_ISR_RSF         /*!<
Register synchronization flag             */
-#define RTC_FLAG_INITS                     RTC_ISR_INITS       /*!< RTC
initialization status flag            */
-#define RTC_FLAG_SHPF                      RTC_ISR_SHPF        /*!< Shift
operation pending flag               */
-#define RTC_FLAG_WUTWF                     RTC_ISR_WUTWF       /*!< WUTR
register write allowance flag         */
-#define RTC_FLAG_ALRBWF                    RTC_ISR_ALRBWF      /*!<
ALRMBR register write allowance flag      */
-#define RTC_FLAG_ALRAWF                    RTC_ISR_ALRAWF      /*!<
ALRMAR register write allowance flag      */
-/**
-  * @}
-  */
-
-/**
```

```
-  * @}
-  */
-
-/* Exported macros ------------------------------------------------------*/
-
-/** @defgroup RTC_Exported_Macros RTC Exported Macros
-  * @{
-  */
-
-/** @brief Reset RTC handle state
-  * @param  __HANDLE__ specifies the RTC handle.
-  * @retval None
-  */
-#if (USE_HAL_RTC_REGISTER_CALLBACKS == 1)
-#define __HAL_RTC_RESET_HANDLE_STATE(__HANDLE__) do {
\
-                                                    (__HANDLE__)-
>State = HAL_RTC_STATE_RESET; \
-                                                    (__HANDLE__)-
>MspInitCallback = NULL;      \
-                                                    (__HANDLE__)-
>MspDeInitCallback = NULL;    \
-                                                  } while(0U)
-#else
-#define __HAL_RTC_RESET_HANDLE_STATE(__HANDLE__) ((__HANDLE__)->State =
HAL_RTC_STATE_RESET)
-#endif /* USE_HAL_RTC_REGISTER_CALLBACKS */
-
-/**
-  * @brief  Disable the write protection for RTC registers.
-  * @param  __HANDLE__ specifies the RTC handle.
-  * @retval None
-  */
-#define __HAL_RTC_WRITEPROTECTION_DISABLE(__HANDLE__) do {
\
-                                                      (__HANDLE__)-
>Instance->WPR = 0xCAU;  \
-                                                      (__HANDLE__)-
>Instance->WPR = 0x53U;  \
-                                                    } while(0U)
-
-/**
-  * @brief  Enable the write protection for RTC registers.
-  * @param  __HANDLE__ specifies the RTC handle.
-  * @retval None
-  */
-#define __HAL_RTC_WRITEPROTECTION_ENABLE(__HANDLE__) do {
\
-                                                       (__HANDLE__)-
>Instance->WPR = 0xFFU;  \
-                                                     } while(0U)
-
-/**
-  * @brief  Check whether the RTC Calendar is initialized.
-  * @param  __HANDLE__ specifies the RTC handle.
-  * @retval None
-  */
```

```
-#define __HAL_RTC_IS_CALENDAR_INITIALIZED(__HANDLE__)
(((((__HANDLE__)->Instance->ISR) & (RTC_FLAG_INITS)) == RTC_FLAG_INITS) ?
1U : 0U)
-
-/**
-  * @brief  Enable the RTC ALARMA peripheral.
-  * @param  __HANDLE__ specifies the RTC handle.
-  * @retval None
-  */
-#define __HAL_RTC_ALARMA_ENABLE(__HANDLE__)
((__HANDLE__)->Instance->CR |= (RTC_CR_ALRAE))
-
-/**
-  * @brief  Disable the RTC ALARMA peripheral.
-  * @param  __HANDLE__ specifies the RTC handle.
-  * @retval None
-  */
-#define __HAL_RTC_ALARMA_DISABLE(__HANDLE__)
((__HANDLE__)->Instance->CR &= ~(RTC_CR_ALRAE))
-
-/**
-  * @brief  Enable the RTC ALARMB peripheral.
-  * @param  __HANDLE__ specifies the RTC handle.
-  * @retval None
-  */
-#define __HAL_RTC_ALARMB_ENABLE(__HANDLE__)
((__HANDLE__)->Instance->CR |= (RTC_CR_ALRBE))
-
-/**
-  * @brief  Disable the RTC ALARMB peripheral.
-  * @param  __HANDLE__ specifies the RTC handle.
-  * @retval None
-  */
-#define __HAL_RTC_ALARMB_DISABLE(__HANDLE__)
((__HANDLE__)->Instance->CR &= ~(RTC_CR_ALRBE))
-
-/**
-  * @brief  Enable the RTC Alarm interrupt.
-  * @param  __HANDLE__ specifies the RTC handle.
-  * @param  __INTERRUPT__ specifies the RTC Alarm interrupt sources to
be enabled or disabled.
-  *          This parameter can be any combination of the following
values:
-  *             @arg RTC_IT_ALRA: Alarm A interrupt
-  *             @arg RTC_IT_ALRB: Alarm B interrupt
-  * @retval None
-  */
-#define __HAL_RTC_ALARM_ENABLE_IT(__HANDLE__, __INTERRUPT__)
((__HANDLE__)->Instance->CR |= (__INTERRUPT__))
-
-/**
-  * @brief  Disable the RTC Alarm interrupt.
-  * @param  __HANDLE__ specifies the RTC handle.
-  * @param  __INTERRUPT__ specifies the RTC Alarm interrupt sources to
be enabled or disabled.
-  *          This parameter can be any combination of the following
values:
-  *             @arg RTC_IT_ALRA: Alarm A interrupt
```

```
-  *             @arg RTC_IT_ALRB: Alarm B interrupt
-  * @retval None
-  */
-#define __HAL_RTC_ALARM_DISABLE_IT(__HANDLE__, __INTERRUPT__)
((__HANDLE__)->Instance->CR &= ~(__INTERRUPT__))
-
-/**
-  * @brief  Check whether the specified RTC Alarm interrupt has occurred
or not.
-  * @param  __HANDLE__ specifies the RTC handle.
-  * @param  __INTERRUPT__ specifies the RTC Alarm interrupt to check.
-  *         This parameter can be:
-  *            @arg RTC_IT_ALRA: Alarm A interrupt
-  *            @arg RTC_IT_ALRB: Alarm B interrupt
-  * @retval None
-  */
-#define __HAL_RTC_ALARM_GET_IT(__HANDLE__, __INTERRUPT__)
(((((__HANDLE__)->Instance->ISR) & ((__INTERRUPT__) >> 4U)) != 0U) ? 1U :
0U)
-
-/**
-  * @brief  Get the selected RTC Alarm's flag status.
-  * @param  __HANDLE__ specifies the RTC handle.
-  * @param  __FLAG__ specifies the RTC Alarm Flag to check.
-  *         This parameter can be:
-  *            @arg RTC_FLAG_ALRAF: Alarm A interrupt flag
-  *            @arg RTC_FLAG_ALRAWF: Alarm A 'write allowed' flag
-  *            @arg RTC_FLAG_ALRBF: Alarm B interrupt flag
-  *            @arg RTC_FLAG_ALRBWF: Alarm B 'write allowed' flag
-  * @retval None
-  */
-#define __HAL_RTC_ALARM_GET_FLAG(__HANDLE__, __FLAG__)
(((((__HANDLE__)->Instance->ISR) & (__FLAG__)) != 0U) ? 1U : 0U)
-
-/**
-  * @brief  Clear the RTC Alarm's pending flags.
-  * @param  __HANDLE__ specifies the RTC handle.
-  * @param  __FLAG__ specifies the RTC Alarm flag to be cleared.
-  *          This parameter can be:
-  *            @arg RTC_FLAG_ALRAF
-  *            @arg RTC_FLAG_ALRBF
-  * @retval None
-  */
-#define __HAL_RTC_ALARM_CLEAR_FLAG(__HANDLE__, __FLAG__)
((__HANDLE__)->Instance->ISR) = (~((__FLAG__) |
RTC_ISR_INIT)|((__HANDLE__)->Instance->ISR & RTC_ISR_INIT))
-
-/**
-  * @brief  Check whether the specified RTC Alarm interrupt has been
enabled or not.
-  * @param  __HANDLE__ specifies the RTC handle.
-  * @param  __INTERRUPT__ specifies the RTC Alarm interrupt sources to
check.
-  *         This parameter can be:
-  *            @arg RTC_IT_ALRA: Alarm A interrupt
-  *            @arg RTC_IT_ALRB: Alarm B interrupt
-  * @retval None
-  */
```

```
-#define __HAL_RTC_ALARM_GET_IT_SOURCE(__HANDLE__, __INTERRUPT__)
(((((__HANDLE__)->Instance->CR) & (__INTERRUPT__)) != 0U) ? 1U : 0U)
-
-/**
-  * @brief  Enable interrupt on the RTC Alarm associated EXTI line.
-  * @retval None
-  */
-#define __HAL_RTC_ALARM_EXTI_ENABLE_IT()            (EXTI->IMR |=
RTC_EXTI_LINE_ALARM_EVENT)
-
-/**
-  * @brief  Disable interrupt on the RTC Alarm associated EXTI line.
-  * @retval None
-  */
-#define __HAL_RTC_ALARM_EXTI_DISABLE_IT()           (EXTI->IMR &=
~RTC_EXTI_LINE_ALARM_EVENT)
-
-/**
-  * @brief  Enable event on the RTC Alarm associated EXTI line.
-  * @retval None.
-  */
-#define __HAL_RTC_ALARM_EXTI_ENABLE_EVENT()          (EXTI->EMR |=
RTC_EXTI_LINE_ALARM_EVENT)
-
-/**
-  * @brief  Disable event on the RTC Alarm associated EXTI line.
-  * @retval None.
-  */
-#define __HAL_RTC_ALARM_EXTI_DISABLE_EVENT()         (EXTI->EMR &=
~RTC_EXTI_LINE_ALARM_EVENT)
-
-/**
-  * @brief  Enable falling edge trigger on the RTC Alarm associated EXTI
line.
-  * @retval None.
-  */
-#define __HAL_RTC_ALARM_EXTI_ENABLE_FALLING_EDGE()   (EXTI->FTSR |=
RTC_EXTI_LINE_ALARM_EVENT)
-
-/**
-  * @brief  Disable falling edge trigger on the RTC Alarm associated
EXTI line.
-  * @retval None.
-  */
-#define __HAL_RTC_ALARM_EXTI_DISABLE_FALLING_EDGE()  (EXTI->FTSR &=
~RTC_EXTI_LINE_ALARM_EVENT)
-
-/**
-  * @brief  Enable rising edge trigger on the RTC Alarm associated EXTI
line.
-  * @retval None.
-  */
-#define __HAL_RTC_ALARM_EXTI_ENABLE_RISING_EDGE()    (EXTI->RTSR |=
RTC_EXTI_LINE_ALARM_EVENT)
-
-/**
-  * @brief  Disable rising edge trigger on the RTC Alarm associated EXTI
line.
```

```
-  * @retval None.
-  */
-#define __HAL_RTC_ALARM_EXTI_DISABLE_RISING_EDGE()    (EXTI->RTSR &=
~RTC_EXTI_LINE_ALARM_EVENT)
-
-/**
-  * @brief  Enable rising & falling edge trigger on the RTC Alarm
associated EXTI line.
-  * @retval None.
-  */
-#define __HAL_RTC_ALARM_EXTI_ENABLE_RISING_FALLING_EDGE() do {
\
-
__HAL_RTC_ALARM_EXTI_ENABLE_RISING_EDGE();  \
-
__HAL_RTC_ALARM_EXTI_ENABLE_FALLING_EDGE(); \
-                                                                  } while(0U)
-
-/**
-  * @brief  Disable rising & falling edge trigger on the RTC Alarm
associated EXTI line.
-  * @retval None.
-  */
-#define __HAL_RTC_ALARM_EXTI_DISABLE_RISING_FALLING_EDGE() do {
\
-
__HAL_RTC_ALARM_EXTI_DISABLE_RISING_EDGE();  \
-
__HAL_RTC_ALARM_EXTI_DISABLE_FALLING_EDGE(); \
-                                                                   }
while(0U)
-
-/**
-  * @brief Check whether the RTC Alarm associated EXTI line interrupt
flag is set or not.
-  * @retval Line Status.
-  */
-#define __HAL_RTC_ALARM_EXTI_GET_FLAG()              (EXTI->PR &
RTC_EXTI_LINE_ALARM_EVENT)
-
-/**
-  * @brief Clear the RTC Alarm associated EXTI line flag.
-  * @retval None.
-  */
-#define __HAL_RTC_ALARM_EXTI_CLEAR_FLAG()            (EXTI->PR =
RTC_EXTI_LINE_ALARM_EVENT)
-
-/**
-  * @brief Generate a Software interrupt on RTC Alarm associated EXTI
line.
-  * @retval None.
-  */
-#define __HAL_RTC_ALARM_EXTI_GENERATE_SWIT()         (EXTI->SWIER |=
RTC_EXTI_LINE_ALARM_EVENT)
-/**
-  * @}
-  */
-
```

```
-/* Include RTC HAL Extended module */
-#include "stm32f4xx_hal_rtc_ex.h"
-
-/* Exported functions ---------------------------------------------------
------*/
-
-/** @addtogroup RTC_Exported_Functions
-  * @{
-  */
-
-/** @addtogroup RTC_Exported_Functions_Group1
-  * @{
-  */
-/* Initialization and de-initialization functions
****************************/
-HAL_StatusTypeDef HAL_RTC_Init(RTC_HandleTypeDef *hrtc);
-HAL_StatusTypeDef HAL_RTC_DeInit(RTC_HandleTypeDef *hrtc);
-void              HAL_RTC_MspInit(RTC_HandleTypeDef *hrtc);
-void              HAL_RTC_MspDeInit(RTC_HandleTypeDef *hrtc);
-
-/* Callbacks Register/UnRegister functions
**********************************/
-#if (USE_HAL_RTC_REGISTER_CALLBACKS == 1)
-HAL_StatusTypeDef HAL_RTC_RegisterCallback(RTC_HandleTypeDef *hrtc,
HAL_RTC_CallbackIDTypeDef CallbackID, pRTC_CallbackTypeDef pCallback);
-HAL_StatusTypeDef HAL_RTC_UnRegisterCallback(RTC_HandleTypeDef *hrtc,
HAL_RTC_CallbackIDTypeDef CallbackID);
-#endif /* USE_HAL_RTC_REGISTER_CALLBACKS */
-/**
-  * @}
-  */
-
-/** @addtogroup RTC_Exported_Functions_Group2
-  * @{
-  */
-/* RTC Time and Date functions
*************************************************/
-HAL_StatusTypeDef HAL_RTC_SetTime(RTC_HandleTypeDef *hrtc,
RTC_TimeTypeDef *sTime, uint32_t Format);
-HAL_StatusTypeDef HAL_RTC_GetTime(RTC_HandleTypeDef *hrtc,
RTC_TimeTypeDef *sTime, uint32_t Format);
-HAL_StatusTypeDef HAL_RTC_SetDate(RTC_HandleTypeDef *hrtc,
RTC_DateTypeDef *sDate, uint32_t Format);
-HAL_StatusTypeDef HAL_RTC_GetDate(RTC_HandleTypeDef *hrtc,
RTC_DateTypeDef *sDate, uint32_t Format);
-/**
-  * @}
-  */
-
-/** @addtogroup RTC_Exported_Functions_Group3
-  * @{
-  */
-/* RTC Alarm functions
*****************************************************/
-HAL_StatusTypeDef HAL_RTC_SetAlarm(RTC_HandleTypeDef *hrtc,
RTC_AlarmTypeDef *sAlarm, uint32_t Format);
-HAL_StatusTypeDef HAL_RTC_SetAlarm_IT(RTC_HandleTypeDef *hrtc,
RTC_AlarmTypeDef *sAlarm, uint32_t Format);
```

```
-HAL_StatusTypeDef HAL_RTC_DeactivateAlarm(RTC_HandleTypeDef *hrtc,
uint32_t Alarm);
-HAL_StatusTypeDef HAL_RTC_GetAlarm(RTC_HandleTypeDef *hrtc,
RTC_AlarmTypeDef *sAlarm, uint32_t Alarm, uint32_t Format);
-void              HAL_RTC_AlarmIRQHandler(RTC_HandleTypeDef *hrtc);
-HAL_StatusTypeDef HAL_RTC_PollForAlarmAEvent(RTC_HandleTypeDef *hrtc,
uint32_t Timeout);
-void              HAL_RTC_AlarmAEventCallback(RTC_HandleTypeDef *hrtc);
-/**
-  * @}
-  */
-
-/** @addtogroup RTC_Exported_Functions_Group4
-  * @{
-  */
-/* Peripheral Control functions
*******************************************/
-HAL_StatusTypeDef   HAL_RTC_WaitForSynchro(RTC_HandleTypeDef *hrtc);
-
-/* RTC Daylight Saving Time functions
****************************************/
-void              HAL_RTC_DST_Add1Hour(RTC_HandleTypeDef *hrtc);
-void              HAL_RTC_DST_Sub1Hour(RTC_HandleTypeDef *hrtc);
-void              HAL_RTC_DST_SetStoreOperation(RTC_HandleTypeDef
*hrtc);
-void              HAL_RTC_DST_ClearStoreOperation(RTC_HandleTypeDef
*hrtc);
-uint32_t          HAL_RTC_DST_ReadStoreOperation(RTC_HandleTypeDef
*hrtc);
-/**
-  * @}
-  */
-
-/** @addtogroup RTC_Exported_Functions_Group5
-  * @{
-  */
-/* Peripheral State functions
**************************************************/
-HAL_RTCStateTypeDef HAL_RTC_GetState(RTC_HandleTypeDef *hrtc);
-/**
-  * @}
-  */
-
-/**
-  * @}
-  */
-
-/* Private types -----------------------------------------------------------
------*/
-/* Private variables -------------------------------------------------------
------*/
-/* Private constants -------------------------------------------------------
------*/
-
-/** @defgroup RTC_Private_Constants RTC Private Constants
-  * @{
-  */
-/* Masks Definition */
```

```
-#define RTC_TR_RESERVED_MASK    ((uint32_t)(RTC_TR_HT  | RTC_TR_HU  | \
-                                            RTC_TR_MNT | RTC_TR_MNU | \
-                                            RTC_TR_ST  | RTC_TR_SU  | \
-                                            RTC_TR_PM))
-#define RTC_DR_RESERVED_MASK    ((uint32_t)(RTC_DR_YT | RTC_DR_YU | \
-                                            RTC_DR_MT | RTC_DR_MU | \
-                                            RTC_DR_DT | RTC_DR_DU | \
-                                            RTC_DR_WDU))
-#define RTC_ISR_RESERVED_MASK   ((uint32_t)(RTC_FLAGS_MASK |
RTC_ISR_INIT))
-#define RTC_INIT_MASK           0xFFFFFFFFU
-#define RTC_RSF_MASK            ((uint32_t)~(RTC_ISR_INIT |
RTC_ISR_RSF))
-#define RTC_FLAGS_MASK          ((uint32_t)(RTC_FLAG_INITF   |
RTC_FLAG_INITS  | \
-                                            RTC_FLAG_ALRAF   |
RTC_FLAG_ALRAWF | \
-                                            RTC_FLAG_ALRBF   |
RTC_FLAG_ALRBWF | \
-                                            RTC_FLAG_WUTF    |
RTC_FLAG_WUTWF  | \
-                                            RTC_FLAG_RECALPF |
RTC_FLAG_SHPF   | \
-                                            RTC_FLAG_TSF     |
RTC_FLAG_TSOVF  | \
-                                            RTC_FLAG_RSF     |
RTC_TAMPER_FLAGS_MASK))
-
-#define RTC_TIMEOUT_VALUE       1000U
-
-#define RTC_EXTI_LINE_ALARM_EVENT  EXTI_IMR_MR17  /*!< External
interrupt line 17 connected to the RTC Alarm event */
-/**
-  * @}
-  */
-
-/* Private macros ------------------------------------------------------
------*/
-
-/** @defgroup RTC_Private_Macros RTC Private Macros
-  * @{
-  */
-
-/** @defgroup RTC_IS_RTC_Definitions RTC Private macros to check input
parameters
-  * @{
-  */
-#define IS_RTC_HOUR_FORMAT(FORMAT)     (((FORMAT) == RTC_HOURFORMAT_12)
|| \
-                                        ((FORMAT) == RTC_HOURFORMAT_24))
-
-#define IS_RTC_OUTPUT(OUTPUT) (((OUTPUT) == RTC_OUTPUT_DISABLE) || \
-                               ((OUTPUT) == RTC_OUTPUT_ALARMA)  || \
-                               ((OUTPUT) == RTC_OUTPUT_ALARMB)  || \
-                               ((OUTPUT) == RTC_OUTPUT_WAKEUP))
-
-#define IS_RTC_OUTPUT_POL(POL) (((POL) == RTC_OUTPUT_POLARITY_HIGH) || \
-                                ((POL) == RTC_OUTPUT_POLARITY_LOW))
```

```
-
-#define IS_RTC_OUTPUT_TYPE(TYPE) (((TYPE) == RTC_OUTPUT_TYPE_OPENDRAIN)
|| \
-                                  ((TYPE) == RTC_OUTPUT_TYPE_PUSHPULL))
-
-#define IS_RTC_ASYNCH_PREDIV(PREDIV)   ((PREDIV) <= 0x7FU)
-#define IS_RTC_SYNCH_PREDIV(PREDIV)    ((PREDIV) <= 0x7FFFU)
-
-#define IS_RTC_HOUR12(HOUR)            (((HOUR) > 0U) && ((HOUR) <=
12U))
-#define IS_RTC_HOUR24(HOUR)            ((HOUR) <= 23U)
-#define IS_RTC_MINUTES(MINUTES)        ((MINUTES) <= 59U)
-#define IS_RTC_SECONDS(SECONDS)        ((SECONDS) <= 59U)
-
-#define IS_RTC_HOURFORMAT12(PM)  (((PM) == RTC_HOURFORMAT12_AM) || \
-                                  ((PM) == RTC_HOURFORMAT12_PM))
-
-#define IS_RTC_DAYLIGHT_SAVING(SAVE) (((SAVE) ==
RTC_DAYLIGHTSAVING_SUB1H) || \
-                                      ((SAVE) ==
RTC_DAYLIGHTSAVING_ADD1H) || \
-                                      ((SAVE) ==
RTC_DAYLIGHTSAVING_NONE))
-
-#define IS_RTC_STORE_OPERATION(OPERATION) (((OPERATION) ==
RTC_STOREOPERATION_RESET) || \
-                                           ((OPERATION) ==
RTC_STOREOPERATION_SET))
-
-#define IS_RTC_FORMAT(FORMAT) (((FORMAT) == RTC_FORMAT_BIN) || ((FORMAT)
== RTC_FORMAT_BCD))
-
-#define IS_RTC_YEAR(YEAR)              ((YEAR) <= 99U)
-#define IS_RTC_MONTH(MONTH)            (((MONTH) >= 1U) && ((MONTH) <=
12U))
-#define IS_RTC_DATE(DATE)              (((DATE) >= 1U) && ((DATE) <=
31U))
-
-#define IS_RTC_WEEKDAY(WEEKDAY) (((WEEKDAY) == RTC_WEEKDAY_MONDAY)    ||
\
-                                  ((WEEKDAY) == RTC_WEEKDAY_TUESDAY)   ||
\
-                                  ((WEEKDAY) == RTC_WEEKDAY_WEDNESDAY) ||
\
-                                  ((WEEKDAY) == RTC_WEEKDAY_THURSDAY)  ||
\
-                                  ((WEEKDAY) == RTC_WEEKDAY_FRIDAY)    ||
\
-                                  ((WEEKDAY) == RTC_WEEKDAY_SATURDAY)  ||
\
-                                  ((WEEKDAY) == RTC_WEEKDAY_SUNDAY))
-
-#define IS_RTC_ALARM_DATE_WEEKDAY_DATE(DATE) (((DATE) > 0U) && ((DATE)
<= 31U))
-
-#define IS_RTC_ALARM_DATE_WEEKDAY_WEEKDAY(WEEKDAY) (((WEEKDAY) ==
RTC_WEEKDAY_MONDAY)    || \
```

```
-                                                    ((WEEKDAY) ==
RTC_WEEKDAY_TUESDAY)   || \
-                                                    ((WEEKDAY) ==
RTC_WEEKDAY_WEDNESDAY) || \
-                                                    ((WEEKDAY) ==
RTC_WEEKDAY_THURSDAY)  || \
-                                                    ((WEEKDAY) ==
RTC_WEEKDAY_FRIDAY)    || \
-                                                    ((WEEKDAY) ==
RTC_WEEKDAY_SATURDAY)  || \
-                                                    ((WEEKDAY) ==
RTC_WEEKDAY_SUNDAY))
-
-#define IS_RTC_ALARM_DATE_WEEKDAY_SEL(SEL) (((SEL) ==
RTC_ALARMDATEWEEKDAYSEL_DATE) || \
-                                            ((SEL) ==
RTC_ALARMDATEWEEKDAYSEL_WEEKDAY))
-
-#define IS_RTC_ALARM_MASK(MASK)  (((MASK) &
((uint32_t)~RTC_ALARMMASK_ALL)) == 0U)
-
-#define IS_RTC_ALARM(ALARM)      (((ALARM) == RTC_ALARM_A) || ((ALARM)
== RTC_ALARM_B))
-
-#define IS_RTC_ALARM_SUB_SECOND_VALUE(VALUE) ((VALUE) <=
RTC_ALRMASSR_SS)
-
-#define IS_RTC_ALARM_SUB_SECOND_MASK(MASK)   ((MASK) ==
RTC_ALARMSUBSECONDMASK_ALL)     || \
-                                              ((MASK) ==
RTC_ALARMSUBSECONDMASK_SS14_1)  || \
-                                              ((MASK) ==
RTC_ALARMSUBSECONDMASK_SS14_2)  || \
-                                              ((MASK) ==
RTC_ALARMSUBSECONDMASK_SS14_3)  || \
-                                              ((MASK) ==
RTC_ALARMSUBSECONDMASK_SS14_4)  || \
-                                              ((MASK) ==
RTC_ALARMSUBSECONDMASK_SS14_5)  || \
-                                              ((MASK) ==
RTC_ALARMSUBSECONDMASK_SS14_6)  || \
-                                              ((MASK) ==
RTC_ALARMSUBSECONDMASK_SS14_7)  || \
-                                              ((MASK) ==
RTC_ALARMSUBSECONDMASK_SS14_8)  || \
-                                              ((MASK) ==
RTC_ALARMSUBSECONDMASK_SS14_9)  || \
-                                              ((MASK) ==
RTC_ALARMSUBSECONDMASK_SS14_10) || \
-                                              ((MASK) ==
RTC_ALARMSUBSECONDMASK_SS14_11) || \
-                                              ((MASK) ==
RTC_ALARMSUBSECONDMASK_SS14_12) || \
-                                              ((MASK) ==
RTC_ALARMSUBSECONDMASK_SS14_13) || \
-                                              ((MASK) ==
RTC_ALARMSUBSECONDMASK_SS14)    || \
```

```
-                                                ((MASK) ==
RTC_ALARMSUBSECONDMASK_NONE))
-/**
-  * @}
-  */
-
-/**
-  * @}
-  */
-
-/* Private functions ---------------------------------------------------
------*/
-
-/** @defgroup RTC_Private_Functions RTC Private Functions
-  * @{
-  */
-HAL_StatusTypeDef  RTC_EnterInitMode(RTC_HandleTypeDef *hrtc);
-HAL_StatusTypeDef  RTC_ExitInitMode(RTC_HandleTypeDef *hrtc);
-uint8_t            RTC_ByteToBcd2(uint8_t number);
-uint8_t            RTC_Bcd2ToByte(uint8_t number);
-/**
-  * @}
-  */
-
-/**
-  * @}
-  */
-
-/**
-  * @}
-  */
-
-#ifdef __cplusplus
-}
-#endif
-
-#endif /* STM32F4xx_HAL_RTC_H */
diff --git a/Drivers/STM32F4xx_HAL_Driver/Inc/stm32f4xx_hal_rtc_ex.h
b/Drivers/STM32F4xx_HAL_Driver/Inc/stm32f4xx_hal_rtc_ex.h
deleted file mode 100644
index dbedc86..0000000
--- a/Drivers/STM32F4xx_HAL_Driver/Inc/stm32f4xx_hal_rtc_ex.h
+++ /dev/null
@@ -1,1069 +0,0 @@
-/**
-
******************************************************************************
*****
-  * @file    stm32f4xx_hal_rtc_ex.h
-  * @author  MCD Application Team
-  * @brief   Header file of RTC HAL Extended module.
-
******************************************************************************
*****
-  * @attention
-  *
-  * Copyright (c) 2016 STMicroelectronics.
-  * All rights reserved.
```

```
-  *
-  * This software is licensed under terms that can be found in the
LICENSE file
-  * in the root directory of this software component.
-  * If no LICENSE file comes with this software, it is provided AS-IS.
-  *
-
*******************************************************************************
-  */
-
-/* Define to prevent recursive inclusion -------------------------------
------*/
-#ifndef STM32F4xx_HAL_RTC_EX_H
-#define STM32F4xx_HAL_RTC_EX_H
-
-#ifdef __cplusplus
-extern "C" {
-#endif
-
-/* Includes ------------------------------------------------------------
------*/
-
-#include "stm32f4xx_hal_def.h"
-
-/** @addtogroup STM32F4xx_HAL_Driver
-  * @{
-  */
-
-/** @addtogroup RTCEx
-  * @{
-  */
-
-/* Exported types ------------------------------------------------------
------*/
-
-/** @defgroup RTCEx_Exported_Types RTCEx Exported Types
-  * @{
-  */
-
-/**
-  * @brief  RTC Tamper structure definition
-  */
-typedef struct
-{
-  uint32_t Tamper;                      /*!< Specifies the Tamper Pin.
-                                            This parameter can be a
value of @ref RTCEx_Tamper_Pin_Definitions */
-
-  uint32_t PinSelection;                /*!< Specifies the Tamper Pin.
-                                            This parameter can be a
value of @ref RTCEx_Tamper_Pin_Selection */
-
-  uint32_t Trigger;                     /*!< Specifies the Tamper
Trigger.
-                                            This parameter can be a
value of @ref RTCEx_Tamper_Trigger_Definitions */
-
```

```
-  uint32_t Filter;                     /*!< Specifies the RTC Filter
Tamper.
-                                           This parameter can be a
value of @ref RTCEx_Tamper_Filter_Definitions */
-
-  uint32_t SamplingFrequency;          /*!< Specifies the sampling
frequency.
-                                           This parameter can be a
value of @ref RTCEx_Tamper_Sampling_Frequencies_Definitions */
-
-  uint32_t PrechargeDuration;          /*!< Specifies the Precharge
Duration .
-                                           This parameter can be a
value of @ref RTCEx_Tamper_Pin_Precharge_Duration_Definitions */
-
-  uint32_t TamperPullUp;               /*!< Specifies the Tamper PullUp
.
-                                           This parameter can be a
value of @ref RTCEx_Tamper_Pull_Up_Definitions */
-
-  uint32_t TimeStampOnTamperDetection;  /*!< Specifies the
TimeStampOnTamperDetection.
-                                           This parameter can be a
value of @ref RTCEx_Tamper_TimeStampOnTamperDetection_Definitions */
-} RTC_TamperTypeDef;
-/**
-  * @}
-  */
-
-/* Exported constants --------------------------------------------------
------*/
-
-/** @defgroup RTCEx_Exported_Constants RTCEx Exported Constants
-  * @{
-  */
-
-/** @defgroup RTCEx_Backup_Registers_Definitions RTCEx Backup Registers
Definitions
-  * @{
-  */
-#define RTC_BKP_DR0                         0x00000000U
-#define RTC_BKP_DR1                         0x00000001U
-#define RTC_BKP_DR2                         0x00000002U
-#define RTC_BKP_DR3                         0x00000003U
-#define RTC_BKP_DR4                         0x00000004U
-#define RTC_BKP_DR5                         0x00000005U
-#define RTC_BKP_DR6                         0x00000006U
-#define RTC_BKP_DR7                         0x00000007U
-#define RTC_BKP_DR8                         0x00000008U
-#define RTC_BKP_DR9                         0x00000009U
-#define RTC_BKP_DR10                        0x0000000AU
-#define RTC_BKP_DR11                        0x0000000BU
-#define RTC_BKP_DR12                        0x0000000CU
-#define RTC_BKP_DR13                        0x0000000DU
-#define RTC_BKP_DR14                        0x0000000EU
-#define RTC_BKP_DR15                        0x0000000FU
-#define RTC_BKP_DR16                        0x00000010U
-#define RTC_BKP_DR17                        0x00000011U
```

```
-#define RTC_BKP_DR18                        0x00000012U
-#define RTC_BKP_DR19                        0x00000013U
-/**
-  * @}
-  */
-
-/** @defgroup RTCEx_Timestamp_Edges_Definitions RTCEx Timestamp Edges
Definitions
-  * @{
-  */
-#define RTC_TIMESTAMPEDGE_RISING          0x00000000U
-#define RTC_TIMESTAMPEDGE_FALLING         RTC_CR_TSEDGE
-/**
-  * @}
-  */
-
-/** @defgroup RTCEx_Timestamp_Pin_Selection RTC Timestamp Pin Selection
-  * @{
-  */
-#define RTC_TIMESTAMPPIN_DEFAULT            0x00000000U
-#if defined(RTC_AF2_SUPPORT)
-#define RTC_TIMESTAMPPIN_POS1               RTC_TAFCR_TSINSEL
-#endif /* RTC_AF2_SUPPORT */
-/**
-  * @}
-  */
-
-/** @defgroup RTCEx_Tamper_Pin_Definitions RTCEx Tamper Pins Definitions
-  * @{
-  */
-#define RTC_TAMPER_1                    RTC_TAFCR_TAMP1E
-#if defined(RTC_TAMPER2_SUPPORT)
-#define RTC_TAMPER_2                    RTC_TAFCR_TAMP2E
-#endif /* RTC_TAMPER2_SUPPORT */
-/**
-  * @}
-  */
-
-/** @defgroup RTCEx_Tamper_Pin_Selection RTC tamper Pins Selection
-  * @{
-  */
-#define RTC_TAMPERPIN_DEFAULT               0x00000000U
-#if defined(RTC_AF2_SUPPORT)
-#define RTC_TAMPERPIN_POS1                  RTC_TAFCR_TAMP1INSEL
-#endif /* RTC_AF2_SUPPORT */
-/**
-  * @}
-  */
-
-/** @defgroup RTCEx_Tamper_Interrupt_Definitions RTCEx Tamper Interrupt
Definitions
-  * @{
-  */
-#define RTC_IT_TAMP                         RTC_TAFCR_TAMPIE   /*!< Enable
global Tamper Interrupt           */
-/**
-  * @}
-  */
```

```
-
-/** @defgroup RTCEx_Tamper_Trigger_Definitions RTCEx Tamper Triggers
Definitions
-  * @{
-  */
-#define RTC_TAMPERTRIGGER_RISINGEDGE       0x00000000U
-#define RTC_TAMPERTRIGGER_FALLINGEDGE      0x00000002U
-#define RTC_TAMPERTRIGGER_LOWLEVEL         RTC_TAMPERTRIGGER_RISINGEDGE
-#define RTC_TAMPERTRIGGER_HIGHLEVEL        RTC_TAMPERTRIGGER_FALLINGEDGE
-/**
-  * @}
-  */
-
-/** @defgroup RTCEx_Tamper_Filter_Definitions RTCEx Tamper Filter
Definitions
-  * @{
-  */
-#define RTC_TAMPERFILTER_DISABLE   0x00000000U             /*!< Tamper
filter is disabled */
-
-#define RTC_TAMPERFILTER_2SAMPLE   RTC_TAFCR_TAMPFLT_0    /*!< Tamper is
activated after 2
-
consecutive samples at the active level */
-#define RTC_TAMPERFILTER_4SAMPLE   RTC_TAFCR_TAMPFLT_1    /*!< Tamper is
activated after 4
-
consecutive samples at the active level */
-#define RTC_TAMPERFILTER_8SAMPLE   RTC_TAFCR_TAMPFLT      /*!< Tamper is
activated after 8
-
consecutive samples at the active level */
-#define RTC_TAMPERFILTER_MASK      RTC_TAFCR_TAMPFLT      /*!< Masking
all bits except those of
-                                                                  field
TAMPFLT                            */
-/**
-  * @}
-  */
-
-/** @defgroup RTCEx_Tamper_Sampling_Frequencies_Definitions RTCEx Tamper
Sampling Frequencies Definitions
-  * @{
-  */
-#define RTC_TAMPERSAMPLINGFREQ_RTCCLK_DIV32768  0x00000000U
/*!< Each of the tamper inputs are sampled
-
with a frequency =  RTCCLK / 32768 */
-#define RTC_TAMPERSAMPLINGFREQ_RTCCLK_DIV16384  RTC_TAFCR_TAMPFREQ_0
/*!< Each of the tamper inputs are sampled
-
with a frequency =  RTCCLK / 16384 */
-#define RTC_TAMPERSAMPLINGFREQ_RTCCLK_DIV8192   RTC_TAFCR_TAMPFREQ_1
/*!< Each of the tamper inputs are sampled
-
with a frequency =  RTCCLK / 8192  */
-#define RTC_TAMPERSAMPLINGFREQ_RTCCLK_DIV4096   (RTC_TAFCR_TAMPFREQ_0 |
RTC_TAFCR_TAMPFREQ_1) /*!< Each of the tamper inputs are sampled
```

```
-
with a frequency =  RTCCLK / 4096  */
-#define RTC_TAMPERSAMPLINGFREQ_RTCCLK_DIV2048   RTC_TAFCR_TAMPFREQ_2
/*!< Each of the tamper inputs are sampled
-
with a frequency =  RTCCLK / 2048  */
-#define RTC_TAMPERSAMPLINGFREQ_RTCCLK_DIV1024   (RTC_TAFCR_TAMPFREQ_0 |
RTC_TAFCR_TAMPFREQ_2) /*!< Each of the tamper inputs are sampled
-
with a frequency =  RTCCLK / 1024  */
-#define RTC_TAMPERSAMPLINGFREQ_RTCCLK_DIV512    (RTC_TAFCR_TAMPFREQ_1 |
RTC_TAFCR_TAMPFREQ_2) /*!< Each of the tamper inputs are sampled
-
with a frequency =  RTCCLK / 512   */
-#define RTC_TAMPERSAMPLINGFREQ_RTCCLK_DIV256    RTC_TAFCR_TAMPFREQ
/*!< Each of the tamper inputs are sampled
-
with a frequency =  RTCCLK / 256   */
-#define RTC_TAMPERSAMPLINGFREQ_RTCCLK_MASK      RTC_TAFCR_TAMPFREQ
/*!< Masking all bits except those of
-
field TAMPFREQ                     */
-/**
-  * @}
-  */
-
-/** @defgroup RTCEx_Tamper_Pin_Precharge_Duration_Definitions RTCEx
Tamper Pin Precharge Duration Definitions
-  * @{
-  */
-#define RTC_TAMPERPRECHARGEDURATION_1RTCCLK     0x00000000U
/*!< Tamper pins are pre-charged before
-
sampling during 1 RTCCLK cycle  */
-#define RTC_TAMPERPRECHARGEDURATION_2RTCCLK     RTC_TAFCR_TAMPPRCH_0
/*!< Tamper pins are pre-charged before
-
sampling during 2 RTCCLK cycles */
-#define RTC_TAMPERPRECHARGEDURATION_4RTCCLK     RTC_TAFCR_TAMPPRCH_1
/*!< Tamper pins are pre-charged before
-
sampling during 4 RTCCLK cycles */
-#define RTC_TAMPERPRECHARGEDURATION_8RTCCLK     RTC_TAFCR_TAMPPRCH
/*!< Tamper pins are pre-charged before
-
sampling during 8 RTCCLK cycles */
-#define RTC_TAMPERPRECHARGEDURATION_MASK        RTC_TAFCR_TAMPPRCH
/*!< Masking all bits except those of
-
field TAMPPRCH                  */
-/**
-  * @}
-  */
-
-/** @defgroup  RTCEx_Tamper_Pull_Up_Definitions RTCEx Tamper Pull Up
Definitions
-  * @{
-  */
```

```
-#define RTC_TAMPER_PULLUP_ENABLE  0x00000000U          /*!< Tamper pins
are pre-charged before sampling     */
-#define RTC_TAMPER_PULLUP_DISABLE RTC_TAFCR_TAMPPUDIS   /*!< Tamper pins
are not pre-charged before sampling */
-#define RTC_TAMPER_PULLUP_MASK    RTC_TAFCR_TAMPPUDIS   /*!< Masking all
bits except bit TAMPPUDIS           */
-/**
-  * @}
-  */
-
-/** @defgroup RTCEx_Tamper_TimeStampOnTamperDetection_Definitions RTCEx
Tamper TimeStamp On Tamper Detection Definitions
-  * @{
-  */
-#define RTC_TIMESTAMPONTAMPERDETECTION_ENABLE  RTC_TAFCR_TAMPTS  /*!<
TimeStamp on Tamper Detection event saved        */
-#define RTC_TIMESTAMPONTAMPERDETECTION_DISABLE 0x00000000U       /*!<
TimeStamp on Tamper Detection event is not saved */
-#define RTC_TIMESTAMPONTAMPERDETECTION_MASK    RTC_TAFCR_TAMPTS  /*!<
Masking all bits except bit TAMPTS               */
-/**
-  * @}
-  */
-
-/** @defgroup RTCEx_Wakeup_Timer_Definitions RTCEx Wakeup Timer
Definitions
-  * @{
-  */
-#define RTC_WAKEUPCLOCK_RTCCLK_DIV16        0x00000000U
-#define RTC_WAKEUPCLOCK_RTCCLK_DIV8         RTC_CR_WUCKSEL_0
-#define RTC_WAKEUPCLOCK_RTCCLK_DIV4         RTC_CR_WUCKSEL_1
-#define RTC_WAKEUPCLOCK_RTCCLK_DIV2         (RTC_CR_WUCKSEL_0 |
RTC_CR_WUCKSEL_1)
-#define RTC_WAKEUPCLOCK_CK_SPRE_16BITS      RTC_CR_WUCKSEL_2
-#define RTC_WAKEUPCLOCK_CK_SPRE_17BITS      (RTC_CR_WUCKSEL_1 |
RTC_CR_WUCKSEL_2)
-/**
-  * @}
-  */
-
-/** @defgroup RTCEx_Coarse_Calibration_Definitions RTCEx Coarse Calib
Definitions
-  * @{
-  */
-#define RTC_CALIBSIGN_POSITIVE            0x00000000U
-#define RTC_CALIBSIGN_NEGATIVE            RTC_CALIBR_DCS
-/**
-  * @}
-  */
-
-/** @defgroup RTCEx_Smooth_calib_period_Definitions RTCEx Smooth Calib
Period Definitions
-  * @{
-  */
-#define RTC_SMOOTHCALIB_PERIOD_32SEC   0x00000000U      /*!< If RTCCLK =
32768 Hz, smooth calibration
-                                                           period is
32s, otherwise 2^20 RTCCLK pulses */
```

```
-#define RTC_SMOOTHCALIB_PERIOD_16SEC   RTC_CALR_CALW16  /*!< If RTCCLK =
32768 Hz, smooth calibration
-                                                        period is
16s, otherwise 2^19 RTCCLK pulses */
-#define RTC_SMOOTHCALIB_PERIOD_8SEC    RTC_CALR_CALW8   /*!< If RTCCLK =
32768 Hz, smooth calibration
-                                                        period is
8s, otherwise 2^18 RTCCLK pulses  */
-/**
-  * @}
-  */
-
-/** @defgroup RTCEx_Smooth_calib_Plus_pulses_Definitions RTCEx Smooth
Calib Plus Pulses Definitions
-  * @{
-  */
-#define RTC_SMOOTHCALIB_PLUSPULSES_SET    RTC_CALR_CALP           /*!<
The number of RTCCLK pulses added
-
during a X -second window = Y - CALM[8:0]
-
with Y = 512, 256, 128 when X = 32, 16, 8 */
-#define RTC_SMOOTHCALIB_PLUSPULSES_RESET  0x00000000U             /*!<
The number of RTCCLK pulses subbstited
-
during a 32-second window = CALM[8:0] */
-/**
-  * @}
-  */
-
-/** @defgroup RTCEx_Add_1_Second_Parameter_Definitions RTCEx Add 1
Second Parameter Definitions
-  * @{
-  */
-#define RTC_SHIFTADD1S_RESET      0x00000000U
-#define RTC_SHIFTADD1S_SET        RTC_SHIFTR_ADD1S
-/**
-  * @}
-  */
-
-/** @defgroup RTCEx_Calib_Output_selection_Definitions RTCEx Calib
Output Selection Definitions
-  * @{
-  */
-#define RTC_CALIBOUTPUT_512HZ            0x00000000U
-#define RTC_CALIBOUTPUT_1HZ              RTC_CR_COSEL
-/**
-  * @}
-  */
-
-/**
-  * @}
-  */
-
-/* Exported macros ----------------------------------------------------
------*/
-
-/** @defgroup RTCEx_Exported_Macros RTCEx Exported Macros
```

```
-  * @{
-  */
-
-/* ---------------------------------WAKEUPTIMER-------------------------
------*/
-
-/** @defgroup RTCEx_WakeUp_Timer RTCEx WakeUp Timer
-  * @{
-  */
-
-/**
-  * @brief  Enable the RTC WakeUp Timer peripheral.
-  * @param  __HANDLE__ specifies the RTC handle.
-  * @retval None
-  */
-#define __HAL_RTC_WAKEUPTIMER_ENABLE(__HANDLE__)
((__HANDLE__)->Instance->CR |= (RTC_CR_WUTE))
-
-/**
-  * @brief  Disable the RTC Wakeup Timer peripheral.
-  * @param  __HANDLE__ specifies the RTC handle.
-  * @retval None
-  */
-#define __HAL_RTC_WAKEUPTIMER_DISABLE(__HANDLE__)
((__HANDLE__)->Instance->CR &= ~(RTC_CR_WUTE))
-
-/**
-  * @brief  Enable the RTC Wakeup Timer interrupt.
-  * @param  __HANDLE__ specifies the RTC handle.
-  * @param  __INTERRUPT__ specifies the RTC Wakeup Timer interrupt
sources to be enabled or disabled.
-  *         This parameter can be:
-  *            @arg RTC_IT_WUT: Wakeup Timer interrupt
-  * @retval None
-  */
-#define __HAL_RTC_WAKEUPTIMER_ENABLE_IT(__HANDLE__, __INTERRUPT__)
((__HANDLE__)->Instance->CR |= (__INTERRUPT__))
-
-/**
-  * @brief  Disable the RTC Wakeup Timer interrupt.
-  * @param  __HANDLE__ specifies the RTC handle.
-  * @param  __INTERRUPT__ specifies the RTC Wakeup Timer interrupt
sources to be enabled or disabled.
-  *         This parameter can be:
-  *            @arg RTC_IT_WUT: Wakeup Timer interrupt
-  * @retval None
-  */
-#define __HAL_RTC_WAKEUPTIMER_DISABLE_IT(__HANDLE__, __INTERRUPT__)
((__HANDLE__)->Instance->CR &= ~(__INTERRUPT__))
-
-/**
-  * @brief  Check whether the specified RTC Wakeup Timer interrupt has
occurred or not.
-  * @param  __HANDLE__ specifies the RTC handle.
-  * @param  __INTERRUPT__ specifies the RTC Wakeup Timer interrupt to
check.
-  *         This parameter can be:
-  *            @arg RTC_IT_WUT: Wakeup Timer interrupt
```

```
-  * @retval None
-  */
-#define __HAL_RTC_WAKEUPTIMER_GET_IT(__HANDLE__, __INTERRUPT__)
(((((__HANDLE__)->Instance->ISR) & ((__INTERRUPT__) >> 4U)) != 0U) ? 1U :
0U)
-
-/**
-  * @brief  Check whether the specified RTC Wakeup timer interrupt has
been enabled or not.
-  * @param  __HANDLE__ specifies the RTC handle.
-  * @param  __INTERRUPT__ specifies the RTC Wakeup timer interrupt
sources to check.
-  *         This parameter can be:
-  *           @arg RTC_IT_WUT: WakeUpTimer interrupt
-  * @retval None
-  */
-#define __HAL_RTC_WAKEUPTIMER_GET_IT_SOURCE(__HANDLE__, __INTERRUPT__)
(((((__HANDLE__)->Instance->CR) & (__INTERRUPT__)) != 0U) ? 1U : 0U)
-
-/**
-  * @brief  Get the selected RTC Wakeup Timer's flag status.
-  * @param  __HANDLE__ specifies the RTC handle.
-  * @param  __FLAG__ specifies the RTC Wakeup Timer flag to check.
-  *          This parameter can be:
-  *             @arg RTC_FLAG_WUTF: Wakeup Timer interrupt flag
-  *             @arg RTC_FLAG_WUTWF: Wakeup Timer 'write allowed' flag
-  * @retval None
-  */
-#define __HAL_RTC_WAKEUPTIMER_GET_FLAG(__HANDLE__, __FLAG__)
(((((__HANDLE__)->Instance->ISR) & (__FLAG__)) != 0U)? 1U : 0U)
-
-/**
-  * @brief  Clear the RTC Wakeup timer's pending flags.
-  * @param  __HANDLE__ specifies the RTC handle.
-  * @param  __FLAG__ specifies the RTC Wakeup Timer Flag to clear.
-  *         This parameter can be:
-  *             @arg RTC_FLAG_WUTF: Wakeup Timer interrupt Flag
-  * @retval None
-  */
-#define __HAL_RTC_WAKEUPTIMER_CLEAR_FLAG(__HANDLE__, __FLAG__)
((__HANDLE__)->Instance->ISR) = (~((__FLAG__) |
RTC_ISR_INIT)|((__HANDLE__)->Instance->ISR & RTC_ISR_INIT))
-
-/**
-  * @brief  Enable interrupt on the RTC Wakeup Timer associated EXTI
line.
-  * @retval None
-  */
-#define __HAL_RTC_WAKEUPTIMER_EXTI_ENABLE_IT()       (EXTI->IMR |=
RTC_EXTI_LINE_WAKEUPTIMER_EVENT)
-
-/**
-  * @brief  Disable interrupt on the RTC Wakeup Timer associated EXTI
line.
-  * @retval None
-  */
-#define __HAL_RTC_WAKEUPTIMER_EXTI_DISABLE_IT()      (EXTI->IMR &=
~RTC_EXTI_LINE_WAKEUPTIMER_EVENT)
```

```
-
-/**
-  * @brief  Enable event on the RTC Wakeup Timer associated EXTI line.
-  * @retval None.
-  */
-#define __HAL_RTC_WAKEUPTIMER_EXTI_ENABLE_EVENT()    (EXTI->EMR |=
RTC_EXTI_LINE_WAKEUPTIMER_EVENT)
-
-/**
-  * @brief  Disable event on the RTC Wakeup Timer associated EXTI line.
-  * @retval None.
-  */
-#define __HAL_RTC_WAKEUPTIMER_EXTI_DISABLE_EVENT()   (EXTI->EMR &=
~RTC_EXTI_LINE_WAKEUPTIMER_EVENT)
-
-/**
-  * @brief  Enable falling edge trigger on the RTC Wakeup Timer
associated EXTI line.
-  * @retval None.
-  */
-#define __HAL_RTC_WAKEUPTIMER_EXTI_ENABLE_FALLING_EDGE()   (EXTI->FTSR
|= RTC_EXTI_LINE_WAKEUPTIMER_EVENT)
-
-/**
-  * @brief  Disable falling edge trigger on the RTC Wakeup Timer
associated EXTI line.
-  * @retval None.
-  */
-#define __HAL_RTC_WAKEUPTIMER_EXTI_DISABLE_FALLING_EDGE()  (EXTI->FTSR
&= ~RTC_EXTI_LINE_WAKEUPTIMER_EVENT)
-
-/**
-  * @brief  Enable rising edge trigger on the RTC Wakeup Timer
associated EXTI line.
-  * @retval None.
-  */
-#define __HAL_RTC_WAKEUPTIMER_EXTI_ENABLE_RISING_EDGE()    (EXTI->RTSR
|= RTC_EXTI_LINE_WAKEUPTIMER_EVENT)
-
-/**
-  * @brief  Disable rising edge trigger on the RTC Wakeup Timer
associated EXTI line.
-  * @retval None.
-  */
-#define __HAL_RTC_WAKEUPTIMER_EXTI_DISABLE_RISING_EDGE()   (EXTI->RTSR
&= ~RTC_EXTI_LINE_WAKEUPTIMER_EVENT)
-
-/**
-  * @brief  Enable rising & falling edge trigger on the RTC Wakeup Timer
associated EXTI line.
-  * @retval None.
-  */
-#define __HAL_RTC_WAKEUPTIMER_EXTI_ENABLE_RISING_FALLING_EDGE() do {
\
-
__HAL_RTC_WAKEUPTIMER_EXTI_ENABLE_RISING_EDGE();  \
-
__HAL_RTC_WAKEUPTIMER_EXTI_ENABLE_FALLING_EDGE(); \
```

```
-                                                                       }
while(0U)
-
-/**
-  * @brief  Disable rising & falling edge trigger on the RTC Wakeup
Timer associated EXTI line.
-  * This parameter can be:
-  * @retval None.
-  */
-#define __HAL_RTC_WAKEUPTIMER_EXTI_DISABLE_RISING_FALLING_EDGE() do {
\
-
__HAL_RTC_WAKEUPTIMER_EXTI_DISABLE_RISING_EDGE();  \
-
__HAL_RTC_WAKEUPTIMER_EXTI_DISABLE_FALLING_EDGE(); \
-                                                                        }
while(0U)
-
-/**
-  * @brief Check whether the RTC Wakeup Timer associated EXTI line
interrupt flag is set or not.
-  * @retval Line Status.
-  */
-#define __HAL_RTC_WAKEUPTIMER_EXTI_GET_FLAG()              (EXTI->PR &
RTC_EXTI_LINE_WAKEUPTIMER_EVENT)
-
-/**
-  * @brief Clear the RTC Wakeup Timer associated EXTI line flag.
-  * @retval None.
-  */
-#define __HAL_RTC_WAKEUPTIMER_EXTI_CLEAR_FLAG()            (EXTI->PR =
RTC_EXTI_LINE_WAKEUPTIMER_EVENT)
-
-/**
-  * @brief Generate a Software interrupt on the RTC Wakeup Timer
associated EXTI line.
-  * @retval None.
-  */
-#define __HAL_RTC_WAKEUPTIMER_EXTI_GENERATE_SWIT()         (EXTI->SWIER
|= RTC_EXTI_LINE_WAKEUPTIMER_EVENT)
-
-/**
-  * @}
-  */
-
-/* ---------------------------------TIMESTAMP----------------------------
------*/
-
-/** @defgroup RTCEx_Timestamp RTCEx Timestamp
-  * @{
-  */
-
-/**
-  * @brief  Enable the RTC Timestamp peripheral.
-  * @param  __HANDLE__ specifies the RTC handle.
-  * @retval None
-  */
```

```
-#define __HAL_RTC_TIMESTAMP_ENABLE(__HANDLE__)
((__HANDLE__)->Instance->CR |= (RTC_CR_TSE))
-
-/**
-  * @brief  Disable the RTC Timestamp peripheral.
-  * @param  __HANDLE__ specifies the RTC handle.
-  * @retval None
-  */
-#define __HAL_RTC_TIMESTAMP_DISABLE(__HANDLE__)
((__HANDLE__)->Instance->CR &= ~(RTC_CR_TSE))
-
-/**
-  * @brief  Enable the RTC Timestamp interrupt.
-  * @param  __HANDLE__ specifies the RTC handle.
-  * @param  __INTERRUPT__ specifies the RTC Timestamp interrupt sources
to be enabled or disabled.
-  *         This parameter can be:
-  *            @arg RTC_IT_TS: TimeStamp interrupt
-  * @retval None
-  */
-#define __HAL_RTC_TIMESTAMP_ENABLE_IT(__HANDLE__, __INTERRUPT__)
((__HANDLE__)->Instance->CR |= (__INTERRUPT__))
-
-/**
-  * @brief  Disable the RTC Timestamp interrupt.
-  * @param  __HANDLE__ specifies the RTC handle.
-  * @param  __INTERRUPT__ specifies the RTC Timestamp interrupt sources
to be enabled or disabled.
-  *         This parameter can be:
-  *            @arg RTC_IT_TS: TimeStamp interrupt
-  * @retval None
-  */
-#define __HAL_RTC_TIMESTAMP_DISABLE_IT(__HANDLE__, __INTERRUPT__)
((__HANDLE__)->Instance->CR &= ~(__INTERRUPT__))
-
-/**
-  * @brief  Check whether the specified RTC Timestamp interrupt has
occurred or not.
-  * @param  __HANDLE__ specifies the RTC handle.
-  * @param  __INTERRUPT__ specifies the RTC Timestamp interrupt to
check.
-  *         This parameter can be:
-  *            @arg RTC_IT_TS: TimeStamp interrupt
-  * @retval None
-  */
-#define __HAL_RTC_TIMESTAMP_GET_IT(__HANDLE__, __INTERRUPT__)
(((((__HANDLE__)->Instance->ISR) & ((__INTERRUPT__) >> 4U)) != 0U) ? 1U :
0U)
-
-/**
-  * @brief  Check whether the specified RTC Timestamp interrupt has been
enabled or not.
-  * @param  __HANDLE__ specifies the RTC handle.
-  * @param  __INTERRUPT__ specifies the RTC Timestamp interrupt source
to check.
-  *         This parameter can be:
-  *            @arg RTC_IT_TS: TimeStamp interrupt
-  * @retval None
```

```
-  */
-#define __HAL_RTC_TIMESTAMP_GET_IT_SOURCE(__HANDLE__, __INTERRUPT__)
(((((__HANDLE__)->Instance->CR) & (__INTERRUPT__)) != 0U) ? 1U : 0U)
-
-/**
-  * @brief  Get the selected RTC Timestamp's flag status.
-  * @param  __HANDLE__ specifies the RTC handle.
-  * @param  __FLAG__ specifies the RTC Timestamp flag to check.
-  *         This parameter can be:
-  *            @arg RTC_FLAG_TSF: Timestamp interrupt flag
-  *            @arg RTC_FLAG_TSOVF: Timestamp overflow flag
-  * @retval None
-  */
-#define __HAL_RTC_TIMESTAMP_GET_FLAG(__HANDLE__, __FLAG__)
(((((__HANDLE__)->Instance->ISR) & (__FLAG__)) != 0U)? 1U : 0U)
-
-/**
-  * @brief  Clear the RTC Timestamp's pending flags.
-  * @param  __HANDLE__ specifies the RTC handle.
-  * @param  __FLAG__ specifies the RTC Timestamp flag to clear.
-  *         This parameter can be:
-  *            @arg RTC_FLAG_TSF: Timestamp interrupt flag
-  *            @arg RTC_FLAG_TSOVF: Timestamp overflow flag
-  * @retval None
-  */
-#define __HAL_RTC_TIMESTAMP_CLEAR_FLAG(__HANDLE__, __FLAG__)
((__HANDLE__)->Instance->ISR) = (~((__FLAG__) |
RTC_ISR_INIT)|((__HANDLE__)->Instance->ISR & RTC_ISR_INIT))
-
-/**
-  * @}
-  */
-
-/* ---------------------------------TAMPER-------------------------------
------*/
-
-/** @defgroup RTCEx_Tamper RTCEx Tamper
-  * @{
-  */
-
-/**
-  * @brief  Enable the RTC Tamper1 input detection.
-  * @param  __HANDLE__ specifies the RTC handle.
-  * @retval None
-  */
-#define __HAL_RTC_TAMPER1_ENABLE(__HANDLE__)
((__HANDLE__)->Instance->TAFCR |= (RTC_TAFCR_TAMP1E))
-
-/**
-  * @brief  Disable the RTC Tamper1 input detection.
-  * @param  __HANDLE__ specifies the RTC handle.
-  * @retval None
-  */
-#define __HAL_RTC_TAMPER1_DISABLE(__HANDLE__)
((__HANDLE__)->Instance->TAFCR &= ~(RTC_TAFCR_TAMP1E))
-
-#if defined(RTC_TAMPER2_SUPPORT)
-/**
```

```
-  * @brief  Enable the RTC Tamper2 input detection.
-  * @param  __HANDLE__ specifies the RTC handle.
-  * @retval None
-  */
-#define __HAL_RTC_TAMPER2_ENABLE(__HANDLE__)
((__HANDLE__)->Instance->TAFCR |= (RTC_TAFCR_TAMP2E))
-
-/**
-  * @brief  Disable the RTC Tamper2 input detection.
-  * @param  __HANDLE__ specifies the RTC handle.
-  * @retval None
-  */
-#define __HAL_RTC_TAMPER2_DISABLE(__HANDLE__)
((__HANDLE__)->Instance->TAFCR &= ~(RTC_TAFCR_TAMP2E))
-#endif /* RTC_TAMPER2_SUPPORT */
-
-/**
-  * @brief  Enable the RTC Tamper interrupt.
-  * @param  __HANDLE__ specifies the RTC handle.
-  * @param  __INTERRUPT__ specifies the RTC Tamper interrupt sources to
be enabled.
-  *          This parameter can be any combination of the following
values:
-  *            @arg RTC_IT_TAMP: Tamper global interrupt
-  * @retval None
-  */
-#define __HAL_RTC_TAMPER_ENABLE_IT(__HANDLE__, __INTERRUPT__)
((__HANDLE__)->Instance->TAFCR |= (__INTERRUPT__))
-
-/**
-  * @brief  Disable the RTC Tamper interrupt.
-  * @param  __HANDLE__ specifies the RTC handle.
-  * @param  __INTERRUPT__ specifies the RTC Tamper interrupt sources to
be disabled.
-  *         This parameter can be any combination of the following
values:
-  *            @arg RTC_IT_TAMP: Tamper global interrupt
-  * @retval None
-  */
-#define __HAL_RTC_TAMPER_DISABLE_IT(__HANDLE__, __INTERRUPT__)
((__HANDLE__)->Instance->TAFCR &= ~(__INTERRUPT__))
-
-/**
-  * @brief  Check whether the specified RTC Tamper interrupt has been
enabled or not.
-  * @param  __HANDLE__ specifies the RTC handle.
-  * @param  __INTERRUPT__ specifies the RTC Tamper interrupt source to
check.
-  *         This parameter can be:
-  *            @arg RTC_IT_TAMP: Tamper global interrupt
-  * @retval None
-  */
-#define __HAL_RTC_TAMPER_GET_IT_SOURCE(__HANDLE__, __INTERRUPT__)
(((((__HANDLE__)->Instance->TAFCR) & (__INTERRUPT__)) != 0U) ? 1U : 0U)
-
-/**
-  * @brief  Get the selected RTC Tamper's flag status.
-  * @param  __HANDLE__ specifies the RTC handle.
```

```
-  * @param  __FLAG__ specifies the RTC Tamper flag to be checked.
-  *          This parameter can be:
-  *             @arg RTC_FLAG_TAMP1F: Tamper 1 interrupt flag
-  *             @arg RTC_FLAG_TAMP2F: Tamper 2 interrupt flag (*)
-  *
-  *         (*) value not applicable to all devices.
-  * @retval None
-  */
-#define __HAL_RTC_TAMPER_GET_FLAG(__HANDLE__, __FLAG__)
(((((__HANDLE__)->Instance->ISR) & (__FLAG__)) != 0U)? 1U : 0U)
-
-/**
-  * @brief  Clear the RTC Tamper's pending flags.
-  * @param  __HANDLE__ specifies the RTC handle.
-  * @param  __FLAG__ specifies the RTC Tamper Flag to clear.
-  *          This parameter can be:
-  *             @arg RTC_FLAG_TAMP1F: Tamper 1 interrupt flag
-  *             @arg RTC_FLAG_TAMP2F: Tamper 2 interrupt flag (*)
-  *
-  *         (*) value not applicable to all devices.
-  * @retval None
-  */
-#define __HAL_RTC_TAMPER_CLEAR_FLAG(__HANDLE__, __FLAG__)
((__HANDLE__)->Instance->ISR) = (~((__FLAG__) |
RTC_ISR_INIT)|((__HANDLE__)->Instance->ISR & RTC_ISR_INIT))
-/**
-  * @}
-  */
-
-/* ---------------------------TAMPER/TIMESTAMP----------------------------
------*/
-/** @defgroup RTCEx_Tamper_Timestamp EXTI RTC Tamper Timestamp EXTI
-  * @{
-  */
-
-/**
-  * @brief  Enable interrupt on the RTC Tamper and Timestamp associated
EXTI line.
-  * @retval None
-  */
-#define __HAL_RTC_TAMPER_TIMESTAMP_EXTI_ENABLE_IT()        (EXTI->IMR |=
RTC_EXTI_LINE_TAMPER_TIMESTAMP_EVENT)
-
-/**
-  * @brief  Disable interrupt on the RTC Tamper and Timestamp associated
EXTI line.
-  * @retval None
-  */
-#define __HAL_RTC_TAMPER_TIMESTAMP_EXTI_DISABLE_IT()       (EXTI->IMR &=
~RTC_EXTI_LINE_TAMPER_TIMESTAMP_EVENT)
-
-/**
-  * @brief  Enable event on the RTC Tamper and Timestamp associated EXTI
line.
-  * @retval None.
-  */
-#define __HAL_RTC_TAMPER_TIMESTAMP_EXTI_ENABLE_EVENT()     (EXTI->EMR |=
RTC_EXTI_LINE_TAMPER_TIMESTAMP_EVENT)
```

```
-
-/**
-  * @brief  Disable event on the RTC Tamper and Timestamp associated
EXTI line.
-  * @retval None.
-  */
-#define __HAL_RTC_TAMPER_TIMESTAMP_EXTI_DISABLE_EVENT()    (EXTI->EMR &=
~RTC_EXTI_LINE_TAMPER_TIMESTAMP_EVENT)
-
-/**
-  * @brief  Enable falling edge trigger on the RTC Tamper and Timestamp
associated EXTI line.
-  * @retval None.
-  */
-#define __HAL_RTC_TAMPER_TIMESTAMP_EXTI_ENABLE_FALLING_EDGE()   (EXTI-
>FTSR |= RTC_EXTI_LINE_TAMPER_TIMESTAMP_EVENT)
-
-/**
-  * @brief  Disable falling edge trigger on the RTC Tamper and Timestamp
associated EXTI line.
-  * @retval None.
-  */
-#define __HAL_RTC_TAMPER_TIMESTAMP_EXTI_DISABLE_FALLING_EDGE()  (EXTI-
>FTSR &= ~RTC_EXTI_LINE_TAMPER_TIMESTAMP_EVENT)
-
-/**
-  * @brief  Enable rising edge trigger on the RTC Tamper and Timestamp
associated EXTI line.
-  * @retval None.
-  */
-#define __HAL_RTC_TAMPER_TIMESTAMP_EXTI_ENABLE_RISING_EDGE()    (EXTI-
>RTSR |= RTC_EXTI_LINE_TAMPER_TIMESTAMP_EVENT)
-
-/**
-  * @brief  Disable rising edge trigger on the RTC Tamper and Timestamp
associated EXTI line.
-  * @retval None.
-  */
-#define __HAL_RTC_TAMPER_TIMESTAMP_EXTI_DISABLE_RISING_EDGE()   (EXTI-
>RTSR &= ~RTC_EXTI_LINE_TAMPER_TIMESTAMP_EVENT)
-
-/**
-  * @brief  Enable rising & falling edge trigger on the RTC Tamper and
Timestamp associated EXTI line.
-  * @retval None.
-  */
-#define __HAL_RTC_TAMPER_TIMESTAMP_EXTI_ENABLE_RISING_FALLING_EDGE() do
{                                                           \
-
__HAL_RTC_TAMPER_TIMESTAMP_EXTI_ENABLE_RISING_EDGE();  \
-
__HAL_RTC_TAMPER_TIMESTAMP_EXTI_ENABLE_FALLING_EDGE(); \
-
} while(0U)
-
-/**
-  * @brief  Disable rising & falling edge trigger on the RTC Tamper and
Timestamp associated EXTI line.
```

```
-  * This parameter can be:
-  * @retval None.
-  */
-#define __HAL_RTC_TAMPER_TIMESTAMP_EXTI_DISABLE_RISING_FALLING_EDGE() do
{                                                                    \
-
__HAL_RTC_TAMPER_TIMESTAMP_EXTI_DISABLE_RISING_EDGE();  \
-
__HAL_RTC_TAMPER_TIMESTAMP_EXTI_DISABLE_FALLING_EDGE(); \
-
} while(0U)
-
-/**
-  * @brief Check whether the RTC Tamper and Timestamp associated EXTI
line interrupt flag is set or not.
-  * @retval Line Status.
-  */
-#define __HAL_RTC_TAMPER_TIMESTAMP_EXTI_GET_FLAG()         (EXTI->PR &
RTC_EXTI_LINE_TAMPER_TIMESTAMP_EVENT)
-
-/**
-  * @brief Clear the RTC Tamper and Timestamp associated EXTI line flag.
-  * @retval None.
-  */
-#define __HAL_RTC_TAMPER_TIMESTAMP_EXTI_CLEAR_FLAG()       (EXTI->PR =
RTC_EXTI_LINE_TAMPER_TIMESTAMP_EVENT)
-
-/**
-  * @brief Generate a Software interrupt on the RTC Tamper and Timestamp
associated EXTI line
-  * @retval None.
-  */
-#define __HAL_RTC_TAMPER_TIMESTAMP_EXTI_GENERATE_SWIT()    (EXTI->SWIER
|= RTC_EXTI_LINE_TAMPER_TIMESTAMP_EVENT)
-/**
-  * @}
-  */
-
-/* ------------------------------CALIBRATION--------------------------
------*/
-
-/** @defgroup RTCEx_Calibration RTCEx Calibration
-  * @{
-  */
-
-/**
-  * @brief  Enable the Coarse calibration process.
-  * @param  __HANDLE__ specifies the RTC handle.
-  * @retval None
-  */
-#define __HAL_RTC_COARSE_CALIB_ENABLE(__HANDLE__)
((__HANDLE__)->Instance->CR |= (RTC_CR_DCE))
-
-/**
-  * @brief  Disable the Coarse calibration process.
-  * @param  __HANDLE__ specifies the RTC handle.
-  * @retval None
-  */
```

```
-#define __HAL_RTC_COARSE_CALIB_DISABLE(__HANDLE__)
((__HANDLE__)->Instance->CR &= ~(RTC_CR_DCE))
-
-/**
-  * @brief  Enable the RTC calibration output.
-  * @param  __HANDLE__ specifies the RTC handle.
-  * @retval None
-  */
-#define __HAL_RTC_CALIBRATION_OUTPUT_ENABLE(__HANDLE__)
((__HANDLE__)->Instance->CR |= (RTC_CR_COE))
-
-/**
-  * @brief  Disable the calibration output.
-  * @param  __HANDLE__ specifies the RTC handle.
-  * @retval None
-  */
-#define __HAL_RTC_CALIBRATION_OUTPUT_DISABLE(__HANDLE__)
((__HANDLE__)->Instance->CR &= ~(RTC_CR_COE))
-
-/**
-  * @brief  Enable the clock reference detection.
-  * @param  __HANDLE__ specifies the RTC handle.
-  * @retval None
-  */
-#define __HAL_RTC_CLOCKREF_DETECTION_ENABLE(__HANDLE__)
((__HANDLE__)->Instance->CR |= (RTC_CR_REFCKON))
-
-/**
-  * @brief  Disable the clock reference detection.
-  * @param  __HANDLE__ specifies the RTC handle.
-  * @retval None
-  */
-#define __HAL_RTC_CLOCKREF_DETECTION_DISABLE(__HANDLE__)
((__HANDLE__)->Instance->CR &= ~(RTC_CR_REFCKON))
-
-/**
-  * @brief  Get the selected RTC shift operation's flag status.
-  * @param  __HANDLE__ specifies the RTC handle.
-  * @param  __FLAG__ specifies the RTC shift operation Flag is pending
or not.
-  *          This parameter can be:
-  *             @arg RTC_FLAG_SHPF: Shift pending flag
-  * @retval None
-  */
-#define __HAL_RTC_SHIFT_GET_FLAG(__HANDLE__, __FLAG__)
(((((__HANDLE__)->Instance->ISR) & (__FLAG__)) != 0U)? 1U : 0U)
-/**
-  * @}
-  */
-
-/**
-  * @}
-  */
-
-/* Exported functions ---------------------------------------------------
------*/
-
-/** @defgroup RTCEx_Exported_Functions RTCEx Exported Functions
```

```
-  * @{
-  */
-
-/** @addtogroup RTCEx_Exported_Functions_Group1
-  * @{
-  */
-/* RTC Timestamp and Tamper functions
*****************************************/
-HAL_StatusTypeDef HAL_RTCEx_SetTimeStamp(RTC_HandleTypeDef *hrtc,
uint32_t RTC_TimeStampEdge, uint32_t RTC_TimeStampPin);
-HAL_StatusTypeDef HAL_RTCEx_SetTimeStamp_IT(RTC_HandleTypeDef *hrtc,
uint32_t RTC_TimeStampEdge, uint32_t RTC_TimeStampPin);
-HAL_StatusTypeDef HAL_RTCEx_DeactivateTimeStamp(RTC_HandleTypeDef
*hrtc);
-HAL_StatusTypeDef HAL_RTCEx_GetTimeStamp(RTC_HandleTypeDef *hrtc,
RTC_TimeTypeDef *sTimeStamp, RTC_DateTypeDef *sTimeStampDate, uint32_t
Format);
-
-HAL_StatusTypeDef HAL_RTCEx_SetTamper(RTC_HandleTypeDef *hrtc,
RTC_TamperTypeDef *sTamper);
-HAL_StatusTypeDef HAL_RTCEx_SetTamper_IT(RTC_HandleTypeDef *hrtc,
RTC_TamperTypeDef *sTamper);
-HAL_StatusTypeDef HAL_RTCEx_DeactivateTamper(RTC_HandleTypeDef *hrtc,
uint32_t Tamper);
-void              HAL_RTCEx_TamperTimeStampIRQHandler(RTC_HandleTypeDef
*hrtc);
-
-void              HAL_RTCEx_Tamper1EventCallback(RTC_HandleTypeDef
*hrtc);
-#if defined(RTC_TAMPER2_SUPPORT)
-void              HAL_RTCEx_Tamper2EventCallback(RTC_HandleTypeDef
*hrtc);
-#endif /* RTC_TAMPER2_SUPPORT */
-void              HAL_RTCEx_TimeStampEventCallback(RTC_HandleTypeDef
*hrtc);
-HAL_StatusTypeDef HAL_RTCEx_PollForTimeStampEvent(RTC_HandleTypeDef
*hrtc, uint32_t Timeout);
-HAL_StatusTypeDef HAL_RTCEx_PollForTamper1Event(RTC_HandleTypeDef *hrtc,
uint32_t Timeout);
-#if defined(RTC_TAMPER2_SUPPORT)
-HAL_StatusTypeDef HAL_RTCEx_PollForTamper2Event(RTC_HandleTypeDef *hrtc,
uint32_t Timeout);
-#endif /* RTC_TAMPER2_SUPPORT */
-/**
-  * @}
-  */
-
-/** @addtogroup RTCEx_Exported_Functions_Group2
-  * @{
-  */
-/* RTC Wakeup functions
************************************************/
-HAL_StatusTypeDef HAL_RTCEx_SetWakeUpTimer(RTC_HandleTypeDef *hrtc,
uint32_t WakeUpCounter, uint32_t WakeUpClock);
-HAL_StatusTypeDef HAL_RTCEx_SetWakeUpTimer_IT(RTC_HandleTypeDef *hrtc,
uint32_t WakeUpCounter, uint32_t WakeUpClock);
-HAL_StatusTypeDef HAL_RTCEx_DeactivateWakeUpTimer(RTC_HandleTypeDef
*hrtc);
```

```
-uint32_t          HAL_RTCEx_GetWakeUpTimer(RTC_HandleTypeDef *hrtc);
-void              HAL_RTCEx_WakeUpTimerIRQHandler(RTC_HandleTypeDef
*hrtc);
-void              HAL_RTCEx_WakeUpTimerEventCallback(RTC_HandleTypeDef
*hrtc);
-HAL_StatusTypeDef HAL_RTCEx_PollForWakeUpTimerEvent(RTC_HandleTypeDef
*hrtc, uint32_t Timeout);
-/**
-  * @}
-  */
-
-/** @addtogroup RTCEx_Exported_Functions_Group3
-  * @{
-  */
-/* Extended Control functions
************************************************/
-void              HAL_RTCEx_BKUPWrite(RTC_HandleTypeDef *hrtc, uint32_t
BackupRegister, uint32_t Data);
-uint32_t          HAL_RTCEx_BKUPRead(RTC_HandleTypeDef *hrtc, uint32_t
BackupRegister);
-
-HAL_StatusTypeDef HAL_RTCEx_SetCoarseCalib(RTC_HandleTypeDef *hrtc,
uint32_t CalibSign, uint32_t Value);
-HAL_StatusTypeDef HAL_RTCEx_DeactivateCoarseCalib(RTC_HandleTypeDef
*hrtc);
-HAL_StatusTypeDef HAL_RTCEx_SetSmoothCalib(RTC_HandleTypeDef *hrtc,
uint32_t SmoothCalibPeriod, uint32_t SmoothCalibPlusPulses, uint32_t
SmoothCalibMinusPulsesValue);
-HAL_StatusTypeDef HAL_RTCEx_SetSynchroShift(RTC_HandleTypeDef *hrtc,
uint32_t ShiftAdd1S, uint32_t ShiftSubFS);
-HAL_StatusTypeDef HAL_RTCEx_SetCalibrationOutPut(RTC_HandleTypeDef
*hrtc, uint32_t CalibOutput);
-HAL_StatusTypeDef
HAL_RTCEx_DeactivateCalibrationOutPut(RTC_HandleTypeDef *hrtc);
-HAL_StatusTypeDef HAL_RTCEx_SetRefClock(RTC_HandleTypeDef *hrtc);
-HAL_StatusTypeDef HAL_RTCEx_DeactivateRefClock(RTC_HandleTypeDef *hrtc);
-HAL_StatusTypeDef HAL_RTCEx_EnableBypassShadow(RTC_HandleTypeDef *hrtc);
-HAL_StatusTypeDef HAL_RTCEx_DisableBypassShadow(RTC_HandleTypeDef
*hrtc);
-/**
-  * @}
-  */
-
-/** @addtogroup RTCEx_Exported_Functions_Group4
-  * @{
-  */
-/* Extended RTC features functions
*****************************************/
-void              HAL_RTCEx_AlarmBEventCallback(RTC_HandleTypeDef
*hrtc);
-HAL_StatusTypeDef HAL_RTCEx_PollForAlarmBEvent(RTC_HandleTypeDef *hrtc,
uint32_t Timeout);
-/**
-  * @}
-  */
-
-/**
-  * @}
```

```
-  */
-
-/* Private types -----------------------------------------------------------
------*/
-/* Private variables -------------------------------------------------------
------*/
-/* Private constants -------------------------------------------------------
------*/
-
-/** @defgroup RTCEx_Private_Constants RTCEx Private Constants
-  * @{
-  */
-#define RTC_EXTI_LINE_TAMPER_TIMESTAMP_EVENT  EXTI_IMR_MR21  /*!<
External interrupt line 21 Connected to the RTC Tamper and Timestamp
event */
-#define RTC_EXTI_LINE_WAKEUPTIMER_EVENT       EXTI_IMR_MR22  /*!<
External interrupt line 22 Connected to the RTC Wakeup event */
-/**
-  * @}
-  */
-
-/** @defgroup RTCEx_Private_Constants RTCEx Private Constants
-  * @{
-  */
-/* Masks Definition */
-#if defined(RTC_TAMPER2_SUPPORT)
-#define RTC_TAMPER_ENABLE_BITS_MASK         ((uint32_t) (RTC_TAMPER_1 |
\
-                                                         RTC_TAMPER_2))
-
-#define RTC_TAMPER_FLAGS_MASK               ((uint32_t) (RTC_FLAG_TAMP1F
| \
-
RTC_FLAG_TAMP2F))
-#else /* RTC_TAMPER2_SUPPORT */
-#define RTC_TAMPER_ENABLE_BITS_MASK                      RTC_TAMPER_1
-
-#define RTC_TAMPER_FLAGS_MASK                            RTC_FLAG_TAMP1F
-#endif /* RTC_TAMPER2_SUPPORT */
-/**
-  * @}
-  */
-
-/* Private macros ----------------------------------------------------------
------*/
-
-/** @defgroup RTCEx_Private_Macros RTCEx Private Macros
-  * @{
-  */
-
-/** @defgroup RTCEx_IS_RTC_Definitions Private macros to check input
parameters
-  * @{
-  */
-#define IS_RTC_BKP(BKP)     ((BKP) < (uint32_t) RTC_BKP_NUMBER)
-
-#define IS_TIMESTAMP_EDGE(EDGE) (((EDGE) == RTC_TIMESTAMPEDGE_RISING) ||
\
```

```
-                                ((EDGE) == RTC_TIMESTAMPEDGE_FALLING))
-
-#define IS_RTC_TAMPER(TAMPER) ((((TAMPER) &
((uint32_t)~RTC_TAMPER_ENABLE_BITS_MASK)) == 0x00U) && ((TAMPER) != 0U))
-
-#if defined(RTC_AF2_SUPPORT)
-#define IS_RTC_TAMPER_PIN(PIN) (((PIN) == RTC_TAMPERPIN_DEFAULT) || \
-                                ((PIN) == RTC_TAMPERPIN_POS1))
-#else /* RTC_AF2_SUPPORT */
-#define IS_RTC_TAMPER_PIN(PIN) ((PIN) == RTC_TAMPERPIN_DEFAULT)
-#endif /* RTC_AF2_SUPPORT */
-
-#if defined(RTC_AF2_SUPPORT)
-#define IS_RTC_TIMESTAMP_PIN(PIN) (((PIN) == RTC_TIMESTAMPPIN_DEFAULT)
|| \
-                                   ((PIN) == RTC_TIMESTAMPPIN_POS1))
-#else /* RTC_AF2_SUPPORT */
-#define IS_RTC_TIMESTAMP_PIN(PIN) ((PIN) == RTC_TIMESTAMPPIN_DEFAULT)
-#endif /* RTC_AF2_SUPPORT */
-
-#define IS_RTC_TAMPER_TRIGGER(TRIGGER) (((TRIGGER) ==
RTC_TAMPERTRIGGER_RISINGEDGE)  || \
-                                        ((TRIGGER) ==
RTC_TAMPERTRIGGER_FALLINGEDGE) || \
-                                        ((TRIGGER) ==
RTC_TAMPERTRIGGER_LOWLEVEL)    || \
-                                        ((TRIGGER) ==
RTC_TAMPERTRIGGER_HIGHLEVEL))
-
-#define IS_RTC_TAMPER_FILTER(FILTER)  (((FILTER) ==
RTC_TAMPERFILTER_DISABLE) || \
-                                       ((FILTER) ==
RTC_TAMPERFILTER_2SAMPLE) || \
-                                       ((FILTER) ==
RTC_TAMPERFILTER_4SAMPLE) || \
-                                       ((FILTER) ==
RTC_TAMPERFILTER_8SAMPLE))
-
-#define IS_RTC_TAMPER_FILTER_CONFIG_CORRECT(FILTER, TRIGGER)
\
-                       (  (  ((FILTER) != RTC_TAMPERFILTER_DISABLE)
\
-                          && (  ((TRIGGER) ==
RTC_TAMPERTRIGGER_LOWLEVEL)    \
-                             || ((TRIGGER) ==
RTC_TAMPERTRIGGER_HIGHLEVEL))) \
-                       || (  ((FILTER) == RTC_TAMPERFILTER_DISABLE)
\
-                          && (  ((TRIGGER) ==
RTC_TAMPERTRIGGER_RISINGEDGE)  \
-                             || ((TRIGGER) ==
RTC_TAMPERTRIGGER_FALLINGEDGE))))
-
-#define IS_RTC_TAMPER_SAMPLING_FREQ(FREQ) (((FREQ) ==
RTC_TAMPERSAMPLINGFREQ_RTCCLK_DIV32768)|| \
-                                           ((FREQ) ==
RTC_TAMPERSAMPLINGFREQ_RTCCLK_DIV16384)|| \
```

```
-                                             ((FREQ) ==
RTC_TAMPERSAMPLINGFREQ_RTCCLK_DIV8192) || \
-                                             ((FREQ) ==
RTC_TAMPERSAMPLINGFREQ_RTCCLK_DIV4096) || \
-                                             ((FREQ) ==
RTC_TAMPERSAMPLINGFREQ_RTCCLK_DIV2048) || \
-                                             ((FREQ) ==
RTC_TAMPERSAMPLINGFREQ_RTCCLK_DIV1024) || \
-                                             ((FREQ) ==
RTC_TAMPERSAMPLINGFREQ_RTCCLK_DIV512)  || \
-                                             ((FREQ) ==
RTC_TAMPERSAMPLINGFREQ_RTCCLK_DIV256))
-
-#define IS_RTC_TAMPER_PRECHARGE_DURATION(DURATION) (((DURATION) ==
RTC_TAMPERPRECHARGEDURATION_1RTCCLK) || \
-                                                    ((DURATION) ==
RTC_TAMPERPRECHARGEDURATION_2RTCCLK) || \
-                                                    ((DURATION) ==
RTC_TAMPERPRECHARGEDURATION_4RTCCLK) || \
-                                                    ((DURATION) ==
RTC_TAMPERPRECHARGEDURATION_8RTCCLK))
-
-#define IS_RTC_TAMPER_PULLUP_STATE(STATE) (((STATE) ==
RTC_TAMPER_PULLUP_ENABLE) || \
-                                           ((STATE) ==
RTC_TAMPER_PULLUP_DISABLE))
-
-#define IS_RTC_TAMPER_TIMESTAMPONTAMPER_DETECTION(DETECTION)
(((DETECTION) == RTC_TIMESTAMPONTAMPERDETECTION_ENABLE) || \
-
((DETECTION) == RTC_TIMESTAMPONTAMPERDETECTION_DISABLE))
-
-#define IS_RTC_WAKEUP_CLOCK(CLOCK) (((CLOCK) ==
RTC_WAKEUPCLOCK_RTCCLK_DIV16)   || \
-                                    ((CLOCK) ==
RTC_WAKEUPCLOCK_RTCCLK_DIV8)    || \
-                                    ((CLOCK) ==
RTC_WAKEUPCLOCK_RTCCLK_DIV4)    || \
-                                    ((CLOCK) ==
RTC_WAKEUPCLOCK_RTCCLK_DIV2)    || \
-                                    ((CLOCK) ==
RTC_WAKEUPCLOCK_CK_SPRE_16BITS) || \
-                                    ((CLOCK) ==
RTC_WAKEUPCLOCK_CK_SPRE_17BITS))
-
-#define IS_RTC_WAKEUP_COUNTER(COUNTER)  ((COUNTER) <= RTC_WUTR_WUT)
-
-#define IS_RTC_CALIB_SIGN(SIGN) (((SIGN) == RTC_CALIBSIGN_POSITIVE) || \
-                                 ((SIGN) == RTC_CALIBSIGN_NEGATIVE))
-
-#define IS_RTC_CALIB_VALUE(VALUE) ((VALUE) < 0x20U)
-
-#define IS_RTC_SMOOTH_CALIB_PERIOD(PERIOD) (((PERIOD) ==
RTC_SMOOTHCALIB_PERIOD_32SEC) || \
-                                            ((PERIOD) ==
RTC_SMOOTHCALIB_PERIOD_16SEC) || \
-                                            ((PERIOD) ==
RTC_SMOOTHCALIB_PERIOD_8SEC))
```

```
-
-#define IS_RTC_SMOOTH_CALIB_PLUS(PLUS) (((PLUS) ==
RTC_SMOOTHCALIB_PLUSPULSES_SET) || \
-                                        ((PLUS) ==
RTC_SMOOTHCALIB_PLUSPULSES_RESET))
-
-#define  IS_RTC_SMOOTH_CALIB_MINUS(VALUE) ((VALUE) <= RTC_CALR_CALM)
-
-#define IS_RTC_SHIFT_ADD1S(SEL) (((SEL) == RTC_SHIFTADD1S_RESET) || \
-                                 ((SEL) == RTC_SHIFTADD1S_SET))
-
-#define IS_RTC_SHIFT_SUBFS(FS) ((FS) <= RTC_SHIFTR_SUBFS)
-
-#define IS_RTC_CALIB_OUTPUT(OUTPUT)  (((OUTPUT) ==
RTC_CALIBOUTPUT_512HZ) || \
-                                      ((OUTPUT) == RTC_CALIBOUTPUT_1HZ))
-/**
-  * @}
-  */
-
-/**
-  * @}
-  */
-
-/**
-  * @}
-  */
-
-/**
-  * @}
-  */
-
-#ifdef __cplusplus
-}
-#endif
-
-#endif /* STM32F4xx_HAL_RTC_EX_H */
diff --git a/Drivers/STM32F4xx_HAL_Driver/Inc/stm32f4xx_ll_rtc.h
b/Drivers/STM32F4xx_HAL_Driver/Inc/stm32f4xx_ll_rtc.h
deleted file mode 100644
index 8f4ac94..0000000
--- a/Drivers/STM32F4xx_HAL_Driver/Inc/stm32f4xx_ll_rtc.h
+++ /dev/null
@@ -1,3663 +0,0 @@
-/**
-
******************************************************************************
*****
-  * @file    stm32f4xx_ll_rtc.h
-  * @author  MCD Application Team
-  * @brief   Header file of RTC LL module.
-
******************************************************************************
*****
-  * @attention
-  *
-  * Copyright (c) 2016 STMicroelectronics.
-  * All rights reserved.
```

```
-  *
-  * This software is licensed under terms that can be found in the
LICENSE file
-  * in the root directory of this software component.
-  * If no LICENSE file comes with this software, it is provided AS-IS.
-  *
-
*******************************************************************************
-  */
-
-/* Define to prevent recursive inclusion -------------------------------
------*/
-#ifndef STM32F4xx_LL_RTC_H
-#define STM32F4xx_LL_RTC_H
-
-#ifdef __cplusplus
-extern "C" {
-#endif
-
-/* Includes ------------------------------------------------------------
------*/
-#include "stm32f4xx.h"
-
-/** @addtogroup STM32F4xx_LL_Driver
-  * @{
-  */
-
-#if defined(RTC)
-
-/** @defgroup RTC_LL RTC
-  * @{
-  */
-
-/* Private types -------------------------------------------------------
------*/
-/* Private variables ---------------------------------------------------
------*/
-/* Private constants ---------------------------------------------------
------*/
-/** @defgroup RTC_LL_Private_Constants RTC Private Constants
-  * @{
-  */
-/* Masks Definition */
-#define RTC_INIT_MASK                 0xFFFFFFFFU
-#define RTC_RSF_MASK                  ((uint32_t)~(RTC_ISR_INIT |
RTC_ISR_RSF))
-
-/* Write protection defines */
-#define RTC_WRITE_PROTECTION_DISABLE  ((uint8_t)0xFFU)
-#define RTC_WRITE_PROTECTION_ENABLE_1 ((uint8_t)0xCAU)
-#define RTC_WRITE_PROTECTION_ENABLE_2 ((uint8_t)0x53U)
-
-/* Defines used to combine date & time */
-#define RTC_OFFSET_WEEKDAY            24U
-#define RTC_OFFSET_DAY                16U
-#define RTC_OFFSET_MONTH              8U
-#define RTC_OFFSET_HOUR               16U
```

```
-#define RTC_OFFSET_MINUTE             8U
-
-/**
-  * @}
-  */
-
-/* Private macros ------------------------------------------------------------
------*/
-#if defined(USE_FULL_LL_DRIVER)
-/** @defgroup RTC_LL_Private_Macros RTC Private Macros
-  * @{
-  */
-/**
-  * @}
-  */
-#endif /*USE_FULL_LL_DRIVER*/
-
-/* Exported types ------------------------------------------------------------
------*/
-#if defined(USE_FULL_LL_DRIVER)
-/** @defgroup RTC_LL_ES_INIT RTC Exported Init structure
-  * @{
-  */
-
-/**
-  * @brief  RTC Init structures definition
-  */
-typedef struct
-{
-  uint32_t HourFormat;   /*!< Specifies the RTC Hours Format.
-                              This parameter can be a value of @ref
RTC_LL_EC_HOURFORMAT
-
-                              This feature can be modified afterwards
using unitary function
-                              @ref LL_RTC_SetHourFormat(). */
-
-  uint32_t AsynchPrescaler; /*!< Specifies the RTC Asynchronous
Predivider value.
-                              This parameter must be a number between
Min_Data = 0x00 and Max_Data = 0x7F
-
-                              This feature can be modified afterwards
using unitary function
-                              @ref LL_RTC_SetAsynchPrescaler(). */
-
-  uint32_t SynchPrescaler;  /*!< Specifies the RTC Synchronous
Predivider value.
-                              This parameter must be a number between
Min_Data = 0x00 and Max_Data = 0x7FFF
-
-                              This feature can be modified afterwards
using unitary function
-                              @ref LL_RTC_SetSynchPrescaler(). */
-} LL_RTC_InitTypeDef;
-
-/**
-  * @brief  RTC Time structure definition
```

```
-  */
-typedef struct
-{
-  uint32_t TimeFormat; /*!< Specifies the RTC AM/PM Time.
-                            This parameter can be a value of @ref
RTC_LL_EC_TIME_FORMAT
-
-                            This feature can be modified afterwards
using unitary function @ref LL_RTC_TIME_SetFormat(). */
-
-  uint8_t Hours;       /*!< Specifies the RTC Time Hours.
-                            This parameter must be a number between
Min_Data = 0 and Max_Data = 12 if the @ref LL_RTC_TIME_FORMAT_PM is
selected.
-                            This parameter must be a number between
Min_Data = 0 and Max_Data = 23 if the @ref LL_RTC_TIME_FORMAT_AM_OR_24 is
selected.
-
-                            This feature can be modified afterwards
using unitary function @ref LL_RTC_TIME_SetHour(). */
-
-  uint8_t Minutes;     /*!< Specifies the RTC Time Minutes.
-                            This parameter must be a number between
Min_Data = 0 and Max_Data = 59
-
-                            This feature can be modified afterwards
using unitary function @ref LL_RTC_TIME_SetMinute(). */
-
-  uint8_t Seconds;     /*!< Specifies the RTC Time Seconds.
-                            This parameter must be a number between
Min_Data = 0 and Max_Data = 59
-
-                            This feature can be modified afterwards
using unitary function @ref LL_RTC_TIME_SetSecond(). */
-} LL_RTC_TimeTypeDef;
-
-/**
-  * @brief  RTC Date structure definition
-  */
-typedef struct
-{
-  uint8_t WeekDay;  /*!< Specifies the RTC Date WeekDay.
-                         This parameter can be a value of @ref
RTC_LL_EC_WEEKDAY
-
-                         This feature can be modified afterwards using
unitary function @ref LL_RTC_DATE_SetWeekDay(). */
-
-  uint8_t Month;    /*!< Specifies the RTC Date Month.
-                         This parameter can be a value of @ref
RTC_LL_EC_MONTH
-
-                         This feature can be modified afterwards using
unitary function @ref LL_RTC_DATE_SetMonth(). */
-
-  uint8_t Day;      /*!< Specifies the RTC Date Day.
-                         This parameter must be a number between
Min_Data = 1 and Max_Data = 31
```

```
-
-                          This feature can be modified afterwards using
unitary function @ref LL_RTC_DATE_SetDay(). */
-
-  uint8_t Year;     /*!< Specifies the RTC Date Year.
-                          This parameter must be a number between
Min_Data = 0 and Max_Data = 99
-
-                          This feature can be modified afterwards using
unitary function @ref LL_RTC_DATE_SetYear(). */
-} LL_RTC_DateTypeDef;
-
-/**
-  * @brief  RTC Alarm structure definition
-  */
-typedef struct
-{
-  LL_RTC_TimeTypeDef AlarmTime;  /*!< Specifies the RTC Alarm Time
members. */
-
-  uint32_t AlarmMask;            /*!< Specifies the RTC Alarm Masks.
-                                      This parameter can be a value of
@ref RTC_LL_EC_ALMA_MASK for ALARM A or @ref RTC_LL_EC_ALMB_MASK for
ALARM B.
-
-                                      This feature can be modified
afterwards using unitary function @ref LL_RTC_ALMA_SetMask() for ALARM A
-                                      or @ref LL_RTC_ALMB_SetMask() for
ALARM B.
-                                  */
-
-  uint32_t AlarmDateWeekDaySel;  /*!< Specifies the RTC Alarm is on day
or WeekDay.
-                                      This parameter can be a value of
@ref RTC_LL_EC_ALMA_WEEKDAY_SELECTION for ALARM A or @ref
RTC_LL_EC_ALMB_WEEKDAY_SELECTION for ALARM B
-
-                                      This feature can be modified
afterwards using unitary function @ref LL_RTC_ALMA_EnableWeekday() or
@ref LL_RTC_ALMA_DisableWeekday()
-                                      for ALARM A or @ref
LL_RTC_ALMB_EnableWeekday() or @ref LL_RTC_ALMB_DisableWeekday() for
ALARM B
-                                  */
-
-  uint8_t AlarmDateWeekDay;      /*!< Specifies the RTC Alarm
Day/WeekDay.
-                                      If AlarmDateWeekDaySel set to day,
this parameter  must be a number between Min_Data = 1 and Max_Data = 31.
-
-                                      This feature can be modified
afterwards using unitary function @ref LL_RTC_ALMA_SetDay()
-                                      for ALARM A or @ref
LL_RTC_ALMB_SetDay() for ALARM B.
-
-                                      If AlarmDateWeekDaySel set to
Weekday, this parameter can be a value of @ref RTC_LL_EC_WEEKDAY.
-
```

```
-                                    This feature can be modified
afterwards using unitary function @ref LL_RTC_ALMA_SetWeekDay()
-                                    for ALARM A or @ref
LL_RTC_ALMB_SetWeekDay() for ALARM B.
-                                  */
-} LL_RTC_AlarmTypeDef;
-
-/**
-  * @}
-  */
-#endif /* USE_FULL_LL_DRIVER */
-
-/* Exported constants --------------------------------------------------
------*/
-/** @defgroup RTC_LL_Exported_Constants RTC Exported Constants
-  * @{
-  */
-
-#if defined(USE_FULL_LL_DRIVER)
-/** @defgroup RTC_LL_EC_FORMAT FORMAT
-  * @{
-  */
-#define LL_RTC_FORMAT_BIN                  0x00000000U /*!< Binary data
format */
-#define LL_RTC_FORMAT_BCD                  0x00000001U /*!< BCD data
format */
-/**
-  * @}
-  */
-
-/** @defgroup RTC_LL_EC_ALMA_WEEKDAY_SELECTION RTC Alarm A Date WeekDay
-  * @{
-  */
-#define LL_RTC_ALMA_DATEWEEKDAYSEL_DATE    0x00000000U             /*!<
Alarm A Date is selected */
-#define LL_RTC_ALMA_DATEWEEKDAYSEL_WEEKDAY RTC_ALRMAR_WDSEL        /*!<
Alarm A WeekDay is selected */
-/**
-  * @}
-  */
-
-/** @defgroup RTC_LL_EC_ALMB_WEEKDAY_SELECTION RTC Alarm B Date WeekDay
-  * @{
-  */
-#define LL_RTC_ALMB_DATEWEEKDAYSEL_DATE    0x00000000U             /*!<
Alarm B Date is selected */
-#define LL_RTC_ALMB_DATEWEEKDAYSEL_WEEKDAY RTC_ALRMBR_WDSEL        /*!<
Alarm B WeekDay is selected */
-/**
-  * @}
-  */
-
-#endif /* USE_FULL_LL_DRIVER */
-
-/** @defgroup RTC_LL_EC_GET_FLAG Get Flags Defines
-  * @brief    Flags defines which can be used with LL_RTC_ReadReg
function
-  * @{
```

```
-  */
-#define LL_RTC_ISR_RECALPF                 RTC_ISR_RECALPF
-#if defined(RTC_TAMPER2_SUPPORT)
-#define LL_RTC_ISR_TAMP2F                  RTC_ISR_TAMP2F
-#endif /* RTC_TAMPER2_SUPPORT */
-#define LL_RTC_ISR_TAMP1F                  RTC_ISR_TAMP1F
-#define LL_RTC_ISR_TSOVF                   RTC_ISR_TSOVF
-#define LL_RTC_ISR_TSF                     RTC_ISR_TSF
-#define LL_RTC_ISR_WUTF                    RTC_ISR_WUTF
-#define LL_RTC_ISR_ALRBF                   RTC_ISR_ALRBF
-#define LL_RTC_ISR_ALRAF                   RTC_ISR_ALRAF
-#define LL_RTC_ISR_INITF                   RTC_ISR_INITF
-#define LL_RTC_ISR_RSF                     RTC_ISR_RSF
-#define LL_RTC_ISR_INITS                   RTC_ISR_INITS
-#define LL_RTC_ISR_SHPF                    RTC_ISR_SHPF
-#define LL_RTC_ISR_WUTWF                   RTC_ISR_WUTWF
-#define LL_RTC_ISR_ALRBWF                  RTC_ISR_ALRBWF
-#define LL_RTC_ISR_ALRAWF                  RTC_ISR_ALRAWF
-/**
-  * @}
-  */
-
-/** @defgroup RTC_LL_EC_IT IT Defines
-  * @brief    IT defines which can be used with LL_RTC_ReadReg and
LL_RTC_WriteReg functions
-  * @{
-  */
-#define LL_RTC_CR_TSIE                     RTC_CR_TSIE
-#define LL_RTC_CR_WUTIE                    RTC_CR_WUTIE
-#define LL_RTC_CR_ALRBIE                   RTC_CR_ALRBIE
-#define LL_RTC_CR_ALRAIE                   RTC_CR_ALRAIE
-#define LL_RTC_TAFCR_TAMPIE               RTC_TAFCR_TAMPIE
-/**
-  * @}
-  */
-
-/** @defgroup RTC_LL_EC_WEEKDAY  WEEK DAY
-  * @{
-  */
-#define LL_RTC_WEEKDAY_MONDAY              ((uint8_t)0x01U) /*!< Monday
*/
-#define LL_RTC_WEEKDAY_TUESDAY             ((uint8_t)0x02U) /*!< Tuesday
*/
-#define LL_RTC_WEEKDAY_WEDNESDAY           ((uint8_t)0x03U) /*!<
Wednesday */
-#define LL_RTC_WEEKDAY_THURSDAY            ((uint8_t)0x04U) /*!<
Thrusday  */
-#define LL_RTC_WEEKDAY_FRIDAY              ((uint8_t)0x05U) /*!< Friday
*/
-#define LL_RTC_WEEKDAY_SATURDAY            ((uint8_t)0x06U) /*!<
Saturday  */
-#define LL_RTC_WEEKDAY_SUNDAY              ((uint8_t)0x07U) /*!< Sunday
*/
-/**
-  * @}
-  */
-
-/** @defgroup RTC_LL_EC_MONTH  MONTH
```

```
-  * @{
-  */
-#define LL_RTC_MONTH_JANUARY               ((uint8_t)0x01U)  /*!<
January   */
-#define LL_RTC_MONTH_FEBRUARY              ((uint8_t)0x02U)  /*!<
February  */
-#define LL_RTC_MONTH_MARCH                 ((uint8_t)0x03U)  /*!< March
*/
-#define LL_RTC_MONTH_APRIL                 ((uint8_t)0x04U)  /*!< April
*/
-#define LL_RTC_MONTH_MAY                   ((uint8_t)0x05U)  /*!< May
*/
-#define LL_RTC_MONTH_JUNE                  ((uint8_t)0x06U)  /*!< June
*/
-#define LL_RTC_MONTH_JULY                  ((uint8_t)0x07U)  /*!< July
*/
-#define LL_RTC_MONTH_AUGUST                ((uint8_t)0x08U)  /*!< August
*/
-#define LL_RTC_MONTH_SEPTEMBER             ((uint8_t)0x09U)  /*!<
September */
-#define LL_RTC_MONTH_OCTOBER               ((uint8_t)0x10U)  /*!<
October   */
-#define LL_RTC_MONTH_NOVEMBER              ((uint8_t)0x11U)  /*!<
November  */
-#define LL_RTC_MONTH_DECEMBER              ((uint8_t)0x12U)  /*!<
December  */
-/**
-  * @}
-  */
-
-/** @defgroup RTC_LL_EC_HOURFORMAT  HOUR FORMAT
-  * @{
-  */
-#define LL_RTC_HOURFORMAT_24HOUR           0x00000000U           /*!< 24
hour/day format */
-#define LL_RTC_HOURFORMAT_AMPM             RTC_CR_FMT            /*!<
AM/PM hour format */
-/**
-  * @}
-  */
-
-/** @defgroup RTC_LL_EC_ALARMOUT  ALARM OUTPUT
-  * @{
-  */
-#define LL_RTC_ALARMOUT_DISABLE            0x00000000U             /*!<
Output disabled */
-#define LL_RTC_ALARMOUT_ALMA               RTC_CR_OSEL_0           /*!<
Alarm A output enabled */
-#define LL_RTC_ALARMOUT_ALMB               RTC_CR_OSEL_1           /*!<
Alarm B output enabled */
-#define LL_RTC_ALARMOUT_WAKEUP             RTC_CR_OSEL             /*!<
Wakeup output enabled */
-/**
-  * @}
-  */
-
-/** @defgroup RTC_LL_EC_ALARM_OUTPUTTYPE  ALARM OUTPUT TYPE
-  * @{
```

```
-  */
-#define LL_RTC_ALARM_OUTPUTTYPE_OPENDRAIN  0x00000000U
/*!< RTC_ALARM, when mapped on PC13, is open-drain output */
-#define LL_RTC_ALARM_OUTPUTTYPE_PUSHPULL   RTC_TAFCR_ALARMOUTTYPE /*!<
RTC_ALARM, when mapped on PC13, is push-pull output */
-/**
-  * @}
-  */
-
-/** @defgroup RTC_LL_EC_OUTPUTPOLARITY_PIN  OUTPUT POLARITY PIN
-  * @{
-  */
-#define LL_RTC_OUTPUTPOLARITY_PIN_HIGH     0x00000000U           /*!<
Pin is high when ALRAF/ALRBF/WUTF is asserted (depending on OSEL)*/
-#define LL_RTC_OUTPUTPOLARITY_PIN_LOW      RTC_CR_POL            /*!<
Pin is low when ALRAF/ALRBF/WUTF is asserted (depending on OSEL) */
-/**
-  * @}
-  */
-
-/** @defgroup RTC_LL_EC_TIME_FORMAT TIME FORMAT
-  * @{
-  */
-#define LL_RTC_TIME_FORMAT_AM_OR_24        0x00000000U           /*!< AM
or 24-hour format */
-#define LL_RTC_TIME_FORMAT_PM              RTC_TR_PM             /*!< PM
*/
-/**
-  * @}
-  */
-
-/** @defgroup RTC_LL_EC_SHIFT_SECOND  SHIFT SECOND
-  * @{
-  */
-#define LL_RTC_SHIFT_SECOND_DELAY          0x00000000U           /*
Delay (seconds) = SUBFS / (PREDIV_S + 1) */
-#define LL_RTC_SHIFT_SECOND_ADVANCE        RTC_SHIFTR_ADD1S      /*
Advance (seconds) = (1 - (SUBFS / (PREDIV_S + 1))) */
-/**
-  * @}
-  */
-
-/** @defgroup RTC_LL_EC_ALMA_MASK  ALARMA MASK
-  * @{
-  */
-#define LL_RTC_ALMA_MASK_NONE              0x00000000U             /*!<
No masks applied on Alarm A*/
-#define LL_RTC_ALMA_MASK_DATEWEEKDAY       RTC_ALRMAR_MSK4         /*!<
Date/day do not care in Alarm A comparison */
-#define LL_RTC_ALMA_MASK_HOURS             RTC_ALRMAR_MSK3         /*!<
Hours do not care in Alarm A comparison */
-#define LL_RTC_ALMA_MASK_MINUTES           RTC_ALRMAR_MSK2         /*!<
Minutes do not care in Alarm A comparison */
-#define LL_RTC_ALMA_MASK_SECONDS           RTC_ALRMAR_MSK1         /*!<
Seconds do not care in Alarm A comparison */
-#define LL_RTC_ALMA_MASK_ALL               (RTC_ALRMAR_MSK4 |
RTC_ALRMAR_MSK3 | RTC_ALRMAR_MSK2 | RTC_ALRMAR_MSK1) /*!< Masks all */
-/**
```

```
-  * @}
-  */
-
-/** @defgroup RTC_LL_EC_ALMA_TIME_FORMAT  ALARMA TIME FORMAT
-  * @{
-  */
-#define LL_RTC_ALMA_TIME_FORMAT_AM         0x00000000U           /*!< AM
or 24-hour format */
-#define LL_RTC_ALMA_TIME_FORMAT_PM         RTC_ALRMAR_PM         /*!< PM
*/
-/**
-  * @}
-  */
-
-/** @defgroup RTC_LL_EC_ALMB_MASK  ALARMB MASK
-  * @{
-  */
-#define LL_RTC_ALMB_MASK_NONE              0x00000000U             /*!<
No masks applied on Alarm B                */
-#define LL_RTC_ALMB_MASK_DATEWEEKDAY       RTC_ALRMBR_MSK4         /*!<
Date/day do not care in Alarm B comparison */
-#define LL_RTC_ALMB_MASK_HOURS             RTC_ALRMBR_MSK3         /*!<
Hours do not care in Alarm B comparison    */
-#define LL_RTC_ALMB_MASK_MINUTES           RTC_ALRMBR_MSK2         /*!<
Minutes do not care in Alarm B comparison  */
-#define LL_RTC_ALMB_MASK_SECONDS           RTC_ALRMBR_MSK1         /*!<
Seconds do not care in Alarm B comparison  */
-#define LL_RTC_ALMB_MASK_ALL               (RTC_ALRMBR_MSK4 |
RTC_ALRMBR_MSK3 | RTC_ALRMBR_MSK2 | RTC_ALRMBR_MSK1) /*!< Masks all */
-/**
-  * @}
-  */
-
-/** @defgroup RTC_LL_EC_ALMB_TIME_FORMAT  ALARMB TIME FORMAT
-  * @{
-  */
-#define LL_RTC_ALMB_TIME_FORMAT_AM         0x00000000U           /*!< AM
or 24-hour format */
-#define LL_RTC_ALMB_TIME_FORMAT_PM         RTC_ALRMBR_PM         /*!< PM
*/
-/**
-  * @}
-  */
-
-/** @defgroup RTC_LL_EC_TIMESTAMP_EDGE  TIMESTAMP EDGE
-  * @{
-  */
-#define LL_RTC_TIMESTAMP_EDGE_RISING       0x00000000U           /*!<
RTC_TS input rising edge generates a time-stamp event  */
-#define LL_RTC_TIMESTAMP_EDGE_FALLING      RTC_CR_TSEDGE         /*!<
RTC_TS input falling edge generates a time-stamp event */
-/**
-  * @}
-  */
-
-/** @defgroup RTC_LL_EC_TS_TIME_FORMAT  TIMESTAMP TIME FORMAT
-  * @{
-  */
```

```
-#define LL_RTC_TS_TIME_FORMAT_AM          0x00000000U          /*!< AM
or 24-hour format */
-#define LL_RTC_TS_TIME_FORMAT_PM          RTC_TSTR_PM          /*!< PM
*/
-/**
-  * @}
-  */
-
-/** @defgroup RTC_LL_EC_TAMPER  TAMPER
-  * @{
-  */
-#define LL_RTC_TAMPER_1                   RTC_TAFCR_TAMP1E /*!<
RTC_TAMP1 input detection */
-#if defined(RTC_TAMPER2_SUPPORT)
-#define LL_RTC_TAMPER_2                   RTC_TAFCR_TAMP2E /*!<
RTC_TAMP2 input detection */
-#endif /* RTC_TAMPER2_SUPPORT */
-/**
-  * @}
-  */
-
-/** @defgroup RTC_LL_EC_TAMPER_DURATION  TAMPER DURATION
-  * @{
-  */
-#define LL_RTC_TAMPER_DURATION_1RTCCLK    0x00000000U
/*!< Tamper pins are pre-charged before sampling during 1 RTCCLK cycle
*/
-#define LL_RTC_TAMPER_DURATION_2RTCCLK    RTC_TAFCR_TAMPPRCH_0  /*!<
Tamper pins are pre-charged before sampling during 2 RTCCLK cycles */
-#define LL_RTC_TAMPER_DURATION_4RTCCLK    RTC_TAFCR_TAMPPRCH_1  /*!<
Tamper pins are pre-charged before sampling during 4 RTCCLK cycles */
-#define LL_RTC_TAMPER_DURATION_8RTCCLK    RTC_TAFCR_TAMPPRCH    /*!<
Tamper pins are pre-charged before sampling during 8 RTCCLK cycles */
-/**
-  * @}
-  */
-
-/** @defgroup RTC_LL_EC_TAMPER_FILTER  TAMPER FILTER
-  * @{
-  */
-#define LL_RTC_TAMPER_FILTER_DISABLE      0x00000000U
/*!< Tamper filter is disabled */
-#define LL_RTC_TAMPER_FILTER_2SAMPLE      RTC_TAFCR_TAMPFLT_0    /*!<
Tamper is activated after 2 consecutive samples at the active level */
-#define LL_RTC_TAMPER_FILTER_4SAMPLE      RTC_TAFCR_TAMPFLT_1    /*!<
Tamper is activated after 4 consecutive samples at the active level */
-#define LL_RTC_TAMPER_FILTER_8SAMPLE      RTC_TAFCR_TAMPFLT      /*!<
Tamper is activated after 8 consecutive samples at the active level. */
-/**
-  * @}
-  */
-
-/** @defgroup RTC_LL_EC_TAMPER_SAMPLFREQDIV  TAMPER SAMPLING FREQUENCY
DIVIDER
-  * @{
-  */
```

```
-#define LL_RTC_TAMPER_SAMPLFREQDIV_32768   0x00000000U
/*!< Each of the tamper inputs are sampled with a frequency =  RTCCLK /
32768 */
-#define LL_RTC_TAMPER_SAMPLFREQDIV_16384   RTC_TAFCR_TAMPFREQ_0
/*!< Each of the tamper inputs are sampled with a frequency =  RTCCLK /
16384 */
-#define LL_RTC_TAMPER_SAMPLFREQDIV_8192    RTC_TAFCR_TAMPFREQ_1
/*!< Each of the tamper inputs are sampled with a frequency =  RTCCLK /
8192 */
-#define LL_RTC_TAMPER_SAMPLFREQDIV_4096    (RTC_TAFCR_TAMPFREQ_1 |
RTC_TAFCR_TAMPFREQ_0) /*!< Each of the tamper inputs are sampled with a
frequency =  RTCCLK / 4096 */
-#define LL_RTC_TAMPER_SAMPLFREQDIV_2048    RTC_TAFCR_TAMPFREQ_2
/*!< Each of the tamper inputs are sampled with a frequency =  RTCCLK /
2048 */
-#define LL_RTC_TAMPER_SAMPLFREQDIV_1024    (RTC_TAFCR_TAMPFREQ_2 |
RTC_TAFCR_TAMPFREQ_0) /*!< Each of the tamper inputs are sampled with a
frequency =  RTCCLK / 1024 */
-#define LL_RTC_TAMPER_SAMPLFREQDIV_512     (RTC_TAFCR_TAMPFREQ_2 |
RTC_TAFCR_TAMPFREQ_1) /*!< Each of the tamper inputs are sampled with a
frequency =  RTCCLK / 512 */
-#define LL_RTC_TAMPER_SAMPLFREQDIV_256     RTC_TAFCR_TAMPFREQ
/*!< Each of the tamper inputs are sampled with a frequency =  RTCCLK /
256 */
-/**
-  * @}
-  */
-
-/** @defgroup RTC_LL_EC_TAMPER_ACTIVELEVEL  TAMPER ACTIVE LEVEL
-  * @{
-  */
-#define LL_RTC_TAMPER_ACTIVELEVEL_TAMP1    RTC_TAFCR_TAMP1TRG /*!<
RTC_TAMP1 input falling edge (if TAMPFLT = 00) or staying high (if
TAMPFLT != 00) triggers a tamper detection event */
-#if defined(RTC_TAMPER2_SUPPORT)
-#define LL_RTC_TAMPER_ACTIVELEVEL_TAMP2    RTC_TAFCR_TAMP2TRG /*!<
RTC_TAMP2 input falling edge (if TAMPFLT = 00) or staying high (if
TAMPFLT != 00) triggers a tamper detection event */
-#endif /* RTC_TAMPER2_SUPPORT */
-/**
-  * @}
-  */
-
-/** @defgroup RTC_LL_EC_WAKEUPCLOCK_DIV  WAKEUP CLOCK DIV
-  * @{
-  */
-#define LL_RTC_WAKEUPCLOCK_DIV_16          0x00000000U
/*!< RTC/16 clock is selected */
-#define LL_RTC_WAKEUPCLOCK_DIV_8           (RTC_CR_WUCKSEL_0)
/*!< RTC/8 clock is selected */
-#define LL_RTC_WAKEUPCLOCK_DIV_4           (RTC_CR_WUCKSEL_1)
/*!< RTC/4 clock is selected */
-#define LL_RTC_WAKEUPCLOCK_DIV_2           (RTC_CR_WUCKSEL_1 |
RTC_CR_WUCKSEL_0) /*!< RTC/2 clock is selected */
-#define LL_RTC_WAKEUPCLOCK_CKSPRE          (RTC_CR_WUCKSEL_2)
/*!< ck_spre (usually 1 Hz) clock is selected */
```

```
-#define LL_RTC_WAKEUPCLOCK_CKSPRE_WUT      (RTC_CR_WUCKSEL_2 |
RTC_CR_WUCKSEL_1) /*!< ck_spre (usually 1 Hz) clock is selected and
2exp16 is added to the WUT counter value*/
-/**
-  * @}
-  */
-
-/** @defgroup RTC_LL_EC_BKP  BACKUP
-  * @{
-  */
-#define LL_RTC_BKP_DR0                     0x00000000U
-#define LL_RTC_BKP_DR1                     0x00000001U
-#define LL_RTC_BKP_DR2                     0x00000002U
-#define LL_RTC_BKP_DR3                     0x00000003U
-#define LL_RTC_BKP_DR4                     0x00000004U
-#define LL_RTC_BKP_DR5                     0x00000005U
-#define LL_RTC_BKP_DR6                     0x00000006U
-#define LL_RTC_BKP_DR7                     0x00000007U
-#define LL_RTC_BKP_DR8                     0x00000008U
-#define LL_RTC_BKP_DR9                     0x00000009U
-#define LL_RTC_BKP_DR10                    0x0000000AU
-#define LL_RTC_BKP_DR11                    0x0000000BU
-#define LL_RTC_BKP_DR12                    0x0000000CU
-#define LL_RTC_BKP_DR13                    0x0000000DU
-#define LL_RTC_BKP_DR14                    0x0000000EU
-#define LL_RTC_BKP_DR15                    0x0000000FU
-#define LL_RTC_BKP_DR16                    0x00000010U
-#define LL_RTC_BKP_DR17                    0x00000011U
-#define LL_RTC_BKP_DR18                    0x00000012U
-#define LL_RTC_BKP_DR19                    0x00000013U
-/**
-  * @}
-  */
-
-/** @defgroup RTC_LL_EC_CALIB_OUTPUT  Calibration output
-  * @{
-  */
-#define LL_RTC_CALIB_OUTPUT_NONE           0x00000000U
/*!< Calibration output disabled */
-#define LL_RTC_CALIB_OUTPUT_1HZ            (RTC_CR_COE | RTC_CR_COSEL)
/*!< Calibration output is 1 Hz */
-#define LL_RTC_CALIB_OUTPUT_512HZ          (RTC_CR_COE)
/*!< Calibration output is 512 Hz */
-/**
-  * @}
-  */
-
-/** @defgroup RTC_LL_EC_CALIB_SIGN Coarse digital calibration sign
-  * @{
-  */
-#define LL_RTC_CALIB_SIGN_POSITIVE         0x00000000U           /*!<
Positive calibration: calendar update frequency is increased */
-#define LL_RTC_CALIB_SIGN_NEGATIVE         RTC_CALIBR_DCS        /*!<
Negative calibration: calendar update frequency is decreased */
-/**
-  * @}
-  */
-
```

```
-/** @defgroup RTC_LL_EC_CALIB_INSERTPULSE  Calibration pulse insertion
-  * @{
-  */
-#define LL_RTC_CALIB_INSERTPULSE_NONE      0x00000000U           /*!< No
RTCCLK pulses are added */
-#define LL_RTC_CALIB_INSERTPULSE_SET       RTC_CALR_CALP         /*!<
One RTCCLK pulse is effectively inserted every 2exp11 pulses (frequency
increased by 488.5 ppm) */
-/**
-  * @}
-  */
-
-/** @defgroup RTC_LL_EC_CALIB_PERIOD  Calibration period
-  * @{
-  */
-#define LL_RTC_CALIB_PERIOD_32SEC          0x00000000U           /*!<
Use a 32-second calibration cycle period */
-#define LL_RTC_CALIB_PERIOD_16SEC          RTC_CALR_CALW16       /*!<
Use a 16-second calibration cycle period */
-#define LL_RTC_CALIB_PERIOD_8SEC           RTC_CALR_CALW8        /*!<
Use a 8-second calibration cycle period */
-/**
-  * @}
-  */
-
-/** @defgroup RTC_LL_EC_TSINSEL  TIMESTAMP mapping
-  * @{
-  */
-#define LL_RTC_TimeStampPin_Default        0x00000000U           /*!<
Use RTC_AF1 as TIMESTAMP */
-#if defined(RTC_AF2_SUPPORT)
-#define LL_RTC_TimeStampPin_Pos1           RTC_TAFCR_TSINSEL     /*!<
Use RTC_AF2 as TIMESTAMP */
-#endif /* RTC_AF2_SUPPORT */
-/**
-  * @}
-  */
-
-/** @defgroup RTC_LL_EC_TAMP1INSEL  TAMPER1 mapping
-  * @{
-  */
-#define LL_RTC_TamperPin_Default           0x00000000U           /*!<
Use RTC_AF1 as TAMPER1 */
-#if defined(RTC_AF2_SUPPORT)
-#define LL_RTC_TamperPin_Pos1              RTC_TAFCR_TAMP1INSEL  /*!<
Use RTC_AF2 as TAMPER1 */
-#endif /* RTC_AF2_SUPPORT */
-/**
-  * @}
-  */
-
-/**
-  * @}
-  */
-
-/* Exported macro ------------------------------------------------------
------*/
-/** @defgroup RTC_LL_Exported_Macros RTC Exported Macros
```

```
-  * @{
-  */
-
-/** @defgroup RTC_LL_EM_WRITE_READ Common Write and read registers
Macros
-  * @{
-  */
-
-/**
-  * @brief  Write a value in RTC register
-  * @param  __INSTANCE__ RTC Instance
-  * @param  __REG__ Register to be written
-  * @param  __VALUE__ Value to be written in the register
-  * @retval None
-  */
-#define LL_RTC_WriteReg(__INSTANCE__, __REG__, __VALUE__)
WRITE_REG(__INSTANCE__->__REG__, (__VALUE__))
-
-/**
-  * @brief  Read a value in RTC register
-  * @param  __INSTANCE__ RTC Instance
-  * @param  __REG__ Register to be read
-  * @retval Register value
-  */
-#define LL_RTC_ReadReg(__INSTANCE__, __REG__) READ_REG(__INSTANCE__-
>__REG__)
-/**
-  * @}
-  */
-
-/** @defgroup RTC_LL_EM_Convert Convert helper Macros
-  * @{
-  */
-
-/**
-  * @brief  Helper macro to convert a value from 2 digit decimal format
to BCD format
-  * @param  __VALUE__ Byte to be converted
-  * @retval Converted byte
-  */
-#define __LL_RTC_CONVERT_BIN2BCD(__VALUE__) (uint8_t)((((__VALUE__) /
10U) << 4U) | ((__VALUE__) % 10U))
-
-/**
-  * @brief  Helper macro to convert a value from BCD format to 2 digit
decimal format
-  * @param  __VALUE__ BCD value to be converted
-  * @retval Converted byte
-  */
-#define __LL_RTC_CONVERT_BCD2BIN(__VALUE__)
(uint8_t)(((uint8_t)((__VALUE__) & (uint8_t)0xF0U) >> (uint8_t)0x4U) *
10U + ((__VALUE__) & (uint8_t)0x0FU))
-
-/**
-  * @}
-  */
-
-/** @defgroup RTC_LL_EM_Date Date helper Macros
```

```
-  * @{
-  */
-
-/**
-  * @brief  Helper macro to retrieve weekday.
-  * @param  __RTC_DATE__ Date returned by @ref  LL_RTC_DATE_Get
function.
-  * @retval Returned value can be one of the following values:
-  *         @arg @ref LL_RTC_WEEKDAY_MONDAY
-  *         @arg @ref LL_RTC_WEEKDAY_TUESDAY
-  *         @arg @ref LL_RTC_WEEKDAY_WEDNESDAY
-  *         @arg @ref LL_RTC_WEEKDAY_THURSDAY
-  *         @arg @ref LL_RTC_WEEKDAY_FRIDAY
-  *         @arg @ref LL_RTC_WEEKDAY_SATURDAY
-  *         @arg @ref LL_RTC_WEEKDAY_SUNDAY
-  */
-#define __LL_RTC_GET_WEEKDAY(__RTC_DATE__) (((__RTC_DATE__) >>
RTC_OFFSET_WEEKDAY) & 0x000000FFU)
-
-/**
-  * @brief  Helper macro to retrieve Year in BCD format
-  * @param  __RTC_DATE__ Value returned by @ref  LL_RTC_DATE_Get
-  * @retval Year in BCD format (0x00 . . . 0x99)
-  */
-#define __LL_RTC_GET_YEAR(__RTC_DATE__) ((__RTC_DATE__) & 0x000000FFU)
-
-/**
-  * @brief  Helper macro to retrieve Month in BCD format
-  * @param  __RTC_DATE__ Value returned by @ref  LL_RTC_DATE_Get
-  * @retval Returned value can be one of the following values:
-  *         @arg @ref LL_RTC_MONTH_JANUARY
-  *         @arg @ref LL_RTC_MONTH_FEBRUARY
-  *         @arg @ref LL_RTC_MONTH_MARCH
-  *         @arg @ref LL_RTC_MONTH_APRIL
-  *         @arg @ref LL_RTC_MONTH_MAY
-  *         @arg @ref LL_RTC_MONTH_JUNE
-  *         @arg @ref LL_RTC_MONTH_JULY
-  *         @arg @ref LL_RTC_MONTH_AUGUST
-  *         @arg @ref LL_RTC_MONTH_SEPTEMBER
-  *         @arg @ref LL_RTC_MONTH_OCTOBER
-  *         @arg @ref LL_RTC_MONTH_NOVEMBER
-  *         @arg @ref LL_RTC_MONTH_DECEMBER
-  */
-#define __LL_RTC_GET_MONTH(__RTC_DATE__) (((__RTC_DATE__)
>>RTC_OFFSET_MONTH) & 0x000000FFU)
-
-/**
-  * @brief  Helper macro to retrieve Day in BCD format
-  * @param  __RTC_DATE__ Value returned by @ref  LL_RTC_DATE_Get
-  * @retval Day in BCD format (0x01 . . . 0x31)
-  */
-#define __LL_RTC_GET_DAY(__RTC_DATE__) (((__RTC_DATE__)
>>RTC_OFFSET_DAY) & 0x000000FFU)
-
-/**
-  * @}
-  */
-
```

```
-/** @defgroup RTC_LL_EM_Time Time helper Macros
-  * @{
-  */
-
-/**
-  * @brief  Helper macro to retrieve hour in BCD format
-  * @param  __RTC_TIME__ RTC time returned by @ref LL_RTC_TIME_Get
function
-  * @retval Hours in BCD format (0x01. . .0x12 or between Min_Data=0x00
and Max_Data=0x23)
-  */
-#define __LL_RTC_GET_HOUR(__RTC_TIME__) (((__RTC_TIME__) >>
RTC_OFFSET_HOUR) & 0x000000FFU)
-
-/**
-  * @brief  Helper macro to retrieve minute in BCD format
-  * @param  __RTC_TIME__ RTC time returned by @ref LL_RTC_TIME_Get
function
-  * @retval Minutes in BCD format (0x00. . .0x59)
-  */
-#define __LL_RTC_GET_MINUTE(__RTC_TIME__) (((__RTC_TIME__) >>
RTC_OFFSET_MINUTE) & 0x000000FFU)
-
-/**
-  * @brief  Helper macro to retrieve second in BCD format
-  * @param  __RTC_TIME__ RTC time returned by @ref LL_RTC_TIME_Get
function
-  * @retval Seconds in  format (0x00. . .0x59)
-  */
-#define __LL_RTC_GET_SECOND(__RTC_TIME__) ((__RTC_TIME__) & 0x000000FFU)
-
-/**
-  * @}
-  */
-
-/**
-  * @}
-  */
-
-/* Exported functions --------------------------------------------------
------*/
-/** @defgroup RTC_LL_Exported_Functions RTC Exported Functions
-  * @{
-  */
-
-/** @defgroup RTC_LL_EF_Configuration Configuration
-  * @{
-  */
-
-/**
-  * @brief  Set Hours format (24 hour/day or AM/PM hour format)
-  * @note   Bit is write-protected. @ref LL_RTC_DisableWriteProtection
function should be called before.
-  * @note   It can be written in initialization mode only (@ref
LL_RTC_EnableInitMode function)
-  * @rmtoll CR           FMT           LL_RTC_SetHourFormat
-  * @param  RTCx RTC Instance
```

```
-  * @param  HourFormat This parameter can be one of the following
values:
-  *         @arg @ref LL_RTC_HOURFORMAT_24HOUR
-  *         @arg @ref LL_RTC_HOURFORMAT_AMPM
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_SetHourFormat(RTC_TypeDef *RTCx, uint32_t
HourFormat)
-{
-  MODIFY_REG(RTCx->CR, RTC_CR_FMT, HourFormat);
-}
-
-/**
-  * @brief  Get Hours format (24 hour/day or AM/PM hour format)
-  * @rmtoll CR           FMT           LL_RTC_GetHourFormat
-  * @param  RTCx RTC Instance
-  * @retval Returned value can be one of the following values:
-  *         @arg @ref LL_RTC_HOURFORMAT_24HOUR
-  *         @arg @ref LL_RTC_HOURFORMAT_AMPM
-  */
-__STATIC_INLINE uint32_t LL_RTC_GetHourFormat(RTC_TypeDef *RTCx)
-{
-  return (uint32_t)(READ_BIT(RTCx->CR, RTC_CR_FMT));
-}
-
-/**
-  * @brief  Select the flag to be routed to RTC_ALARM output
-  * @note   Bit is write-protected. @ref LL_RTC_DisableWriteProtection
function should be called before.
-  * @rmtoll CR           OSEL          LL_RTC_SetAlarmOutEvent
-  * @param  RTCx RTC Instance
-  * @param  AlarmOutput This parameter can be one of the following
values:
-  *         @arg @ref LL_RTC_ALARMOUT_DISABLE
-  *         @arg @ref LL_RTC_ALARMOUT_ALMA
-  *         @arg @ref LL_RTC_ALARMOUT_ALMB
-  *         @arg @ref LL_RTC_ALARMOUT_WAKEUP
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_SetAlarmOutEvent(RTC_TypeDef *RTCx, uint32_t
AlarmOutput)
-{
-  MODIFY_REG(RTCx->CR, RTC_CR_OSEL, AlarmOutput);
-}
-
-/**
-  * @brief  Get the flag to be routed to RTC_ALARM output
-  * @rmtoll CR           OSEL          LL_RTC_GetAlarmOutEvent
-  * @param  RTCx RTC Instance
-  * @retval Returned value can be one of the following values:
-  *         @arg @ref LL_RTC_ALARMOUT_DISABLE
-  *         @arg @ref LL_RTC_ALARMOUT_ALMA
-  *         @arg @ref LL_RTC_ALARMOUT_ALMB
-  *         @arg @ref LL_RTC_ALARMOUT_WAKEUP
-  */
-__STATIC_INLINE uint32_t LL_RTC_GetAlarmOutEvent(RTC_TypeDef *RTCx)
-{
-  return (uint32_t)(READ_BIT(RTCx->CR, RTC_CR_OSEL));
```

```
-}
-
-/**
-  * @brief  Set RTC_ALARM output type (ALARM in push-pull or open-drain
output)
-  * @note   Used only when RTC_ALARM is mapped on PC13
-  * @rmtoll TAFCR        ALARMOUTTYPE  LL_RTC_SetAlarmOutputType
-  * @param  RTCx RTC Instance
-  * @param  Output This parameter can be one of the following values:
-  *         @arg @ref LL_RTC_ALARM_OUTPUTTYPE_OPENDRAIN
-  *         @arg @ref LL_RTC_ALARM_OUTPUTTYPE_PUSHPULL
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_SetAlarmOutputType(RTC_TypeDef *RTCx,
uint32_t Output)
-{
-  MODIFY_REG(RTCx->TAFCR, RTC_TAFCR_ALARMOUTTYPE, Output);
-}
-
-/**
-  * @brief  Get RTC_ALARM output type (ALARM in push-pull or open-drain
output)
-  * @note   used only when RTC_ALARM is mapped on PC13
-  * @rmtoll TAFCR        ALARMOUTTYPE  LL_RTC_GetAlarmOutputType
-  * @param  RTCx RTC Instance
-  * @retval Returned value can be one of the following values:
-  *         @arg @ref LL_RTC_ALARM_OUTPUTTYPE_OPENDRAIN
-  *         @arg @ref LL_RTC_ALARM_OUTPUTTYPE_PUSHPULL
-  */
-__STATIC_INLINE uint32_t LL_RTC_GetAlarmOutputType(RTC_TypeDef *RTCx)
-{
-  return (uint32_t)(READ_BIT(RTCx->TAFCR, RTC_TAFCR_ALARMOUTTYPE));
-}
-
-/**
-  * @brief  Enable initialization mode
-  * @note   Initialization mode is used to program time and date
register (RTC_TR and RTC_DR)
-  *         and prescaler register (RTC_PRER).
-  *         Counters are stopped and start counting from the new value
when INIT is reset.
-  * @rmtoll ISR          INIT          LL_RTC_EnableInitMode
-  * @param  RTCx RTC Instance
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_EnableInitMode(RTC_TypeDef *RTCx)
-{
-  /* Set the Initialization mode */
-  WRITE_REG(RTCx->ISR, RTC_INIT_MASK);
-}
-
-/**
-  * @brief  Disable initialization mode (Free running mode)
-  * @rmtoll ISR          INIT          LL_RTC_DisableInitMode
-  * @param  RTCx RTC Instance
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_DisableInitMode(RTC_TypeDef *RTCx)
```

```
-{
-  /* Exit Initialization mode */
-  WRITE_REG(RTCx->ISR, (uint32_t)~RTC_ISR_INIT);
-}
-
-/**
-  * @brief  Set Output polarity (pin is low when ALRAF/ALRBF/WUTF is
asserted)
-  * @note   Bit is write-protected. @ref LL_RTC_DisableWriteProtection
function should be called before.
-  * @rmtoll CR           POL           LL_RTC_SetOutputPolarity
-  * @param  RTCx RTC Instance
-  * @param  Polarity This parameter can be one of the following values:
-  *         @arg @ref LL_RTC_OUTPUTPOLARITY_PIN_HIGH
-  *         @arg @ref LL_RTC_OUTPUTPOLARITY_PIN_LOW
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_SetOutputPolarity(RTC_TypeDef *RTCx,
uint32_t Polarity)
-{
-  MODIFY_REG(RTCx->CR, RTC_CR_POL, Polarity);
-}
-
-/**
-  * @brief  Get Output polarity
-  * @rmtoll CR           POL           LL_RTC_GetOutputPolarity
-  * @param  RTCx RTC Instance
-  * @retval Returned value can be one of the following values:
-  *         @arg @ref LL_RTC_OUTPUTPOLARITY_PIN_HIGH
-  *         @arg @ref LL_RTC_OUTPUTPOLARITY_PIN_LOW
-  */
-__STATIC_INLINE uint32_t LL_RTC_GetOutputPolarity(RTC_TypeDef *RTCx)
-{
-  return (uint32_t)(READ_BIT(RTCx->CR, RTC_CR_POL));
-}
-
-/**
-  * @brief  Enable Bypass the shadow registers
-  * @note   Bit is write-protected. @ref LL_RTC_DisableWriteProtection
function should be called before.
-  * @rmtoll CR           BYPSHAD       LL_RTC_EnableShadowRegBypass
-  * @param  RTCx RTC Instance
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_EnableShadowRegBypass(RTC_TypeDef *RTCx)
-{
-  SET_BIT(RTCx->CR, RTC_CR_BYPSHAD);
-}
-
-/**
-  * @brief  Disable Bypass the shadow registers
-  * @rmtoll CR           BYPSHAD       LL_RTC_DisableShadowRegBypass
-  * @param  RTCx RTC Instance
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_DisableShadowRegBypass(RTC_TypeDef *RTCx)
-{
-  CLEAR_BIT(RTCx->CR, RTC_CR_BYPSHAD);
```

```
-}
-
-/**
-  * @brief  Check if Shadow registers bypass is enabled or not.
-  * @rmtoll CR           BYPSHAD       LL_RTC_IsShadowRegBypassEnabled
-  * @param  RTCx RTC Instance
-  * @retval State of bit (1 or 0).
-  */
-__STATIC_INLINE uint32_t LL_RTC_IsShadowRegBypassEnabled(RTC_TypeDef
*RTCx)
-{
-  return ((READ_BIT(RTCx->CR, RTC_CR_BYPSHAD) == (RTC_CR_BYPSHAD)) ? 1UL
: 0UL);
-}
-
-/**
-  * @brief  Enable RTC_REFIN reference clock detection (50 or 60 Hz)
-  * @note   Bit is write-protected. @ref LL_RTC_DisableWriteProtection
function should be called before.
-  * @note   It can be written in initialization mode only (@ref
LL_RTC_EnableInitMode function)
-  * @rmtoll CR           REFCKON       LL_RTC_EnableRefClock
-  * @param  RTCx RTC Instance
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_EnableRefClock(RTC_TypeDef *RTCx)
-{
-  SET_BIT(RTCx->CR, RTC_CR_REFCKON);
-}
-
-/**
-  * @brief  Disable RTC_REFIN reference clock detection (50 or 60 Hz)
-  * @note   Bit is write-protected. @ref LL_RTC_DisableWriteProtection
function should be called before.
-  * @note   It can be written in initialization mode only (@ref
LL_RTC_EnableInitMode function)
-  * @rmtoll CR           REFCKON       LL_RTC_DisableRefClock
-  * @param  RTCx RTC Instance
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_DisableRefClock(RTC_TypeDef *RTCx)
-{
-  CLEAR_BIT(RTCx->CR, RTC_CR_REFCKON);
-}
-
-/**
-  * @brief  Set Asynchronous prescaler factor
-  * @rmtoll PRER         PREDIV_A      LL_RTC_SetAsynchPrescaler
-  * @param  RTCx RTC Instance
-  * @param  AsynchPrescaler Value between Min_Data = 0 and Max_Data =
0x7F
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_SetAsynchPrescaler(RTC_TypeDef *RTCx,
uint32_t AsynchPrescaler)
-{
-  MODIFY_REG(RTCx->PRER, RTC_PRER_PREDIV_A, AsynchPrescaler <<
RTC_PRER_PREDIV_A_Pos);
```

```
-}
-
-/**
-  * @brief  Set Synchronous prescaler factor
-  * @rmtoll PRER          PREDIV_S      LL_RTC_SetSynchPrescaler
-  * @param  RTCx RTC Instance
-  * @param  SynchPrescaler Value between Min_Data = 0 and Max_Data =
0x7FFF
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_SetSynchPrescaler(RTC_TypeDef *RTCx,
uint32_t SynchPrescaler)
-{
-  MODIFY_REG(RTCx->PRER, RTC_PRER_PREDIV_S, SynchPrescaler);
-}
-
-/**
-  * @brief  Get Asynchronous prescaler factor
-  * @rmtoll PRER          PREDIV_A      LL_RTC_GetAsynchPrescaler
-  * @param  RTCx RTC Instance
-  * @retval Value between Min_Data = 0 and Max_Data = 0x7F
-  */
-__STATIC_INLINE uint32_t LL_RTC_GetAsynchPrescaler(RTC_TypeDef *RTCx)
-{
-  return (uint32_t)(READ_BIT(RTCx->PRER, RTC_PRER_PREDIV_A) >>
RTC_PRER_PREDIV_A_Pos);
-}
-
-/**
-  * @brief  Get Synchronous prescaler factor
-  * @rmtoll PRER          PREDIV_S      LL_RTC_GetSynchPrescaler
-  * @param  RTCx RTC Instance
-  * @retval Value between Min_Data = 0 and Max_Data = 0x7FFF
-  */
-__STATIC_INLINE uint32_t LL_RTC_GetSynchPrescaler(RTC_TypeDef *RTCx)
-{
-  return (uint32_t)(READ_BIT(RTCx->PRER, RTC_PRER_PREDIV_S));
-}
-
-/**
-  * @brief  Enable the write protection for RTC registers.
-  * @rmtoll WPR           KEY           LL_RTC_EnableWriteProtection
-  * @param  RTCx RTC Instance
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_EnableWriteProtection(RTC_TypeDef *RTCx)
-{
-  WRITE_REG(RTCx->WPR, RTC_WRITE_PROTECTION_DISABLE);
-}
-
-/**
-  * @brief  Disable the write protection for RTC registers.
-  * @rmtoll WPR           KEY           LL_RTC_DisableWriteProtection
-  * @param  RTCx RTC Instance
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_DisableWriteProtection(RTC_TypeDef *RTCx)
-{
```

```
-  WRITE_REG(RTCx->WPR, RTC_WRITE_PROTECTION_ENABLE_1);
-  WRITE_REG(RTCx->WPR, RTC_WRITE_PROTECTION_ENABLE_2);
-}
-
-/**
-  * @}
-  */
-
-/** @defgroup RTC_LL_EF_Time Time
-  * @{
-  */
-
-/**
-  * @brief  Set time format (AM/24-hour or PM notation)
-  * @note   Bit is write-protected. @ref LL_RTC_DisableWriteProtection
function should be called before.
-  * @note   It can be written in initialization mode only (@ref
LL_RTC_EnableInitMode function)
-  * @rmtoll TR           PM            LL_RTC_TIME_SetFormat
-  * @param  RTCx RTC Instance
-  * @param  TimeFormat This parameter can be one of the following
values:
-  *         @arg @ref LL_RTC_TIME_FORMAT_AM_OR_24
-  *         @arg @ref LL_RTC_TIME_FORMAT_PM
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_TIME_SetFormat(RTC_TypeDef *RTCx, uint32_t
TimeFormat)
-{
-  MODIFY_REG(RTCx->TR, RTC_TR_PM, TimeFormat);
-}
-
-/**
-  * @brief  Get time format (AM or PM notation)
-  * @note if RTC shadow registers are not bypassed (BYPSHAD=0), need to
check if RSF flag is set
-  *       before reading this bit
-  * @note Read either RTC_SSR or RTC_TR locks the values in the higher-
order calendar
-  *       shadow registers until RTC_DR is read (LL_RTC_ReadReg(RTC,
DR)).
-  * @rmtoll TR           PM            LL_RTC_TIME_GetFormat
-  * @param  RTCx RTC Instance
-  * @retval Returned value can be one of the following values:
-  *         @arg @ref LL_RTC_TIME_FORMAT_AM_OR_24
-  *         @arg @ref LL_RTC_TIME_FORMAT_PM
-  */
-__STATIC_INLINE uint32_t LL_RTC_TIME_GetFormat(RTC_TypeDef *RTCx)
-{
-  return (uint32_t)(READ_BIT(RTCx->TR, RTC_TR_PM));
-}
-
-/**
-  * @brief  Set Hours in BCD format
-  * @note   Bit is write-protected. @ref LL_RTC_DisableWriteProtection
function should be called before.
-  * @note   It can be written in initialization mode only (@ref
LL_RTC_EnableInitMode function)
```

```
-  * @note helper macro __LL_RTC_CONVERT_BIN2BCD is available to convert
hour from binary to BCD format
-  * @rmtoll TR          HT            LL_RTC_TIME_SetHour\n
-  *         TR          HU            LL_RTC_TIME_SetHour
-  * @param  RTCx RTC Instance
-  * @param  Hours Value between Min_Data=0x01 and Max_Data=0x12 or
between Min_Data=0x00 and Max_Data=0x23
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_TIME_SetHour(RTC_TypeDef *RTCx, uint32_t
Hours)
-{
-  MODIFY_REG(RTCx->TR, (RTC_TR_HT | RTC_TR_HU),
-             (((Hours & 0xF0U) << (RTC_TR_HT_Pos - 4U)) | ((Hours &
0x0FU) << RTC_TR_HU_Pos)));
-}
-
-/**
-  * @brief  Get Hours in BCD format
-  * @note if RTC shadow registers are not bypassed (BYPSHAD=0), need to
check if RSF flag is set
-  *       before reading this bit
-  * @note Read either RTC_SSR or RTC_TR locks the values in the higher-
order calendar
-  *       shadow registers until RTC_DR is read (LL_RTC_ReadReg(RTC,
DR)).
-  * @note helper macro __LL_RTC_CONVERT_BCD2BIN is available to convert
hour from BCD to
-  *       Binary format
-  * @rmtoll TR          HT            LL_RTC_TIME_GetHour\n
-  *         TR          HU            LL_RTC_TIME_GetHour
-  * @param  RTCx RTC Instance
-  * @retval Value between Min_Data=0x01 and Max_Data=0x12 or between
Min_Data=0x00 and Max_Data=0x23
-  */
-__STATIC_INLINE uint32_t LL_RTC_TIME_GetHour(RTC_TypeDef *RTCx)
-{
-  return (uint32_t)((READ_BIT(RTCx->TR, (RTC_TR_HT | RTC_TR_HU))) >>
RTC_TR_HU_Pos);
-}
-
-/**
-  * @brief  Set Minutes in BCD format
-  * @note   Bit is write-protected. @ref LL_RTC_DisableWriteProtection
function should be called before.
-  * @note   It can be written in initialization mode only (@ref
LL_RTC_EnableInitMode function)
-  * @note helper macro __LL_RTC_CONVERT_BIN2BCD is available to convert
Minutes from binary to BCD format
-  * @rmtoll TR          MNT           LL_RTC_TIME_SetMinute\n
-  *         TR          MNU           LL_RTC_TIME_SetMinute
-  * @param  RTCx RTC Instance
-  * @param  Minutes Value between Min_Data=0x00 and Max_Data=0x59
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_TIME_SetMinute(RTC_TypeDef *RTCx, uint32_t
Minutes)
-{
```

```
-  MODIFY_REG(RTCx->TR, (RTC_TR_MNT | RTC_TR_MNU),
-             (((Minutes & 0xF0U) << (RTC_TR_MNT_Pos - 4U)) | ((Minutes &
0x0FU) << RTC_TR_MNU_Pos)));
-}
-
-/**
-  * @brief  Get Minutes in BCD format
-  * @note if RTC shadow registers are not bypassed (BYPSHAD=0), need to
check if RSF flag is set
-  *       before reading this bit
-  * @note Read either RTC_SSR or RTC_TR locks the values in the higher-
order calendar
-  *       shadow registers until RTC_DR is read (LL_RTC_ReadReg(RTC,
DR)).
-  * @note helper macro __LL_RTC_CONVERT_BCD2BIN is available to convert
minute from BCD
-  *       to Binary format
-  * @rmtoll TR           MNT           LL_RTC_TIME_GetMinute\n
-  *         TR           MNU           LL_RTC_TIME_GetMinute
-  * @param  RTCx RTC Instance
-  * @retval Value between Min_Data=0x00 and Max_Data=0x59
-  */
-__STATIC_INLINE uint32_t LL_RTC_TIME_GetMinute(RTC_TypeDef *RTCx)
-{
-  return (uint32_t)(READ_BIT(RTCx->TR, (RTC_TR_MNT | RTC_TR_MNU)) >>
RTC_TR_MNU_Pos);
-}
-
-/**
-  * @brief  Set Seconds in BCD format
-  * @note   Bit is write-protected. @ref LL_RTC_DisableWriteProtection
function should be called before.
-  * @note   It can be written in initialization mode only (@ref
LL_RTC_EnableInitMode function)
-  * @note helper macro __LL_RTC_CONVERT_BIN2BCD is available to convert
Seconds from binary to BCD format
-  * @rmtoll TR           ST            LL_RTC_TIME_SetSecond\n
-  *         TR           SU            LL_RTC_TIME_SetSecond
-  * @param  RTCx RTC Instance
-  * @param  Seconds Value between Min_Data=0x00 and Max_Data=0x59
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_TIME_SetSecond(RTC_TypeDef *RTCx, uint32_t
Seconds)
-{
-  MODIFY_REG(RTCx->TR, (RTC_TR_ST | RTC_TR_SU),
-             (((Seconds & 0xF0U) << (RTC_TR_ST_Pos - 4U)) | ((Seconds &
0x0FU) << RTC_TR_SU_Pos)));
-}
-
-/**
-  * @brief  Get Seconds in BCD format
-  * @note if RTC shadow registers are not bypassed (BYPSHAD=0), need to
check if RSF flag is set
-  *       before reading this bit
-  * @note Read either RTC_SSR or RTC_TR locks the values in the higher-
order calendar
```

```
-  *       shadow registers until RTC_DR is read (LL_RTC_ReadReg(RTC,
DR)).
-  * @note helper macro __LL_RTC_CONVERT_BCD2BIN is available to convert
Seconds from BCD
-  *       to Binary format
-  * @rmtoll TR           ST            LL_RTC_TIME_GetSecond\n
-  *         TR           SU            LL_RTC_TIME_GetSecond
-  * @param  RTCx RTC Instance
-  * @retval Value between Min_Data=0x00 and Max_Data=0x59
-  */
-__STATIC_INLINE uint32_t LL_RTC_TIME_GetSecond(RTC_TypeDef *RTCx)
-{
-  return (uint32_t)(READ_BIT(RTCx->TR, (RTC_TR_ST | RTC_TR_SU)) >>
RTC_TR_SU_Pos);
-}
-
-/**
-  * @brief  Set time (hour, minute and second) in BCD format
-  * @note   Bit is write-protected. @ref LL_RTC_DisableWriteProtection
function should be called before.
-  * @note   It can be written in initialization mode only (@ref
LL_RTC_EnableInitMode function)
-  * @note TimeFormat and Hours should follow the same format
-  * @rmtoll TR           PM            LL_RTC_TIME_Config\n
-  *         TR           HT            LL_RTC_TIME_Config\n
-  *         TR           HU            LL_RTC_TIME_Config\n
-  *         TR           MNT           LL_RTC_TIME_Config\n
-  *         TR           MNU           LL_RTC_TIME_Config\n
-  *         TR           ST            LL_RTC_TIME_Config\n
-  *         TR           SU            LL_RTC_TIME_Config
-  * @param  RTCx RTC Instance
-  * @param  Format12_24 This parameter can be one of the following
values:
-  *         @arg @ref LL_RTC_TIME_FORMAT_AM_OR_24
-  *         @arg @ref LL_RTC_TIME_FORMAT_PM
-  * @param  Hours Value between Min_Data=0x01 and Max_Data=0x12 or
between Min_Data=0x00 and Max_Data=0x23
-  * @param  Minutes Value between Min_Data=0x00 and Max_Data=0x59
-  * @param  Seconds Value between Min_Data=0x00 and Max_Data=0x59
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_TIME_Config(RTC_TypeDef *RTCx, uint32_t
Format12_24, uint32_t Hours, uint32_t Minutes, uint32_t Seconds)
-{
-  uint32_t temp;
-
-  temp = Format12_24
| \
-         (((Hours   & 0xF0U) << (RTC_TR_HT_Pos  - 4U)) | ((Hours &
0x0FU) << RTC_TR_HU_Pos))    | \
-         (((Minutes & 0xF0U) << (RTC_TR_MNT_Pos - 4U)) | ((Minutes &
0x0FU) << RTC_TR_MNU_Pos)) | \
-         (((Seconds & 0xF0U) << (RTC_TR_ST_Pos  - 4U)) | ((Seconds &
0x0FU) << RTC_TR_SU_Pos));
-  MODIFY_REG(RTCx->TR, (RTC_TR_PM | RTC_TR_HT | RTC_TR_HU | RTC_TR_MNT |
RTC_TR_MNU | RTC_TR_ST | RTC_TR_SU), temp);
-}
-
```

```
-/**
-  * @brief  Get time (hour, minute and second) in BCD format
-  * @note if RTC shadow registers are not bypassed (BYPSHAD=0), need to
check if RSF flag is set
-  *       before reading this bit
-  * @note Read either RTC_SSR or RTC_TR locks the values in the higher-
order calendar
-  *       shadow registers until RTC_DR is read (LL_RTC_ReadReg(RTC,
DR)).
-  * @note helper macros __LL_RTC_GET_HOUR, __LL_RTC_GET_MINUTE and
__LL_RTC_GET_SECOND
-  *       are available to get independently each parameter.
-  * @rmtoll TR           HT            LL_RTC_TIME_Get\n
-  *         TR           HU            LL_RTC_TIME_Get\n
-  *         TR           MNT           LL_RTC_TIME_Get\n
-  *         TR           MNU           LL_RTC_TIME_Get\n
-  *         TR           ST            LL_RTC_TIME_Get\n
-  *         TR           SU            LL_RTC_TIME_Get
-  * @param  RTCx RTC Instance
-  * @retval Combination of hours, minutes and seconds (Format:
0x00HHMMSS).
-  */
-__STATIC_INLINE uint32_t LL_RTC_TIME_Get(RTC_TypeDef *RTCx)
-{
-  return (uint32_t)(READ_BIT(RTCx->TR, (RTC_TR_HT | RTC_TR_HU |
RTC_TR_MNT | RTC_TR_MNU | RTC_TR_ST | RTC_TR_SU)));
-}
-
-/**
-  * @brief  Memorize whether the daylight saving time change has been
performed
-  * @note   Bit is write-protected. @ref LL_RTC_DisableWriteProtection
function should be called before.
-  * @rmtoll CR           BKP           LL_RTC_TIME_EnableDayLightStore
-  * @param  RTCx RTC Instance
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_TIME_EnableDayLightStore(RTC_TypeDef *RTCx)
-{
-  SET_BIT(RTCx->CR, RTC_CR_BKP);
-}
-
-/**
-  * @brief  Disable memorization whether the daylight saving time change
has been performed.
-  * @note   Bit is write-protected. @ref LL_RTC_DisableWriteProtection
function should be called before.
-  * @rmtoll CR           BKP           LL_RTC_TIME_DisableDayLightStore
-  * @param  RTCx RTC Instance
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_TIME_DisableDayLightStore(RTC_TypeDef *RTCx)
-{
-  CLEAR_BIT(RTCx->CR, RTC_CR_BKP);
-}
-
-/**
```

```
-  * @brief  Check if RTC Day Light Saving stored operation has been
enabled or not
-  * @rmtoll CR           BKP
LL_RTC_TIME_IsDayLightStoreEnabled
-  * @param  RTCx RTC Instance
-  * @retval State of bit (1 or 0).
-  */
-__STATIC_INLINE uint32_t LL_RTC_TIME_IsDayLightStoreEnabled(RTC_TypeDef
*RTCx)
-{
-  return ((READ_BIT(RTCx->CR, RTC_CR_BKP) == (RTC_CR_BKP)) ? 1UL : 0UL);
-}
-
-/**
-  * @brief  Subtract 1 hour (winter time change)
-  * @note   Bit is write-protected. @ref LL_RTC_DisableWriteProtection
function should be called before.
-  * @rmtoll CR           SUB1H         LL_RTC_TIME_DecHour
-  * @param  RTCx RTC Instance
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_TIME_DecHour(RTC_TypeDef *RTCx)
-{
-  SET_BIT(RTCx->CR, RTC_CR_SUB1H);
-}
-
-/**
-  * @brief  Add 1 hour (summer time change)
-  * @note   Bit is write-protected. @ref LL_RTC_DisableWriteProtection
function should be called before.
-  * @rmtoll CR           ADD1H         LL_RTC_TIME_IncHour
-  * @param  RTCx RTC Instance
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_TIME_IncHour(RTC_TypeDef *RTCx)
-{
-  SET_BIT(RTCx->CR, RTC_CR_ADD1H);
-}
-
-/**
-  * @brief  Get subseconds value in the synchronous prescaler counter.
-  * @note  You can use both SubSeconds value and SecondFraction
(PREDIV_S through
-  *        LL_RTC_GetSynchPrescaler function) terms returned to convert
Calendar
-  *        SubSeconds value in second fraction ratio with time unit
following
-  *        generic formula:
-  *          ==> Seconds fraction ratio * time_unit =
-  *                 [(SecondFraction-SubSeconds)/(SecondFraction+1)] *
time_unit
-  *        This conversion can be performed only if no shift operation
is pending
-  *        (ie. SHFP=0) when PREDIV_S >= SS.
-  * @rmtoll SSR          SS            LL_RTC_TIME_GetSubSecond
-  * @param  RTCx RTC Instance
-  * @retval Subseconds value (number between 0 and 65535)
-  */
```

```
-__STATIC_INLINE uint32_t LL_RTC_TIME_GetSubSecond(RTC_TypeDef *RTCx)
-{
-  return (uint32_t)(READ_BIT(RTCx->SSR, RTC_SSR_SS));
-}
-
-/**
-  * @brief  Synchronize to a remote clock with a high degree of
precision.
-  * @note   This operation effectively subtracts from (delays) or
advance the clock of a fraction of a second.
-  * @note   Bit is write-protected. @ref LL_RTC_DisableWriteProtection
function should be called before.
-  * @note   When REFCKON is set, firmware must not write to Shift
control register.
-  * @rmtoll SHIFTR       ADD1S         LL_RTC_TIME_Synchronize\n
-  *         SHIFTR       SUBFS         LL_RTC_TIME_Synchronize
-  * @param  RTCx RTC Instance
-  * @param  ShiftSecond This parameter can be one of the following
values:
-  *         @arg @ref LL_RTC_SHIFT_SECOND_DELAY
-  *         @arg @ref LL_RTC_SHIFT_SECOND_ADVANCE
-  * @param  Fraction Number of Seconds Fractions (any value from 0 to
0x7FFF)
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_TIME_Synchronize(RTC_TypeDef *RTCx, uint32_t
ShiftSecond, uint32_t Fraction)
-{
-  WRITE_REG(RTCx->SHIFTR, ShiftSecond | Fraction);
-}
-
-/**
-  * @}
-  */
-
-/** @defgroup RTC_LL_EF_Date Date
-  * @{
-  */
-
-/**
-  * @brief  Set Year in BCD format
-  * @note helper macro __LL_RTC_CONVERT_BIN2BCD is available to convert
Year from binary to BCD format
-  * @rmtoll DR           YT            LL_RTC_DATE_SetYear\n
-  *         DR           YU            LL_RTC_DATE_SetYear
-  * @param  RTCx RTC Instance
-  * @param  Year Value between Min_Data=0x00 and Max_Data=0x99
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_DATE_SetYear(RTC_TypeDef *RTCx, uint32_t
Year)
-{
-  MODIFY_REG(RTCx->DR, (RTC_DR_YT | RTC_DR_YU),
-             (((Year & 0xF0U) << (RTC_DR_YT_Pos - 4U)) | ((Year & 0x0FU)
<< RTC_DR_YU_Pos)));
-}
-
-/**
```

```
-  * @brief  Get Year in BCD format
-  * @note if RTC shadow registers are not bypassed (BYPSHAD=0), need to
check if RSF flag is set
-  *       before reading this bit
-  * @note helper macro __LL_RTC_CONVERT_BCD2BIN is available to convert
Year from BCD to Binary format
-  * @rmtoll DR           YT            LL_RTC_DATE_GetYear\n
-  *         DR           YU            LL_RTC_DATE_GetYear
-  * @param  RTCx RTC Instance
-  * @retval Value between Min_Data=0x00 and Max_Data=0x99
-  */
-__STATIC_INLINE uint32_t LL_RTC_DATE_GetYear(RTC_TypeDef *RTCx)
-{
-  return (uint32_t)((READ_BIT(RTCx->DR, (RTC_DR_YT | RTC_DR_YU))) >>
RTC_DR_YU_Pos);
-}
-
-/**
-  * @brief  Set Week day
-  * @rmtoll DR           WDU           LL_RTC_DATE_SetWeekDay
-  * @param  RTCx RTC Instance
-  * @param  WeekDay This parameter can be one of the following values:
-  *         @arg @ref LL_RTC_WEEKDAY_MONDAY
-  *         @arg @ref LL_RTC_WEEKDAY_TUESDAY
-  *         @arg @ref LL_RTC_WEEKDAY_WEDNESDAY
-  *         @arg @ref LL_RTC_WEEKDAY_THURSDAY
-  *         @arg @ref LL_RTC_WEEKDAY_FRIDAY
-  *         @arg @ref LL_RTC_WEEKDAY_SATURDAY
-  *         @arg @ref LL_RTC_WEEKDAY_SUNDAY
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_DATE_SetWeekDay(RTC_TypeDef *RTCx, uint32_t
WeekDay)
-{
-  MODIFY_REG(RTCx->DR, RTC_DR_WDU, WeekDay << RTC_DR_WDU_Pos);
-}
-
-/**
-  * @brief  Get Week day
-  * @note if RTC shadow registers are not bypassed (BYPSHAD=0), need to
check if RSF flag is set
-  *       before reading this bit
-  * @rmtoll DR           WDU           LL_RTC_DATE_GetWeekDay
-  * @param  RTCx RTC Instance
-  * @retval Returned value can be one of the following values:
-  *         @arg @ref LL_RTC_WEEKDAY_MONDAY
-  *         @arg @ref LL_RTC_WEEKDAY_TUESDAY
-  *         @arg @ref LL_RTC_WEEKDAY_WEDNESDAY
-  *         @arg @ref LL_RTC_WEEKDAY_THURSDAY
-  *         @arg @ref LL_RTC_WEEKDAY_FRIDAY
-  *         @arg @ref LL_RTC_WEEKDAY_SATURDAY
-  *         @arg @ref LL_RTC_WEEKDAY_SUNDAY
-  */
-__STATIC_INLINE uint32_t LL_RTC_DATE_GetWeekDay(RTC_TypeDef *RTCx)
-{
-  return (uint32_t)(READ_BIT(RTCx->DR, RTC_DR_WDU) >> RTC_DR_WDU_Pos);
-}
-
```

```
-/**
-  * @brief  Set Month in BCD format
-  * @note helper macro __LL_RTC_CONVERT_BIN2BCD is available to convert
Month from binary to BCD format
-  * @rmtoll DR          MT            LL_RTC_DATE_SetMonth\n
-  *         DR          MU            LL_RTC_DATE_SetMonth
-  * @param  RTCx RTC Instance
-  * @param  Month This parameter can be one of the following values:
-  *         @arg @ref LL_RTC_MONTH_JANUARY
-  *         @arg @ref LL_RTC_MONTH_FEBRUARY
-  *         @arg @ref LL_RTC_MONTH_MARCH
-  *         @arg @ref LL_RTC_MONTH_APRIL
-  *         @arg @ref LL_RTC_MONTH_MAY
-  *         @arg @ref LL_RTC_MONTH_JUNE
-  *         @arg @ref LL_RTC_MONTH_JULY
-  *         @arg @ref LL_RTC_MONTH_AUGUST
-  *         @arg @ref LL_RTC_MONTH_SEPTEMBER
-  *         @arg @ref LL_RTC_MONTH_OCTOBER
-  *         @arg @ref LL_RTC_MONTH_NOVEMBER
-  *         @arg @ref LL_RTC_MONTH_DECEMBER
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_DATE_SetMonth(RTC_TypeDef *RTCx, uint32_t
Month)
-{
-  MODIFY_REG(RTCx->DR, (RTC_DR_MT | RTC_DR_MU),
-             (((Month & 0xF0U) << (RTC_DR_MT_Pos - 4U)) | ((Month &
0x0FU) << RTC_DR_MU_Pos)));
-}
-
-/**
-  * @brief  Get Month in BCD format
-  * @note if RTC shadow registers are not bypassed (BYPSHAD=0), need to
check if RSF flag is set
-  *       before reading this bit
-  * @note helper macro __LL_RTC_CONVERT_BCD2BIN is available to convert
Month from BCD to Binary format
-  * @rmtoll DR          MT            LL_RTC_DATE_GetMonth\n
-  *         DR          MU            LL_RTC_DATE_GetMonth
-  * @param  RTCx RTC Instance
-  * @retval Returned value can be one of the following values:
-  *         @arg @ref LL_RTC_MONTH_JANUARY
-  *         @arg @ref LL_RTC_MONTH_FEBRUARY
-  *         @arg @ref LL_RTC_MONTH_MARCH
-  *         @arg @ref LL_RTC_MONTH_APRIL
-  *         @arg @ref LL_RTC_MONTH_MAY
-  *         @arg @ref LL_RTC_MONTH_JUNE
-  *         @arg @ref LL_RTC_MONTH_JULY
-  *         @arg @ref LL_RTC_MONTH_AUGUST
-  *         @arg @ref LL_RTC_MONTH_SEPTEMBER
-  *         @arg @ref LL_RTC_MONTH_OCTOBER
-  *         @arg @ref LL_RTC_MONTH_NOVEMBER
-  *         @arg @ref LL_RTC_MONTH_DECEMBER
-  */
-__STATIC_INLINE uint32_t LL_RTC_DATE_GetMonth(RTC_TypeDef *RTCx)
-{
-  return (uint32_t)((READ_BIT(RTCx->DR, (RTC_DR_MT | RTC_DR_MU))) >>
RTC_DR_MU_Pos);
```

```
-}
-
-/**
-  * @brief  Set Day in BCD format
-  * @note helper macro __LL_RTC_CONVERT_BIN2BCD is available to convert
Day from binary to BCD format
-  * @rmtoll DR           DT            LL_RTC_DATE_SetDay\n
-  *         DR           DU            LL_RTC_DATE_SetDay
-  * @param  RTCx RTC Instance
-  * @param  Day Value between Min_Data=0x01 and Max_Data=0x31
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_DATE_SetDay(RTC_TypeDef *RTCx, uint32_t Day)
-{
-  MODIFY_REG(RTCx->DR, (RTC_DR_DT | RTC_DR_DU),
-             (((Day & 0xF0U) << (RTC_DR_DT_Pos - 4U)) | ((Day & 0x0FU)
<< RTC_DR_DU_Pos)));
-}
-
-/**
-  * @brief  Get Day in BCD format
-  * @note if RTC shadow registers are not bypassed (BYPSHAD=0), need to
check if RSF flag is set
-  *       before reading this bit
-  * @note helper macro __LL_RTC_CONVERT_BCD2BIN is available to convert
Day from BCD to Binary format
-  * @rmtoll DR           DT            LL_RTC_DATE_GetDay\n
-  *         DR           DU            LL_RTC_DATE_GetDay
-  * @param  RTCx RTC Instance
-  * @retval Value between Min_Data=0x01 and Max_Data=0x31
-  */
-__STATIC_INLINE uint32_t LL_RTC_DATE_GetDay(RTC_TypeDef *RTCx)
-{
-  return (uint32_t)((READ_BIT(RTCx->DR, (RTC_DR_DT | RTC_DR_DU))) >>
RTC_DR_DU_Pos);
-}
-
-/**
-  * @brief  Set date (WeekDay, Day, Month and Year) in BCD format
-  * @rmtoll DR           WDU           LL_RTC_DATE_Config\n
-  *         DR           MT            LL_RTC_DATE_Config\n
-  *         DR           MU            LL_RTC_DATE_Config\n
-  *         DR           DT            LL_RTC_DATE_Config\n
-  *         DR           DU            LL_RTC_DATE_Config\n
-  *         DR           YT            LL_RTC_DATE_Config\n
-  *         DR           YU            LL_RTC_DATE_Config
-  * @param  RTCx RTC Instance
-  * @param  WeekDay This parameter can be one of the following values:
-  *         @arg @ref LL_RTC_WEEKDAY_MONDAY
-  *         @arg @ref LL_RTC_WEEKDAY_TUESDAY
-  *         @arg @ref LL_RTC_WEEKDAY_WEDNESDAY
-  *         @arg @ref LL_RTC_WEEKDAY_THURSDAY
-  *         @arg @ref LL_RTC_WEEKDAY_FRIDAY
-  *         @arg @ref LL_RTC_WEEKDAY_SATURDAY
-  *         @arg @ref LL_RTC_WEEKDAY_SUNDAY
-  * @param  Day Value between Min_Data=0x01 and Max_Data=0x31
-  * @param  Month This parameter can be one of the following values:
-  *         @arg @ref LL_RTC_MONTH_JANUARY
```

```
-  *         @arg @ref LL_RTC_MONTH_FEBRUARY
-  *         @arg @ref LL_RTC_MONTH_MARCH
-  *         @arg @ref LL_RTC_MONTH_APRIL
-  *         @arg @ref LL_RTC_MONTH_MAY
-  *         @arg @ref LL_RTC_MONTH_JUNE
-  *         @arg @ref LL_RTC_MONTH_JULY
-  *         @arg @ref LL_RTC_MONTH_AUGUST
-  *         @arg @ref LL_RTC_MONTH_SEPTEMBER
-  *         @arg @ref LL_RTC_MONTH_OCTOBER
-  *         @arg @ref LL_RTC_MONTH_NOVEMBER
-  *         @arg @ref LL_RTC_MONTH_DECEMBER
-  * @param  Year Value between Min_Data=0x00 and Max_Data=0x99
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_DATE_Config(RTC_TypeDef *RTCx, uint32_t
WeekDay, uint32_t Day, uint32_t Month, uint32_t Year)
-{
-  uint32_t temp;
-
-  temp = (  WeekDay
<< RTC_DR_WDU_Pos) | \
-          (((Year  & 0xF0U) << (RTC_DR_YT_Pos - 4U)) | ((Year  & 0x0FU)
<< RTC_DR_YU_Pos)) | \
-          (((Month & 0xF0U) << (RTC_DR_MT_Pos - 4U)) | ((Month & 0x0FU)
<< RTC_DR_MU_Pos)) | \
-          (((Day   & 0xF0U) << (RTC_DR_DT_Pos - 4U)) | ((Day   & 0x0FU)
<< RTC_DR_DU_Pos));
-
-  MODIFY_REG(RTCx->DR, (RTC_DR_WDU | RTC_DR_MT | RTC_DR_MU | RTC_DR_DT |
RTC_DR_DU | RTC_DR_YT | RTC_DR_YU), temp);
-}
-
-/**
-  * @brief  Get date (WeekDay, Day, Month and Year) in BCD format
-  * @note if RTC shadow registers are not bypassed (BYPSHAD=0), need to
check if RSF flag is set
-  *       before reading this bit
-  * @note helper macros __LL_RTC_GET_WEEKDAY, __LL_RTC_GET_YEAR,
__LL_RTC_GET_MONTH,
-  * and __LL_RTC_GET_DAY are available to get independently each
parameter.
-  * @rmtoll DR           WDU           LL_RTC_DATE_Get\n
-  *         DR           MT            LL_RTC_DATE_Get\n
-  *         DR           MU            LL_RTC_DATE_Get\n
-  *         DR           DT            LL_RTC_DATE_Get\n
-  *         DR           DU            LL_RTC_DATE_Get\n
-  *         DR           YT            LL_RTC_DATE_Get\n
-  *         DR           YU            LL_RTC_DATE_Get
-  * @param  RTCx RTC Instance
-  * @retval Combination of WeekDay, Day, Month and Year (Format:
0xWWDDMMYY).
-  */
-__STATIC_INLINE uint32_t LL_RTC_DATE_Get(RTC_TypeDef *RTCx)
-{
-  uint32_t temp;
-
-  temp = READ_BIT(RTCx->DR, (RTC_DR_WDU | RTC_DR_MT | RTC_DR_MU |
RTC_DR_DT | RTC_DR_DU | RTC_DR_YT | RTC_DR_YU));
```

```
-
-  return (uint32_t)((((temp &              RTC_DR_WDU) >>
RTC_DR_WDU_Pos) << RTC_OFFSET_WEEKDAY) | \
-                    (((temp & (RTC_DR_DT | RTC_DR_DU)) >> RTC_DR_DU_Pos)
<< RTC_OFFSET_DAY)     | \
-                    (((temp & (RTC_DR_MT | RTC_DR_MU)) >> RTC_DR_MU_Pos)
<< RTC_OFFSET_MONTH)   | \
-                     ((temp & (RTC_DR_YT | RTC_DR_YU)) >>
RTC_DR_YU_Pos));
-}
-
-/**
-  * @}
-  */
-
-/** @defgroup RTC_LL_EF_ALARMA ALARMA
-  * @{
-  */
-
-/**
-  * @brief  Enable Alarm A
-  * @note   Bit is write-protected. @ref LL_RTC_DisableWriteProtection
function should be called before.
-  * @rmtoll CR           ALRAE         LL_RTC_ALMA_Enable
-  * @param  RTCx RTC Instance
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_ALMA_Enable(RTC_TypeDef *RTCx)
-{
-  SET_BIT(RTCx->CR, RTC_CR_ALRAE);
-}
-
-/**
-  * @brief  Disable Alarm A
-  * @note   Bit is write-protected. @ref LL_RTC_DisableWriteProtection
function should be called before.
-  * @rmtoll CR           ALRAE         LL_RTC_ALMA_Disable
-  * @param  RTCx RTC Instance
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_ALMA_Disable(RTC_TypeDef *RTCx)
-{
-  CLEAR_BIT(RTCx->CR, RTC_CR_ALRAE);
-}
-
-/**
-  * @brief  Specify the Alarm A masks.
-  * @rmtoll ALRMAR       MSK4          LL_RTC_ALMA_SetMask\n
-  *         ALRMAR       MSK3          LL_RTC_ALMA_SetMask\n
-  *         ALRMAR       MSK2          LL_RTC_ALMA_SetMask\n
-  *         ALRMAR       MSK1          LL_RTC_ALMA_SetMask
-  * @param  RTCx RTC Instance
-  * @param  Mask This parameter can be a combination of the following
values:
-  *         @arg @ref LL_RTC_ALMA_MASK_NONE
-  *         @arg @ref LL_RTC_ALMA_MASK_DATEWEEKDAY
-  *         @arg @ref LL_RTC_ALMA_MASK_HOURS
-  *         @arg @ref LL_RTC_ALMA_MASK_MINUTES
```

```
-  *         @arg @ref LL_RTC_ALMA_MASK_SECONDS
-  *         @arg @ref LL_RTC_ALMA_MASK_ALL
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_ALMA_SetMask(RTC_TypeDef *RTCx, uint32_t
Mask)
-{
-  MODIFY_REG(RTCx->ALRMAR, RTC_ALRMAR_MSK4 | RTC_ALRMAR_MSK3 |
RTC_ALRMAR_MSK2 | RTC_ALRMAR_MSK1, Mask);
-}
-
-/**
-  * @brief  Get the Alarm A masks.
-  * @rmtoll ALRMAR       MSK4          LL_RTC_ALMA_GetMask\n
-  *         ALRMAR       MSK3          LL_RTC_ALMA_GetMask\n
-  *         ALRMAR       MSK2          LL_RTC_ALMA_GetMask\n
-  *         ALRMAR       MSK1          LL_RTC_ALMA_GetMask
-  * @param  RTCx RTC Instance
-  * @retval Returned value can be can be a combination of the following
values:
-  *         @arg @ref LL_RTC_ALMA_MASK_NONE
-  *         @arg @ref LL_RTC_ALMA_MASK_DATEWEEKDAY
-  *         @arg @ref LL_RTC_ALMA_MASK_HOURS
-  *         @arg @ref LL_RTC_ALMA_MASK_MINUTES
-  *         @arg @ref LL_RTC_ALMA_MASK_SECONDS
-  *         @arg @ref LL_RTC_ALMA_MASK_ALL
-  */
-__STATIC_INLINE uint32_t LL_RTC_ALMA_GetMask(RTC_TypeDef *RTCx)
-{
-  return (uint32_t)(READ_BIT(RTCx->ALRMAR, RTC_ALRMAR_MSK4 |
RTC_ALRMAR_MSK3 | RTC_ALRMAR_MSK2 | RTC_ALRMAR_MSK1));
-}
-
-/**
-  * @brief  Enable AlarmA Week day selection (DU[3:0] represents the
week day. DT[1:0] is do not care)
-  * @rmtoll ALRMAR       WDSEL         LL_RTC_ALMA_EnableWeekday
-  * @param  RTCx RTC Instance
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_ALMA_EnableWeekday(RTC_TypeDef *RTCx)
-{
-  SET_BIT(RTCx->ALRMAR, RTC_ALRMAR_WDSEL);
-}
-
-/**
-  * @brief  Disable AlarmA Week day selection (DU[3:0] represents the
date )
-  * @rmtoll ALRMAR       WDSEL         LL_RTC_ALMA_DisableWeekday
-  * @param  RTCx RTC Instance
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_ALMA_DisableWeekday(RTC_TypeDef *RTCx)
-{
-  CLEAR_BIT(RTCx->ALRMAR, RTC_ALRMAR_WDSEL);
-}
-
-/**
```

```
-  * @brief  Set ALARM A Day in BCD format
-  * @note helper macro __LL_RTC_CONVERT_BIN2BCD is available to convert
Day from binary to BCD format
-  * @rmtoll ALRMAR       DT            LL_RTC_ALMA_SetDay\n
-  *         ALRMAR       DU            LL_RTC_ALMA_SetDay
-  * @param  RTCx RTC Instance
-  * @param  Day Value between Min_Data=0x01 and Max_Data=0x31
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_ALMA_SetDay(RTC_TypeDef *RTCx, uint32_t Day)
-{
-  MODIFY_REG(RTCx->ALRMAR, (RTC_ALRMAR_DT | RTC_ALRMAR_DU),
-             (((Day & 0xF0U) << (RTC_ALRMAR_DT_Pos - 4U)) | ((Day &
0x0FU) << RTC_ALRMAR_DU_Pos)));
-}
-
-/**
-  * @brief  Get ALARM A Day in BCD format
-  * @note helper macro __LL_RTC_CONVERT_BCD2BIN is available to convert
Day from BCD to Binary format
-  * @rmtoll ALRMAR       DT            LL_RTC_ALMA_GetDay\n
-  *         ALRMAR       DU            LL_RTC_ALMA_GetDay
-  * @param  RTCx RTC Instance
-  * @retval Value between Min_Data=0x01 and Max_Data=0x31
-  */
-__STATIC_INLINE uint32_t LL_RTC_ALMA_GetDay(RTC_TypeDef *RTCx)
-{
-  return (uint32_t)((READ_BIT(RTCx->ALRMAR, (RTC_ALRMAR_DT |
RTC_ALRMAR_DU))) >> RTC_ALRMAR_DU_Pos);
-}
-
-/**
-  * @brief  Set ALARM A Weekday
-  * @rmtoll ALRMAR       DU            LL_RTC_ALMA_SetWeekDay
-  * @param  RTCx RTC Instance
-  * @param  WeekDay This parameter can be one of the following values:
-  *         @arg @ref LL_RTC_WEEKDAY_MONDAY
-  *         @arg @ref LL_RTC_WEEKDAY_TUESDAY
-  *         @arg @ref LL_RTC_WEEKDAY_WEDNESDAY
-  *         @arg @ref LL_RTC_WEEKDAY_THURSDAY
-  *         @arg @ref LL_RTC_WEEKDAY_FRIDAY
-  *         @arg @ref LL_RTC_WEEKDAY_SATURDAY
-  *         @arg @ref LL_RTC_WEEKDAY_SUNDAY
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_ALMA_SetWeekDay(RTC_TypeDef *RTCx, uint32_t
WeekDay)
-{
-  MODIFY_REG(RTCx->ALRMAR, RTC_ALRMAR_DU, WeekDay << RTC_ALRMAR_DU_Pos);
-}
-
-/**
-  * @brief  Get ALARM A Weekday
-  * @rmtoll ALRMAR       DU            LL_RTC_ALMA_GetWeekDay
-  * @param  RTCx RTC Instance
-  * @retval Returned value can be one of the following values:
-  *         @arg @ref LL_RTC_WEEKDAY_MONDAY
-  *         @arg @ref LL_RTC_WEEKDAY_TUESDAY
```

```
-  *         @arg @ref LL_RTC_WEEKDAY_WEDNESDAY
-  *         @arg @ref LL_RTC_WEEKDAY_THURSDAY
-  *         @arg @ref LL_RTC_WEEKDAY_FRIDAY
-  *         @arg @ref LL_RTC_WEEKDAY_SATURDAY
-  *         @arg @ref LL_RTC_WEEKDAY_SUNDAY
-  */
-__STATIC_INLINE uint32_t LL_RTC_ALMA_GetWeekDay(RTC_TypeDef *RTCx)
-{
-  return (uint32_t)(READ_BIT(RTCx->ALRMAR, RTC_ALRMAR_DU) >>
RTC_ALRMAR_DU_Pos);
-}
-
-/**
-  * @brief  Set Alarm A time format (AM/24-hour or PM notation)
-  * @rmtoll ALRMAR       PM            LL_RTC_ALMA_SetTimeFormat
-  * @param  RTCx RTC Instance
-  * @param  TimeFormat This parameter can be one of the following
values:
-  *         @arg @ref LL_RTC_ALMA_TIME_FORMAT_AM
-  *         @arg @ref LL_RTC_ALMA_TIME_FORMAT_PM
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_ALMA_SetTimeFormat(RTC_TypeDef *RTCx,
uint32_t TimeFormat)
-{
-  MODIFY_REG(RTCx->ALRMAR, RTC_ALRMAR_PM, TimeFormat);
-}
-
-/**
-  * @brief  Get Alarm A time format (AM or PM notation)
-  * @rmtoll ALRMAR       PM            LL_RTC_ALMA_GetTimeFormat
-  * @param  RTCx RTC Instance
-  * @retval Returned value can be one of the following values:
-  *         @arg @ref LL_RTC_ALMA_TIME_FORMAT_AM
-  *         @arg @ref LL_RTC_ALMA_TIME_FORMAT_PM
-  */
-__STATIC_INLINE uint32_t LL_RTC_ALMA_GetTimeFormat(RTC_TypeDef *RTCx)
-{
-  return (uint32_t)(READ_BIT(RTCx->ALRMAR, RTC_ALRMAR_PM));
-}
-
-/**
-  * @brief  Set ALARM A Hours in BCD format
-  * @note helper macro __LL_RTC_CONVERT_BIN2BCD is available to convert
Hours from binary to BCD format
-  * @rmtoll ALRMAR       HT            LL_RTC_ALMA_SetHour\n
-  *         ALRMAR       HU            LL_RTC_ALMA_SetHour
-  * @param  RTCx RTC Instance
-  * @param  Hours Value between Min_Data=0x01 and Max_Data=0x12 or
between Min_Data=0x00 and Max_Data=0x23
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_ALMA_SetHour(RTC_TypeDef *RTCx, uint32_t
Hours)
-{
-  MODIFY_REG(RTCx->ALRMAR, (RTC_ALRMAR_HT | RTC_ALRMAR_HU),
-             (((Hours & 0xF0U) << (RTC_ALRMAR_HT_Pos - 4U)) | ((Hours &
0x0FU) << RTC_ALRMAR_HU_Pos)));
```

```
-}
-
-/**
-  * @brief  Get ALARM A Hours in BCD format
-  * @note helper macro __LL_RTC_CONVERT_BCD2BIN is available to convert
Hours from BCD to Binary format
-  * @rmtoll ALRMAR       HT            LL_RTC_ALMA_GetHour\n
-  *         ALRMAR       HU            LL_RTC_ALMA_GetHour
-  * @param  RTCx RTC Instance
-  * @retval Value between Min_Data=0x01 and Max_Data=0x12 or between
Min_Data=0x00 and Max_Data=0x23
-  */
-__STATIC_INLINE uint32_t LL_RTC_ALMA_GetHour(RTC_TypeDef *RTCx)
-{
-  return (uint32_t)((READ_BIT(RTCx->ALRMAR, (RTC_ALRMAR_HT |
RTC_ALRMAR_HU))) >> RTC_ALRMAR_HU_Pos);
-}
-
-/**
-  * @brief  Set ALARM A Minutes in BCD format
-  * @note helper macro __LL_RTC_CONVERT_BIN2BCD is available to convert
Minutes from binary to BCD format
-  * @rmtoll ALRMAR       MNT           LL_RTC_ALMA_SetMinute\n
-  *         ALRMAR       MNU           LL_RTC_ALMA_SetMinute
-  * @param  RTCx RTC Instance
-  * @param  Minutes Value between Min_Data=0x00 and Max_Data=0x59
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_ALMA_SetMinute(RTC_TypeDef *RTCx, uint32_t
Minutes)
-{
-  MODIFY_REG(RTCx->ALRMAR, (RTC_ALRMAR_MNT | RTC_ALRMAR_MNU),
-             (((Minutes & 0xF0U) << (RTC_ALRMAR_MNT_Pos - 4U)) |
((Minutes & 0x0FU) << RTC_ALRMAR_MNU_Pos)));
-}
-
-/**
-  * @brief  Get ALARM A Minutes in BCD format
-  * @note helper macro __LL_RTC_CONVERT_BCD2BIN is available to convert
Minutes from BCD to Binary format
-  * @rmtoll ALRMAR       MNT           LL_RTC_ALMA_GetMinute\n
-  *         ALRMAR       MNU           LL_RTC_ALMA_GetMinute
-  * @param  RTCx RTC Instance
-  * @retval Value between Min_Data=0x00 and Max_Data=0x59
-  */
-__STATIC_INLINE uint32_t LL_RTC_ALMA_GetMinute(RTC_TypeDef *RTCx)
-{
-  return (uint32_t)((READ_BIT(RTCx->ALRMAR, (RTC_ALRMAR_MNT |
RTC_ALRMAR_MNU))) >> RTC_ALRMAR_MNU_Pos);
-}
-
-/**
-  * @brief  Set ALARM A Seconds in BCD format
-  * @note helper macro __LL_RTC_CONVERT_BIN2BCD is available to convert
Seconds from binary to BCD format
-  * @rmtoll ALRMAR       ST            LL_RTC_ALMA_SetSecond\n
-  *         ALRMAR       SU            LL_RTC_ALMA_SetSecond
-  * @param  RTCx RTC Instance
```

```
-  * @param  Seconds Value between Min_Data=0x00 and Max_Data=0x59
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_ALMA_SetSecond(RTC_TypeDef *RTCx, uint32_t
Seconds)
-{
-  MODIFY_REG(RTCx->ALRMAR, (RTC_ALRMAR_ST | RTC_ALRMAR_SU),
-             (((Seconds & 0xF0U) << (RTC_ALRMAR_ST_Pos - 4U)) |
((Seconds & 0x0FU) << RTC_ALRMAR_SU_Pos)));
-}
-
-/**
-  * @brief  Get ALARM A Seconds in BCD format
-  * @note helper macro __LL_RTC_CONVERT_BCD2BIN is available to convert
Seconds from BCD to Binary format
-  * @rmtoll ALRMAR       ST            LL_RTC_ALMA_GetSecond\n
-  *         ALRMAR       SU            LL_RTC_ALMA_GetSecond
-  * @param  RTCx RTC Instance
-  * @retval Value between Min_Data=0x00 and Max_Data=0x59
-  */
-__STATIC_INLINE uint32_t LL_RTC_ALMA_GetSecond(RTC_TypeDef *RTCx)
-{
-  return (uint32_t)((READ_BIT(RTCx->ALRMAR, (RTC_ALRMAR_ST |
RTC_ALRMAR_SU))) >> RTC_ALRMAR_SU_Pos);
-}
-
-/**
-  * @brief  Set Alarm A Time (hour, minute and second) in BCD format
-  * @rmtoll ALRMAR       PM            LL_RTC_ALMA_ConfigTime\n
-  *         ALRMAR       HT            LL_RTC_ALMA_ConfigTime\n
-  *         ALRMAR       HU            LL_RTC_ALMA_ConfigTime\n
-  *         ALRMAR       MNT           LL_RTC_ALMA_ConfigTime\n
-  *         ALRMAR       MNU           LL_RTC_ALMA_ConfigTime\n
-  *         ALRMAR       ST            LL_RTC_ALMA_ConfigTime\n
-  *         ALRMAR       SU            LL_RTC_ALMA_ConfigTime
-  * @param  RTCx RTC Instance
-  * @param  Format12_24 This parameter can be one of the following
values:
-  *         @arg @ref LL_RTC_ALMA_TIME_FORMAT_AM
-  *         @arg @ref LL_RTC_ALMA_TIME_FORMAT_PM
-  * @param  Hours Value between Min_Data=0x01 and Max_Data=0x12 or
between Min_Data=0x00 and Max_Data=0x23
-  * @param  Minutes Value between Min_Data=0x00 and Max_Data=0x59
-  * @param  Seconds Value between Min_Data=0x00 and Max_Data=0x59
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_ALMA_ConfigTime(RTC_TypeDef *RTCx, uint32_t
Format12_24, uint32_t Hours, uint32_t Minutes, uint32_t Seconds)
-{
-  uint32_t temp;
-
-  temp = Format12_24
| \
-         (((Hours   & 0xF0U) << (RTC_ALRMAR_HT_Pos  - 4U)) | ((Hours   &
0x0FU) << RTC_ALRMAR_HU_Pos))  | \
-         (((Minutes & 0xF0U) << (RTC_ALRMAR_MNT_Pos - 4U)) | ((Minutes &
0x0FU) << RTC_ALRMAR_MNU_Pos)) | \
```

```
-         (((Seconds & 0xF0U) << (RTC_ALRMAR_ST_Pos  - 4U)) | ((Seconds &
0x0FU) << RTC_ALRMAR_SU_Pos));
-
-  MODIFY_REG(RTCx->ALRMAR, RTC_ALRMAR_PM | RTC_ALRMAR_HT | RTC_ALRMAR_HU
| RTC_ALRMAR_MNT | RTC_ALRMAR_MNU | RTC_ALRMAR_ST | RTC_ALRMAR_SU, temp);
-}
-
-/**
-  * @brief  Get Alarm B Time (hour, minute and second) in BCD format
-  * @note helper macros __LL_RTC_GET_HOUR, __LL_RTC_GET_MINUTE and
__LL_RTC_GET_SECOND
-  * are available to get independently each parameter.
-  * @rmtoll ALRMAR       HT            LL_RTC_ALMA_GetTime\n
-  *         ALRMAR       HU            LL_RTC_ALMA_GetTime\n
-  *         ALRMAR       MNT           LL_RTC_ALMA_GetTime\n
-  *         ALRMAR       MNU           LL_RTC_ALMA_GetTime\n
-  *         ALRMAR       ST            LL_RTC_ALMA_GetTime\n
-  *         ALRMAR       SU            LL_RTC_ALMA_GetTime
-  * @param  RTCx RTC Instance
-  * @retval Combination of hours, minutes and seconds.
-  */
-__STATIC_INLINE uint32_t LL_RTC_ALMA_GetTime(RTC_TypeDef *RTCx)
-{
-  return (uint32_t)((LL_RTC_ALMA_GetHour(RTCx) << RTC_OFFSET_HOUR) |
(LL_RTC_ALMA_GetMinute(RTCx) << RTC_OFFSET_MINUTE) |
LL_RTC_ALMA_GetSecond(RTCx));
-}
-
-/**
-  * @brief  Mask the most-significant bits of the subseconds field
starting from
-  *         the bit specified in parameter Mask
-  * @note This register can be written only when ALRAE is reset in
RTC_CR register,
-  *       or in initialization mode.
-  * @rmtoll ALRMASSR     MASKSS        LL_RTC_ALMA_SetSubSecondMask
-  * @param  RTCx RTC Instance
-  * @param  Mask Value between Min_Data=0x00 and Max_Data=0xF
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_ALMA_SetSubSecondMask(RTC_TypeDef *RTCx,
uint32_t Mask)
-{
-  MODIFY_REG(RTCx->ALRMASSR, RTC_ALRMASSR_MASKSS, Mask <<
RTC_ALRMASSR_MASKSS_Pos);
-}
-
-/**
-  * @brief  Get Alarm A subseconds mask
-  * @rmtoll ALRMASSR     MASKSS        LL_RTC_ALMA_GetSubSecondMask
-  * @param  RTCx RTC Instance
-  * @retval Value between Min_Data=0x00 and Max_Data=0xF
-  */
-__STATIC_INLINE uint32_t LL_RTC_ALMA_GetSubSecondMask(RTC_TypeDef *RTCx)
-{
-  return (uint32_t)(READ_BIT(RTCx->ALRMASSR, RTC_ALRMASSR_MASKSS) >>
RTC_ALRMASSR_MASKSS_Pos);
-}
```

```
-
-/**
-  * @brief  Set Alarm A subseconds value
-  * @rmtoll ALRMASSR     SS            LL_RTC_ALMA_SetSubSecond
-  * @param  RTCx RTC Instance
-  * @param  Subsecond Value between Min_Data=0x00 and Max_Data=0x7FFF
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_ALMA_SetSubSecond(RTC_TypeDef *RTCx,
uint32_t Subsecond)
-{
-  MODIFY_REG(RTCx->ALRMASSR, RTC_ALRMASSR_SS, Subsecond);
-}
-
-/**
-  * @brief  Get Alarm A subseconds value
-  * @rmtoll ALRMASSR     SS            LL_RTC_ALMA_GetSubSecond
-  * @param  RTCx RTC Instance
-  * @retval Value between Min_Data=0x00 and Max_Data=0x7FFF
-  */
-__STATIC_INLINE uint32_t LL_RTC_ALMA_GetSubSecond(RTC_TypeDef *RTCx)
-{
-  return (uint32_t)(READ_BIT(RTCx->ALRMASSR, RTC_ALRMASSR_SS));
-}
-
-/**
-  * @}
-  */
-
-/** @defgroup RTC_LL_EF_ALARMB ALARMB
-  * @{
-  */
-
-/**
-  * @brief  Enable Alarm B
-  * @note   Bit is write-protected. @ref LL_RTC_DisableWriteProtection
function should be called before.
-  * @rmtoll CR           ALRBE         LL_RTC_ALMB_Enable
-  * @param  RTCx RTC Instance
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_ALMB_Enable(RTC_TypeDef *RTCx)
-{
-  SET_BIT(RTCx->CR, RTC_CR_ALRBE);
-}
-
-/**
-  * @brief  Disable Alarm B
-  * @note   Bit is write-protected. @ref LL_RTC_DisableWriteProtection
function should be called before.
-  * @rmtoll CR           ALRBE         LL_RTC_ALMB_Disable
-  * @param  RTCx RTC Instance
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_ALMB_Disable(RTC_TypeDef *RTCx)
-{
-  CLEAR_BIT(RTCx->CR, RTC_CR_ALRBE);
-}
```

```
-
-/**
-  * @brief  Specify the Alarm B masks.
-  * @rmtoll ALRMBR       MSK4          LL_RTC_ALMB_SetMask\n
-  *         ALRMBR       MSK3          LL_RTC_ALMB_SetMask\n
-  *         ALRMBR       MSK2          LL_RTC_ALMB_SetMask\n
-  *         ALRMBR       MSK1          LL_RTC_ALMB_SetMask
-  * @param  RTCx RTC Instance
-  * @param  Mask This parameter can be a combination of the following
values:
-  *         @arg @ref LL_RTC_ALMB_MASK_NONE
-  *         @arg @ref LL_RTC_ALMB_MASK_DATEWEEKDAY
-  *         @arg @ref LL_RTC_ALMB_MASK_HOURS
-  *         @arg @ref LL_RTC_ALMB_MASK_MINUTES
-  *         @arg @ref LL_RTC_ALMB_MASK_SECONDS
-  *         @arg @ref LL_RTC_ALMB_MASK_ALL
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_ALMB_SetMask(RTC_TypeDef *RTCx, uint32_t
Mask)
-{
-  MODIFY_REG(RTCx->ALRMBR, RTC_ALRMBR_MSK4 | RTC_ALRMBR_MSK3 |
RTC_ALRMBR_MSK2 | RTC_ALRMBR_MSK1, Mask);
-}
-
-/**
-  * @brief  Get the Alarm B masks.
-  * @rmtoll ALRMBR       MSK4          LL_RTC_ALMB_GetMask\n
-  *         ALRMBR       MSK3          LL_RTC_ALMB_GetMask\n
-  *         ALRMBR       MSK2          LL_RTC_ALMB_GetMask\n
-  *         ALRMBR       MSK1          LL_RTC_ALMB_GetMask
-  * @param  RTCx RTC Instance
-  * @retval Returned value can be can be a combination of the following
values:
-  *         @arg @ref LL_RTC_ALMB_MASK_NONE
-  *         @arg @ref LL_RTC_ALMB_MASK_DATEWEEKDAY
-  *         @arg @ref LL_RTC_ALMB_MASK_HOURS
-  *         @arg @ref LL_RTC_ALMB_MASK_MINUTES
-  *         @arg @ref LL_RTC_ALMB_MASK_SECONDS
-  *         @arg @ref LL_RTC_ALMB_MASK_ALL
-  */
-__STATIC_INLINE uint32_t LL_RTC_ALMB_GetMask(RTC_TypeDef *RTCx)
-{
-  return (uint32_t)(READ_BIT(RTCx->ALRMBR, RTC_ALRMBR_MSK4 |
RTC_ALRMBR_MSK3 | RTC_ALRMBR_MSK2 | RTC_ALRMBR_MSK1));
-}
-
-/**
-  * @brief  Enable AlarmB Week day selection (DU[3:0] represents the
week day. DT[1:0] is do not care)
-  * @rmtoll ALRMBR       WDSEL         LL_RTC_ALMB_EnableWeekday
-  * @param  RTCx RTC Instance
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_ALMB_EnableWeekday(RTC_TypeDef *RTCx)
-{
-  SET_BIT(RTCx->ALRMBR, RTC_ALRMBR_WDSEL);
-}
```

```
-
-/**
-  * @brief  Disable AlarmB Week day selection (DU[3:0] represents the
date )
-  * @rmtoll ALRMBR       WDSEL         LL_RTC_ALMB_DisableWeekday
-  * @param  RTCx RTC Instance
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_ALMB_DisableWeekday(RTC_TypeDef *RTCx)
-{
-  CLEAR_BIT(RTCx->ALRMBR, RTC_ALRMBR_WDSEL);
-}
-
-/**
-  * @brief  Set ALARM B Day in BCD format
-  * @note helper macro __LL_RTC_CONVERT_BIN2BCD is available to convert
Day from binary to BCD format
-  * @rmtoll ALRMBR       DT            LL_RTC_ALMB_SetDay\n
-  *         ALRMBR       DU            LL_RTC_ALMB_SetDay
-  * @param  RTCx RTC Instance
-  * @param  Day Value between Min_Data=0x01 and Max_Data=0x31
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_ALMB_SetDay(RTC_TypeDef *RTCx, uint32_t Day)
-{
-  MODIFY_REG(RTCx->ALRMBR, (RTC_ALRMBR_DT | RTC_ALRMBR_DU),
-             (((Day & 0xF0U) << (RTC_ALRMBR_DT_Pos - 4U)) | ((Day &
0x0FU) << RTC_ALRMBR_DU_Pos)));
-}
-
-/**
-  * @brief  Get ALARM B Day in BCD format
-  * @note helper macro __LL_RTC_CONVERT_BCD2BIN is available to convert
Day from BCD to Binary format
-  * @rmtoll ALRMBR       DT            LL_RTC_ALMB_GetDay\n
-  *         ALRMBR       DU            LL_RTC_ALMB_GetDay
-  * @param  RTCx RTC Instance
-  * @retval Value between Min_Data=0x01 and Max_Data=0x31
-  */
-__STATIC_INLINE uint32_t LL_RTC_ALMB_GetDay(RTC_TypeDef *RTCx)
-{
-  return (uint32_t)((READ_BIT(RTCx->ALRMBR, (RTC_ALRMBR_DT |
RTC_ALRMBR_DU))) >> RTC_ALRMBR_DU_Pos);
-}
-
-/**
-  * @brief  Set ALARM B Weekday
-  * @rmtoll ALRMBR       DU            LL_RTC_ALMB_SetWeekDay
-  * @param  RTCx RTC Instance
-  * @param  WeekDay This parameter can be one of the following values:
-  *         @arg @ref LL_RTC_WEEKDAY_MONDAY
-  *         @arg @ref LL_RTC_WEEKDAY_TUESDAY
-  *         @arg @ref LL_RTC_WEEKDAY_WEDNESDAY
-  *         @arg @ref LL_RTC_WEEKDAY_THURSDAY
-  *         @arg @ref LL_RTC_WEEKDAY_FRIDAY
-  *         @arg @ref LL_RTC_WEEKDAY_SATURDAY
-  *         @arg @ref LL_RTC_WEEKDAY_SUNDAY
-  * @retval None
```

```
-  */
-__STATIC_INLINE void LL_RTC_ALMB_SetWeekDay(RTC_TypeDef *RTCx, uint32_t
WeekDay)
-{
-  MODIFY_REG(RTCx->ALRMBR, RTC_ALRMBR_DU, WeekDay << RTC_ALRMBR_DU_Pos);
-}
-
-/**
-  * @brief  Get ALARM B Weekday
-  * @rmtoll ALRMBR       DU            LL_RTC_ALMB_GetWeekDay
-  * @param  RTCx RTC Instance
-  * @retval Returned value can be one of the following values:
-  *         @arg @ref LL_RTC_WEEKDAY_MONDAY
-  *         @arg @ref LL_RTC_WEEKDAY_TUESDAY
-  *         @arg @ref LL_RTC_WEEKDAY_WEDNESDAY
-  *         @arg @ref LL_RTC_WEEKDAY_THURSDAY
-  *         @arg @ref LL_RTC_WEEKDAY_FRIDAY
-  *         @arg @ref LL_RTC_WEEKDAY_SATURDAY
-  *         @arg @ref LL_RTC_WEEKDAY_SUNDAY
-  */
-__STATIC_INLINE uint32_t LL_RTC_ALMB_GetWeekDay(RTC_TypeDef *RTCx)
-{
-  return (uint32_t)(READ_BIT(RTCx->ALRMBR, RTC_ALRMBR_DU) >>
RTC_ALRMBR_DU_Pos);
-}
-
-/**
-  * @brief  Set ALARM B time format (AM/24-hour or PM notation)
-  * @rmtoll ALRMBR       PM            LL_RTC_ALMB_SetTimeFormat
-  * @param  RTCx RTC Instance
-  * @param  TimeFormat This parameter can be one of the following
values:
-  *         @arg @ref LL_RTC_ALMB_TIME_FORMAT_AM
-  *         @arg @ref LL_RTC_ALMB_TIME_FORMAT_PM
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_ALMB_SetTimeFormat(RTC_TypeDef *RTCx,
uint32_t TimeFormat)
-{
-  MODIFY_REG(RTCx->ALRMBR, RTC_ALRMBR_PM, TimeFormat);
-}
-
-/**
-  * @brief  Get ALARM B time format (AM or PM notation)
-  * @rmtoll ALRMBR       PM            LL_RTC_ALMB_GetTimeFormat
-  * @param  RTCx RTC Instance
-  * @retval Returned value can be one of the following values:
-  *         @arg @ref LL_RTC_ALMB_TIME_FORMAT_AM
-  *         @arg @ref LL_RTC_ALMB_TIME_FORMAT_PM
-  */
-__STATIC_INLINE uint32_t LL_RTC_ALMB_GetTimeFormat(RTC_TypeDef *RTCx)
-{
-  return (uint32_t)(READ_BIT(RTCx->ALRMBR, RTC_ALRMBR_PM));
-}
-
-/**
-  * @brief  Set ALARM B Hours in BCD format
```

```
-  * @note helper macro __LL_RTC_CONVERT_BIN2BCD is available to convert
Hours from binary to BCD format
-  * @rmtoll ALRMBR       HT            LL_RTC_ALMB_SetHour\n
-  *         ALRMBR       HU            LL_RTC_ALMB_SetHour
-  * @param  RTCx RTC Instance
-  * @param  Hours Value between Min_Data=0x01 and Max_Data=0x12 or
between Min_Data=0x00 and Max_Data=0x23
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_ALMB_SetHour(RTC_TypeDef *RTCx, uint32_t
Hours)
-{
-  MODIFY_REG(RTCx->ALRMBR, (RTC_ALRMBR_HT | RTC_ALRMBR_HU),
-             (((Hours & 0xF0U) << (RTC_ALRMBR_HT_Pos - 4U)) | ((Hours &
0x0FU) << RTC_ALRMBR_HU_Pos)));
-}
-
-/**
-  * @brief  Get ALARM B Hours in BCD format
-  * @note helper macro __LL_RTC_CONVERT_BCD2BIN is available to convert
Hours from BCD to Binary format
-  * @rmtoll ALRMBR       HT            LL_RTC_ALMB_GetHour\n
-  *         ALRMBR       HU            LL_RTC_ALMB_GetHour
-  * @param  RTCx RTC Instance
-  * @retval Value between Min_Data=0x01 and Max_Data=0x12 or between
Min_Data=0x00 and Max_Data=0x23
-  */
-__STATIC_INLINE uint32_t LL_RTC_ALMB_GetHour(RTC_TypeDef *RTCx)
-{
-  return (uint32_t)((READ_BIT(RTCx->ALRMBR, (RTC_ALRMBR_HT |
RTC_ALRMBR_HU))) >> RTC_ALRMBR_HU_Pos);
-}
-
-/**
-  * @brief  Set ALARM B Minutes in BCD format
-  * @note helper macro __LL_RTC_CONVERT_BIN2BCD is available to convert
Minutes from binary to BCD format
-  * @rmtoll ALRMBR       MNT           LL_RTC_ALMB_SetMinute\n
-  *         ALRMBR       MNU           LL_RTC_ALMB_SetMinute
-  * @param  RTCx RTC Instance
-  * @param  Minutes between Min_Data=0x00 and Max_Data=0x59
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_ALMB_SetMinute(RTC_TypeDef *RTCx, uint32_t
Minutes)
-{
-  MODIFY_REG(RTCx->ALRMBR, (RTC_ALRMBR_MNT | RTC_ALRMBR_MNU),
-             (((Minutes & 0xF0U) << (RTC_ALRMBR_MNT_Pos - 4U)) |
((Minutes & 0x0FU) << RTC_ALRMBR_MNU_Pos)));
-}
-
-/**
-  * @brief  Get ALARM B Minutes in BCD format
-  * @note helper macro __LL_RTC_CONVERT_BCD2BIN is available to convert
Minutes from BCD to Binary format
-  * @rmtoll ALRMBR       MNT           LL_RTC_ALMB_GetMinute\n
-  *         ALRMBR       MNU           LL_RTC_ALMB_GetMinute
-  * @param  RTCx RTC Instance
```

```
-  * @retval Value between Min_Data=0x00 and Max_Data=0x59
-  */
-__STATIC_INLINE uint32_t LL_RTC_ALMB_GetMinute(RTC_TypeDef *RTCx)
-{
-  return (uint32_t)((READ_BIT(RTCx->ALRMBR, (RTC_ALRMBR_MNT |
RTC_ALRMBR_MNU))) >> RTC_ALRMBR_MNU_Pos);
-}
-
-/**
-  * @brief  Set ALARM B Seconds in BCD format
-  * @note helper macro __LL_RTC_CONVERT_BIN2BCD is available to convert
Seconds from binary to BCD format
-  * @rmtoll ALRMBR       ST            LL_RTC_ALMB_SetSecond\n
-  *         ALRMBR       SU            LL_RTC_ALMB_SetSecond
-  * @param  RTCx RTC Instance
-  * @param  Seconds Value between Min_Data=0x00 and Max_Data=0x59
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_ALMB_SetSecond(RTC_TypeDef *RTCx, uint32_t
Seconds)
-{
-  MODIFY_REG(RTCx->ALRMBR, (RTC_ALRMBR_ST | RTC_ALRMBR_SU),
-             (((Seconds & 0xF0U) << (RTC_ALRMBR_ST_Pos - 4U)) |
((Seconds & 0x0FU) << RTC_ALRMBR_SU_Pos)));
-}
-
-/**
-  * @brief  Get ALARM B Seconds in BCD format
-  * @note helper macro __LL_RTC_CONVERT_BCD2BIN is available to convert
Seconds from BCD to Binary format
-  * @rmtoll ALRMBR       ST            LL_RTC_ALMB_GetSecond\n
-  *         ALRMBR       SU            LL_RTC_ALMB_GetSecond
-  * @param  RTCx RTC Instance
-  * @retval Value between Min_Data=0x00 and Max_Data=0x59
-  */
-__STATIC_INLINE uint32_t LL_RTC_ALMB_GetSecond(RTC_TypeDef *RTCx)
-{
-  return (uint32_t)((READ_BIT(RTCx->ALRMBR, (RTC_ALRMBR_ST |
RTC_ALRMBR_SU))) >> RTC_ALRMBR_SU_Pos);
-}
-
-/**
-  * @brief  Set Alarm B Time (hour, minute and second) in BCD format
-  * @rmtoll ALRMBR       PM            LL_RTC_ALMB_ConfigTime\n
-  *         ALRMBR       HT            LL_RTC_ALMB_ConfigTime\n
-  *         ALRMBR       HU            LL_RTC_ALMB_ConfigTime\n
-  *         ALRMBR       MNT           LL_RTC_ALMB_ConfigTime\n
-  *         ALRMBR       MNU           LL_RTC_ALMB_ConfigTime\n
-  *         ALRMBR       ST            LL_RTC_ALMB_ConfigTime\n
-  *         ALRMBR       SU            LL_RTC_ALMB_ConfigTime
-  * @param  RTCx RTC Instance
-  * @param  Format12_24 This parameter can be one of the following
values:
-  *         @arg @ref LL_RTC_ALMB_TIME_FORMAT_AM
-  *         @arg @ref LL_RTC_ALMB_TIME_FORMAT_PM
-  * @param  Hours Value between Min_Data=0x01 and Max_Data=0x12 or
between Min_Data=0x00 and Max_Data=0x23
-  * @param  Minutes Value between Min_Data=0x00 and Max_Data=0x59
```

```
-  * @param  Seconds Value between Min_Data=0x00 and Max_Data=0x59
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_ALMB_ConfigTime(RTC_TypeDef *RTCx, uint32_t
Format12_24, uint32_t Hours, uint32_t Minutes, uint32_t Seconds)
-{
-  uint32_t temp;
-
-  temp = Format12_24
| \
-         (((Hours   & 0xF0U) << (RTC_ALRMBR_HT_Pos  - 4U)) | ((Hours   &
0x0FU) << RTC_ALRMBR_HU_Pos))  | \
-         (((Minutes & 0xF0U) << (RTC_ALRMBR_MNT_Pos - 4U)) | ((Minutes &
0x0FU) << RTC_ALRMBR_MNU_Pos)) | \
-         (((Seconds & 0xF0U) << (RTC_ALRMBR_ST_Pos  - 4U)) | ((Seconds &
0x0FU) << RTC_ALRMBR_SU_Pos));
-
-  MODIFY_REG(RTCx->ALRMBR, RTC_ALRMBR_PM | RTC_ALRMBR_HT | RTC_ALRMBR_HU
| RTC_ALRMBR_MNT | RTC_ALRMBR_MNU | RTC_ALRMBR_ST | RTC_ALRMBR_SU, temp);
-}
-
-/**
-  * @brief  Get Alarm B Time (hour, minute and second) in BCD format
-  * @note helper macros __LL_RTC_GET_HOUR, __LL_RTC_GET_MINUTE and
__LL_RTC_GET_SECOND
-  * are available to get independently each parameter.
-  * @rmtoll ALRMBR       HT            LL_RTC_ALMB_GetTime\n
-  *         ALRMBR       HU            LL_RTC_ALMB_GetTime\n
-  *         ALRMBR       MNT           LL_RTC_ALMB_GetTime\n
-  *         ALRMBR       MNU           LL_RTC_ALMB_GetTime\n
-  *         ALRMBR       ST            LL_RTC_ALMB_GetTime\n
-  *         ALRMBR       SU            LL_RTC_ALMB_GetTime
-  * @param  RTCx RTC Instance
-  * @retval Combination of hours, minutes and seconds.
-  */
-__STATIC_INLINE uint32_t LL_RTC_ALMB_GetTime(RTC_TypeDef *RTCx)
-{
-  return (uint32_t)((LL_RTC_ALMB_GetHour(RTCx) << RTC_OFFSET_HOUR) |
(LL_RTC_ALMB_GetMinute(RTCx) << RTC_OFFSET_MINUTE) |
LL_RTC_ALMB_GetSecond(RTCx));
-}
-
-/**
-  * @brief  Mask the most-significant bits of the subseconds field
starting from
-  *         the bit specified in parameter Mask
-  * @note This register can be written only when ALRBE is reset in
RTC_CR register,
-  *       or in initialization mode.
-  * @rmtoll ALRMBSSR     MASKSS        LL_RTC_ALMB_SetSubSecondMask
-  * @param  RTCx RTC Instance
-  * @param  Mask Value between Min_Data=0x00 and Max_Data=0xF
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_ALMB_SetSubSecondMask(RTC_TypeDef *RTCx,
uint32_t Mask)
-{
```

```
-  MODIFY_REG(RTCx->ALRMBSSR, RTC_ALRMBSSR_MASKSS, Mask <<
RTC_ALRMBSSR_MASKSS_Pos);
-}
-
-/**
-  * @brief  Get Alarm B subseconds mask
-  * @rmtoll ALRMBSSR     MASKSS        LL_RTC_ALMB_GetSubSecondMask
-  * @param  RTCx RTC Instance
-  * @retval Value between Min_Data=0x00 and Max_Data=0xF
-  */
-__STATIC_INLINE uint32_t LL_RTC_ALMB_GetSubSecondMask(RTC_TypeDef *RTCx)
-{
-  return (uint32_t)(READ_BIT(RTCx->ALRMBSSR, RTC_ALRMBSSR_MASKSS)  >>
RTC_ALRMBSSR_MASKSS_Pos);
-}
-
-/**
-  * @brief  Set Alarm B subseconds value
-  * @rmtoll ALRMBSSR     SS            LL_RTC_ALMB_SetSubSecond
-  * @param  RTCx RTC Instance
-  * @param  Subsecond Value between Min_Data=0x00 and Max_Data=0x7FFF
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_ALMB_SetSubSecond(RTC_TypeDef *RTCx,
uint32_t Subsecond)
-{
-  MODIFY_REG(RTCx->ALRMBSSR, RTC_ALRMBSSR_SS, Subsecond);
-}
-
-/**
-  * @brief  Get Alarm B subseconds value
-  * @rmtoll ALRMBSSR     SS            LL_RTC_ALMB_GetSubSecond
-  * @param  RTCx RTC Instance
-  * @retval Value between Min_Data=0x00 and Max_Data=0x7FFF
-  */
-__STATIC_INLINE uint32_t LL_RTC_ALMB_GetSubSecond(RTC_TypeDef *RTCx)
-{
-  return (uint32_t)(READ_BIT(RTCx->ALRMBSSR, RTC_ALRMBSSR_SS));
-}
-
-/**
-  * @}
-  */
-
-/** @defgroup RTC_LL_EF_Timestamp Timestamp
-  * @{
-  */
-
-/**
-  * @brief  Enable Timestamp
-  * @note   Bit is write-protected. @ref LL_RTC_DisableWriteProtection
function should be called before.
-  * @rmtoll CR           TSE           LL_RTC_TS_Enable
-  * @param  RTCx RTC Instance
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_TS_Enable(RTC_TypeDef *RTCx)
-{
```

```
-  SET_BIT(RTCx->CR, RTC_CR_TSE);
-}
-
-/**
-  * @brief  Disable Timestamp
-  * @note   Bit is write-protected. @ref LL_RTC_DisableWriteProtection
function should be called before.
-  * @rmtoll CR           TSE           LL_RTC_TS_Disable
-  * @param  RTCx RTC Instance
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_TS_Disable(RTC_TypeDef *RTCx)
-{
-  CLEAR_BIT(RTCx->CR, RTC_CR_TSE);
-}
-
-/**
-  * @brief  Set Time-stamp event active edge
-  * @note   Bit is write-protected. @ref LL_RTC_DisableWriteProtection
function should be called before.
-  * @note TSE must be reset when TSEDGE is changed to avoid unwanted TSF
setting
-  * @rmtoll CR           TSEDGE        LL_RTC_TS_SetActiveEdge
-  * @param  RTCx RTC Instance
-  * @param  Edge This parameter can be one of the following values:
-  *         @arg @ref LL_RTC_TIMESTAMP_EDGE_RISING
-  *         @arg @ref LL_RTC_TIMESTAMP_EDGE_FALLING
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_TS_SetActiveEdge(RTC_TypeDef *RTCx, uint32_t
Edge)
-{
-  MODIFY_REG(RTCx->CR, RTC_CR_TSEDGE, Edge);
-}
-
-/**
-  * @brief  Get Time-stamp event active edge
-  * @note   Bit is write-protected. @ref LL_RTC_DisableWriteProtection
function should be called before.
-  * @rmtoll CR           TSEDGE        LL_RTC_TS_GetActiveEdge
-  * @param  RTCx RTC Instance
-  * @retval Returned value can be one of the following values:
-  *         @arg @ref LL_RTC_TIMESTAMP_EDGE_RISING
-  *         @arg @ref LL_RTC_TIMESTAMP_EDGE_FALLING
-  */
-__STATIC_INLINE uint32_t LL_RTC_TS_GetActiveEdge(RTC_TypeDef *RTCx)
-{
-  return (uint32_t)(READ_BIT(RTCx->CR, RTC_CR_TSEDGE));
-}
-
-/**
-  * @brief  Get Timestamp AM/PM notation (AM or 24-hour format)
-  * @rmtoll TSTR         PM            LL_RTC_TS_GetTimeFormat
-  * @param  RTCx RTC Instance
-  * @retval Returned value can be one of the following values:
-  *         @arg @ref LL_RTC_TS_TIME_FORMAT_AM
-  *         @arg @ref LL_RTC_TS_TIME_FORMAT_PM
-  */
```

```
-__STATIC_INLINE uint32_t LL_RTC_TS_GetTimeFormat(RTC_TypeDef *RTCx)
-{
-  return (uint32_t)(READ_BIT(RTCx->TSTR, RTC_TSTR_PM));
-}
-
-/**
-  * @brief  Get Timestamp Hours in BCD format
-  * @note helper macro __LL_RTC_CONVERT_BCD2BIN is available to convert
Hours from BCD to Binary format
-  * @rmtoll TSTR         HT            LL_RTC_TS_GetHour\n
-  *         TSTR         HU            LL_RTC_TS_GetHour
-  * @param  RTCx RTC Instance
-  * @retval Value between Min_Data=0x01 and Max_Data=0x12 or between
Min_Data=0x00 and Max_Data=0x23
-  */
-__STATIC_INLINE uint32_t LL_RTC_TS_GetHour(RTC_TypeDef *RTCx)
-{
-  return (uint32_t)(READ_BIT(RTCx->TSTR, RTC_TSTR_HT | RTC_TSTR_HU) >>
RTC_TSTR_HU_Pos);
-}
-
-/**
-  * @brief  Get Timestamp Minutes in BCD format
-  * @note helper macro __LL_RTC_CONVERT_BCD2BIN is available to convert
Minutes from BCD to Binary format
-  * @rmtoll TSTR         MNT           LL_RTC_TS_GetMinute\n
-  *         TSTR         MNU           LL_RTC_TS_GetMinute
-  * @param  RTCx RTC Instance
-  * @retval Value between Min_Data=0x00 and Max_Data=0x59
-  */
-__STATIC_INLINE uint32_t LL_RTC_TS_GetMinute(RTC_TypeDef *RTCx)
-{
-  return (uint32_t)(READ_BIT(RTCx->TSTR, RTC_TSTR_MNT | RTC_TSTR_MNU) >>
RTC_TSTR_MNU_Pos);
-}
-
-/**
-  * @brief  Get Timestamp Seconds in BCD format
-  * @note helper macro __LL_RTC_CONVERT_BCD2BIN is available to convert
Seconds from BCD to Binary format
-  * @rmtoll TSTR         ST            LL_RTC_TS_GetSecond\n
-  *         TSTR         SU            LL_RTC_TS_GetSecond
-  * @param  RTCx RTC Instance
-  * @retval Value between Min_Data=0x00 and Max_Data=0x59
-  */
-__STATIC_INLINE uint32_t LL_RTC_TS_GetSecond(RTC_TypeDef *RTCx)
-{
-  return (uint32_t)(READ_BIT(RTCx->TSTR, RTC_TSTR_ST | RTC_TSTR_SU));
-}
-
-/**
-  * @brief  Get Timestamp time (hour, minute and second) in BCD format
-  * @note helper macros __LL_RTC_GET_HOUR, __LL_RTC_GET_MINUTE and
__LL_RTC_GET_SECOND
-  * are available to get independently each parameter.
-  * @rmtoll TSTR         HT            LL_RTC_TS_GetTime\n
-  *         TSTR         HU            LL_RTC_TS_GetTime\n
-  *         TSTR         MNT           LL_RTC_TS_GetTime\n
```

```
-  *         TSTR         MNU           LL_RTC_TS_GetTime\n
-  *         TSTR         ST            LL_RTC_TS_GetTime\n
-  *         TSTR         SU            LL_RTC_TS_GetTime
-  * @param  RTCx RTC Instance
-  * @retval Combination of hours, minutes and seconds.
-  */
-__STATIC_INLINE uint32_t LL_RTC_TS_GetTime(RTC_TypeDef *RTCx)
-{
-  return (uint32_t)(READ_BIT(RTCx->TSTR,
-                             RTC_TSTR_HT | RTC_TSTR_HU | RTC_TSTR_MNT |
RTC_TSTR_MNU | RTC_TSTR_ST | RTC_TSTR_SU));
-}
-
-/**
-  * @brief  Get Timestamp Week day
-  * @rmtoll TSDR         WDU           LL_RTC_TS_GetWeekDay
-  * @param  RTCx RTC Instance
-  * @retval Returned value can be one of the following values:
-  *         @arg @ref LL_RTC_WEEKDAY_MONDAY
-  *         @arg @ref LL_RTC_WEEKDAY_TUESDAY
-  *         @arg @ref LL_RTC_WEEKDAY_WEDNESDAY
-  *         @arg @ref LL_RTC_WEEKDAY_THURSDAY
-  *         @arg @ref LL_RTC_WEEKDAY_FRIDAY
-  *         @arg @ref LL_RTC_WEEKDAY_SATURDAY
-  *         @arg @ref LL_RTC_WEEKDAY_SUNDAY
-  */
-__STATIC_INLINE uint32_t LL_RTC_TS_GetWeekDay(RTC_TypeDef *RTCx)
-{
-  return (uint32_t)(READ_BIT(RTCx->TSDR, RTC_TSDR_WDU) >>
RTC_TSDR_WDU_Pos);
-}
-
-/**
-  * @brief  Get Timestamp Month in BCD format
-  * @note helper macro __LL_RTC_CONVERT_BCD2BIN is available to convert
Month from BCD to Binary format
-  * @rmtoll TSDR         MT            LL_RTC_TS_GetMonth\n
-  *         TSDR         MU            LL_RTC_TS_GetMonth
-  * @param  RTCx RTC Instance
-  * @retval Returned value can be one of the following values:
-  *         @arg @ref LL_RTC_MONTH_JANUARY
-  *         @arg @ref LL_RTC_MONTH_FEBRUARY
-  *         @arg @ref LL_RTC_MONTH_MARCH
-  *         @arg @ref LL_RTC_MONTH_APRIL
-  *         @arg @ref LL_RTC_MONTH_MAY
-  *         @arg @ref LL_RTC_MONTH_JUNE
-  *         @arg @ref LL_RTC_MONTH_JULY
-  *         @arg @ref LL_RTC_MONTH_AUGUST
-  *         @arg @ref LL_RTC_MONTH_SEPTEMBER
-  *         @arg @ref LL_RTC_MONTH_OCTOBER
-  *         @arg @ref LL_RTC_MONTH_NOVEMBER
-  *         @arg @ref LL_RTC_MONTH_DECEMBER
-  */
-__STATIC_INLINE uint32_t LL_RTC_TS_GetMonth(RTC_TypeDef *RTCx)
-{
-  return (uint32_t)(READ_BIT(RTCx->TSDR, RTC_TSDR_MT | RTC_TSDR_MU) >>
RTC_TSDR_MU_Pos);
-}
```

```
-
-/**
-  * @brief  Get Timestamp Day in BCD format
-  * @note helper macro __LL_RTC_CONVERT_BCD2BIN is available to convert
Day from BCD to Binary format
-  * @rmtoll TSDR         DT            LL_RTC_TS_GetDay\n
-  *         TSDR         DU            LL_RTC_TS_GetDay
-  * @param  RTCx RTC Instance
-  * @retval Value between Min_Data=0x01 and Max_Data=0x31
-  */
-__STATIC_INLINE uint32_t LL_RTC_TS_GetDay(RTC_TypeDef *RTCx)
-{
-  return (uint32_t)(READ_BIT(RTCx->TSDR, RTC_TSDR_DT | RTC_TSDR_DU));
-}
-
-/**
-  * @brief  Get Timestamp date (WeekDay, Day and Month) in BCD format
-  * @note helper macros __LL_RTC_GET_WEEKDAY, __LL_RTC_GET_MONTH,
-  * and __LL_RTC_GET_DAY are available to get independently each
parameter.
-  * @rmtoll TSDR         WDU           LL_RTC_TS_GetDate\n
-  *         TSDR         MT            LL_RTC_TS_GetDate\n
-  *         TSDR         MU            LL_RTC_TS_GetDate\n
-  *         TSDR         DT            LL_RTC_TS_GetDate\n
-  *         TSDR         DU            LL_RTC_TS_GetDate
-  * @param  RTCx RTC Instance
-  * @retval Combination of Weekday, Day and Month
-  */
-__STATIC_INLINE uint32_t LL_RTC_TS_GetDate(RTC_TypeDef *RTCx)
-{
-  return (uint32_t)(READ_BIT(RTCx->TSDR, RTC_TSDR_WDU | RTC_TSDR_MT |
RTC_TSDR_MU | RTC_TSDR_DT | RTC_TSDR_DU));
-}
-
-/**
-  * @brief  Get time-stamp subseconds value
-  * @rmtoll TSSSR        SS            LL_RTC_TS_GetSubSecond
-  * @param  RTCx RTC Instance
-  * @retval Value between Min_Data=0x00 and Max_Data=0xFFFF
-  */
-__STATIC_INLINE uint32_t LL_RTC_TS_GetSubSecond(RTC_TypeDef *RTCx)
-{
-  return (uint32_t)(READ_BIT(RTCx->TSSSR, RTC_TSSSR_SS));
-}
-
-#if defined(RTC_TAFCR_TAMPTS)
-/**
-  * @brief  Activate timestamp on tamper detection event
-  * @rmtoll TAFCR       TAMPTS        LL_RTC_TS_EnableOnTamper
-  * @param  RTCx RTC Instance
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_TS_EnableOnTamper(RTC_TypeDef *RTCx)
-{
-  SET_BIT(RTCx->TAFCR, RTC_TAFCR_TAMPTS);
-}
-
-/**
```

```
-  * @brief  Disable timestamp on tamper detection event
-  * @rmtoll TAFCR       TAMPTS        LL_RTC_TS_DisableOnTamper
-  * @param  RTCx RTC Instance
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_TS_DisableOnTamper(RTC_TypeDef *RTCx)
-{
-  CLEAR_BIT(RTCx->TAFCR, RTC_TAFCR_TAMPTS);
-}
-#endif /* RTC_TAFCR_TAMPTS */
-
-/**
-  * @brief  Set timestamp Pin
-  * @rmtoll TAFCR       TSINSEL      LL_RTC_TS_SetPin
-  * @param  RTCx RTC Instance
-  * @param  TSPin specifies the RTC Timestamp Pin.
-  *          This parameter can be one of the following values:
-  *            @arg LL_RTC_TimeStampPin_Default: RTC_AF1 is used as RTC
Timestamp Pin.
-  *            @arg LL_RTC_TimeStampPin_Pos1: RTC_AF2 is used as RTC
Timestamp Pin. (*)
-  *
-  *            (*) value not applicable to all devices.
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_TS_SetPin(RTC_TypeDef *RTCx, uint32_t TSPin)
-{
-  MODIFY_REG(RTCx->TAFCR, RTC_TAFCR_TSINSEL, TSPin);
-}
-
-/**
-  * @brief  Get timestamp Pin
-  * @rmtoll TAFCR       TSINSEL      LL_RTC_TS_GetPin
-  * @param  RTCx RTC Instance
-  * @retval Returned value can be one of the following values:
-  *            @arg LL_RTC_TimeStampPin_Default: RTC_AF1 is used as RTC
Timestamp Pin.
-  *            @arg LL_RTC_TimeStampPin_Pos1: RTC_AF2 is used as RTC
Timestamp Pin. (*)
-  *
-  *            (*) value not applicable to all devices.
-  * @retval None
-  */
-__STATIC_INLINE uint32_t LL_RTC_TS_GetPin(RTC_TypeDef *RTCx)
-{
-  return (uint32_t)(READ_BIT(RTCx->TAFCR, RTC_TAFCR_TSINSEL));
-}
-
-/**
-  * @}
-  */
-
-/** @defgroup RTC_LL_EF_Tamper Tamper
-  * @{
-  */
-
-/**
-  * @brief  Enable RTC_TAMPx input detection
```

```
-  * @rmtoll TAFCR       TAMP1E        LL_RTC_TAMPER_Enable\n
-  *         TAFCR       TAMP2E        LL_RTC_TAMPER_Enable\n
-  * @param  RTCx RTC Instance
-  * @param  Tamper This parameter can be a combination of the following
values:
-  *         @arg @ref LL_RTC_TAMPER_1
-  *         @arg @ref LL_RTC_TAMPER_2 (*)
-  *
-  *         (*) value not applicable to all devices.
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_TAMPER_Enable(RTC_TypeDef *RTCx, uint32_t
Tamper)
-{
-  SET_BIT(RTCx->TAFCR, Tamper);
-}
-
-/**
-  * @brief  Clear RTC_TAMPx input detection
-  * @rmtoll TAFCR       TAMP1E        LL_RTC_TAMPER_Disable\n
-  *         TAFCR       TAMP2E        LL_RTC_TAMPER_Disable\n
-  * @param  RTCx RTC Instance
-  * @param  Tamper This parameter can be a combination of the following
values:
-  *         @arg @ref LL_RTC_TAMPER_1
-  *         @arg @ref LL_RTC_TAMPER_2 (*)
-  *
-  *         (*) value not applicable to all devices.
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_TAMPER_Disable(RTC_TypeDef *RTCx, uint32_t
Tamper)
-{
-  CLEAR_BIT(RTCx->TAFCR, Tamper);
-}
-
-/**
-  * @brief  Disable RTC_TAMPx pull-up disable (Disable precharge of
RTC_TAMPx pins)
-  * @rmtoll TAFCR       TAMPPUDIS     LL_RTC_TAMPER_DisablePullUp
-  * @param  RTCx RTC Instance
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_TAMPER_DisablePullUp(RTC_TypeDef *RTCx)
-{
-  SET_BIT(RTCx->TAFCR, RTC_TAFCR_TAMPPUDIS);
-}
-
-/**
-  * @brief  Enable RTC_TAMPx pull-up disable ( Precharge RTC_TAMPx pins
before sampling)
-  * @rmtoll TAFCR       TAMPPUDIS     LL_RTC_TAMPER_EnablePullUp
-  * @param  RTCx RTC Instance
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_TAMPER_EnablePullUp(RTC_TypeDef *RTCx)
-{
-  CLEAR_BIT(RTCx->TAFCR, RTC_TAFCR_TAMPPUDIS);
```

```
-}
-
-/**
-  * @brief  Set RTC_TAMPx precharge duration
-  * @rmtoll TAFCR       TAMPPRCH      LL_RTC_TAMPER_SetPrecharge
-  * @param  RTCx RTC Instance
-  * @param  Duration This parameter can be one of the following values:
-  *         @arg @ref LL_RTC_TAMPER_DURATION_1RTCCLK
-  *         @arg @ref LL_RTC_TAMPER_DURATION_2RTCCLK
-  *         @arg @ref LL_RTC_TAMPER_DURATION_4RTCCLK
-  *         @arg @ref LL_RTC_TAMPER_DURATION_8RTCCLK
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_TAMPER_SetPrecharge(RTC_TypeDef *RTCx,
uint32_t Duration)
-{
-  MODIFY_REG(RTCx->TAFCR, RTC_TAFCR_TAMPPRCH, Duration);
-}
-
-/**
-  * @brief  Get RTC_TAMPx precharge duration
-  * @rmtoll TAFCR       TAMPPRCH      LL_RTC_TAMPER_GetPrecharge
-  * @param  RTCx RTC Instance
-  * @retval Returned value can be one of the following values:
-  *         @arg @ref LL_RTC_TAMPER_DURATION_1RTCCLK
-  *         @arg @ref LL_RTC_TAMPER_DURATION_2RTCCLK
-  *         @arg @ref LL_RTC_TAMPER_DURATION_4RTCCLK
-  *         @arg @ref LL_RTC_TAMPER_DURATION_8RTCCLK
-  */
-__STATIC_INLINE uint32_t LL_RTC_TAMPER_GetPrecharge(RTC_TypeDef *RTCx)
-{
-  return (uint32_t)(READ_BIT(RTCx->TAFCR, RTC_TAFCR_TAMPPRCH));
-}
-
-/**
-  * @brief  Set RTC_TAMPx filter count
-  * @rmtoll TAFCR       TAMPFLT       LL_RTC_TAMPER_SetFilterCount
-  * @param  RTCx RTC Instance
-  * @param  FilterCount This parameter can be one of the following
values:
-  *         @arg @ref LL_RTC_TAMPER_FILTER_DISABLE
-  *         @arg @ref LL_RTC_TAMPER_FILTER_2SAMPLE
-  *         @arg @ref LL_RTC_TAMPER_FILTER_4SAMPLE
-  *         @arg @ref LL_RTC_TAMPER_FILTER_8SAMPLE
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_TAMPER_SetFilterCount(RTC_TypeDef *RTCx,
uint32_t FilterCount)
-{
-  MODIFY_REG(RTCx->TAFCR, RTC_TAFCR_TAMPFLT, FilterCount);
-}
-
-/**
-  * @brief  Get RTC_TAMPx filter count
-  * @rmtoll TAFCR       TAMPFLT       LL_RTC_TAMPER_GetFilterCount
-  * @param  RTCx RTC Instance
-  * @retval Returned value can be one of the following values:
-  *         @arg @ref LL_RTC_TAMPER_FILTER_DISABLE
```

```
-  *         @arg @ref LL_RTC_TAMPER_FILTER_2SAMPLE
-  *         @arg @ref LL_RTC_TAMPER_FILTER_4SAMPLE
-  *         @arg @ref LL_RTC_TAMPER_FILTER_8SAMPLE
-  */
-__STATIC_INLINE uint32_t LL_RTC_TAMPER_GetFilterCount(RTC_TypeDef *RTCx)
-{
-  return (uint32_t)(READ_BIT(RTCx->TAFCR, RTC_TAFCR_TAMPFLT));
-}
-
-/**
-  * @brief  Set Tamper sampling frequency
-  * @rmtoll TAFCR       TAMPFREQ      LL_RTC_TAMPER_SetSamplingFreq
-  * @param  RTCx RTC Instance
-  * @param  SamplingFreq This parameter can be one of the following
values:
-  *         @arg @ref LL_RTC_TAMPER_SAMPLFREQDIV_32768
-  *         @arg @ref LL_RTC_TAMPER_SAMPLFREQDIV_16384
-  *         @arg @ref LL_RTC_TAMPER_SAMPLFREQDIV_8192
-  *         @arg @ref LL_RTC_TAMPER_SAMPLFREQDIV_4096
-  *         @arg @ref LL_RTC_TAMPER_SAMPLFREQDIV_2048
-  *         @arg @ref LL_RTC_TAMPER_SAMPLFREQDIV_1024
-  *         @arg @ref LL_RTC_TAMPER_SAMPLFREQDIV_512
-  *         @arg @ref LL_RTC_TAMPER_SAMPLFREQDIV_256
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_TAMPER_SetSamplingFreq(RTC_TypeDef *RTCx,
uint32_t SamplingFreq)
-{
-  MODIFY_REG(RTCx->TAFCR, RTC_TAFCR_TAMPFREQ, SamplingFreq);
-}
-
-/**
-  * @brief  Get Tamper sampling frequency
-  * @rmtoll TAFCR       TAMPFREQ      LL_RTC_TAMPER_GetSamplingFreq
-  * @param  RTCx RTC Instance
-  * @retval Returned value can be one of the following values:
-  *         @arg @ref LL_RTC_TAMPER_SAMPLFREQDIV_32768
-  *         @arg @ref LL_RTC_TAMPER_SAMPLFREQDIV_16384
-  *         @arg @ref LL_RTC_TAMPER_SAMPLFREQDIV_8192
-  *         @arg @ref LL_RTC_TAMPER_SAMPLFREQDIV_4096
-  *         @arg @ref LL_RTC_TAMPER_SAMPLFREQDIV_2048
-  *         @arg @ref LL_RTC_TAMPER_SAMPLFREQDIV_1024
-  *         @arg @ref LL_RTC_TAMPER_SAMPLFREQDIV_512
-  *         @arg @ref LL_RTC_TAMPER_SAMPLFREQDIV_256
-  */
-__STATIC_INLINE uint32_t LL_RTC_TAMPER_GetSamplingFreq(RTC_TypeDef
*RTCx)
-{
-  return (uint32_t)(READ_BIT(RTCx->TAFCR, RTC_TAFCR_TAMPFREQ));
-}
-
-/**
-  * @brief  Enable Active level for Tamper input
-  * @rmtoll TAFCR       TAMP1TRG      LL_RTC_TAMPER_EnableActiveLevel\n
-  *         TAFCR       TAMP2TRG      LL_RTC_TAMPER_EnableActiveLevel\n
-  * @param  RTCx RTC Instance
-  * @param  Tamper This parameter can be a combination of the following
values:
```

```
-  *         @arg @ref LL_RTC_TAMPER_ACTIVELEVEL_TAMP1
-  *         @arg @ref LL_RTC_TAMPER_ACTIVELEVEL_TAMP2 (*)
-  *
-  *         (*) value not applicable to all devices.
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_TAMPER_EnableActiveLevel(RTC_TypeDef *RTCx,
uint32_t Tamper)
-{
-  SET_BIT(RTCx->TAFCR, Tamper);
-}
-
-/**
-  * @brief  Disable Active level for Tamper input
-  * @rmtoll TAFCR       TAMP1TRG      LL_RTC_TAMPER_DisableActiveLevel\n
-  *         TAFCR       TAMP2TRG      LL_RTC_TAMPER_DisableActiveLevel\n
-  * @param  RTCx RTC Instance
-  * @param  Tamper This parameter can be a combination of the following
values:
-  *         @arg @ref LL_RTC_TAMPER_ACTIVELEVEL_TAMP1
-  *         @arg @ref LL_RTC_TAMPER_ACTIVELEVEL_TAMP2 (*)
-  *
-  *         (*) value not applicable to all devices.
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_TAMPER_DisableActiveLevel(RTC_TypeDef *RTCx,
uint32_t Tamper)
-{
-  CLEAR_BIT(RTCx->TAFCR, Tamper);
-}
-
-/**
-  * @brief  Set Tamper Pin
-  * @rmtoll TAFCR       TAMP1INSEL      LL_RTC_TAMPER_SetPin
-  * @param  RTCx RTC Instance
-  * @param  TamperPin specifies the RTC Tamper Pin.
-  *          This parameter can be one of the following values:
-  *            @arg LL_RTC_TamperPin_Default: RTC_AF1 is used as RTC
Tamper Pin.
-  *            @arg LL_RTC_TamperPin_Pos1: RTC_AF2 is used as RTC Tamper
Pin. (*)
-  *
-  *            (*) value not applicable to all devices.
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_TAMPER_SetPin(RTC_TypeDef *RTCx, uint32_t
TamperPin)
-{
-  MODIFY_REG(RTCx->TAFCR, RTC_TAFCR_TAMP1INSEL, TamperPin);
-}
-
-/**
-  * @brief  Get Tamper Pin
-  * @rmtoll TAFCR       TAMP1INSEL      LL_RTC_TAMPER_GetPin
-  * @param  RTCx RTC Instance
-  * @retval Returned value can be one of the following values:
-  *            @arg LL_RTC_TamperPin_Default: RTC_AF1 is used as RTC
Tamper Pin.
```

```
-  *            @arg LL_RTC_TamperPin_Pos1: RTC_AF2 is selected as RTC
Tamper Pin. (*)
-  *
-  *            (*) value not applicable to all devices.
-  * @retval None
-  */
-
-__STATIC_INLINE uint32_t LL_RTC_TAMPER_GetPin(RTC_TypeDef *RTCx)
-{
-  return (uint32_t)(READ_BIT(RTCx->TAFCR, RTC_TAFCR_TAMP1INSEL));
-}
-
-/**
-  * @}
-  */
-
-/** @defgroup RTC_LL_EF_Wakeup Wakeup
-  * @{
-  */
-
-/**
-  * @brief  Enable Wakeup timer
-  * @note   Bit is write-protected. @ref LL_RTC_DisableWriteProtection
function should be called before.
-  * @rmtoll CR           WUTE          LL_RTC_WAKEUP_Enable
-  * @param  RTCx RTC Instance
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_WAKEUP_Enable(RTC_TypeDef *RTCx)
-{
-  SET_BIT(RTCx->CR, RTC_CR_WUTE);
-}
-
-/**
-  * @brief  Disable Wakeup timer
-  * @note   Bit is write-protected. @ref LL_RTC_DisableWriteProtection
function should be called before.
-  * @rmtoll CR           WUTE          LL_RTC_WAKEUP_Disable
-  * @param  RTCx RTC Instance
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_WAKEUP_Disable(RTC_TypeDef *RTCx)
-{
-  CLEAR_BIT(RTCx->CR, RTC_CR_WUTE);
-}
-
-/**
-  * @brief  Check if Wakeup timer is enabled or not
-  * @rmtoll CR           WUTE          LL_RTC_WAKEUP_IsEnabled
-  * @param  RTCx RTC Instance
-  * @retval State of bit (1 or 0).
-  */
-__STATIC_INLINE uint32_t LL_RTC_WAKEUP_IsEnabled(RTC_TypeDef *RTCx)
-{
-  return ((READ_BIT(RTCx->CR, RTC_CR_WUTE) == (RTC_CR_WUTE)) ? 1UL :
0UL);
-}
-
```

```
-/**
-  * @brief  Select Wakeup clock
-  * @note   Bit is write-protected. @ref LL_RTC_DisableWriteProtection
function should be called before.
-  * @note   Bit can be written only when RTC_CR WUTE bit = 0 and RTC_ISR
WUTWF bit = 1
-  * @rmtoll CR           WUCKSEL       LL_RTC_WAKEUP_SetClock
-  * @param  RTCx RTC Instance
-  * @param  WakeupClock This parameter can be one of the following
values:
-  *         @arg @ref LL_RTC_WAKEUPCLOCK_DIV_16
-  *         @arg @ref LL_RTC_WAKEUPCLOCK_DIV_8
-  *         @arg @ref LL_RTC_WAKEUPCLOCK_DIV_4
-  *         @arg @ref LL_RTC_WAKEUPCLOCK_DIV_2
-  *         @arg @ref LL_RTC_WAKEUPCLOCK_CKSPRE
-  *         @arg @ref LL_RTC_WAKEUPCLOCK_CKSPRE_WUT
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_WAKEUP_SetClock(RTC_TypeDef *RTCx, uint32_t
WakeupClock)
-{
-  MODIFY_REG(RTCx->CR, RTC_CR_WUCKSEL, WakeupClock);
-}
-
-/**
-  * @brief  Get Wakeup clock
-  * @rmtoll CR           WUCKSEL       LL_RTC_WAKEUP_GetClock
-  * @param  RTCx RTC Instance
-  * @retval Returned value can be one of the following values:
-  *         @arg @ref LL_RTC_WAKEUPCLOCK_DIV_16
-  *         @arg @ref LL_RTC_WAKEUPCLOCK_DIV_8
-  *         @arg @ref LL_RTC_WAKEUPCLOCK_DIV_4
-  *         @arg @ref LL_RTC_WAKEUPCLOCK_DIV_2
-  *         @arg @ref LL_RTC_WAKEUPCLOCK_CKSPRE
-  *         @arg @ref LL_RTC_WAKEUPCLOCK_CKSPRE_WUT
-  */
-__STATIC_INLINE uint32_t LL_RTC_WAKEUP_GetClock(RTC_TypeDef *RTCx)
-{
-  return (uint32_t)(READ_BIT(RTCx->CR, RTC_CR_WUCKSEL));
-}
-
-/**
-  * @brief  Set Wakeup auto-reload value
-  * @note   Bit can be written only when WUTWF is set to 1 in RTC_ISR
-  * @rmtoll WUTR         WUT           LL_RTC_WAKEUP_SetAutoReload
-  * @param  RTCx RTC Instance
-  * @param  Value Value between Min_Data=0x00 and Max_Data=0xFFFF
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_WAKEUP_SetAutoReload(RTC_TypeDef *RTCx,
uint32_t Value)
-{
-  MODIFY_REG(RTCx->WUTR, RTC_WUTR_WUT, Value);
-}
-
-/**
-  * @brief  Get Wakeup auto-reload value
-  * @rmtoll WUTR         WUT           LL_RTC_WAKEUP_GetAutoReload
```

```
-  * @param  RTCx RTC Instance
-  * @retval Value between Min_Data=0x00 and Max_Data=0xFFFF
-  */
-__STATIC_INLINE uint32_t LL_RTC_WAKEUP_GetAutoReload(RTC_TypeDef *RTCx)
-{
-  return (uint32_t)(READ_BIT(RTCx->WUTR, RTC_WUTR_WUT));
-}
-
-/**
-  * @}
-  */
-
-/** @defgroup RTC_LL_EF_Backup_Registers Backup_Registers
-  * @{
-  */
-
-/**
-  * @brief  Writes a data in a specified RTC Backup data register.
-  * @rmtoll BKPxR        BKP           LL_RTC_BAK_SetRegister
-  * @param  RTCx RTC Instance
-  * @param  BackupRegister This parameter can be one of the following
values:
-  *         @arg @ref LL_RTC_BKP_DR0
-  *         @arg @ref LL_RTC_BKP_DR1
-  *         @arg @ref LL_RTC_BKP_DR2
-  *         @arg @ref LL_RTC_BKP_DR3
-  *         @arg @ref LL_RTC_BKP_DR4
-  *         @arg @ref LL_RTC_BKP_DR5
-  *         @arg @ref LL_RTC_BKP_DR6
-  *         @arg @ref LL_RTC_BKP_DR7
-  *         @arg @ref LL_RTC_BKP_DR8
-  *         @arg @ref LL_RTC_BKP_DR9
-  *         @arg @ref LL_RTC_BKP_DR10
-  *         @arg @ref LL_RTC_BKP_DR11
-  *         @arg @ref LL_RTC_BKP_DR12
-  *         @arg @ref LL_RTC_BKP_DR13
-  *         @arg @ref LL_RTC_BKP_DR14
-  *         @arg @ref LL_RTC_BKP_DR15
-  *         @arg @ref LL_RTC_BKP_DR16
-  *         @arg @ref LL_RTC_BKP_DR17
-  *         @arg @ref LL_RTC_BKP_DR18
-  *         @arg @ref LL_RTC_BKP_DR19
-  * @param  Data Value between Min_Data=0x00 and Max_Data=0xFFFFFFFF
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_BAK_SetRegister(RTC_TypeDef *RTCx, uint32_t
BackupRegister, uint32_t Data)
-{
-  uint32_t temp;
-
-  temp = (uint32_t)(&(RTCx->BKP0R));
-  temp += (BackupRegister * 4U);
-
-  /* Write the specified register */
-  *(__IO uint32_t *)temp = (uint32_t)Data;
-}
-
-/**
```

```
-  * @brief  Reads data from the specified RTC Backup data Register.
-  * @rmtoll BKPxR        BKP           LL_RTC_BAK_GetRegister
-  * @param  RTCx RTC Instance
-  * @param  BackupRegister This parameter can be one of the following
values:
-  *         @arg @ref LL_RTC_BKP_DR0
-  *         @arg @ref LL_RTC_BKP_DR1
-  *         @arg @ref LL_RTC_BKP_DR2
-  *         @arg @ref LL_RTC_BKP_DR3
-  *         @arg @ref LL_RTC_BKP_DR4
-  *         @arg @ref LL_RTC_BKP_DR5
-  *         @arg @ref LL_RTC_BKP_DR6
-  *         @arg @ref LL_RTC_BKP_DR7
-  *         @arg @ref LL_RTC_BKP_DR8
-  *         @arg @ref LL_RTC_BKP_DR9
-  *         @arg @ref LL_RTC_BKP_DR10
-  *         @arg @ref LL_RTC_BKP_DR11
-  *         @arg @ref LL_RTC_BKP_DR12
-  *         @arg @ref LL_RTC_BKP_DR13
-  *         @arg @ref LL_RTC_BKP_DR14
-  *         @arg @ref LL_RTC_BKP_DR15
-  *         @arg @ref LL_RTC_BKP_DR16
-  *         @arg @ref LL_RTC_BKP_DR17
-  *         @arg @ref LL_RTC_BKP_DR18
-  *         @arg @ref LL_RTC_BKP_DR19
-  * @retval Value between Min_Data=0x00 and Max_Data=0xFFFFFFFF
-  */
-__STATIC_INLINE uint32_t LL_RTC_BAK_GetRegister(RTC_TypeDef *RTCx,
uint32_t BackupRegister)
-{
-  uint32_t temp;
-
-  temp = (uint32_t)(&(RTCx->BKP0R));
-  temp += (BackupRegister * 4U);
-
-  /* Read the specified register */
-  return (*(__IO uint32_t *)temp);
-}
-
-/**
-  * @}
-  */
-
-/** @defgroup RTC_LL_EF_Calibration Calibration
-  * @{
-  */
-
-/**
-  * @brief  Set Calibration output frequency (1 Hz or 512 Hz)
-  * @note Bits are write-protected. @ref LL_RTC_DisableWriteProtection
function should be called before.
-  * @rmtoll CR           COE           LL_RTC_CAL_SetOutputFreq\n
-  *         CR           COSEL         LL_RTC_CAL_SetOutputFreq
-  * @param  RTCx RTC Instance
-  * @param  Frequency This parameter can be one of the following values:
-  *         @arg @ref LL_RTC_CALIB_OUTPUT_NONE
-  *         @arg @ref LL_RTC_CALIB_OUTPUT_1HZ
-  *         @arg @ref LL_RTC_CALIB_OUTPUT_512HZ
```

```
-  *
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_CAL_SetOutputFreq(RTC_TypeDef *RTCx,
uint32_t Frequency)
-{
-  MODIFY_REG(RTCx->CR, RTC_CR_COE | RTC_CR_COSEL, Frequency);
-}
-
-/**
-  * @brief  Get Calibration output frequency (1 Hz or 512 Hz)
-  * @rmtoll CR           COE           LL_RTC_CAL_GetOutputFreq\n
-  *         CR           COSEL         LL_RTC_CAL_GetOutputFreq
-  * @param  RTCx RTC Instance
-  * @retval Returned value can be one of the following values:
-  *         @arg @ref LL_RTC_CALIB_OUTPUT_NONE
-  *         @arg @ref LL_RTC_CALIB_OUTPUT_1HZ
-  *         @arg @ref LL_RTC_CALIB_OUTPUT_512HZ
-  *
-  */
-__STATIC_INLINE uint32_t LL_RTC_CAL_GetOutputFreq(RTC_TypeDef *RTCx)
-{
-  return (uint32_t)(READ_BIT(RTCx->CR, RTC_CR_COE | RTC_CR_COSEL));
-}
-
-/**
-  * @brief  Enable Coarse digital calibration
-  * @note   Bit is write-protected. @ref LL_RTC_DisableWriteProtection
function should be called before.
-  * @note   It can be written in initialization mode only (@ref
LL_RTC_EnableInitMode function)
-  * @rmtoll CR           DCE           LL_RTC_CAL_EnableCoarseDigital
-  * @param  RTCx RTC Instance
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_CAL_EnableCoarseDigital(RTC_TypeDef *RTCx)
-{
-  SET_BIT(RTCx->CR, RTC_CR_DCE);
-}
-
-/**
-  * @brief  Disable Coarse digital calibration
-  * @note   Bit is write-protected. @ref LL_RTC_DisableWriteProtection
function should be called before.
-  * @note   It can be written in initialization mode only (@ref
LL_RTC_EnableInitMode function)
-  * @rmtoll CR           DCE           LL_RTC_CAL_DisableCoarseDigital
-  * @param  RTCx RTC Instance
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_CAL_DisableCoarseDigital(RTC_TypeDef  *RTCx)
-{
-  CLEAR_BIT(RTCx->CR, RTC_CR_DCE);
-}
-
-/**
-  * @brief  Set the coarse digital calibration
```

```
-  * @note   Bit is write-protected. @ref LL_RTC_DisableWriteProtection
function should be called before.
-  * @note   It can be written in initialization mode only (@ref
LL_RTC_EnableInitMode function)
-  * @rmtoll CALIBR       DCS           LL_RTC_CAL_ConfigCoarseDigital\n
-  *         CALIBR       DC            LL_RTC_CAL_ConfigCoarseDigital
-  * @param  RTCx RTC Instance
-  * @param  Sign This parameter can be one of the following values:
-  *         @arg @ref LL_RTC_CALIB_SIGN_POSITIVE
-  *         @arg @ref LL_RTC_CALIB_SIGN_NEGATIVE
-  * @param  Value value of coarse calibration expressed in ppm (coded on
5 bits)
-  * @note   This Calibration value should be between 0 and 63 when using
negative sign with a 2-ppm step.
-  * @note   This Calibration value should be between 0 and 126 when
using positive sign with a 4-ppm step.
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_CAL_ConfigCoarseDigital(RTC_TypeDef *RTCx,
uint32_t Sign, uint32_t Value)
-{
-  MODIFY_REG(RTCx->CALIBR, RTC_CALIBR_DCS | RTC_CALIBR_DC, Sign |
Value);
-}
-
-/**
-  * @brief  Get the coarse digital calibration value
-  * @rmtoll CALIBR       DC            LL_RTC_CAL_GetCoarseDigitalValue
-  * @param  RTCx RTC Instance
-  * @retval value of coarse calibration expressed in ppm (coded on 5
bits)
-  */
-__STATIC_INLINE uint32_t LL_RTC_CAL_GetCoarseDigitalValue(RTC_TypeDef
*RTCx)
-{
-  return (uint32_t)(READ_BIT(RTCx->CALIBR, RTC_CALIBR_DC));
-}
-
-/**
-  * @brief  Get the coarse digital calibration sign
-  * @rmtoll CALIBR       DCS           LL_RTC_CAL_GetCoarseDigitalSign
-  * @param  RTCx RTC Instance
-  * @retval Returned value can be one of the following values:
-  *         @arg @ref LL_RTC_CALIB_SIGN_POSITIVE
-  *         @arg @ref LL_RTC_CALIB_SIGN_NEGATIVE
-  */
-__STATIC_INLINE uint32_t LL_RTC_CAL_GetCoarseDigitalSign(RTC_TypeDef
*RTCx)
-{
-  return (uint32_t)(READ_BIT(RTCx->CALIBR, RTC_CALIBR_DCS));
-}
-
-/**
-  * @brief  Insert or not One RTCCLK pulse every 2exp11 pulses
(frequency increased by 488.5 ppm)
-  * @note   Bit is write-protected. @ref LL_RTC_DisableWriteProtection
function should be called before.
-  * @note   Bit can be written only when RECALPF is set to 0 in RTC_ISR
```

```
-  * @rmtoll CALR         CALP          LL_RTC_CAL_SetPulse
-  * @param  RTCx RTC Instance
-  * @param  Pulse This parameter can be one of the following values:
-  *         @arg @ref LL_RTC_CALIB_INSERTPULSE_NONE
-  *         @arg @ref LL_RTC_CALIB_INSERTPULSE_SET
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_CAL_SetPulse(RTC_TypeDef *RTCx, uint32_t
Pulse)
-{
-  MODIFY_REG(RTCx->CALR, RTC_CALR_CALP, Pulse);
-}
-
-/**
-  * @brief  Check if one RTCCLK has been inserted or not every 2exp11
pulses (frequency increased by 488.5 ppm)
-  * @rmtoll CALR         CALP          LL_RTC_CAL_IsPulseInserted
-  * @param  RTCx RTC Instance
-  * @retval State of bit (1 or 0).
-  */
-__STATIC_INLINE uint32_t LL_RTC_CAL_IsPulseInserted(RTC_TypeDef *RTCx)
-{
-  return ((READ_BIT(RTCx->CALR, RTC_CALR_CALP) == (RTC_CALR_CALP)) ? 1UL
: 0UL);
-}
-
-/**
-  * @brief  Set smooth calibration cycle period
-  * @note   Bit is write-protected. @ref LL_RTC_DisableWriteProtection
function should be called before.
-  * @note   Bit can be written only when RECALPF is set to 0 in RTC_ISR
-  * @rmtoll CALR         CALW8         LL_RTC_CAL_SetPeriod\n
-  *         CALR         CALW16        LL_RTC_CAL_SetPeriod
-  * @param  RTCx RTC Instance
-  * @param  Period This parameter can be one of the following values:
-  *         @arg @ref LL_RTC_CALIB_PERIOD_32SEC
-  *         @arg @ref LL_RTC_CALIB_PERIOD_16SEC
-  *         @arg @ref LL_RTC_CALIB_PERIOD_8SEC
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_CAL_SetPeriod(RTC_TypeDef *RTCx, uint32_t
Period)
-{
-  MODIFY_REG(RTCx->CALR, RTC_CALR_CALW8 | RTC_CALR_CALW16, Period);
-}
-
-/**
-  * @brief  Get smooth calibration cycle period
-  * @rmtoll CALR         CALW8         LL_RTC_CAL_GetPeriod\n
-  *         CALR         CALW16        LL_RTC_CAL_GetPeriod
-  * @param  RTCx RTC Instance
-  * @retval Returned value can be one of the following values:
-  *         @arg @ref LL_RTC_CALIB_PERIOD_32SEC
-  *         @arg @ref LL_RTC_CALIB_PERIOD_16SEC
-  *         @arg @ref LL_RTC_CALIB_PERIOD_8SEC
-  */
-__STATIC_INLINE uint32_t LL_RTC_CAL_GetPeriod(RTC_TypeDef *RTCx)
-{
```

```
-  return (uint32_t)(READ_BIT(RTCx->CALR, RTC_CALR_CALW8 |
RTC_CALR_CALW16));
-}
-
-/**
-  * @brief  Set smooth Calibration minus
-  * @note   Bit is write-protected. @ref LL_RTC_DisableWriteProtection
function should be called before.
-  * @note   Bit can be written only when RECALPF is set to 0 in RTC_ISR
-  * @rmtoll CALR         CALM          LL_RTC_CAL_SetMinus
-  * @param  RTCx RTC Instance
-  * @param  CalibMinus Value between Min_Data=0x00 and Max_Data=0x1FF
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_CAL_SetMinus(RTC_TypeDef *RTCx, uint32_t
CalibMinus)
-{
-  MODIFY_REG(RTCx->CALR, RTC_CALR_CALM, CalibMinus);
-}
-
-/**
-  * @brief  Get smooth Calibration minus
-  * @rmtoll CALR         CALM          LL_RTC_CAL_GetMinus
-  * @param  RTCx RTC Instance
-  * @retval Value between Min_Data=0x00 and Max_Data= 0x1FF
-  */
-__STATIC_INLINE uint32_t LL_RTC_CAL_GetMinus(RTC_TypeDef *RTCx)
-{
-  return (uint32_t)(READ_BIT(RTCx->CALR, RTC_CALR_CALM));
-}
-
-/**
-  * @}
-  */
-
-/** @defgroup RTC_LL_EF_FLAG_Management FLAG_Management
-  * @{
-  */
-
-/**
-  * @brief  Get Recalibration pending Flag
-  * @rmtoll ISR          RECALPF       LL_RTC_IsActiveFlag_RECALP
-  * @param  RTCx RTC Instance
-  * @retval State of bit (1 or 0).
-  */
-__STATIC_INLINE uint32_t LL_RTC_IsActiveFlag_RECALP(RTC_TypeDef *RTCx)
-{
-  return ((READ_BIT(RTCx->ISR, RTC_ISR_RECALPF) == (RTC_ISR_RECALPF)) ?
1UL : 0UL);
-}
-
-#if defined(RTC_TAMPER2_SUPPORT)
-/**
-  * @brief  Get RTC_TAMP2 detection flag
-  * @rmtoll ISR          TAMP2F        LL_RTC_IsActiveFlag_TAMP2
-  * @param  RTCx RTC Instance
-  * @retval State of bit (1 or 0).
-  */
```

```
-__STATIC_INLINE uint32_t LL_RTC_IsActiveFlag_TAMP2(RTC_TypeDef *RTCx)
-{
-  return ((READ_BIT(RTCx->ISR, RTC_ISR_TAMP2F) == (RTC_ISR_TAMP2F)) ?
1UL : 0UL);
-}
-#endif /* RTC_TAMPER2_SUPPORT */
-
-/**
-  * @brief  Get RTC_TAMP1 detection flag
-  * @rmtoll ISR          TAMP1F        LL_RTC_IsActiveFlag_TAMP1
-  * @param  RTCx RTC Instance
-  * @retval State of bit (1 or 0).
-  */
-__STATIC_INLINE uint32_t LL_RTC_IsActiveFlag_TAMP1(RTC_TypeDef *RTCx)
-{
-  return ((READ_BIT(RTCx->ISR, RTC_ISR_TAMP1F) == (RTC_ISR_TAMP1F)) ?
1UL : 0UL);
-}
-
-/**
-  * @brief  Get Time-stamp overflow flag
-  * @rmtoll ISR          TSOVF         LL_RTC_IsActiveFlag_TSOV
-  * @param  RTCx RTC Instance
-  * @retval State of bit (1 or 0).
-  */
-__STATIC_INLINE uint32_t LL_RTC_IsActiveFlag_TSOV(RTC_TypeDef *RTCx)
-{
-  return ((READ_BIT(RTCx->ISR, RTC_ISR_TSOVF) == (RTC_ISR_TSOVF)) ? 1UL
: 0UL);
-}
-
-/**
-  * @brief  Get Time-stamp flag
-  * @rmtoll ISR          TSF           LL_RTC_IsActiveFlag_TS
-  * @param  RTCx RTC Instance
-  * @retval State of bit (1 or 0).
-  */
-__STATIC_INLINE uint32_t LL_RTC_IsActiveFlag_TS(RTC_TypeDef *RTCx)
-{
-  return ((READ_BIT(RTCx->ISR, RTC_ISR_TSF) == (RTC_ISR_TSF)) ? 1UL :
0UL);
-}
-
-/**
-  * @brief  Get Wakeup timer flag
-  * @rmtoll ISR          WUTF          LL_RTC_IsActiveFlag_WUT
-  * @param  RTCx RTC Instance
-  * @retval State of bit (1 or 0).
-  */
-__STATIC_INLINE uint32_t LL_RTC_IsActiveFlag_WUT(RTC_TypeDef *RTCx)
-{
-  return ((READ_BIT(RTCx->ISR, RTC_ISR_WUTF) == (RTC_ISR_WUTF)) ? 1UL :
0UL);
-}
-
-/**
-  * @brief  Get Alarm B flag
-  * @rmtoll ISR          ALRBF         LL_RTC_IsActiveFlag_ALRB
```

```
-  * @param  RTCx RTC Instance
-  * @retval State of bit (1 or 0).
-  */
-__STATIC_INLINE uint32_t LL_RTC_IsActiveFlag_ALRB(RTC_TypeDef *RTCx)
-{
-  return ((READ_BIT(RTCx->ISR, RTC_ISR_ALRBF) == (RTC_ISR_ALRBF)) ? 1UL
: 0UL);
-}
-
-/**
-  * @brief  Get Alarm A flag
-  * @rmtoll ISR          ALRAF         LL_RTC_IsActiveFlag_ALRA
-  * @param  RTCx RTC Instance
-  * @retval State of bit (1 or 0).
-  */
-__STATIC_INLINE uint32_t LL_RTC_IsActiveFlag_ALRA(RTC_TypeDef *RTCx)
-{
-  return ((READ_BIT(RTCx->ISR, RTC_ISR_ALRAF) == (RTC_ISR_ALRAF)) ? 1UL
: 0UL);
-}
-
-#if defined(RTC_TAMPER2_SUPPORT)
-/**
-  * @brief  Clear RTC_TAMP2 detection flag
-  * @rmtoll ISR          TAMP2F        LL_RTC_ClearFlag_TAMP2
-  * @param  RTCx RTC Instance
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_ClearFlag_TAMP2(RTC_TypeDef *RTCx)
-{
-  WRITE_REG(RTCx->ISR, (~((RTC_ISR_TAMP2F | RTC_ISR_INIT) & 0x0000FFFFU)
| (RTCx->ISR & RTC_ISR_INIT)));
-}
-#endif /* RTC_TAMPER2_SUPPORT */
-
-/**
-  * @brief  Clear RTC_TAMP1 detection flag
-  * @rmtoll ISR          TAMP1F        LL_RTC_ClearFlag_TAMP1
-  * @param  RTCx RTC Instance
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_ClearFlag_TAMP1(RTC_TypeDef *RTCx)
-{
-  WRITE_REG(RTCx->ISR, (~((RTC_ISR_TAMP1F | RTC_ISR_INIT) & 0x0000FFFFU)
| (RTCx->ISR & RTC_ISR_INIT)));
-}
-
-/**
-  * @brief  Clear Time-stamp overflow flag
-  * @rmtoll ISR          TSOVF         LL_RTC_ClearFlag_TSOV
-  * @param  RTCx RTC Instance
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_ClearFlag_TSOV(RTC_TypeDef *RTCx)
-{
-  WRITE_REG(RTCx->ISR, (~((RTC_ISR_TSOVF | RTC_ISR_INIT) & 0x0000FFFFU)
| (RTCx->ISR & RTC_ISR_INIT)));
-}
```

```
-
-/**
-  * @brief  Clear Time-stamp flag
-  * @rmtoll ISR          TSF           LL_RTC_ClearFlag_TS
-  * @param  RTCx RTC Instance
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_ClearFlag_TS(RTC_TypeDef *RTCx)
-{
-  WRITE_REG(RTCx->ISR, (~((RTC_ISR_TSF | RTC_ISR_INIT) & 0x0000FFFFU) |
(RTCx->ISR & RTC_ISR_INIT)));
-}
-
-/**
-  * @brief  Clear Wakeup timer flag
-  * @rmtoll ISR          WUTF          LL_RTC_ClearFlag_WUT
-  * @param  RTCx RTC Instance
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_ClearFlag_WUT(RTC_TypeDef *RTCx)
-{
-  WRITE_REG(RTCx->ISR, (~((RTC_ISR_WUTF | RTC_ISR_INIT) & 0x0000FFFFU) |
(RTCx->ISR & RTC_ISR_INIT)));
-}
-
-/**
-  * @brief  Clear Alarm B flag
-  * @rmtoll ISR          ALRBF         LL_RTC_ClearFlag_ALRB
-  * @param  RTCx RTC Instance
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_ClearFlag_ALRB(RTC_TypeDef *RTCx)
-{
-  WRITE_REG(RTCx->ISR, (~((RTC_ISR_ALRBF | RTC_ISR_INIT) & 0x0000FFFFU)
| (RTCx->ISR & RTC_ISR_INIT)));
-}
-
-/**
-  * @brief  Clear Alarm A flag
-  * @rmtoll ISR          ALRAF         LL_RTC_ClearFlag_ALRA
-  * @param  RTCx RTC Instance
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_ClearFlag_ALRA(RTC_TypeDef *RTCx)
-{
-  WRITE_REG(RTCx->ISR, (~((RTC_ISR_ALRAF | RTC_ISR_INIT) & 0x0000FFFFU)
| (RTCx->ISR & RTC_ISR_INIT)));
-}
-
-/**
-  * @brief  Get Initialization flag
-  * @rmtoll ISR          INITF         LL_RTC_IsActiveFlag_INIT
-  * @param  RTCx RTC Instance
-  * @retval State of bit (1 or 0).
-  */
-__STATIC_INLINE uint32_t LL_RTC_IsActiveFlag_INIT(RTC_TypeDef *RTCx)
-{
```

```
-  return ((READ_BIT(RTCx->ISR, RTC_ISR_INITF) == (RTC_ISR_INITF)) ? 1UL
: 0UL);
-}
-
-/**
-  * @brief  Get Registers synchronization flag
-  * @rmtoll ISR          RSF           LL_RTC_IsActiveFlag_RS
-  * @param  RTCx RTC Instance
-  * @retval State of bit (1 or 0).
-  */
-__STATIC_INLINE uint32_t LL_RTC_IsActiveFlag_RS(RTC_TypeDef *RTCx)
-{
-  return ((READ_BIT(RTCx->ISR, RTC_ISR_RSF) == (RTC_ISR_RSF)) ? 1UL :
0UL);
-}
-
-/**
-  * @brief  Clear Registers synchronization flag
-  * @rmtoll ISR          RSF           LL_RTC_ClearFlag_RS
-  * @param  RTCx RTC Instance
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_ClearFlag_RS(RTC_TypeDef *RTCx)
-{
-  WRITE_REG(RTCx->ISR, (~((RTC_ISR_RSF | RTC_ISR_INIT) & 0x0000FFFFU) |
(RTCx->ISR & RTC_ISR_INIT)));
-}
-
-/**
-  * @brief  Get Initialization status flag
-  * @rmtoll ISR          INITS         LL_RTC_IsActiveFlag_INITS
-  * @param  RTCx RTC Instance
-  * @retval State of bit (1 or 0).
-  */
-__STATIC_INLINE uint32_t LL_RTC_IsActiveFlag_INITS(RTC_TypeDef *RTCx)
-{
-  return ((READ_BIT(RTCx->ISR, RTC_ISR_INITS) == (RTC_ISR_INITS)) ? 1UL
: 0UL);
-}
-
-/**
-  * @brief  Get Shift operation pending flag
-  * @rmtoll ISR          SHPF          LL_RTC_IsActiveFlag_SHP
-  * @param  RTCx RTC Instance
-  * @retval State of bit (1 or 0).
-  */
-__STATIC_INLINE uint32_t LL_RTC_IsActiveFlag_SHP(RTC_TypeDef *RTCx)
-{
-  return ((READ_BIT(RTCx->ISR, RTC_ISR_SHPF) == (RTC_ISR_SHPF)) ? 1UL :
0UL);
-}
-
-/**
-  * @brief  Get Wakeup timer write flag
-  * @rmtoll ISR          WUTWF         LL_RTC_IsActiveFlag_WUTW
-  * @param  RTCx RTC Instance
-  * @retval State of bit (1 or 0).
-  */
```

```
-__STATIC_INLINE uint32_t LL_RTC_IsActiveFlag_WUTW(RTC_TypeDef *RTCx)
-{
-  return ((READ_BIT(RTCx->ISR, RTC_ISR_WUTWF) == (RTC_ISR_WUTWF)) ? 1UL
: 0UL);
-}
-
-/**
-  * @brief  Get Alarm B write flag
-  * @rmtoll ISR          ALRBWF        LL_RTC_IsActiveFlag_ALRBW
-  * @param  RTCx RTC Instance
-  * @retval State of bit (1 or 0).
-  */
-__STATIC_INLINE uint32_t LL_RTC_IsActiveFlag_ALRBW(RTC_TypeDef *RTCx)
-{
-  return ((READ_BIT(RTCx->ISR, RTC_ISR_ALRBWF) == (RTC_ISR_ALRBWF)) ?
1UL : 0UL);
-}
-
-/**
-  * @brief  Get Alarm A write flag
-  * @rmtoll ISR          ALRAWF        LL_RTC_IsActiveFlag_ALRAW
-  * @param  RTCx RTC Instance
-  * @retval State of bit (1 or 0).
-  */
-__STATIC_INLINE uint32_t LL_RTC_IsActiveFlag_ALRAW(RTC_TypeDef *RTCx)
-{
-  return ((READ_BIT(RTCx->ISR, RTC_ISR_ALRAWF) == (RTC_ISR_ALRAWF)) ?
1UL : 0UL);
-}
-
-/**
-  * @}
-  */
-
-/** @defgroup RTC_LL_EF_IT_Management IT_Management
-  * @{
-  */
-
-/**
-  * @brief  Enable Time-stamp interrupt
-  * @note   Bit is write-protected. @ref LL_RTC_DisableWriteProtection
function should be called before.
-  * @rmtoll CR           TSIE          LL_RTC_EnableIT_TS
-  * @param  RTCx RTC Instance
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_EnableIT_TS(RTC_TypeDef *RTCx)
-{
-  SET_BIT(RTCx->CR, RTC_CR_TSIE);
-}
-
-/**
-  * @brief  Disable Time-stamp interrupt
-  * @note   Bit is write-protected. @ref LL_RTC_DisableWriteProtection
function should be called before.
-  * @rmtoll CR           TSIE          LL_RTC_DisableIT_TS
-  * @param  RTCx RTC Instance
-  * @retval None
```

```
-  */
-__STATIC_INLINE void LL_RTC_DisableIT_TS(RTC_TypeDef *RTCx)
-{
-  CLEAR_BIT(RTCx->CR, RTC_CR_TSIE);
-}
-
-/**
-  * @brief  Enable Wakeup timer interrupt
-  * @note   Bit is write-protected. @ref LL_RTC_DisableWriteProtection
function should be called before.
-  * @rmtoll CR           WUTIE         LL_RTC_EnableIT_WUT
-  * @param  RTCx RTC Instance
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_EnableIT_WUT(RTC_TypeDef *RTCx)
-{
-  SET_BIT(RTCx->CR, RTC_CR_WUTIE);
-}
-
-/**
-  * @brief  Disable Wakeup timer interrupt
-  * @note   Bit is write-protected. @ref LL_RTC_DisableWriteProtection
function should be called before.
-  * @rmtoll CR           WUTIE         LL_RTC_DisableIT_WUT
-  * @param  RTCx RTC Instance
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_DisableIT_WUT(RTC_TypeDef *RTCx)
-{
-  CLEAR_BIT(RTCx->CR, RTC_CR_WUTIE);
-}
-
-/**
-  * @brief  Enable Alarm B interrupt
-  * @note   Bit is write-protected. @ref LL_RTC_DisableWriteProtection
function should be called before.
-  * @rmtoll CR           ALRBIE        LL_RTC_EnableIT_ALRB
-  * @param  RTCx RTC Instance
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_EnableIT_ALRB(RTC_TypeDef *RTCx)
-{
-  SET_BIT(RTCx->CR, RTC_CR_ALRBIE);
-}
-
-/**
-  * @brief  Disable Alarm B interrupt
-  * @note   Bit is write-protected. @ref LL_RTC_DisableWriteProtection
function should be called before.
-  * @rmtoll CR           ALRBIE        LL_RTC_DisableIT_ALRB
-  * @param  RTCx RTC Instance
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_DisableIT_ALRB(RTC_TypeDef *RTCx)
-{
-  CLEAR_BIT(RTCx->CR, RTC_CR_ALRBIE);
-}
-
```

```
-/**
-  * @brief  Enable Alarm A interrupt
-  * @note   Bit is write-protected. @ref LL_RTC_DisableWriteProtection
function should be called before.
-  * @rmtoll CR           ALRAIE        LL_RTC_EnableIT_ALRA
-  * @param  RTCx RTC Instance
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_EnableIT_ALRA(RTC_TypeDef *RTCx)
-{
-  SET_BIT(RTCx->CR, RTC_CR_ALRAIE);
-}
-
-/**
-  * @brief  Disable Alarm A interrupt
-  * @note   Bit is write-protected. @ref LL_RTC_DisableWriteProtection
function should be called before.
-  * @rmtoll CR           ALRAIE        LL_RTC_DisableIT_ALRA
-  * @param  RTCx RTC Instance
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_DisableIT_ALRA(RTC_TypeDef *RTCx)
-{
-  CLEAR_BIT(RTCx->CR, RTC_CR_ALRAIE);
-}
-
-/**
-  * @brief  Enable all Tamper Interrupt
-  * @rmtoll TAFCR       TAMPIE        LL_RTC_EnableIT_TAMP
-  * @param  RTCx RTC Instance
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_EnableIT_TAMP(RTC_TypeDef *RTCx)
-{
-  SET_BIT(RTCx->TAFCR, RTC_TAFCR_TAMPIE);
-}
-
-/**
-  * @brief  Disable all Tamper Interrupt
-  * @rmtoll TAFCR       TAMPIE        LL_RTC_DisableIT_TAMP
-  * @param  RTCx RTC Instance
-  * @retval None
-  */
-__STATIC_INLINE void LL_RTC_DisableIT_TAMP(RTC_TypeDef *RTCx)
-{
-  CLEAR_BIT(RTCx->TAFCR, RTC_TAFCR_TAMPIE);
-}
-
-/**
-  * @brief  Check if  Time-stamp interrupt is enabled or not
-  * @rmtoll CR           TSIE          LL_RTC_IsEnabledIT_TS
-  * @param  RTCx RTC Instance
-  * @retval State of bit (1 or 0).
-  */
-__STATIC_INLINE uint32_t LL_RTC_IsEnabledIT_TS(RTC_TypeDef *RTCx)
-{
-  return ((READ_BIT(RTCx->CR, RTC_CR_TSIE) == (RTC_CR_TSIE)) ? 1UL :
0UL);
```

```
-}
-
-/**
-  * @brief  Check if  Wakeup timer interrupt is enabled or not
-  * @rmtoll CR           WUTIE         LL_RTC_IsEnabledIT_WUT
-  * @param  RTCx RTC Instance
-  * @retval State of bit (1 or 0).
-  */
-__STATIC_INLINE uint32_t LL_RTC_IsEnabledIT_WUT(RTC_TypeDef *RTCx)
-{
-  return ((READ_BIT(RTCx->CR, RTC_CR_WUTIE) == (RTC_CR_WUTIE)) ? 1UL :
0UL);
-}
-
-/**
-  * @brief  Check if  Alarm B interrupt is enabled or not
-  * @rmtoll CR           ALRBIE        LL_RTC_IsEnabledIT_ALRB
-  * @param  RTCx RTC Instance
-  * @retval State of bit (1 or 0).
-  */
-__STATIC_INLINE uint32_t LL_RTC_IsEnabledIT_ALRB(RTC_TypeDef *RTCx)
-{
-  return ((READ_BIT(RTCx->CR, RTC_CR_ALRBIE) == (RTC_CR_ALRBIE)) ? 1UL :
0UL);
-}
-
-/**
-  * @brief  Check if  Alarm A interrupt is enabled or not
-  * @rmtoll CR           ALRAIE        LL_RTC_IsEnabledIT_ALRA
-  * @param  RTCx RTC Instance
-  * @retval State of bit (1 or 0).
-  */
-__STATIC_INLINE uint32_t LL_RTC_IsEnabledIT_ALRA(RTC_TypeDef *RTCx)
-{
-  return ((READ_BIT(RTCx->CR, RTC_CR_ALRAIE) == (RTC_CR_ALRAIE)) ? 1UL :
0UL);
-}
-
-/**
-  * @brief  Check if all the TAMPER interrupts are enabled or not
-  * @rmtoll TAFCR       TAMPIE        LL_RTC_IsEnabledIT_TAMP
-  * @param  RTCx RTC Instance
-  * @retval State of bit (1 or 0).
-  */
-__STATIC_INLINE uint32_t LL_RTC_IsEnabledIT_TAMP(RTC_TypeDef *RTCx)
-{
-  return ((READ_BIT(RTCx->TAFCR,
-                    RTC_TAFCR_TAMPIE) == (RTC_TAFCR_TAMPIE)) ? 1UL :
0UL);
-}
-
-/**
-  * @}
-  */
-
-#if defined(USE_FULL_LL_DRIVER)
-/** @defgroup RTC_LL_EF_Init Initialization and de-initialization
functions
```

```
-  * @{
-  */
-
-ErrorStatus LL_RTC_DeInit(RTC_TypeDef *RTCx);
-ErrorStatus LL_RTC_Init(RTC_TypeDef *RTCx, LL_RTC_InitTypeDef
*RTC_InitStruct);
-void        LL_RTC_StructInit(LL_RTC_InitTypeDef *RTC_InitStruct);
-ErrorStatus LL_RTC_TIME_Init(RTC_TypeDef *RTCx, uint32_t RTC_Format,
LL_RTC_TimeTypeDef *RTC_TimeStruct);
-void        LL_RTC_TIME_StructInit(LL_RTC_TimeTypeDef *RTC_TimeStruct);
-ErrorStatus LL_RTC_DATE_Init(RTC_TypeDef *RTCx, uint32_t RTC_Format,
LL_RTC_DateTypeDef *RTC_DateStruct);
-void        LL_RTC_DATE_StructInit(LL_RTC_DateTypeDef *RTC_DateStruct);
-ErrorStatus LL_RTC_ALMA_Init(RTC_TypeDef *RTCx, uint32_t RTC_Format,
LL_RTC_AlarmTypeDef *RTC_AlarmStruct);
-ErrorStatus LL_RTC_ALMB_Init(RTC_TypeDef *RTCx, uint32_t RTC_Format,
LL_RTC_AlarmTypeDef *RTC_AlarmStruct);
-void        LL_RTC_ALMA_StructInit(LL_RTC_AlarmTypeDef
*RTC_AlarmStruct);
-void        LL_RTC_ALMB_StructInit(LL_RTC_AlarmTypeDef
*RTC_AlarmStruct);
-ErrorStatus LL_RTC_EnterInitMode(RTC_TypeDef *RTCx);
-ErrorStatus LL_RTC_ExitInitMode(RTC_TypeDef *RTCx);
-ErrorStatus LL_RTC_WaitForSynchro(RTC_TypeDef *RTCx);
-
-/**
-  * @}
-  */
-#endif /* USE_FULL_LL_DRIVER */
-
-/**
-  * @}
-  */
-
-/**
-  * @}
-  */
-
-#endif /* defined(RTC) */
-
-/**
-  * @}
-  */
-
-#ifdef __cplusplus
-}
-#endif
-
-#endif /* STM32F4xx_LL_RTC_H */
diff --git a/Drivers/STM32F4xx_HAL_Driver/Src/stm32f4xx_hal_rtc.c
b/Drivers/STM32F4xx_HAL_Driver/Src/stm32f4xx_hal_rtc.c
deleted file mode 100644
index 96d70da..0000000
--- a/Drivers/STM32F4xx_HAL_Driver/Src/stm32f4xx_hal_rtc.c
+++ /dev/null
@@ -1,1920 +0,0 @@
-/**
```

```
-
*******************************************************************************
-  * @file    stm32f4xx_hal_rtc.c
-  * @author  MCD Application Team
-  * @brief   RTC HAL module driver.
-  *          This file provides firmware functions to manage the following
-  *          functionalities of the Real-Time Clock (RTC) peripheral:
-  *           + Initialization and de-initialization functions
-  *           + Calendar (Time and Date) configuration functions
-  *           + Alarms (Alarm A and Alarm B) configuration functions
-  *           + Peripheral Control functions
-  *           + Peripheral State functions
-  *
-
*******************************************************************************
-  * @attention
-  *
-  * Copyright (c) 2016 STMicroelectronics.
-  * All rights reserved.
-  *
-  * This software is licensed under terms that can be found in the LICENSE file
-  * in the root directory of this software component.
-  * If no LICENSE file comes with this software, it is provided AS-IS.
-  *
-
*******************************************************************************
-  @verbatim
-
==============================================================================
-               ##### RTC and Backup Domain Operating Condition #####
-
==============================================================================
-  [..] The real-time clock (RTC), the RTC backup registers, and the backup
-       SRAM (BKP SRAM) can be powered from the VBAT voltage when the main
-       VDD supply is powered off.
-       To retain the content of the RTC backup registers, BKP SRAM, and supply
-       the RTC when VDD is turned off, VBAT pin can be connected to an optional
-       standby voltage supplied by a battery or by another source.
-
-  [..] To allow the RTC operating even when the main digital supply (VDD) is turned
-       off, the VBAT pin powers the following blocks:
-    (#) The RTC
-    (#) The LSE oscillator
-    (#) The BKP SRAM when the low power backup regulator is enabled
-    (#) PC13 to PC15 I/Os, plus PI8 I/O (when available)
-
```

```
-  [..] When the backup domain is supplied by VDD (analog switch
connected to VDD),
-       the following pins are available:
-    (#) PC14 and PC15 can be used as either GPIO or LSE pins
-    (#) PC13 can be used as a GPIO or as the RTC_AF1 pin
-    (#) PI8 can be used as a GPIO or as the RTC_AF2 pin
-
-  [..] When the backup domain is supplied by VBAT (analog switch
connected to VBAT
-       because VDD is not present), the following pins are available:
-    (#) PC14 and PC15 can be used as LSE pins only
-    (#) PC13 can be used as the RTC_AF1 pin
-    (#) PI8 can be used as the RTC_AF2 pin
-
-                    ##### Backup Domain Reset #####
-  ==================================================================
-  [..] The backup domain reset sets all RTC registers and the RCC_BDCR
register
-       to their reset values.
-       The BKP SRAM is not affected by this reset. The only way to reset
the BKP
-       SRAM is through the Flash interface by requesting a protection
level
-       change from 1 to 0.
-  [..] A backup domain reset is generated when one of the following
events occurs:
-    (#) Software reset, triggered by setting the BDRST bit in the
-        RCC Backup domain control register (RCC_BDCR).
-    (#) VDD or VBAT power on, if both supplies have previously been
powered off.
-
-                    ##### Backup Domain Access #####
-  ==================================================================
-  [..] After reset, the backup domain (RTC registers, RTC backup data
registers
-       and BKP SRAM) is protected against possible unwanted write
accesses.
-  [..] To enable access to the RTC Domain and RTC registers, proceed as
follows:
-    (+) Enable the Power Controller (PWR) APB1 interface clock using the
-        __HAL_RCC_PWR_CLK_ENABLE() macro.
-    (+) Enable access to RTC domain using the HAL_PWR_EnableBkUpAccess()
function.
-    (+) Select the RTC clock source using the __HAL_RCC_RTC_CONFIG()
macro.
-    (+) Enable RTC Clock using the __HAL_RCC_RTC_ENABLE() macro.
-
-
================================================================================
=====
-                        ##### How to use this driver #####
-
================================================================================
=====
-  [..]
-    (+) Enable the RTC domain access (see description in the section
above).
```

```
-    (+) Configure the RTC Prescaler (Asynchronous and Synchronous) and
RTC hour
-        format using the HAL_RTC_Init() function.
-
-  *** Time and Date configuration ***
-  ==================================
-  [..]
-    (+) To configure the RTC Calendar (Time and Date) use the
HAL_RTC_SetTime()
-        and HAL_RTC_SetDate() functions.
-    (+) To read the RTC Calendar, use the HAL_RTC_GetTime() and
HAL_RTC_GetDate()
-        functions.
-    (+) To manage the RTC summer or winter time change, use the
following
-        functions:
-        (++) HAL_RTC_DST_Add1Hour() or HAL_RTC_DST_Sub1Hour to add or
subtract
-             1 hour from the calendar time.
-        (++) HAL_RTC_DST_SetStoreOperation() or
HAL_RTC_DST_ClearStoreOperation
-             to memorize whether the time change has been performed or
not.
-
-  *** Alarm configuration ***
-  ===========================
-  [..]
-    (+) To configure the RTC Alarm use the HAL_RTC_SetAlarm() function.
-        You can also configure the RTC Alarm with interrupt mode using
the
-        HAL_RTC_SetAlarm_IT() function.
-    (+) To read the RTC Alarm, use the HAL_RTC_GetAlarm() function.
-
-                  ##### RTC and low power modes #####
-  ================================================================
-  [..] The MCU can be woken up from a low power mode by an RTC alternate
-       function.
-  [..] The RTC alternate functions are the RTC alarms (Alarm A and Alarm
B),
-       RTC wakeup, RTC tamper event detection and RTC timestamp event
detection.
-       These RTC alternate functions can wake up the system from the
Stop and
-       Standby low power modes.
-  [..] The system can also wake up from low power modes without
depending
-       on an external interrupt (Auto-wakeup mode), by using the RTC
alarm
-       or the RTC wakeup events.
-  [..] The RTC provides a programmable time base for waking up from the
-       Stop or Standby mode at regular intervals.
-       Wakeup from STOP and STANDBY modes is possible only when the RTC
clock
-       source is LSE or LSI.
-
-  *** Callback registration ***
-  =============================================
-  [..]
```

```
-  When the compilation define USE_HAL_RTC_REGISTER_CALLBACKS is set to 0
or
-  not defined, the callback registration feature is not available and
all
-  callbacks are set to the corresponding weak functions.
-  This is the recommended configuration in order to optimize memory/code
-  consumption footprint/performances.
-  [..]
-  The compilation define  USE_HAL_RTC_REGISTER_CALLBACKS when set to 1
-  allows the user to configure dynamically the driver callbacks.
-  Use Function HAL_RTC_RegisterCallback() to register an interrupt
callback.
-  [..]
-  Function HAL_RTC_RegisterCallback() allows to register following
callbacks:
-    (+) AlarmAEventCallback          : RTC Alarm A Event callback.
-    (+) AlarmBEventCallback          : RTC Alarm B Event callback.
-    (+) TimeStampEventCallback       : RTC Timestamp Event callback.
-    (+) WakeUpTimerEventCallback     : RTC WakeUpTimer Event callback.
-    (+) Tamper1EventCallback         : RTC Tamper 1 Event callback.
-    (+) Tamper2EventCallback         : RTC Tamper 2 Event callback. (*)
-    (+) MspInitCallback              : RTC MspInit callback.
-    (+) MspDeInitCallback            : RTC MspDeInit callback.
-
-  (*) value not applicable to all devices.
-  [..]
-  This function takes as parameters the HAL peripheral handle, the
Callback ID
-  and a pointer to the user callback function.
-  [..]
-  Use function HAL_RTC_UnRegisterCallback() to reset a callback to the
default
-  weak function.
-  HAL_RTC_UnRegisterCallback() takes as parameters the HAL peripheral
handle,
-  and the Callback ID.
-  This function allows to reset following callbacks:
-    (+) AlarmAEventCallback          : RTC Alarm A Event callback.
-    (+) AlarmBEventCallback          : RTC Alarm B Event callback.
-    (+) TimeStampEventCallback       : RTC Timestamp Event callback.
-    (+) WakeUpTimerEventCallback     : RTC WakeUpTimer Event callback.
-    (+) Tamper1EventCallback         : RTC Tamper 1 Event callback.
-    (+) Tamper2EventCallback         : RTC Tamper 2 Event callback. (*)
-    (+) MspInitCallback              : RTC MspInit callback.
-    (+) MspDeInitCallback            : RTC MspDeInit callback.
-
-  (*) value not applicable to all devices.
-  [..]
-  By default, after the HAL_RTC_Init() and when the state is
HAL_RTC_STATE_RESET,
-  all callbacks are set to the corresponding weak functions:
-  examples AlarmAEventCallback(), TimeStampEventCallback().
-  Exception done for MspInit() and MspDeInit() callbacks that are reset
to the
-  legacy weak function in the HAL_RTC_Init()/HAL_RTC_DeInit() only when
these
-  callbacks are null (not registered beforehand).
```

```
-  If not, MspInit() or MspDeInit() are not null,
HAL_RTC_Init()/HAL_RTC_DeInit()
-  keep and use the user MspInit()/MspDeInit() callbacks (registered
beforehand).
-  [..]
-  Callbacks can be registered/unregistered in HAL_RTC_STATE_READY state
only.
-  Exception done for MspInit() and MspDeInit() that can be
registered/unregistered
-  in HAL_RTC_STATE_READY or HAL_RTC_STATE_RESET state.
-  Thus registered (user) MspInit()/MspDeInit() callbacks can be used
during the
-  Init/DeInit.
-  In that case first register the MspInit()/MspDeInit() user callbacks
using
-  HAL_RTC_RegisterCallback() before calling HAL_RTC_DeInit() or
HAL_RTC_Init()
-  functions.
-
-  @endverbatim
-
******************************************************************************
-  */
-
-/* Includes ------------------------------------------------------------------*/
-#include "stm32f4xx_hal.h"
-
-/** @addtogroup STM32F4xx_HAL_Driver
-  * @{
-  */
-
-/** @defgroup RTC RTC
-  * @brief    RTC HAL module driver
-  * @{
-  */
-
-#ifdef HAL_RTC_MODULE_ENABLED
-
-/* Private typedef -----------------------------------------------------------*/
-/* Private define ------------------------------------------------------------*/
-/* Private macro -------------------------------------------------------------*/
-/* Private variables ---------------------------------------------------------*/
-/* Private function prototypes -----------------------------------------------*/
-/* Exported functions ---------------------------------------------------------*/
-
-/** @defgroup RTC_Exported_Functions RTC Exported Functions
-  * @{
-  */
-
```

```
-/** @defgroup RTC_Exported_Functions_Group1 Initialization and de-
initialization functions
-  * @brief    Initialization and Configuration functions
-  *
-@verbatim
-
==============================================================================
======
-              ##### Initialization and de-initialization functions #####
-
==============================================================================
======
-   [..] This section provides functions allowing to initialize and
configure the
-         RTC Prescaler (Synchronous and Asynchronous), RTC Hour format,
disable
-         RTC registers Write protection, enter and exit the RTC
initialization mode,
-         RTC registers synchronization check and reference clock
detection enable.
-         (#) The RTC Prescaler is programmed to generate the RTC 1Hz
time base.
-             It is split into 2 programmable prescalers to minimize
power consumption.
-             (++) A 7-bit asynchronous prescaler and a 15-bit
synchronous prescaler.
-             (++) When both prescalers are used, it is recommended to
configure the
-                 asynchronous prescaler to a high value to minimize
power consumption.
-         (#) All RTC registers are Write protected. Writing to the RTC
registers
-             is enabled by writing a key into the Write Protection
register, RTC_WPR.
-         (#) To configure the RTC Calendar, user application should
enter
-             initialization mode. In this mode, the calendar counter is
stopped
-             and its value can be updated. When the initialization
sequence is
-             complete, the calendar restarts counting after 4 RTCCLK
cycles.
-         (#) To read the calendar through the shadow registers after
Calendar
-             initialization, calendar update or after wakeup from low
power modes
-             the software must first clear the RSF flag. The software
must then
-             wait until it is set again before reading the calendar,
which means
-             that the calendar registers have been correctly copied into
the
-             RTC_TR and RTC_DR shadow registers. The
HAL_RTC_WaitForSynchro() function
-             implements the above software sequence (RSF clear and RSF
check).
-
-@endverbatim
```

```
-  * @{
-  */
-
-/**
-  * @brief  Initializes the RTC peripheral
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
-  *                the configuration information for RTC.
-  * @retval HAL status
-  */
-HAL_StatusTypeDef HAL_RTC_Init(RTC_HandleTypeDef *hrtc)
-{
-  HAL_StatusTypeDef status;
-
-  /* Check RTC handler validity */
-  if (hrtc == NULL)
-  {
-    return HAL_ERROR;
-  }
-
-  /* Check the parameters */
-  assert_param(IS_RTC_ALL_INSTANCE(hrtc->Instance));
-  assert_param(IS_RTC_HOUR_FORMAT(hrtc->Init.HourFormat));
-  assert_param(IS_RTC_ASYNCH_PREDIV(hrtc->Init.AsynchPrediv));
-  assert_param(IS_RTC_SYNCH_PREDIV(hrtc->Init.SynchPrediv));
-  assert_param(IS_RTC_OUTPUT(hrtc->Init.OutPut));
-  assert_param(IS_RTC_OUTPUT_POL(hrtc->Init.OutPutPolarity));
-  assert_param(IS_RTC_OUTPUT_TYPE(hrtc->Init.OutPutType));
-
-#if (USE_HAL_RTC_REGISTER_CALLBACKS == 1)
-  if (hrtc->State == HAL_RTC_STATE_RESET)
-  {
-    /* Allocate lock resource and initialize it */
-    hrtc->Lock = HAL_UNLOCKED;
-
-    hrtc->AlarmAEventCallback          =  HAL_RTC_AlarmAEventCallback;
/* Legacy weak AlarmAEventCallback      */
-    hrtc->AlarmBEventCallback          =  HAL_RTCEx_AlarmBEventCallback;
/* Legacy weak AlarmBEventCallback      */
-    hrtc->TimeStampEventCallback       =
HAL_RTCEx_TimeStampEventCallback;   /* Legacy weak TimeStampEventCallback
*/
-    hrtc->WakeUpTimerEventCallback     =
HAL_RTCEx_WakeUpTimerEventCallback; /* Legacy weak
WakeUpTimerEventCallback */
-    hrtc->Tamper1EventCallback         =
HAL_RTCEx_Tamper1EventCallback;     /* Legacy weak Tamper1EventCallback
*/
-#if defined(RTC_TAMPER2_SUPPORT)
-    hrtc->Tamper2EventCallback         =
HAL_RTCEx_Tamper2EventCallback;     /* Legacy weak Tamper2EventCallback
*/
-#endif /* RTC_TAMPER2_SUPPORT */
-
-    if (hrtc->MspInitCallback == NULL)
-    {
-      hrtc->MspInitCallback = HAL_RTC_MspInit;
-    }
-    /* Init the low level hardware */
```

```
-    hrtc->MspInitCallback(hrtc);
-
-    if (hrtc->MspDeInitCallback == NULL)
-    {
-      hrtc->MspDeInitCallback = HAL_RTC_MspDeInit;
-    }
-  }
-#else /* USE_HAL_RTC_REGISTER_CALLBACKS */
-  if (hrtc->State == HAL_RTC_STATE_RESET)
-  {
-    /* Allocate lock resource and initialize it */
-    hrtc->Lock = HAL_UNLOCKED;
-
-    /* Initialize RTC MSP */
-    HAL_RTC_MspInit(hrtc);
-  }
-#endif /* USE_HAL_RTC_REGISTER_CALLBACKS */
-
-  /* Set RTC state */
-  hrtc->State = HAL_RTC_STATE_BUSY;
-
-  /* Check whether the calendar needs to be initialized */
-  if (__HAL_RTC_IS_CALENDAR_INITIALIZED(hrtc) == 0U)
-  {
-    /* Disable the write protection for RTC registers */
-    __HAL_RTC_WRITEPROTECTION_DISABLE(hrtc);
-
-    /* Enter Initialization mode */
-    status = RTC_EnterInitMode(hrtc);
-
-    if (status == HAL_OK)
-    {
-      /* Clear RTC_CR FMT, OSEL and POL Bits */
-      hrtc->Instance->CR &= ((uint32_t)~(RTC_CR_FMT | RTC_CR_OSEL |
RTC_CR_POL));
-      /* Set RTC_CR register */
-      hrtc->Instance->CR |= (uint32_t)(hrtc->Init.HourFormat | hrtc-
>Init.OutPut | hrtc->Init.OutPutPolarity);
-
-      /* Configure the RTC PRER */
-      hrtc->Instance->PRER = (uint32_t)(hrtc->Init.SynchPrediv);
-      hrtc->Instance->PRER |= (uint32_t)(hrtc->Init.AsynchPrediv <<
RTC_PRER_PREDIV_A_Pos);
-
-      /* Exit Initialization mode */
-      status = RTC_ExitInitMode(hrtc);
-    }
-
-    if (status == HAL_OK)
-    {
-      hrtc->Instance->TAFCR &= (uint32_t)~RTC_OUTPUT_TYPE_PUSHPULL;
-      hrtc->Instance->TAFCR |= (uint32_t)(hrtc->Init.OutPutType);
-    }
-
-    /* Enable the write protection for RTC registers */
-    __HAL_RTC_WRITEPROTECTION_ENABLE(hrtc);
-  }
-  else
```

```
-  {
-    /* The calendar is already initialized */
-    status = HAL_OK;
-  }
-
-  if (status == HAL_OK)
-  {
-    hrtc->State = HAL_RTC_STATE_READY;
-  }
-
-  return status;
-}
-
-/**
-  * @brief  DeInitializes the RTC peripheral
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
-  *                the configuration information for RTC.
-  * @note   This function does not reset the RTC Backup Data registers.
-  * @retval HAL status
-  */
-HAL_StatusTypeDef HAL_RTC_DeInit(RTC_HandleTypeDef *hrtc)
-{
-  HAL_StatusTypeDef status;
-
-  /* Check the parameters */
-  assert_param(IS_RTC_ALL_INSTANCE(hrtc->Instance));
-
-  /* Set RTC state */
-  hrtc->State = HAL_RTC_STATE_BUSY;
-
-  /* Disable the write protection for RTC registers */
-  __HAL_RTC_WRITEPROTECTION_DISABLE(hrtc);
-
-  /* Enter Initialization mode */
-  status = RTC_EnterInitMode(hrtc);
-
-  if (status == HAL_OK)
-  {
-    /* Reset RTC registers */
-    hrtc->Instance->TR = 0x00000000U;
-    hrtc->Instance->DR = (RTC_DR_WDU_0 | RTC_DR_MU_0 | RTC_DR_DU_0);
-    hrtc->Instance->CR = 0x00000000U;
-    hrtc->Instance->WUTR = RTC_WUTR_WUT;
-    hrtc->Instance->PRER = (uint32_t)(RTC_PRER_PREDIV_A | 0x000000FFU);
-    hrtc->Instance->CALIBR   = 0x00000000U;
-    hrtc->Instance->ALRMAR   = 0x00000000U;
-    hrtc->Instance->ALRMBR   = 0x00000000U;
-    hrtc->Instance->CALR     = 0x00000000U;
-    hrtc->Instance->SHIFTR   = 0x00000000U;
-    hrtc->Instance->ALRMASSR = 0x00000000U;
-    hrtc->Instance->ALRMBSSR = 0x00000000U;
-
-    /* Exit Initialization mode */
-    status = RTC_ExitInitMode(hrtc);
-  }
-
-  /* Enable the write protection for RTC registers */
-  __HAL_RTC_WRITEPROTECTION_ENABLE(hrtc);
```

```
-
-  if (status == HAL_OK)
-  {
-    /* Reset Tamper and alternate functions configuration register */
-    hrtc->Instance->TAFCR = 0x00000000U;
-
-#if (USE_HAL_RTC_REGISTER_CALLBACKS == 1)
-    if (hrtc->MspDeInitCallback == NULL)
-    {
-      hrtc->MspDeInitCallback = HAL_RTC_MspDeInit;
-    }
-
-    /* DeInit the low level hardware: CLOCK, NVIC.*/
-    hrtc->MspDeInitCallback(hrtc);
-#else /* USE_HAL_RTC_REGISTER_CALLBACKS */
-    /* De-Initialize RTC MSP */
-    HAL_RTC_MspDeInit(hrtc);
-#endif /* USE_HAL_RTC_REGISTER_CALLBACKS */
-
-    hrtc->State = HAL_RTC_STATE_RESET;
-  }
-
-  /* Release Lock */
-  __HAL_UNLOCK(hrtc);
-
-  return status;
-}
-
-#if (USE_HAL_RTC_REGISTER_CALLBACKS == 1)
-/**
-  * @brief  Registers a User RTC Callback
-  *         To be used instead of the weak predefined callback
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
-  *                the configuration information for RTC.
-  * @param  CallbackID ID of the callback to be registered
-  *         This parameter can be one of the following values:
-  *          @arg @ref HAL_RTC_ALARM_A_EVENT_CB_ID          Alarm A
Event Callback ID
-  *          @arg @ref HAL_RTC_ALARM_B_EVENT_CB_ID          Alarm B
Event Callback ID
-  *          @arg @ref HAL_RTC_TIMESTAMP_EVENT_CB_ID        Timestamp
Event Callback ID
-  *          @arg @ref HAL_RTC_WAKEUPTIMER_EVENT_CB_ID      Wakeup Timer
Event Callback ID
-  *          @arg @ref HAL_RTC_TAMPER1_EVENT_CB_ID          Tamper 1
Event Callback ID
-  *          @arg @ref HAL_RTC_TAMPER2_EVENT_CB_ID          Tamper 2
Event Callback ID (*)
-  *          @arg @ref HAL_RTC_MSPINIT_CB_ID                MSP Init
callback ID
-  *          @arg @ref HAL_RTC_MSPDEINIT_CB_ID              MSP DeInit
callback ID
-  *
-  *         (*) value not applicable to all devices.
-  * @param  pCallback pointer to the Callback function
-  * @retval HAL status
-  */
```

```
-HAL_StatusTypeDef HAL_RTC_RegisterCallback(RTC_HandleTypeDef *hrtc,
HAL_RTC_CallbackIDTypeDef CallbackID, pRTC_CallbackTypeDef pCallback)
-{
-  HAL_StatusTypeDef status = HAL_OK;
-
-  if (pCallback == NULL)
-  {
-    return HAL_ERROR;
-  }
-
-  /* Process locked */
-  __HAL_LOCK(hrtc);
-
-  if (HAL_RTC_STATE_READY == hrtc->State)
-  {
-    switch (CallbackID)
-    {
-      case HAL_RTC_ALARM_A_EVENT_CB_ID :
-        hrtc->AlarmAEventCallback = pCallback;
-        break;
-
-      case HAL_RTC_ALARM_B_EVENT_CB_ID :
-        hrtc->AlarmBEventCallback = pCallback;
-        break;
-
-      case HAL_RTC_TIMESTAMP_EVENT_CB_ID :
-        hrtc->TimeStampEventCallback = pCallback;
-        break;
-
-      case HAL_RTC_WAKEUPTIMER_EVENT_CB_ID :
-        hrtc->WakeUpTimerEventCallback = pCallback;
-        break;
-
-      case HAL_RTC_TAMPER1_EVENT_CB_ID :
-        hrtc->Tamper1EventCallback = pCallback;
-        break;
-
-#if defined(RTC_TAMPER2_SUPPORT)
-      case HAL_RTC_TAMPER2_EVENT_CB_ID :
-        hrtc->Tamper2EventCallback = pCallback;
-        break;
-#endif /* RTC_TAMPER2_SUPPORT */
-
-      case HAL_RTC_MSPINIT_CB_ID :
-        hrtc->MspInitCallback = pCallback;
-        break;
-
-      case HAL_RTC_MSPDEINIT_CB_ID :
-        hrtc->MspDeInitCallback = pCallback;
-        break;
-
-      default :
-        /* Return error status */
-        status =  HAL_ERROR;
-        break;
-    }
-  }
-  else if (HAL_RTC_STATE_RESET == hrtc->State)
```

```
-  {
-    switch (CallbackID)
-    {
-      case HAL_RTC_MSPINIT_CB_ID :
-        hrtc->MspInitCallback = pCallback;
-        break;
-
-      case HAL_RTC_MSPDEINIT_CB_ID :
-        hrtc->MspDeInitCallback = pCallback;
-        break;
-
-      default :
-        /* Return error status */
-        status =  HAL_ERROR;
-        break;
-    }
-  }
-  else
-  {
-    /* Return error status */
-    status =  HAL_ERROR;
-  }
-
-  /* Release Lock */
-  __HAL_UNLOCK(hrtc);
-
-  return status;
-}
-
-/**
-  * @brief  Unregisters an RTC Callback
-  *         RTC callback is redirected to the weak predefined callback
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
-  *                the configuration information for RTC.
-  * @param  CallbackID ID of the callback to be unregistered
-  *         This parameter can be one of the following values:
-  *          @arg @ref HAL_RTC_ALARM_A_EVENT_CB_ID          Alarm A
Event Callback ID
-  *          @arg @ref HAL_RTC_ALARM_B_EVENT_CB_ID          Alarm B
Event Callback ID
-  *          @arg @ref HAL_RTC_TIMESTAMP_EVENT_CB_ID        Timestamp
Event Callback ID
-  *          @arg @ref HAL_RTC_WAKEUPTIMER_EVENT_CB_ID      Wakeup Timer
Event Callback ID
-  *          @arg @ref HAL_RTC_TAMPER1_EVENT_CB_ID          Tamper 1
Event Callback ID
-  *          @arg @ref HAL_RTC_TAMPER2_EVENT_CB_ID          Tamper 2
Event Callback ID (*)
-  *          @arg @ref HAL_RTC_MSPINIT_CB_ID                MSP Init
callback ID
-  *          @arg @ref HAL_RTC_MSPDEINIT_CB_ID              MSP DeInit
callback ID
-  *
-  *         (*) value not applicable to all devices.
-  * @retval HAL status
-  */
-HAL_StatusTypeDef HAL_RTC_UnRegisterCallback(RTC_HandleTypeDef *hrtc,
HAL_RTC_CallbackIDTypeDef CallbackID)
```

```
-{
-  HAL_StatusTypeDef status = HAL_OK;
-
-  /* Process locked */
-  __HAL_LOCK(hrtc);
-
-  if (HAL_RTC_STATE_READY == hrtc->State)
-  {
-    switch (CallbackID)
-    {
-      case HAL_RTC_ALARM_A_EVENT_CB_ID :
-        hrtc->AlarmAEventCallback = HAL_RTC_AlarmAEventCallback;
/* Legacy weak AlarmAEventCallback    */
-        break;
-
-      case HAL_RTC_ALARM_B_EVENT_CB_ID :
-        hrtc->AlarmBEventCallback = HAL_RTCEx_AlarmBEventCallback;
/* Legacy weak AlarmBEventCallback */
-        break;
-
-      case HAL_RTC_TIMESTAMP_EVENT_CB_ID :
-        hrtc->TimeStampEventCallback = HAL_RTCEx_TimeStampEventCallback;
/* Legacy weak TimeStampEventCallback    */
-        break;
-
-      case HAL_RTC_WAKEUPTIMER_EVENT_CB_ID :
-        hrtc->WakeUpTimerEventCallback =
HAL_RTCEx_WakeUpTimerEventCallback; /* Legacy weak
WakeUpTimerEventCallback */
-        break;
-
-      case HAL_RTC_TAMPER1_EVENT_CB_ID :
-        hrtc->Tamper1EventCallback = HAL_RTCEx_Tamper1EventCallback;
/* Legacy weak Tamper1EventCallback   */
-        break;
-
-#if defined(RTC_TAMPER2_SUPPORT)
-      case HAL_RTC_TAMPER2_EVENT_CB_ID :
-        hrtc->Tamper2EventCallback = HAL_RTCEx_Tamper2EventCallback;
/* Legacy weak Tamper2EventCallback         */
-        break;
-#endif /* RTC_TAMPER2_SUPPORT */
-
-      case HAL_RTC_MSPINIT_CB_ID :
-        hrtc->MspInitCallback = HAL_RTC_MspInit;
-        break;
-
-      case HAL_RTC_MSPDEINIT_CB_ID :
-        hrtc->MspDeInitCallback = HAL_RTC_MspDeInit;
-        break;
-
-      default :
-        /* Return error status */
-        status =  HAL_ERROR;
-        break;
-    }
-  }
-  else if (HAL_RTC_STATE_RESET == hrtc->State)
```

```
-  {
-    switch (CallbackID)
-    {
-      case HAL_RTC_MSPINIT_CB_ID :
-        hrtc->MspInitCallback = HAL_RTC_MspInit;
-        break;
-
-      case HAL_RTC_MSPDEINIT_CB_ID :
-        hrtc->MspDeInitCallback = HAL_RTC_MspDeInit;
-        break;
-
-      default :
-        /* Return error status */
-        status =  HAL_ERROR;
-        break;
-    }
-  }
-  else
-  {
-    /* Return error status */
-    status =  HAL_ERROR;
-  }
-
-  /* Release Lock */
-  __HAL_UNLOCK(hrtc);
-
-  return status;
-}
-#endif /* USE_HAL_RTC_REGISTER_CALLBACKS */
-
-/**
-  * @brief  Initializes the RTC MSP.
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
-  *                the configuration information for RTC.
-  * @retval None
-  */
-__weak void HAL_RTC_MspInit(RTC_HandleTypeDef *hrtc)
-{
-  /* Prevent unused argument(s) compilation warning */
-  UNUSED(hrtc);
-
-  /* NOTE: This function should not be modified, when the callback is
needed,
-           the HAL_RTC_MspInit could be implemented in the user file
-   */
-}
-
-/**
-  * @brief  DeInitializes the RTC MSP.
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
-  *                the configuration information for RTC.
-  * @retval None
-  */
-__weak void HAL_RTC_MspDeInit(RTC_HandleTypeDef *hrtc)
-{
-  /* Prevent unused argument(s) compilation warning */
-  UNUSED(hrtc);
-
```

```
-  /* NOTE: This function should not be modified, when the callback is
needed,
-           the HAL_RTC_MspDeInit could be implemented in the user file
-   */
-}
-
-/**
-  * @}
-  */
-
-/** @defgroup RTC_Exported_Functions_Group2 RTC Time and Date functions
-  * @brief    RTC Time and Date functions
-  *
-@verbatim
-
==============================================================================
======
-                 ##### RTC Time and Date functions #####
-
==============================================================================
======
-
- [..] This section provides functions allowing to configure Time and
Date features
-
-@endverbatim
-  * @{
-  */
-
-/**
-  * @brief  Sets RTC current time.
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
-  *                the configuration information for RTC.
-  * @param  sTime Pointer to Time structure
-  * @note   DayLightSaving and StoreOperation interfaces are deprecated.
-  *         To manage Daylight Saving Time, please use HAL_RTC_DST_xxx
functions.
-  * @param  Format Specifies the format of the entered parameters.
-  *          This parameter can be one of the following values:
-  *            @arg RTC_FORMAT_BIN: Binary data format
-  *            @arg RTC_FORMAT_BCD: BCD data format
-  * @retval HAL status
-  */
-HAL_StatusTypeDef HAL_RTC_SetTime(RTC_HandleTypeDef *hrtc,
RTC_TimeTypeDef *sTime, uint32_t Format)
-{
-  uint32_t tmpreg = 0U;
-  HAL_StatusTypeDef status;
-
-  /* Check the parameters */
-  assert_param(IS_RTC_FORMAT(Format));
-  assert_param(IS_RTC_DAYLIGHT_SAVING(sTime->DayLightSaving));
-  assert_param(IS_RTC_STORE_OPERATION(sTime->StoreOperation));
-
-  /* Process Locked */
-  __HAL_LOCK(hrtc);
-
-  hrtc->State = HAL_RTC_STATE_BUSY;
```

```
-
-  if (Format == RTC_FORMAT_BIN)
-  {
-    if ((hrtc->Instance->CR & RTC_CR_FMT) != 0U)
-    {
-      assert_param(IS_RTC_HOUR12(sTime->Hours));
-      assert_param(IS_RTC_HOURFORMAT12(sTime->TimeFormat));
-    }
-    else
-    {
-      sTime->TimeFormat = 0x00U;
-      assert_param(IS_RTC_HOUR24(sTime->Hours));
-    }
-    assert_param(IS_RTC_MINUTES(sTime->Minutes));
-    assert_param(IS_RTC_SECONDS(sTime->Seconds));
-
-    tmpreg = (uint32_t)(( (uint32_t)RTC_ByteToBcd2(sTime->Hours)   <<
RTC_TR_HU_Pos)  | \
-                        ( (uint32_t)RTC_ByteToBcd2(sTime->Minutes) <<
RTC_TR_MNU_Pos) | \
-                        ( (uint32_t)RTC_ByteToBcd2(sTime->Seconds))
| \
-                        (((uint32_t)sTime->TimeFormat)             <<
RTC_TR_PM_Pos));
-  }
-  else
-  {
-    if ((hrtc->Instance->CR & RTC_CR_FMT) != 0U)
-    {
-      assert_param(IS_RTC_HOUR12(RTC_Bcd2ToByte(sTime->Hours)));
-      assert_param(IS_RTC_HOURFORMAT12(sTime->TimeFormat));
-    }
-    else
-    {
-      sTime->TimeFormat = 0x00U;
-      assert_param(IS_RTC_HOUR24(RTC_Bcd2ToByte(sTime->Hours)));
-    }
-    assert_param(IS_RTC_MINUTES(RTC_Bcd2ToByte(sTime->Minutes)));
-    assert_param(IS_RTC_SECONDS(RTC_Bcd2ToByte(sTime->Seconds)));
-    tmpreg = (((uint32_t)(sTime->Hours)      << RTC_TR_HU_Pos)  | \
-              ((uint32_t)(sTime->Minutes)    << RTC_TR_MNU_Pos) | \
-              ((uint32_t) sTime->Seconds)                       | \
-              ((uint32_t)(sTime->TimeFormat) << RTC_TR_PM_Pos));
-  }
-
-  /* Disable the write protection for RTC registers */
-  __HAL_RTC_WRITEPROTECTION_DISABLE(hrtc);
-
-  /* Enter Initialization mode */
-  status = RTC_EnterInitMode(hrtc);
-
-  if (status == HAL_OK)
-  {
-    /* Set the RTC_TR register */
-    hrtc->Instance->TR = (uint32_t)(tmpreg & RTC_TR_RESERVED_MASK);
-
-    /* Clear the bits to be configured (Deprecated. Use HAL_RTC_DST_xxx
functions instead) */
```

```
-    hrtc->Instance->CR &= (uint32_t)~RTC_CR_BKP;
-
-    /* Configure the RTC_CR register (Deprecated. Use HAL_RTC_DST_xxx
functions instead) */
-    hrtc->Instance->CR |= (uint32_t)(sTime->DayLightSaving | sTime-
>StoreOperation);
-
-    /* Exit Initialization mode */
-    status = RTC_ExitInitMode(hrtc);
-  }
-
-  if (status == HAL_OK)
-  {
-    hrtc->State = HAL_RTC_STATE_READY;
-  }
-
-  /* Enable the write protection for RTC registers */
-  __HAL_RTC_WRITEPROTECTION_ENABLE(hrtc);
-
-  /* Process Unlocked */
-  __HAL_UNLOCK(hrtc);
-
-  return status;
-}
-
-/**
-  * @brief  Gets RTC current time.
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
-  *                the configuration information for RTC.
-  * @param  sTime Pointer to Time structure
-  * @param  Format Specifies the format of the entered parameters.
-  *          This parameter can be one of the following values:
-  *            @arg RTC_FORMAT_BIN: Binary data format
-  *            @arg RTC_FORMAT_BCD: BCD data format
-  * @note  You can use SubSeconds and SecondFraction (sTime structure
fields
-  *        returned) to convert SubSeconds value in second fraction
ratio with
-  *        time unit following generic formula:
-  *        Second fraction ratio * time_unit =
-  *           [(SecondFraction - SubSeconds) / (SecondFraction + 1)] *
time_unit
-  *        This conversion can be performed only if no shift operation
is pending
-  *        (ie. SHFP=0) when PREDIV_S >= SS
-  * @note  You must call HAL_RTC_GetDate() after HAL_RTC_GetTime() to
unlock the
-  *        values in the higher-order calendar shadow registers to
ensure
-  *        consistency between the time and date values.
-  *        Reading RTC current time locks the values in calendar shadow
registers
-  *        until current date is read to ensure consistency between the
time and
-  *        date values.
-  * @retval HAL status
-  */
```

```
-HAL_StatusTypeDef HAL_RTC_GetTime(RTC_HandleTypeDef *hrtc,
RTC_TimeTypeDef *sTime, uint32_t Format)
-{
-  uint32_t tmpreg = 0U;
-
-  /* Check the parameters */
-  assert_param(IS_RTC_FORMAT(Format));
-
-  /* Get subseconds value from the corresponding register */
-  sTime->SubSeconds = (uint32_t)(hrtc->Instance->SSR);
-
-  /* Get SecondFraction structure field from the corresponding register
field*/
-  sTime->SecondFraction = (uint32_t)(hrtc->Instance->PRER &
RTC_PRER_PREDIV_S);
-
-  /* Get the TR register */
-  tmpreg = (uint32_t)(hrtc->Instance->TR & RTC_TR_RESERVED_MASK);
-
-  /* Fill the structure fields with the read parameters */
-  sTime->Hours      = (uint8_t)((tmpreg & (RTC_TR_HT  | RTC_TR_HU))  >>
RTC_TR_HU_Pos);
-  sTime->Minutes    = (uint8_t)((tmpreg & (RTC_TR_MNT | RTC_TR_MNU)) >>
RTC_TR_MNU_Pos);
-  sTime->Seconds    = (uint8_t)( tmpreg & (RTC_TR_ST  | RTC_TR_SU));
-  sTime->TimeFormat = (uint8_t)((tmpreg & (RTC_TR_PM))               >>
RTC_TR_PM_Pos);
-
-  /* Check the input parameters format */
-  if (Format == RTC_FORMAT_BIN)
-  {
-    /* Convert the time structure parameters to Binary format */
-    sTime->Hours = (uint8_t)RTC_Bcd2ToByte(sTime->Hours);
-    sTime->Minutes = (uint8_t)RTC_Bcd2ToByte(sTime->Minutes);
-    sTime->Seconds = (uint8_t)RTC_Bcd2ToByte(sTime->Seconds);
-  }
-
-  return HAL_OK;
-}
-
-/**
-  * @brief  Sets RTC current date.
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
-  *                the configuration information for RTC.
-  * @param  sDate Pointer to date structure
-  * @param  Format specifies the format of the entered parameters.
-  *          This parameter can be one of the following values:
-  *            @arg RTC_FORMAT_BIN: Binary data format
-  *            @arg RTC_FORMAT_BCD: BCD data format
-  * @retval HAL status
-  */
-HAL_StatusTypeDef HAL_RTC_SetDate(RTC_HandleTypeDef *hrtc,
RTC_DateTypeDef *sDate, uint32_t Format)
-{
-  uint32_t datetmpreg = 0U;
-  HAL_StatusTypeDef status;
-
-  /* Check the parameters */
```

```
-  assert_param(IS_RTC_FORMAT(Format));
-
-  /* Process Locked */
-  __HAL_LOCK(hrtc);
-
-  hrtc->State = HAL_RTC_STATE_BUSY;
-
-  if ((Format == RTC_FORMAT_BIN) && ((sDate->Month & 0x10U) == 0x10U))
-  {
-    sDate->Month = (uint8_t)((sDate->Month & (uint8_t)~(0x10U)) +
(uint8_t)0x0AU);
-  }
-
-  assert_param(IS_RTC_WEEKDAY(sDate->WeekDay));
-
-  if (Format == RTC_FORMAT_BIN)
-  {
-    assert_param(IS_RTC_YEAR(sDate->Year));
-    assert_param(IS_RTC_MONTH(sDate->Month));
-    assert_param(IS_RTC_DATE(sDate->Date));
-
-    datetmpreg = (((uint32_t)RTC_ByteToBcd2(sDate->Year)  <<
RTC_DR_YU_Pos) | \
-                  ((uint32_t)RTC_ByteToBcd2(sDate->Month) <<
RTC_DR_MU_Pos) | \
-                  ((uint32_t)RTC_ByteToBcd2(sDate->Date))
| \
-                  ((uint32_t)sDate->WeekDay               <<
RTC_DR_WDU_Pos));
-  }
-  else
-  {
-    assert_param(IS_RTC_YEAR(RTC_Bcd2ToByte(sDate->Year)));
-    assert_param(IS_RTC_MONTH(RTC_Bcd2ToByte(sDate->Month)));
-    assert_param(IS_RTC_DATE(RTC_Bcd2ToByte(sDate->Date)));
-
-    datetmpreg = ((((uint32_t)sDate->Year)    << RTC_DR_YU_Pos) | \
-                  (((uint32_t)sDate->Month)   << RTC_DR_MU_Pos) | \
-                  ((uint32_t) sDate->Date)                      | \
-                  (((uint32_t)sDate->WeekDay) << RTC_DR_WDU_Pos));
-  }
-
-  /* Disable the write protection for RTC registers */
-  __HAL_RTC_WRITEPROTECTION_DISABLE(hrtc);
-
-  /* Enter Initialization mode */
-  status = RTC_EnterInitMode(hrtc);
-
-  if (status == HAL_OK)
-  {
-    /* Set the RTC_DR register */
-    hrtc->Instance->DR = (uint32_t)(datetmpreg & RTC_DR_RESERVED_MASK);
-
-    /* Exit Initialization mode */
-    status = RTC_ExitInitMode(hrtc);
-  }
-
-  if (status == HAL_OK)
```

```
-  {
-    hrtc->State = HAL_RTC_STATE_READY;
-  }
-
-  /* Enable the write protection for RTC registers */
-  __HAL_RTC_WRITEPROTECTION_ENABLE(hrtc);
-
-  /* Process Unlocked */
-  __HAL_UNLOCK(hrtc);
-
-  return status;
-}
-
-/**
-  * @brief  Gets RTC current date.
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
-  *                the configuration information for RTC.
-  * @param  sDate Pointer to Date structure
-  * @param  Format Specifies the format of the entered parameters.
-  *          This parameter can be one of the following values:
-  *            @arg RTC_FORMAT_BIN:  Binary data format
-  *            @arg RTC_FORMAT_BCD:  BCD data format
-  * @note  You must call HAL_RTC_GetDate() after HAL_RTC_GetTime() to
unlock the
-  *        values in the higher-order calendar shadow registers to
ensure
-  *        consistency between the time and date values.
-  *        Reading RTC current time locks the values in calendar shadow
registers
-  *        until current date is read to ensure consistency between the
time and
-  *        date values.
-  * @retval HAL status
-  */
-HAL_StatusTypeDef HAL_RTC_GetDate(RTC_HandleTypeDef *hrtc,
RTC_DateTypeDef *sDate, uint32_t Format)
-{
-  uint32_t datetmpreg = 0U;
-
-  /* Check the parameters */
-  assert_param(IS_RTC_FORMAT(Format));
-
-  /* Get the DR register */
-  datetmpreg = (uint32_t)(hrtc->Instance->DR & RTC_DR_RESERVED_MASK);
-
-  /* Fill the structure fields with the read parameters */
-  sDate->Year    = (uint8_t)((datetmpreg & (RTC_DR_YT | RTC_DR_YU)) >>
RTC_DR_YU_Pos);
-  sDate->Month   = (uint8_t)((datetmpreg & (RTC_DR_MT | RTC_DR_MU)) >>
RTC_DR_MU_Pos);
-  sDate->Date    = (uint8_t) (datetmpreg & (RTC_DR_DT | RTC_DR_DU));
-  sDate->WeekDay = (uint8_t)((datetmpreg & (RTC_DR_WDU))            >>
RTC_DR_WDU_Pos);
-
-  /* Check the input parameters format */
-  if (Format == RTC_FORMAT_BIN)
-  {
-    /* Convert the date structure parameters to Binary format */
```

```
-    sDate->Year  = (uint8_t)RTC_Bcd2ToByte(sDate->Year);
-    sDate->Month = (uint8_t)RTC_Bcd2ToByte(sDate->Month);
-    sDate->Date  = (uint8_t)RTC_Bcd2ToByte(sDate->Date);
-  }
-  return HAL_OK;
-}
-
-/**
-  * @}
-  */
-
-/** @defgroup RTC_Exported_Functions_Group3 RTC Alarm functions
-  * @brief    RTC Alarm functions
-  *
-@verbatim
-
==============================================================================
======
-                 ##### RTC Alarm functions #####
-
==============================================================================
======
-
- [..] This section provides functions allowing to configure Alarm
feature
-
-@endverbatim
-  * @{
-  */
-/**
-  * @brief  Sets the specified RTC Alarm.
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
-  *                the configuration information for RTC.
-  * @param  sAlarm Pointer to Alarm structure
-  * @param  Format Specifies the format of the entered parameters.
-  *          This parameter can be one of the following values:
-  *             @arg RTC_FORMAT_BIN: Binary data format
-  *             @arg RTC_FORMAT_BCD: BCD data format
-  * @note   The Alarm register can only be written when the
corresponding Alarm
-  *         is disabled (Use the HAL_RTC_DeactivateAlarm()).
-  * @note   The HAL_RTC_SetTime() must be called before enabling the
Alarm feature.
-  * @retval HAL status
-  */
-HAL_StatusTypeDef HAL_RTC_SetAlarm(RTC_HandleTypeDef *hrtc,
RTC_AlarmTypeDef *sAlarm, uint32_t Format)
-{
-  uint32_t tickstart = 0U;
-  uint32_t tmpreg = 0U;
-  uint32_t subsecondtmpreg = 0U;
-
-  /* Check the parameters */
-  assert_param(IS_RTC_FORMAT(Format));
-  assert_param(IS_RTC_ALARM(sAlarm->Alarm));
-  assert_param(IS_RTC_ALARM_MASK(sAlarm->AlarmMask));
-  assert_param(IS_RTC_ALARM_DATE_WEEKDAY_SEL(sAlarm-
>AlarmDateWeekDaySel));
```

```
-  assert_param(IS_RTC_ALARM_SUB_SECOND_VALUE(sAlarm-
>AlarmTime.SubSeconds));
-  assert_param(IS_RTC_ALARM_SUB_SECOND_MASK(sAlarm-
>AlarmSubSecondMask));
-
-  /* Process Locked */
-  __HAL_LOCK(hrtc);
-
-  /* Change RTC state to BUSY */
-  hrtc->State = HAL_RTC_STATE_BUSY;
-
-  /* Check the data format (binary or BCD) and store the Alarm time and
date
-     configuration accordingly */
-  if (Format == RTC_FORMAT_BIN)
-  {
-    if ((hrtc->Instance->CR & RTC_CR_FMT) != 0U)
-    {
-      assert_param(IS_RTC_HOUR12(sAlarm->AlarmTime.Hours));
-      assert_param(IS_RTC_HOURFORMAT12(sAlarm->AlarmTime.TimeFormat));
-    }
-    else
-    {
-      sAlarm->AlarmTime.TimeFormat = 0x00U;
-      assert_param(IS_RTC_HOUR24(sAlarm->AlarmTime.Hours));
-    }
-    assert_param(IS_RTC_MINUTES(sAlarm->AlarmTime.Minutes));
-    assert_param(IS_RTC_SECONDS(sAlarm->AlarmTime.Seconds));
-
-    if (sAlarm->AlarmDateWeekDaySel == RTC_ALARMDATEWEEKDAYSEL_DATE)
-    {
-      assert_param(IS_RTC_ALARM_DATE_WEEKDAY_DATE(sAlarm-
>AlarmDateWeekDay));
-    }
-    else
-    {
-      assert_param(IS_RTC_ALARM_DATE_WEEKDAY_WEEKDAY(sAlarm-
>AlarmDateWeekDay));
-    }
-
-    tmpreg = (((uint32_t)RTC_ByteToBcd2(sAlarm->AlarmTime.Hours)   <<
RTC_ALRMAR_HU_Pos)  | \
-              ((uint32_t)RTC_ByteToBcd2(sAlarm->AlarmTime.Minutes) <<
RTC_ALRMAR_MNU_Pos) | \
-              ((uint32_t)RTC_ByteToBcd2(sAlarm->AlarmTime.Seconds))
| \
-              ((uint32_t)(sAlarm->AlarmTime.TimeFormat)            <<
RTC_ALRMAR_PM_Pos)  | \
-              ((uint32_t)RTC_ByteToBcd2(sAlarm->AlarmDateWeekDay)  <<
RTC_ALRMAR_DU_Pos)  | \
-              ((uint32_t)sAlarm->AlarmDateWeekDaySel)
| \
-              ((uint32_t)sAlarm->AlarmMask));
-  }
-  else
-  {
-    if ((hrtc->Instance->CR & RTC_CR_FMT) != 0U)
-    {
```

```
-      assert_param(IS_RTC_HOUR12(RTC_Bcd2ToByte(sAlarm-
>AlarmTime.Hours)));
-      assert_param(IS_RTC_HOURFORMAT12(sAlarm->AlarmTime.TimeFormat));
-    }
-    else
-    {
-      sAlarm->AlarmTime.TimeFormat = 0x00U;
-      assert_param(IS_RTC_HOUR24(RTC_Bcd2ToByte(sAlarm-
>AlarmTime.Hours)));
-    }
-
-    assert_param(IS_RTC_MINUTES(RTC_Bcd2ToByte(sAlarm-
>AlarmTime.Minutes)));
-    assert_param(IS_RTC_SECONDS(RTC_Bcd2ToByte(sAlarm-
>AlarmTime.Seconds)));
-
-    if (sAlarm->AlarmDateWeekDaySel == RTC_ALARMDATEWEEKDAYSEL_DATE)
-    {
-      assert_param(IS_RTC_ALARM_DATE_WEEKDAY_DATE(RTC_Bcd2ToByte(sAlarm-
>AlarmDateWeekDay)));
-    }
-    else
-    {
-
assert_param(IS_RTC_ALARM_DATE_WEEKDAY_WEEKDAY(RTC_Bcd2ToByte(sAlarm-
>AlarmDateWeekDay)));
-    }
-
-    tmpreg = (((uint32_t)(sAlarm->AlarmTime.Hours)      <<
RTC_ALRMAR_HU_Pos)  | \
-              ((uint32_t)(sAlarm->AlarmTime.Minutes)    <<
RTC_ALRMAR_MNU_Pos) | \
-              ((uint32_t) sAlarm->AlarmTime.Seconds)
| \
-              ((uint32_t)(sAlarm->AlarmTime.TimeFormat) <<
RTC_ALRMAR_PM_Pos)  | \
-              ((uint32_t)(sAlarm->AlarmDateWeekDay)     <<
RTC_ALRMAR_DU_Pos)  | \
-              ((uint32_t) sAlarm->AlarmDateWeekDaySel)
| \
-              ((uint32_t) sAlarm->AlarmMask));
-  }
-
-  /* Store the Alarm subseconds configuration */
-  subsecondtmpreg = (uint32_t)((uint32_t)(sAlarm->AlarmTime.SubSeconds)
| \
-                               (uint32_t)(sAlarm->AlarmSubSecondMask));
-
-  /* Disable the write protection for RTC registers */
-  __HAL_RTC_WRITEPROTECTION_DISABLE(hrtc);
-
-  if (sAlarm->Alarm == RTC_ALARM_A)
-  {
-    /* Disable Alarm A */
-    __HAL_RTC_ALARMA_DISABLE(hrtc);
-
-    /* In case interrupt mode is used, the interrupt source must be
disabled */
```

```
-    __HAL_RTC_ALARM_DISABLE_IT(hrtc, RTC_IT_ALRA);
-
-    /* Clear Alarm A flag */
-    __HAL_RTC_ALARM_CLEAR_FLAG(hrtc, RTC_FLAG_ALRAF);
-
-    /* Get tick */
-    tickstart = HAL_GetTick();
-
-    /* Wait till RTC ALRAWF flag is set and if timeout is reached exit
*/
-    while (__HAL_RTC_ALARM_GET_FLAG(hrtc, RTC_FLAG_ALRAWF) == 0U)
-    {
-      if ((HAL_GetTick() - tickstart) > RTC_TIMEOUT_VALUE)
-      {
-        /* Enable the write protection for RTC registers */
-        __HAL_RTC_WRITEPROTECTION_ENABLE(hrtc);
-
-        hrtc->State = HAL_RTC_STATE_TIMEOUT;
-
-        /* Process Unlocked */
-        __HAL_UNLOCK(hrtc);
-
-        return HAL_TIMEOUT;
-      }
-    }
-
-    /* Configure Alarm A register */
-    hrtc->Instance->ALRMAR = (uint32_t)tmpreg;
-    /* Configure Alarm A Subseconds register */
-    hrtc->Instance->ALRMASSR = subsecondtmpreg;
-    /* Enable Alarm A */
-    __HAL_RTC_ALARMA_ENABLE(hrtc);
-  }
-  else
-  {
-    /* Disable Alarm B */
-    __HAL_RTC_ALARMB_DISABLE(hrtc);
-
-    /* In case interrupt mode is used, the interrupt source must be
disabled */
-    __HAL_RTC_ALARM_DISABLE_IT(hrtc, RTC_IT_ALRB);
-
-    /* Clear Alarm B flag */
-    __HAL_RTC_ALARM_CLEAR_FLAG(hrtc, RTC_FLAG_ALRBF);
-
-    /* Get tick */
-    tickstart = HAL_GetTick();
-
-    /* Wait till RTC ALRBWF flag is set and if timeout is reached exit
*/
-    while (__HAL_RTC_ALARM_GET_FLAG(hrtc, RTC_FLAG_ALRBWF) == 0U)
-    {
-      if ((HAL_GetTick() - tickstart) > RTC_TIMEOUT_VALUE)
-      {
-        /* Enable the write protection for RTC registers */
-        __HAL_RTC_WRITEPROTECTION_ENABLE(hrtc);
-
-        hrtc->State = HAL_RTC_STATE_TIMEOUT;
```

```
-
-        /* Process Unlocked */
-        __HAL_UNLOCK(hrtc);
-
-        return HAL_TIMEOUT;
-      }
-    }
-
-    /* Configure Alarm B register */
-    hrtc->Instance->ALRMBR = (uint32_t)tmpreg;
-    /* Configure Alarm B Subseconds register */
-    hrtc->Instance->ALRMBSSR = subsecondtmpreg;
-    /* Enable Alarm B */
-    __HAL_RTC_ALARMB_ENABLE(hrtc);
-  }
-
-  /* Enable the write protection for RTC registers */
-  __HAL_RTC_WRITEPROTECTION_ENABLE(hrtc);
-
-  /* Change RTC state back to READY */
-  hrtc->State = HAL_RTC_STATE_READY;
-
-  /* Process Unlocked */
-  __HAL_UNLOCK(hrtc);
-
-  return HAL_OK;
-}
-
-/**
-  * @brief  Sets the specified RTC Alarm with Interrupt.
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
-  *                the configuration information for RTC.
-  * @param  sAlarm Pointer to Alarm structure
-  * @param  Format Specifies the format of the entered parameters.
-  *          This parameter can be one of the following values:
-  *             @arg RTC_FORMAT_BIN: Binary data format
-  *             @arg RTC_FORMAT_BCD: BCD data format
-  * @note   The Alarm register can only be written when the
corresponding Alarm
-  *         is disabled (Use the HAL_RTC_DeactivateAlarm()).
-  * @note   The HAL_RTC_SetTime() must be called before enabling the
Alarm feature.
-  * @retval HAL status
-  */
-HAL_StatusTypeDef HAL_RTC_SetAlarm_IT(RTC_HandleTypeDef *hrtc,
RTC_AlarmTypeDef *sAlarm, uint32_t Format)
-{
-  __IO uint32_t count  = RTC_TIMEOUT_VALUE * (SystemCoreClock / 32U /
1000U);
-       uint32_t tmpreg = 0U;
-       uint32_t subsecondtmpreg = 0U;
-
-  /* Check the parameters */
-  assert_param(IS_RTC_FORMAT(Format));
-  assert_param(IS_RTC_ALARM(sAlarm->Alarm));
-  assert_param(IS_RTC_ALARM_MASK(sAlarm->AlarmMask));
-  assert_param(IS_RTC_ALARM_DATE_WEEKDAY_SEL(sAlarm-
>AlarmDateWeekDaySel));
```

```
-  assert_param(IS_RTC_ALARM_SUB_SECOND_VALUE(sAlarm-
>AlarmTime.SubSeconds));
-  assert_param(IS_RTC_ALARM_SUB_SECOND_MASK(sAlarm-
>AlarmSubSecondMask));
-
-  /* Process Locked */
-  __HAL_LOCK(hrtc);
-
-  /* Change RTC state to BUSY */
-  hrtc->State = HAL_RTC_STATE_BUSY;
-
-  /* Check the data format (binary or BCD) and store the Alarm time and
date
-     configuration accordingly */
-  if (Format == RTC_FORMAT_BIN)
-  {
-    if ((hrtc->Instance->CR & RTC_CR_FMT) != 0U)
-    {
-      assert_param(IS_RTC_HOUR12(sAlarm->AlarmTime.Hours));
-      assert_param(IS_RTC_HOURFORMAT12(sAlarm->AlarmTime.TimeFormat));
-    }
-    else
-    {
-      sAlarm->AlarmTime.TimeFormat = 0x00U;
-      assert_param(IS_RTC_HOUR24(sAlarm->AlarmTime.Hours));
-    }
-    assert_param(IS_RTC_MINUTES(sAlarm->AlarmTime.Minutes));
-    assert_param(IS_RTC_SECONDS(sAlarm->AlarmTime.Seconds));
-
-    if (sAlarm->AlarmDateWeekDaySel == RTC_ALARMDATEWEEKDAYSEL_DATE)
-    {
-      assert_param(IS_RTC_ALARM_DATE_WEEKDAY_DATE(sAlarm-
>AlarmDateWeekDay));
-    }
-    else
-    {
-      assert_param(IS_RTC_ALARM_DATE_WEEKDAY_WEEKDAY(sAlarm-
>AlarmDateWeekDay));
-    }
-
-    tmpreg = (((uint32_t)RTC_ByteToBcd2(sAlarm->AlarmTime.Hours)   <<
RTC_ALRMAR_HU_Pos)  | \
-              ((uint32_t)RTC_ByteToBcd2(sAlarm->AlarmTime.Minutes) <<
RTC_ALRMAR_MNU_Pos) | \
-              ((uint32_t)RTC_ByteToBcd2(sAlarm->AlarmTime.Seconds))
| \
-              ((uint32_t)(sAlarm->AlarmTime.TimeFormat)            <<
RTC_ALRMAR_PM_Pos)  | \
-              ((uint32_t)RTC_ByteToBcd2(sAlarm->AlarmDateWeekDay)  <<
RTC_ALRMAR_DU_Pos)  | \
-              ((uint32_t)sAlarm->AlarmDateWeekDaySel)
| \
-              ((uint32_t)sAlarm->AlarmMask));
-  }
-  else
-  {
-    if ((hrtc->Instance->CR & RTC_CR_FMT) != 0U)
-    {
```

```
-      assert_param(IS_RTC_HOUR12(RTC_Bcd2ToByte(sAlarm-
>AlarmTime.Hours)));
-      assert_param(IS_RTC_HOURFORMAT12(sAlarm->AlarmTime.TimeFormat));
-    }
-    else
-    {
-      sAlarm->AlarmTime.TimeFormat = 0x00U;
-      assert_param(IS_RTC_HOUR24(RTC_Bcd2ToByte(sAlarm-
>AlarmTime.Hours)));
-    }
-
-    assert_param(IS_RTC_MINUTES(RTC_Bcd2ToByte(sAlarm-
>AlarmTime.Minutes)));
-    assert_param(IS_RTC_SECONDS(RTC_Bcd2ToByte(sAlarm-
>AlarmTime.Seconds)));
-
-    if (sAlarm->AlarmDateWeekDaySel == RTC_ALARMDATEWEEKDAYSEL_DATE)
-    {
-      assert_param(IS_RTC_ALARM_DATE_WEEKDAY_DATE(RTC_Bcd2ToByte(sAlarm-
>AlarmDateWeekDay)));
-    }
-    else
-    {
-
assert_param(IS_RTC_ALARM_DATE_WEEKDAY_WEEKDAY(RTC_Bcd2ToByte(sAlarm-
>AlarmDateWeekDay)));
-    }
-
-    tmpreg = (((uint32_t)(sAlarm->AlarmTime.Hours)      <<
RTC_ALRMAR_HU_Pos)  | \
-              ((uint32_t)(sAlarm->AlarmTime.Minutes)    <<
RTC_ALRMAR_MNU_Pos) | \
-              ((uint32_t) sAlarm->AlarmTime.Seconds)
| \
-              ((uint32_t)(sAlarm->AlarmTime.TimeFormat) <<
RTC_ALRMAR_PM_Pos)  | \
-              ((uint32_t)(sAlarm->AlarmDateWeekDay)     <<
RTC_ALRMAR_DU_Pos)  | \
-              ((uint32_t) sAlarm->AlarmDateWeekDaySel)
| \
-              ((uint32_t) sAlarm->AlarmMask));
-  }
-
-  /* Store the Alarm subseconds configuration */
-  subsecondtmpreg = (uint32_t)((uint32_t)(sAlarm->AlarmTime.SubSeconds)
| \
-                               (uint32_t)(sAlarm->AlarmSubSecondMask));
-
-  /* Disable the write protection for RTC registers */
-  __HAL_RTC_WRITEPROTECTION_DISABLE(hrtc);
-
-  if (sAlarm->Alarm == RTC_ALARM_A)
-  {
-    /* Disable Alarm A */
-    __HAL_RTC_ALARMA_DISABLE(hrtc);
-
-    /* Clear Alarm A flag */
-    __HAL_RTC_ALARM_CLEAR_FLAG(hrtc, RTC_FLAG_ALRAF);
```

```
-
-    /* Wait till RTC ALRAWF flag is set and if timeout is reached exit
*/
-    do
-    {
-      count = count - 1U;
-      if (count == 0U)
-      {
-        /* Enable the write protection for RTC registers */
-        __HAL_RTC_WRITEPROTECTION_ENABLE(hrtc);
-
-        hrtc->State = HAL_RTC_STATE_TIMEOUT;
-
-        /* Process Unlocked */
-        __HAL_UNLOCK(hrtc);
-
-        return HAL_TIMEOUT;
-      }
-    } while (__HAL_RTC_ALARM_GET_FLAG(hrtc, RTC_FLAG_ALRAWF) == 0U);
-
-  /* Configure Alarm A register */
-    hrtc->Instance->ALRMAR = (uint32_t)tmpreg;
-    /* Configure Alarm A Subseconds register */
-    hrtc->Instance->ALRMASSR = subsecondtmpreg;
-    /* Enable Alarm A */
-    __HAL_RTC_ALARMA_ENABLE(hrtc);
-    /* Enable Alarm A interrupt */
-    __HAL_RTC_ALARM_ENABLE_IT(hrtc, RTC_IT_ALRA);
-  }
-  else
-  {
-    /* Disable Alarm B */
-    __HAL_RTC_ALARMB_DISABLE(hrtc);
-
-    /* Clear Alarm B flag */
-    __HAL_RTC_ALARM_CLEAR_FLAG(hrtc, RTC_FLAG_ALRBF);
-
-    /* Reload the counter */
-    count = RTC_TIMEOUT_VALUE * (SystemCoreClock / 32U / 1000U);
-
-    /* Wait till RTC ALRBWF flag is set and if timeout is reached exit
*/
-    do
-    {
-      count = count - 1U;
-      if (count == 0U)
-      {
-        /* Enable the write protection for RTC registers */
-        __HAL_RTC_WRITEPROTECTION_ENABLE(hrtc);
-
-        hrtc->State = HAL_RTC_STATE_TIMEOUT;
-
-        /* Process Unlocked */
-        __HAL_UNLOCK(hrtc);
-
-        return HAL_TIMEOUT;
-      }
-    } while (__HAL_RTC_ALARM_GET_FLAG(hrtc, RTC_FLAG_ALRBWF) == 0U);
```

```
-
-    /* Configure Alarm B register */
-    hrtc->Instance->ALRMBR = (uint32_t)tmpreg;
-    /* Configure Alarm B Subseconds register */
-    hrtc->Instance->ALRMBSSR = subsecondtmpreg;
-    /* Enable Alarm B */
-    __HAL_RTC_ALARMB_ENABLE(hrtc);
-    /* Enable Alarm B interrupt */
-    __HAL_RTC_ALARM_ENABLE_IT(hrtc, RTC_IT_ALRB);
-  }
-
-  /* Enable and configure the EXTI line associated to the RTC Alarm
interrupt */
-  __HAL_RTC_ALARM_EXTI_ENABLE_IT();
-  __HAL_RTC_ALARM_EXTI_ENABLE_RISING_EDGE();
-
-  /* Enable the write protection for RTC registers */
-  __HAL_RTC_WRITEPROTECTION_ENABLE(hrtc);
-
-  /* Change RTC state back to READY */
-  hrtc->State = HAL_RTC_STATE_READY;
-
-  /* Process Unlocked */
-  __HAL_UNLOCK(hrtc);
-
-  return HAL_OK;
-}
-
-/**
-  * @brief  Deactivates the specified RTC Alarm.
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
-  *                the configuration information for RTC.
-  * @param  Alarm Specifies the Alarm.
-  *          This parameter can be one of the following values:
-  *            @arg RTC_ALARM_A: Alarm A
-  *            @arg RTC_ALARM_B: Alarm B
-  * @retval HAL status
-  */
-HAL_StatusTypeDef HAL_RTC_DeactivateAlarm(RTC_HandleTypeDef *hrtc,
uint32_t Alarm)
-{
-  uint32_t tickstart = 0U;
-
-  /* Check the parameters */
-  assert_param(IS_RTC_ALARM(Alarm));
-
-  /* Process Locked */
-  __HAL_LOCK(hrtc);
-
-  hrtc->State = HAL_RTC_STATE_BUSY;
-
-  /* Disable the write protection for RTC registers */
-  __HAL_RTC_WRITEPROTECTION_DISABLE(hrtc);
-
-  if (Alarm == RTC_ALARM_A)
-  {
-    /* Disable Alarm A */
-    __HAL_RTC_ALARMA_DISABLE(hrtc);
```

```
-
-    /* In case interrupt mode is used, the interrupt source must be
disabled */
-    __HAL_RTC_ALARM_DISABLE_IT(hrtc, RTC_IT_ALRA);
-
-    /* Get tick */
-    tickstart = HAL_GetTick();
-
-    /* Wait till RTC ALRAWF flag is set and if timeout is reached exit
*/
-    while (__HAL_RTC_ALARM_GET_FLAG(hrtc, RTC_FLAG_ALRAWF) == 0U)
-    {
-      if ((HAL_GetTick() - tickstart) > RTC_TIMEOUT_VALUE)
-      {
-        /* Enable the write protection for RTC registers */
-        __HAL_RTC_WRITEPROTECTION_ENABLE(hrtc);
-
-        hrtc->State = HAL_RTC_STATE_TIMEOUT;
-
-        /* Process Unlocked */
-        __HAL_UNLOCK(hrtc);
-
-        return HAL_TIMEOUT;
-      }
-    }
-  }
-  else
-  {
-    /* Disable Alarm B */
-    __HAL_RTC_ALARMB_DISABLE(hrtc);
-
-    /* In case interrupt mode is used, the interrupt source must be
disabled */
-    __HAL_RTC_ALARM_DISABLE_IT(hrtc, RTC_IT_ALRB);
-
-    /* Get tick */
-    tickstart = HAL_GetTick();
-
-    /* Wait till RTC ALRBWF flag is set and if timeout is reached exit
*/
-    while (__HAL_RTC_ALARM_GET_FLAG(hrtc, RTC_FLAG_ALRBWF) == 0U)
-    {
-      if ((HAL_GetTick() - tickstart) > RTC_TIMEOUT_VALUE)
-      {
-        /* Enable the write protection for RTC registers */
-        __HAL_RTC_WRITEPROTECTION_ENABLE(hrtc);
-
-        hrtc->State = HAL_RTC_STATE_TIMEOUT;
-
-        /* Process Unlocked */
-        __HAL_UNLOCK(hrtc);
-
-        return HAL_TIMEOUT;
-      }
-    }
-  }
-
-  /* Enable the write protection for RTC registers */
```

```
-  __HAL_RTC_WRITEPROTECTION_ENABLE(hrtc);
-
-  hrtc->State = HAL_RTC_STATE_READY;
-
-  /* Process Unlocked */
-  __HAL_UNLOCK(hrtc);
-
-  return HAL_OK;
-}
-
-/**
-  * @brief  Gets the RTC Alarm value and masks.
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
-  *               the configuration information for RTC.
-  * @param  sAlarm Pointer to Date structure
-  * @param  Alarm Specifies the Alarm.
-  *          This parameter can be one of the following values:
-  *            @arg RTC_ALARM_A: Alarm A
-  *            @arg RTC_ALARM_B: Alarm B
-  * @param  Format Specifies the format of the entered parameters.
-  *          This parameter can be one of the following values:
-  *             @arg RTC_FORMAT_BIN: Binary data format
-  *             @arg RTC_FORMAT_BCD: BCD data format
-  * @retval HAL status
-  */
-HAL_StatusTypeDef HAL_RTC_GetAlarm(RTC_HandleTypeDef *hrtc,
RTC_AlarmTypeDef *sAlarm, uint32_t Alarm, uint32_t Format)
-{
-  uint32_t tmpreg = 0U;
-  uint32_t subsecondtmpreg = 0U;
-
-  /* Check the parameters */
-  assert_param(IS_RTC_FORMAT(Format));
-  assert_param(IS_RTC_ALARM(Alarm));
-
-  if (Alarm == RTC_ALARM_A)
-  {
-    sAlarm->Alarm = RTC_ALARM_A;
-
-    tmpreg = (uint32_t)(hrtc->Instance->ALRMAR);
-    subsecondtmpreg = (uint32_t)((hrtc->Instance->ALRMASSR) &
RTC_ALRMASSR_SS);
-  }
-  else
-  {
-    sAlarm->Alarm = RTC_ALARM_B;
-
-    tmpreg = (uint32_t)(hrtc->Instance->ALRMBR);
-    subsecondtmpreg = (uint32_t)((hrtc->Instance->ALRMBSSR) &
RTC_ALRMBSSR_SS);
-  }
-
-  /* Fill the structure with the read parameters */
-  sAlarm->AlarmTime.Hours      = (uint8_t) ((tmpreg & (RTC_ALRMAR_HT  |
RTC_ALRMAR_HU))  >> RTC_ALRMAR_HU_Pos);
-  sAlarm->AlarmTime.Minutes    = (uint8_t) ((tmpreg & (RTC_ALRMAR_MNT |
RTC_ALRMAR_MNU)) >> RTC_ALRMAR_MNU_Pos);
```

```
-  sAlarm->AlarmTime.Seconds    = (uint8_t) ( tmpreg & (RTC_ALRMAR_ST  |
RTC_ALRMAR_SU));
-  sAlarm->AlarmTime.TimeFormat = (uint8_t) ((tmpreg & RTC_ALRMAR_PM)
>> RTC_TR_PM_Pos);
-  sAlarm->AlarmTime.SubSeconds = (uint32_t) subsecondtmpreg;
-  sAlarm->AlarmDateWeekDay     = (uint8_t) ((tmpreg & (RTC_ALRMAR_DT  |
RTC_ALRMAR_DU))  >> RTC_ALRMAR_DU_Pos);
-  sAlarm->AlarmDateWeekDaySel  = (uint32_t) (tmpreg & RTC_ALRMAR_WDSEL);
-  sAlarm->AlarmMask            = (uint32_t) (tmpreg &
RTC_ALARMMASK_ALL);
-
-  if (Format == RTC_FORMAT_BIN)
-  {
-    sAlarm->AlarmTime.Hours   = RTC_Bcd2ToByte(sAlarm->AlarmTime.Hours);
-    sAlarm->AlarmTime.Minutes = RTC_Bcd2ToByte(sAlarm-
>AlarmTime.Minutes);
-    sAlarm->AlarmTime.Seconds = RTC_Bcd2ToByte(sAlarm-
>AlarmTime.Seconds);
-    sAlarm->AlarmDateWeekDay  = RTC_Bcd2ToByte(sAlarm-
>AlarmDateWeekDay);
-  }
-
-  return HAL_OK;
-}
-
-/**
-  * @brief  Handles Alarm interrupt request.
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
-  *                the configuration information for RTC.
-  * @retval None
-  */
-void HAL_RTC_AlarmIRQHandler(RTC_HandleTypeDef *hrtc)
-{
-  /* Clear the EXTI flag associated to the RTC Alarm interrupt */
-  __HAL_RTC_ALARM_EXTI_CLEAR_FLAG();
-
-  /* Get the Alarm A interrupt source enable status */
-  if (__HAL_RTC_ALARM_GET_IT_SOURCE(hrtc, RTC_IT_ALRA) != 0U)
-  {
-    /* Get the pending status of the Alarm A Interrupt */
-    if (__HAL_RTC_ALARM_GET_FLAG(hrtc, RTC_FLAG_ALRAF) != 0U)
-    {
-      /* Clear the Alarm A interrupt pending bit */
-      __HAL_RTC_ALARM_CLEAR_FLAG(hrtc, RTC_FLAG_ALRAF);
-
-      /* Alarm A callback */
-#if (USE_HAL_RTC_REGISTER_CALLBACKS == 1)
-      hrtc->AlarmAEventCallback(hrtc);
-#else
-      HAL_RTC_AlarmAEventCallback(hrtc);
-#endif /* USE_HAL_RTC_REGISTER_CALLBACKS */
-    }
-  }
-
-  /* Get the Alarm B interrupt source enable status */
-  if (__HAL_RTC_ALARM_GET_IT_SOURCE(hrtc, RTC_IT_ALRB) != 0U)
-  {
-    /* Get the pending status of the Alarm B Interrupt */
```

```
-    if (__HAL_RTC_ALARM_GET_FLAG(hrtc, RTC_FLAG_ALRBF) != 0U)
-    {
-      /* Clear the Alarm B interrupt pending bit */
-      __HAL_RTC_ALARM_CLEAR_FLAG(hrtc, RTC_FLAG_ALRBF);
-
-      /* Alarm B callback */
-#if (USE_HAL_RTC_REGISTER_CALLBACKS == 1)
-      hrtc->AlarmBEventCallback(hrtc);
-#else
-      HAL_RTCEx_AlarmBEventCallback(hrtc);
-#endif /* USE_HAL_RTC_REGISTER_CALLBACKS */
-    }
-  }
-
-  /* Change RTC state */
-  hrtc->State = HAL_RTC_STATE_READY;
-}
-
-/**
-  * @brief  Alarm A callback.
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
-  *                the configuration information for RTC.
-  * @retval None
-  */
-__weak void HAL_RTC_AlarmAEventCallback(RTC_HandleTypeDef *hrtc)
-{
-  /* Prevent unused argument(s) compilation warning */
-  UNUSED(hrtc);
-
-  /* NOTE: This function should not be modified, when the callback is
needed,
-           the HAL_RTC_AlarmAEventCallback could be implemented in the
user file
-   */
-}
-
-/**
-  * @brief  Handles Alarm A Polling request.
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
-  *                the configuration information for RTC.
-  * @param  Timeout Timeout duration
-  * @retval HAL status
-  */
-HAL_StatusTypeDef HAL_RTC_PollForAlarmAEvent(RTC_HandleTypeDef *hrtc,
uint32_t Timeout)
-{
-  uint32_t tickstart = 0U;
-
-  /* Get tick */
-  tickstart = HAL_GetTick();
-
-  /* Wait till RTC ALRAF flag is set and if timeout is reached exit */
-  while (__HAL_RTC_ALARM_GET_FLAG(hrtc, RTC_FLAG_ALRAF) == 0U)
-  {
-    if (Timeout != HAL_MAX_DELAY)
-    {
-      if ((Timeout == 0U) || ((HAL_GetTick() - tickstart) > Timeout))
-      {
```

```
-        hrtc->State = HAL_RTC_STATE_TIMEOUT;
-        return HAL_TIMEOUT;
-      }
-    }
-  }
-
-  /* Clear the Alarm flag */
-  __HAL_RTC_ALARM_CLEAR_FLAG(hrtc, RTC_FLAG_ALRAF);
-
-  /* Change RTC state */
-  hrtc->State = HAL_RTC_STATE_READY;
-
-  return HAL_OK;
-}
-
-/**
-  * @}
-  */
-
-/** @defgroup RTC_Exported_Functions_Group4 Peripheral Control functions
-  * @brief    Peripheral Control functions
-  *
-@verbatim
-
==============================================================================
======
-                      ##### Peripheral Control functions #####
-
==============================================================================
======
-    [..]
-    This subsection provides functions allowing to
-      (+) Wait for RTC Time and Date Synchronization
-      (+) Manage RTC Summer or Winter time change
-
-@endverbatim
-  * @{
-  */
-
-/**
-  * @brief  Waits until the RTC Time and Date registers (RTC_TR and
RTC_DR) are
-  *         synchronized with RTC APB clock.
-  * @note   The RTC Resynchronization mode is write protected, use the
-  *         __HAL_RTC_WRITEPROTECTION_DISABLE() before calling this
function.
-  * @note   To read the calendar through the shadow registers after
Calendar
-  *         initialization, calendar update or after wakeup from low
power modes
-  *         the software must first clear the RSF flag.
-  *         The software must then wait until it is set again before
reading
-  *         the calendar, which means that the calendar registers have
been
-  *         correctly copied into the RTC_TR and RTC_DR shadow
registers.
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
```

```
-  *                the configuration information for RTC.
-  * @retval HAL status
-  */
-HAL_StatusTypeDef HAL_RTC_WaitForSynchro(RTC_HandleTypeDef *hrtc)
-{
-  uint32_t tickstart = 0U;
-
-  /* Clear RSF flag, keep reserved bits at reset values (setting other
flags has no effect) */
-  hrtc->Instance->ISR = ((uint32_t)(RTC_RSF_MASK &
RTC_ISR_RESERVED_MASK));
-
-  /* Get tick */
-  tickstart = HAL_GetTick();
-
-  /* Wait the registers to be synchronised */
-  while ((hrtc->Instance->ISR & RTC_ISR_RSF) == 0U)
-  {
-    if ((HAL_GetTick() - tickstart) > RTC_TIMEOUT_VALUE)
-    {
-      return HAL_TIMEOUT;
-    }
-  }
-
-  return HAL_OK;
-}
-
-/**
-  * @brief  Daylight Saving Time, adds one hour to the calendar in one
-  *         single operation without going through the initialization
procedure.
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
-  *                the configuration information for RTC.
-  * @retval None
-  */
-void HAL_RTC_DST_Add1Hour(RTC_HandleTypeDef *hrtc)
-{
-  __HAL_RTC_WRITEPROTECTION_DISABLE(hrtc);
-  SET_BIT(hrtc->Instance->CR, RTC_CR_ADD1H);
-  __HAL_RTC_WRITEPROTECTION_ENABLE(hrtc);
-}
-
-/**
-  * @brief  Daylight Saving Time, subtracts one hour from the calendar
in one
-  *         single operation without going through the initialization
procedure.
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
-  *                the configuration information for RTC.
-  * @retval None
-  */
-void HAL_RTC_DST_Sub1Hour(RTC_HandleTypeDef *hrtc)
-{
-  __HAL_RTC_WRITEPROTECTION_DISABLE(hrtc);
-  SET_BIT(hrtc->Instance->CR, RTC_CR_SUB1H);
-  __HAL_RTC_WRITEPROTECTION_ENABLE(hrtc);
-}
-
```

```
-/**
-  * @brief  Daylight Saving Time, sets the store operation bit.
-  * @note   It can be used by the software in order to memorize the DST
status.
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
-  *                the configuration information for RTC.
-  * @retval None
-  */
-void HAL_RTC_DST_SetStoreOperation(RTC_HandleTypeDef *hrtc)
-{
-  __HAL_RTC_WRITEPROTECTION_DISABLE(hrtc);
-  SET_BIT(hrtc->Instance->CR, RTC_CR_BKP);
-  __HAL_RTC_WRITEPROTECTION_ENABLE(hrtc);
-}
-
-/**
-  * @brief  Daylight Saving Time, clears the store operation bit.
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
-  *                the configuration information for RTC.
-  * @retval None
-  */
-void HAL_RTC_DST_ClearStoreOperation(RTC_HandleTypeDef *hrtc)
-{
-  __HAL_RTC_WRITEPROTECTION_DISABLE(hrtc);
-  CLEAR_BIT(hrtc->Instance->CR, RTC_CR_BKP);
-  __HAL_RTC_WRITEPROTECTION_ENABLE(hrtc);
-}
-
-/**
-  * @brief  Daylight Saving Time, reads the store operation bit.
-  * @param  hrtc RTC handle
-  * @retval operation see RTC_StoreOperation_Definitions
-  */
-uint32_t HAL_RTC_DST_ReadStoreOperation(RTC_HandleTypeDef *hrtc)
-{
-  return READ_BIT(hrtc->Instance->CR, RTC_CR_BKP);
-}
-
-/**
-  * @}
-  */
-
-/** @defgroup RTC_Exported_Functions_Group5 Peripheral State functions
-  * @brief    Peripheral State functions
-  *
-@verbatim
-
==========================================================================
======
-                      ##### Peripheral State functions #####
-
==========================================================================
======
-    [..]
-    This subsection provides functions allowing to
-      (+) Get RTC state
-
-@endverbatim
```

```
-  * @{
-  */
-/**
-  * @brief  Returns the RTC state.
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
-  *                the configuration information for RTC.
-  * @retval HAL state
-  */
-HAL_RTCStateTypeDef HAL_RTC_GetState(RTC_HandleTypeDef *hrtc)
-{
-  return hrtc->State;
-}
-
-/**
-  * @}
-  */
-
-
-/**
-  * @}
-  */
-
-/** @addtogroup RTC_Private_Functions
-  * @{
-  */
-
-/**
-  * @brief  Enters the RTC Initialization mode.
-  * @note   The RTC Initialization mode is write protected, use the
-  *         __HAL_RTC_WRITEPROTECTION_DISABLE() before calling this
function.
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
-  *                the configuration information for RTC.
-  * @retval HAL status
-  */
-HAL_StatusTypeDef RTC_EnterInitMode(RTC_HandleTypeDef *hrtc)
-{
-  uint32_t tickstart = 0U;
-  HAL_StatusTypeDef status = HAL_OK;
-
-  /* Check that Initialization mode is not already set */
-  if (READ_BIT(hrtc->Instance->ISR, RTC_ISR_INITF) == 0U)
-  {
-    /* Set INIT bit to enter Initialization mode */
-    SET_BIT(hrtc->Instance->ISR, RTC_ISR_INIT);
-
-    /* Get tick */
-    tickstart = HAL_GetTick();
-
-    /* Wait till RTC is in INIT state and if timeout is reached exit */
-    while ((READ_BIT(hrtc->Instance->ISR, RTC_ISR_INITF) == 0U) &&
(status != HAL_ERROR))
-    {
-      if ((HAL_GetTick() - tickstart) > RTC_TIMEOUT_VALUE)
-      {
-        /* Set RTC state */
-        hrtc->State = HAL_RTC_STATE_ERROR;
-        status = HAL_ERROR;
```

```
-      }
-    }
-  }
-
-  return status;
-}
-
-/**
-  * @brief  Exits the RTC Initialization mode.
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
-  *                the configuration information for RTC.
-  * @retval HAL status
-  */
-HAL_StatusTypeDef RTC_ExitInitMode(RTC_HandleTypeDef *hrtc)
-{
-  HAL_StatusTypeDef status = HAL_OK;
-
-  /* Clear INIT bit to exit Initialization mode */
-  CLEAR_BIT(hrtc->Instance->ISR, RTC_ISR_INIT);
-
-  /* If CR_BYPSHAD bit = 0, wait for synchro */
-  if (READ_BIT(hrtc->Instance->CR, RTC_CR_BYPSHAD) == 0U)
-  {
-    if (HAL_RTC_WaitForSynchro(hrtc) != HAL_OK)
-    {
-      /* Set RTC state */
-      hrtc->State = HAL_RTC_STATE_ERROR;
-      status = HAL_ERROR;
-    }
-  }
-
-  return status;
-}
-
-/**
-  * @brief  Converts a 2-digit number from decimal to BCD format.
-  * @param  number decimal-formatted number (from 0 to 99) to be
converted
-  * @retval Converted byte
-  */
-uint8_t RTC_ByteToBcd2(uint8_t number)
-{
-  uint32_t bcdhigh = 0U;
-
-  while (number >= 10U)
-  {
-    bcdhigh++;
-    number -= 10U;
-  }
-
-  return ((uint8_t)(bcdhigh << 4U) | number);
-}
-
-/**
-  * @brief  Converts a 2-digit number from BCD to decimal format.
-  * @param  number BCD-formatted number (from 00 to 99) to be converted
-  * @retval Converted word
-  */
```

```
-uint8_t RTC_Bcd2ToByte(uint8_t number)
-{
-  uint32_t tens = 0U;
-  tens = (((uint32_t)number & 0xF0U) >> 4U) * 10U;
-  return (uint8_t)(tens + ((uint32_t)number & 0x0FU));
-}
-
-/**
-  * @}
-  */
-
-#endif /* HAL_RTC_MODULE_ENABLED */
-/**
-  * @}
-  */
-
-/**
-  * @}
-  */
diff --git a/Drivers/STM32F4xx_HAL_Driver/Src/stm32f4xx_hal_rtc_ex.c
b/Drivers/STM32F4xx_HAL_Driver/Src/stm32f4xx_hal_rtc_ex.c
deleted file mode 100644
index 2a032ab..0000000
--- a/Drivers/STM32F4xx_HAL_Driver/Src/stm32f4xx_hal_rtc_ex.c
+++ /dev/null
@@ -1,1882 +0,0 @@
-/**
-
******************************************************************************
*****
-  * @file    stm32f4xx_hal_rtc_ex.c
-  * @author  MCD Application Team
-  * @brief   Extended RTC HAL module driver.
-  *          This file provides firmware functions to manage the
following
-  *          functionalities of the Real-Time Clock (RTC) Extended
peripheral:
-  *           + RTC Timestamp functions
-  *           + RTC Tamper functions
-  *           + RTC Wakeup functions
-  *           + Extended Control functions
-  *           + Extended RTC features functions
-  *
-
******************************************************************************
*****
-  * @attention
-  *
-  * Copyright (c) 2016 STMicroelectronics.
-  * All rights reserved.
-  *
-  * This software is licensed under terms that can be found in the
LICENSE file
-  * in the root directory of this software component.
-  * If no LICENSE file comes with this software, it is provided AS-IS.
-  *
```

```
-
*******************************************************************************
*****
-  @verbatim
-
==============================================================================
=====
-                        ##### How to use this driver #####
-
==============================================================================
=====
-  [..]
-    (+) Enable the RTC domain access.
-    (+) Configure the RTC Prescaler (Asynchronous and Synchronous) and
RTC hour
-        format using the HAL_RTC_Init() function.
-
-  *** RTC Wakeup configuration ***
-  ================================
-  [..]
-    (+) To configure the RTC Wakeup Clock source and Counter use the
-        HAL_RTCEx_SetWakeUpTimer() function.
-        You can also configure the RTC Wakeup timer in interrupt mode
using the
-        HAL_RTCEx_SetWakeUpTimer_IT() function.
-    (+) To read the RTC Wakeup Counter register, use the
HAL_RTCEx_GetWakeUpTimer()
-        function.
-
-  *** Timestamp configuration ***
-  ================================
-  [..]
-    (+) To configure the RTC Timestamp use the HAL_RTCEx_SetTimeStamp()
function.
-        You can also configure the RTC Timestamp with interrupt mode
using the
-        HAL_RTCEx_SetTimeStamp_IT() function.
-    (+) To read the RTC Timestamp Time and Date register, use the
-        HAL_RTCEx_GetTimeStamp() function.
-    (+) The Timestamp alternate function can be mapped either to RTC_AF1
(PC13)
-        or RTC_AF2 (PI8) depending on the value of TSINSEL bit in
RTC_TAFCR
-        register.
-        For STM32F446xx devices RTC_AF2 corresponds to pin PA0 and not
to pin PI8.
-        The corresponding pin is also selected by
HAL_RTCEx_SetTimeStamp()
-        or HAL_RTCEx_SetTimeStamp_IT() functions.
-
-  *** Tamper configuration ***
-  ============================
-  [..]
-    (+) To enable the RTC Tamper and configure the Tamper filter count,
trigger
-        Edge or Level according to the Tamper filter value (if equal to
0 Edge
```

```
-        else Level), sampling frequency, precharge or discharge and
Pull-UP use
-        the HAL_RTCEx_SetTamper() function.
-        You can configure RTC Tamper in interrupt mode using
HAL_RTCEx_SetTamper_IT()
-        function.
-    (+) The TAMPER1 alternate function can be mapped either to RTC_AF1
(PC13)
-        or RTC_AF2 (PI8) depending on the value of TAMP1INSEL bit in
RTC_TAFCR
-        register.
-        The corresponding pin is also selected by HAL_RTCEx_SetTamper()
-        or HAL_RTCEx_SetTamper_IT() functions.
-    (+) The TAMPER2 alternate function is mapped to RTC_AF2 (PI8).
-        For STM32F446xx devices RTC_AF2 corresponds to pin PA0 and not
to pin PI8.
-
-  *** Backup Data Registers configuration ***
-  ============================================
-  [..]
-    (+) To write to the RTC Backup Data registers, use the
HAL_RTCEx_BKUPWrite()
-        function.
-    (+) To read the RTC Backup Data registers, use the
HAL_RTCEx_BKUPRead()
-        function.
-
-  *** Coarse Digital Calibration configuration ***
-  ================================================
-  [..]
-    (+) The Coarse Digital Calibration can be used to compensate crystal
inaccuracy
-        by setting the DCS bit in RTC_CALIBR register.
-    (+) When positive calibration is enabled (DCS = '0'), 2 asynchronous
prescaler
-        clock cycles are added every minute during 2xDC minutes.
-        This causes the calendar to be updated sooner, thereby adjusting
the
-        effective RTC frequency to be a bit higher.
-    (+) When negative calibration is enabled (DCS = '1'), 1 asynchronous
prescaler
-        clock cycle is removed every minute during 2xDC minutes.
-        This causes the calendar to be updated later, thereby adjusting
the
-        effective RTC frequency to be a bit lower.
-    (+) DC is configured through bits DC[4:0] of RTC_CALIBR register.
This number
-        ranges from 0 to 31 corresponding to a time interval (2xDC)
ranging from
-        0 to 62.
-    (+) In order to measure the clock deviation, a 512 Hz clock is
output for
-        calibration.
-    (+) To configure the RTC Coarse Digital Calibration value and sign
use the
-        HAL_RTCEx_SetCoarseCalib() function.
-
-  *** Smooth Digital Calibration configuration ***
```

```
-  ================================================
-  [..]
-    (+) RTC frequency can be digitally calibrated with a resolution of
about
-        0.954 ppm with a range from -487.1 ppm to +488.5 ppm.
-        The correction of the frequency is performed using a series of
small
-        adjustments (adding and/or subtracting individual RTCCLK
pulses).
-    (+) The smooth digital calibration is performed during a cycle of
about 2^20
-        RTCCLK pulses (or 32 seconds) when the input frequency is 32,768
Hz.
-        This cycle is maintained by a 20-bit counter clocked by RTCCLK.
-    (+) The smooth calibration register (RTC_CALR) specifies the number
of RTCCLK
-        clock cycles to be masked during the 32-second cycle.
-    (+) To configure the RTC Smooth Digital Calibration value and the
corresponding
-        calibration cycle period (32s,16s and 8s) use the
HAL_RTCEx_SetSmoothCalib()
-        function.
-
-  @endverbatim
-
**************************************************************************
*****
-  */
-
-/* Includes ------------------------------------------------------------
------*/
-#include "stm32f4xx_hal.h"
-
-/** @addtogroup STM32F4xx_HAL_Driver
-  * @{
-  */
-
-/** @defgroup RTCEx RTCEx
-  * @brief    RTC Extended HAL module driver
-  * @{
-  */
-
-#ifdef HAL_RTC_MODULE_ENABLED
-
-/* Private typedef -----------------------------------------------------
------*/
-/* Private define ------------------------------------------------------
------*/
-/* Private macro -------------------------------------------------------
------*/
-/* Private variables ---------------------------------------------------
------*/
-/* Private function prototypes -----------------------------------------
------*/
-/* Exported functions --------------------------------------------------
------*/
-
-/** @defgroup RTCEx_Exported_Functions RTCEx Exported Functions
```

```
-  * @{
-  */
-
-/** @defgroup RTCEx_Exported_Functions_Group1 RTC Timestamp and Tamper
functions
-  * @brief    RTC Timestamp and Tamper functions
-  *
-@verbatim
-
==============================================================================
======
-                  ##### RTC Timestamp and Tamper functions #####
-
==============================================================================
======
-
- [..] This section provides functions allowing to configure Timestamp
feature
-
-@endverbatim
-  * @{
-  */
-
-/**
-  * @brief  Sets Timestamp.
-  * @note   This API must be called before enabling the Timestamp
feature.
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
-  *                the configuration information for RTC.
-  * @param  RTC_TimeStampEdge Specifies the pin edge on which the
Timestamp is
-  *         activated.
-  *          This parameter can be one of the following values:
-  *             @arg RTC_TIMESTAMPEDGE_RISING: the Timestamp event
occurs on
-  *                                            the rising edge of the
related pin.
-  *             @arg RTC_TIMESTAMPEDGE_FALLING: the Timestamp event
occurs on
-  *                                            the falling edge of the
related pin.
-  * @param  RTC_TimeStampPin Specifies the RTC Timestamp Pin.
-  *          This parameter can be one of the following values:
-  *             @arg RTC_TIMESTAMPPIN_DEFAULT: PC13 is selected as RTC
Timestamp Pin.
-  *             @arg RTC_TIMESTAMPPIN_POS1: PI8 is selected as RTC
Timestamp Pin.
-  * @note RTC_TIMESTAMPPIN_POS1 corresponds to pin PA0 in the case of
-  *       STM32F446xx devices.
-  * @note RTC_TIMESTAMPPIN_POS1 is not applicable to the following list
of devices:
-  *       STM32F412xx, STM32F413xx and STM32F423xx.
-  * @retval HAL status
-  */
-HAL_StatusTypeDef HAL_RTCEx_SetTimeStamp(RTC_HandleTypeDef *hrtc,
uint32_t RTC_TimeStampEdge, uint32_t RTC_TimeStampPin)
-{
-  uint32_t tmpreg = 0U;
```

```
-
-  /* Check the parameters */
-  assert_param(IS_TIMESTAMP_EDGE(RTC_TimeStampEdge));
-  assert_param(IS_RTC_TIMESTAMP_PIN(RTC_TimeStampPin));
-
-  /* Process Locked */
-  __HAL_LOCK(hrtc);
-
-  /* Change RTC state to BUSY */
-  hrtc->State = HAL_RTC_STATE_BUSY;
-
-  hrtc->Instance->TAFCR &= (uint32_t)~RTC_TAFCR_TSINSEL;
-  hrtc->Instance->TAFCR |= (uint32_t)(RTC_TimeStampPin);
-
-  /* Get the RTC_CR register and clear the bits to be configured */
-  tmpreg = (uint32_t)(hrtc->Instance->CR & (uint32_t)~(RTC_CR_TSEDGE |
RTC_CR_TSE));
-
-  /* Configure the Timestamp TSEDGE bit */
-  tmpreg |= RTC_TimeStampEdge;
-
-  /* Disable the write protection for RTC registers */
-  __HAL_RTC_WRITEPROTECTION_DISABLE(hrtc);
-
-  /* Copy the desired configuration into the CR register */
-  hrtc->Instance->CR = (uint32_t)tmpreg;
-
-  /* Clear RTC Timestamp flag */
-  __HAL_RTC_TIMESTAMP_CLEAR_FLAG(hrtc, RTC_FLAG_TSF);
-
-  /* Clear RTC Timestamp overrun Flag */
-  __HAL_RTC_TIMESTAMP_CLEAR_FLAG(hrtc, RTC_FLAG_TSOVF);
-
-  /* Enable the Timestamp saving */
-  __HAL_RTC_TIMESTAMP_ENABLE(hrtc);
-
-  /* Enable the write protection for RTC registers */
-  __HAL_RTC_WRITEPROTECTION_ENABLE(hrtc);
-
-  /* Change RTC state back to READY */
-  hrtc->State = HAL_RTC_STATE_READY;
-
-  /* Process Unlocked */
-  __HAL_UNLOCK(hrtc);
-
-  return HAL_OK;
-}
-
-/**
-  * @brief  Sets Timestamp with Interrupt.
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
-  *                the configuration information for RTC.
-  * @note   This API must be called before enabling the Timestamp
feature.
-  * @param  RTC_TimeStampEdge Specifies the pin edge on which the
Timestamp is
-  *         activated.
-  *          This parameter can be one of the following values:
```

```
-  *             @arg RTC_TIMESTAMPEDGE_RISING: the Timestamp event
occurs on
-  *                                            the rising edge of the
related pin.
-  *             @arg RTC_TIMESTAMPEDGE_FALLING: the Timestamp event
occurs on
-  *                                            the falling edge of the
related pin.
-  * @param  RTC_TimeStampPin Specifies the RTC Timestamp Pin.
-  *          This parameter can be one of the following values:
-  *             @arg RTC_TIMESTAMPPIN_DEFAULT: PC13 is selected as RTC
Timestamp Pin.
-  *             @arg RTC_TIMESTAMPPIN_POS1: PI8 is selected as RTC
Timestamp Pin.
-  * @note RTC_TIMESTAMPPIN_POS1 corresponds to pin PA0 in the case of
-  *       STM32F446xx devices.
-  * @note RTC_TIMESTAMPPIN_POS1 is not applicable to the following list
of devices:
-  *       STM32F412xx, STM32F413xx and STM32F423xx.
-  * @retval HAL status
-  */
-HAL_StatusTypeDef HAL_RTCEx_SetTimeStamp_IT(RTC_HandleTypeDef *hrtc,
uint32_t RTC_TimeStampEdge, uint32_t RTC_TimeStampPin)
-{
-  uint32_t tmpreg = 0U;
-
-  /* Check the parameters */
-  assert_param(IS_TIMESTAMP_EDGE(RTC_TimeStampEdge));
-  assert_param(IS_RTC_TIMESTAMP_PIN(RTC_TimeStampPin));
-
-  /* Process Locked */
-  __HAL_LOCK(hrtc);
-
-  /* Change RTC state to BUSY */
-  hrtc->State = HAL_RTC_STATE_BUSY;
-
-  hrtc->Instance->TAFCR &= (uint32_t)~RTC_TAFCR_TSINSEL;
-  hrtc->Instance->TAFCR |= (uint32_t)(RTC_TimeStampPin);
-
-  /* Get the RTC_CR register and clear the bits to be configured */
-  tmpreg = (uint32_t)(hrtc->Instance->CR & (uint32_t)~(RTC_CR_TSEDGE |
RTC_CR_TSE));
-
-  /* Configure the Timestamp TSEDGE bit */
-  tmpreg |= RTC_TimeStampEdge;
-
-  /* Disable the write protection for RTC registers */
-  __HAL_RTC_WRITEPROTECTION_DISABLE(hrtc);
-
-  /* Copy the desired configuration into the CR register */
-  hrtc->Instance->CR = (uint32_t)tmpreg;
-
-  /* Clear RTC Timestamp flag */
-  __HAL_RTC_TIMESTAMP_CLEAR_FLAG(hrtc, RTC_FLAG_TSF);
-
-  /* Clear RTC Timestamp overrun Flag */
-  __HAL_RTC_TIMESTAMP_CLEAR_FLAG(hrtc, RTC_FLAG_TSOVF);
-
```

```
-  /* Enable the Timestamp saving */
-  __HAL_RTC_TIMESTAMP_ENABLE(hrtc);
-
-  /* Enable IT Timestamp */
-  __HAL_RTC_TIMESTAMP_ENABLE_IT(hrtc, RTC_IT_TS);
-
-  /* Enable the write protection for RTC registers */
-  __HAL_RTC_WRITEPROTECTION_ENABLE(hrtc);
-
-  /* Enable and configure the EXTI line associated to the RTC Timestamp
and Tamper interrupts */
-  __HAL_RTC_TAMPER_TIMESTAMP_EXTI_ENABLE_IT();
-  __HAL_RTC_TAMPER_TIMESTAMP_EXTI_ENABLE_RISING_EDGE();
-
-  /* Change RTC state back to READY */
-  hrtc->State = HAL_RTC_STATE_READY;
-
-  /* Process Unlocked */
-  __HAL_UNLOCK(hrtc);
-
-  return HAL_OK;
-}
-
-/**
-  * @brief  Deactivates Timestamp.
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
-  *                the configuration information for RTC.
-  * @retval HAL status
-  */
-HAL_StatusTypeDef HAL_RTCEx_DeactivateTimeStamp(RTC_HandleTypeDef *hrtc)
-{
-  uint32_t tmpreg = 0U;
-
-  /* Process Locked */
-  __HAL_LOCK(hrtc);
-
-  hrtc->State = HAL_RTC_STATE_BUSY;
-
-  /* Disable the write protection for RTC registers */
-  __HAL_RTC_WRITEPROTECTION_DISABLE(hrtc);
-
-  /* In case interrupt mode is used, the interrupt source must disabled
*/
-  __HAL_RTC_TIMESTAMP_DISABLE_IT(hrtc, RTC_IT_TS);
-
-  /* Get the RTC_CR register and clear the bits to be configured */
-  tmpreg = (uint32_t)(hrtc->Instance->CR & (uint32_t)~(RTC_CR_TSEDGE |
RTC_CR_TSE));
-
-  /* Configure the Timestamp TSEDGE and Enable bits */
-  hrtc->Instance->CR = (uint32_t)tmpreg;
-
-  /* Enable the write protection for RTC registers */
-  __HAL_RTC_WRITEPROTECTION_ENABLE(hrtc);
-
-  hrtc->State = HAL_RTC_STATE_READY;
-
-  /* Process Unlocked */
```

```
-  __HAL_UNLOCK(hrtc);
-
-  return HAL_OK;
-}
-
-/**
-  * @brief  Gets the RTC Timestamp value.
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
-  *                the configuration information for RTC.
-  * @param  sTimeStamp Pointer to Time structure
-  * @param  sTimeStampDate Pointer to Date structure
-  * @param  Format specifies the format of the entered parameters.
-  *          This parameter can be one of the following values:
-  *             @arg RTC_FORMAT_BIN: Binary data format
-  *             @arg RTC_FORMAT_BCD: BCD data format
-  * @retval HAL status
-  */
-HAL_StatusTypeDef HAL_RTCEx_GetTimeStamp(RTC_HandleTypeDef *hrtc,
RTC_TimeTypeDef *sTimeStamp, RTC_DateTypeDef *sTimeStampDate, uint32_t
Format)
-{
-  uint32_t tmptime = 0U;
-  uint32_t tmpdate = 0U;
-
-  /* Check the parameters */
-  assert_param(IS_RTC_FORMAT(Format));
-
-  /* Get the Timestamp time and date registers values */
-  tmptime = (uint32_t)(hrtc->Instance->TSTR & RTC_TR_RESERVED_MASK);
-  tmpdate = (uint32_t)(hrtc->Instance->TSDR & RTC_DR_RESERVED_MASK);
-
-  /* Fill the Time structure fields with the read parameters */
-  sTimeStamp->Hours      = (uint8_t)((tmptime & (RTC_TSTR_HT  |
RTC_TSTR_HU))  >> RTC_TSTR_HU_Pos);
-  sTimeStamp->Minutes    = (uint8_t)((tmptime & (RTC_TSTR_MNT |
RTC_TSTR_MNU)) >> RTC_TSTR_MNU_Pos);
-  sTimeStamp->Seconds    = (uint8_t)((tmptime & (RTC_TSTR_ST  |
RTC_TSTR_SU))  >> RTC_TSTR_SU_Pos);
-  sTimeStamp->TimeFormat = (uint8_t)((tmptime & (RTC_TSTR_PM))
>> RTC_TSTR_PM_Pos);
-  sTimeStamp->SubSeconds = (uint32_t) hrtc->Instance->TSSSR;
-
-  /* Fill the Date structure fields with the read parameters */
-  sTimeStampDate->Year    = 0U;
-  sTimeStampDate->Month   = (uint8_t)((tmpdate & (RTC_TSDR_MT |
RTC_TSDR_MU)) >> RTC_TSDR_MU_Pos);
-  sTimeStampDate->Date    = (uint8_t)((tmpdate & (RTC_TSDR_DT |
RTC_TSDR_DU)) >> RTC_TSDR_DU_Pos);
-  sTimeStampDate->WeekDay = (uint8_t)((tmpdate & (RTC_TSDR_WDU))
>> RTC_TSDR_WDU_Pos);
-
-  /* Check the input parameters format */
-  if (Format == RTC_FORMAT_BIN)
-  {
-    /* Convert the Timestamp structure parameters to Binary format */
-    sTimeStamp->Hours   = (uint8_t)RTC_Bcd2ToByte(sTimeStamp->Hours);
-    sTimeStamp->Minutes = (uint8_t)RTC_Bcd2ToByte(sTimeStamp->Minutes);
-    sTimeStamp->Seconds = (uint8_t)RTC_Bcd2ToByte(sTimeStamp->Seconds);
```

```
-
-    /* Convert the DateTimeStamp structure parameters to Binary format
*/
-    sTimeStampDate->Month   = (uint8_t)RTC_Bcd2ToByte(sTimeStampDate-
>Month);
-    sTimeStampDate->Date    = (uint8_t)RTC_Bcd2ToByte(sTimeStampDate-
>Date);
-    sTimeStampDate->WeekDay = (uint8_t)RTC_Bcd2ToByte(sTimeStampDate-
>WeekDay);
-  }
-
-  /* Clear the Timestamp Flag */
-  __HAL_RTC_TIMESTAMP_CLEAR_FLAG(hrtc, RTC_FLAG_TSF);
-
-  return HAL_OK;
-}
-
-/**
-  * @brief  Sets Tamper.
-  * @note   By calling this API the tamper global interrupt will be
disabled.
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
-  *                the configuration information for RTC.
-  * @param  sTamper Pointer to Tamper Structure.
-  * @retval HAL status
-  */
-HAL_StatusTypeDef HAL_RTCEx_SetTamper(RTC_HandleTypeDef *hrtc,
RTC_TamperTypeDef *sTamper)
-{
-  uint32_t tmpreg = 0U;
-
-  /* Check the parameters */
-  assert_param(IS_RTC_TAMPER(sTamper->Tamper));
-  assert_param(IS_RTC_TAMPER_PIN(sTamper->PinSelection));
-  assert_param(IS_RTC_TAMPER_TRIGGER(sTamper->Trigger));
-  assert_param(IS_RTC_TAMPER_FILTER(sTamper->Filter));
-  assert_param(IS_RTC_TAMPER_FILTER_CONFIG_CORRECT(sTamper->Filter,
sTamper->Trigger));
-  assert_param(IS_RTC_TAMPER_SAMPLING_FREQ(sTamper->SamplingFrequency));
-  assert_param(IS_RTC_TAMPER_PRECHARGE_DURATION(sTamper-
>PrechargeDuration));
-  assert_param(IS_RTC_TAMPER_PULLUP_STATE(sTamper->TamperPullUp));
-  assert_param(IS_RTC_TAMPER_TIMESTAMPONTAMPER_DETECTION(sTamper-
>TimeStampOnTamperDetection));
-
-  /* Process Locked */
-  __HAL_LOCK(hrtc);
-
-  hrtc->State = HAL_RTC_STATE_BUSY;
-
-  /* Copy control register into temporary variable */
-  tmpreg = hrtc->Instance->TAFCR;
-
-  /* Enable selected tamper */
-  tmpreg |= (sTamper->Tamper);
-
-  /* Configure the tamper trigger bit (this bit is just on the right of
the
```

```
-       tamper enable bit, hence the one-time right shift before updating
it) */
-  if (sTamper->Trigger == RTC_TAMPERTRIGGER_FALLINGEDGE)
-  {
-    /* Set the tamper trigger bit (case of falling edge or high level)
*/
-    tmpreg |= (uint32_t)(sTamper->Tamper << 1U);
-  }
-  else
-  {
-    /* Clear the tamper trigger bit (case of rising edge or low level)
*/
-    tmpreg &= (uint32_t)~(sTamper->Tamper << 1U);
-  }
-
-  /* Clear remaining fields before setting them */
-  tmpreg &= ~(RTC_TAMPERFILTER_MASK              | \
-              RTC_TAMPERSAMPLINGFREQ_RTCCLK_MASK | \
-              RTC_TAMPERPRECHARGEDURATION_MASK   | \
-              RTC_TAMPER_PULLUP_MASK             | \
-              RTC_TAFCR_TAMP1INSEL               | \
-              RTC_TIMESTAMPONTAMPERDETECTION_MASK);
-
-  /* Set remaining parameters of desired configuration into temporary
variable */
-  tmpreg |= ((uint32_t)sTamper->Filter            | \
-             (uint32_t)sTamper->SamplingFrequency | \
-             (uint32_t)sTamper->PrechargeDuration | \
-             (uint32_t)sTamper->TamperPullUp      | \
-             (uint32_t)sTamper->PinSelection      | \
-             (uint32_t)sTamper->TimeStampOnTamperDetection);
-
-  /* Disable tamper global interrupt in case it is enabled */
-  tmpreg &= (uint32_t)~RTC_TAFCR_TAMPIE;
-
-  /* Copy desired configuration into configuration register */
-  hrtc->Instance->TAFCR = tmpreg;
-
-  hrtc->State = HAL_RTC_STATE_READY;
-
-  /* Process Unlocked */
-  __HAL_UNLOCK(hrtc);
-
-  return HAL_OK;
-}
-
-/**
-  * @brief  Sets Tamper with interrupt.
-  * @note   By calling this API the tamper global interrupt will be
enabled.
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
-  *                the configuration information for RTC.
-  * @param  sTamper Pointer to RTC Tamper.
-  * @retval HAL status
-  */
-HAL_StatusTypeDef HAL_RTCEx_SetTamper_IT(RTC_HandleTypeDef *hrtc,
RTC_TamperTypeDef *sTamper)
-{
```

```
-  uint32_t tmpreg = 0U;
-
-  /* Check the parameters */
-  assert_param(IS_RTC_TAMPER(sTamper->Tamper));
-  assert_param(IS_RTC_TAMPER_PIN(sTamper->PinSelection));
-  assert_param(IS_RTC_TAMPER_TRIGGER(sTamper->Trigger));
-  assert_param(IS_RTC_TAMPER_FILTER(sTamper->Filter));
-  assert_param(IS_RTC_TAMPER_FILTER_CONFIG_CORRECT(sTamper->Filter,
sTamper->Trigger));
-  assert_param(IS_RTC_TAMPER_SAMPLING_FREQ(sTamper->SamplingFrequency));
-  assert_param(IS_RTC_TAMPER_PRECHARGE_DURATION(sTamper-
>PrechargeDuration));
-  assert_param(IS_RTC_TAMPER_PULLUP_STATE(sTamper->TamperPullUp));
-  assert_param(IS_RTC_TAMPER_TIMESTAMPONTAMPER_DETECTION(sTamper-
>TimeStampOnTamperDetection));
-
-  /* Process Locked */
-  __HAL_LOCK(hrtc);
-
-  hrtc->State = HAL_RTC_STATE_BUSY;
-
-  /* Copy control register into temporary variable */
-  tmpreg = hrtc->Instance->TAFCR;
-
-  /* Enable selected tamper */
-  tmpreg |= (sTamper->Tamper);
-
-  /* Configure the tamper trigger bit (this bit is just on the right of
the
-       tamper enable bit, hence the one-time right shift before updating
it) */
-  if (sTamper->Trigger == RTC_TAMPERTRIGGER_FALLINGEDGE)
-  {
-    /* Set the tamper trigger bit (case of falling edge or high level)
*/
-    tmpreg |= (uint32_t)(sTamper->Tamper << 1U);
-  }
-  else
-  {
-    /* Clear the tamper trigger bit (case of rising edge or low level)
*/
-    tmpreg &= (uint32_t)~(sTamper->Tamper << 1U);
-  }
-
-  /* Clear remaining fields before setting them */
-  tmpreg &= ~(RTC_TAMPERFILTER_MASK              | \
-              RTC_TAMPERSAMPLINGFREQ_RTCCLK_MASK | \
-              RTC_TAMPERPRECHARGEDURATION_MASK   | \
-              RTC_TAMPER_PULLUP_MASK             | \
-              RTC_TAFCR_TAMP1INSEL               | \
-              RTC_TIMESTAMPONTAMPERDETECTION_MASK);
-
-  /* Set remaining parameters of desired configuration into temporary
variable */
-  tmpreg |= ((uint32_t)sTamper->Filter            | \
-             (uint32_t)sTamper->SamplingFrequency | \
-             (uint32_t)sTamper->PrechargeDuration | \
-             (uint32_t)sTamper->TamperPullUp      | \
```

```
-             (uint32_t)sTamper->PinSelection      | \
-             (uint32_t)sTamper->TimeStampOnTamperDetection);
-
-  /* Enable global tamper interrupt */
-  tmpreg |= (uint32_t)RTC_TAFCR_TAMPIE;
-
-  /* Copy desired configuration into configuration register */
-  hrtc->Instance->TAFCR = tmpreg;
-
-  /* Enable and configure the EXTI line associated to the RTC Timestamp
and Tamper interrupts */
-  __HAL_RTC_TAMPER_TIMESTAMP_EXTI_ENABLE_IT();
-  __HAL_RTC_TAMPER_TIMESTAMP_EXTI_ENABLE_RISING_EDGE();
-
-  hrtc->State = HAL_RTC_STATE_READY;
-
-  /* Process Unlocked */
-  __HAL_UNLOCK(hrtc);
-
-  return HAL_OK;
-}
-
-/**
-  * @brief  Deactivates Tamper.
-  * @note   The tamper global interrupt bit will remain unchanged.
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
-  *                the configuration information for RTC.
-  * @param  Tamper Selected tamper pin.
-  *          This parameter can be any combination of the following
values:
-  *            @arg RTC_TAMPER_1:  Tamper 1
-  *            @arg RTC_TAMPER_2:  Tamper 2 (*)
-  *
-  *         (*) value not applicable to all devices.
-  * @retval HAL status
-  */
-HAL_StatusTypeDef HAL_RTCEx_DeactivateTamper(RTC_HandleTypeDef *hrtc,
uint32_t Tamper)
-{
-  assert_param(IS_RTC_TAMPER(Tamper));
-
-  /* Process Locked */
-  __HAL_LOCK(hrtc);
-
-  hrtc->State = HAL_RTC_STATE_BUSY;
-
-  /* Disable the selected Tamper pin */
-  hrtc->Instance->TAFCR &= (uint32_t)~Tamper;
-
-  hrtc->State = HAL_RTC_STATE_READY;
-
-  /* Process Unlocked */
-  __HAL_UNLOCK(hrtc);
-
-  return HAL_OK;
-}
-
-/**
```

```
-  * @brief  Handles Timestamp and Tamper interrupt request.
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
-  *               the configuration information for RTC.
-  * @retval None
-  */
-void HAL_RTCEx_TamperTimeStampIRQHandler(RTC_HandleTypeDef *hrtc)
-{
-  /* Clear the EXTI flag associated to the RTC Timestamp and Tamper
interrupts */
-  __HAL_RTC_TAMPER_TIMESTAMP_EXTI_CLEAR_FLAG();
-
-  /* Get the Timestamp interrupt source enable status */
-  if (__HAL_RTC_TIMESTAMP_GET_IT_SOURCE(hrtc, RTC_IT_TS) != 0U)
-  {
-    /* Get the pending status of the Timestamp Interrupt */
-    if (__HAL_RTC_TIMESTAMP_GET_FLAG(hrtc, RTC_FLAG_TSF) != 0U)
-    {
-      /* Timestamp callback */
-#if (USE_HAL_RTC_REGISTER_CALLBACKS == 1)
-      hrtc->TimeStampEventCallback(hrtc);
-#else
-      HAL_RTCEx_TimeStampEventCallback(hrtc);
-#endif /* USE_HAL_RTC_REGISTER_CALLBACKS */
-
-      /* Clear the Timestamp interrupt pending bit after returning from
callback
-         as RTC_TSTR and RTC_TSDR registers are cleared when TSF bit is
reset */
-      __HAL_RTC_TIMESTAMP_CLEAR_FLAG(hrtc, RTC_FLAG_TSF);
-    }
-  }
-
-  /* Get the Tamper 1 interrupt source enable status */
-  if (__HAL_RTC_TAMPER_GET_IT_SOURCE(hrtc, RTC_IT_TAMP) != 0U)
-  {
-    /* Get the pending status of the Tamper 1 Interrupt */
-    if (__HAL_RTC_TAMPER_GET_FLAG(hrtc, RTC_FLAG_TAMP1F) != 0U)
-    {
-      /* Clear the Tamper interrupt pending bit */
-      __HAL_RTC_TAMPER_CLEAR_FLAG(hrtc, RTC_FLAG_TAMP1F);
-
-      /* Tamper callback */
-#if (USE_HAL_RTC_REGISTER_CALLBACKS == 1)
-      hrtc->Tamper1EventCallback(hrtc);
-#else
-      HAL_RTCEx_Tamper1EventCallback(hrtc);
-#endif /* USE_HAL_RTC_REGISTER_CALLBACKS */
-    }
-  }
-
-#if defined(RTC_TAMPER2_SUPPORT)
-  /* Get the Tamper 2 interrupt source enable status */
-  if (__HAL_RTC_TAMPER_GET_IT_SOURCE(hrtc, RTC_IT_TAMP) != 0U)
-  {
-    /* Get the pending status of the Tamper 2 Interrupt */
-    if (__HAL_RTC_TAMPER_GET_FLAG(hrtc, RTC_FLAG_TAMP2F) != 0U)
-    {
-      /* Clear the Tamper interrupt pending bit */
```

```
-      __HAL_RTC_TAMPER_CLEAR_FLAG(hrtc, RTC_FLAG_TAMP2F);
-
-      /* Tamper callback */
-#if (USE_HAL_RTC_REGISTER_CALLBACKS == 1)
-      hrtc->Tamper2EventCallback(hrtc);
-#else
-      HAL_RTCEx_Tamper2EventCallback(hrtc);
-#endif /* USE_HAL_RTC_REGISTER_CALLBACKS */
-    }
-  }
-#endif /* RTC_TAMPER2_SUPPORT */
-
-  /* Change RTC state */
-  hrtc->State = HAL_RTC_STATE_READY;
-}
-
-/**
-  * @brief  Timestamp callback.
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
-  *                the configuration information for RTC.
-  * @retval None
-  */
-__weak void HAL_RTCEx_TimeStampEventCallback(RTC_HandleTypeDef *hrtc)
-{
-  /* Prevent unused argument(s) compilation warning */
-  UNUSED(hrtc);
-
-  /* NOTE: This function should not be modified, when the callback is
needed,
-           the HAL_RTCEx_TimeStampEventCallback could be implemented in
the user file
-  */
-}
-
-/**
-  * @brief  Tamper 1 callback.
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
-  *                the configuration information for RTC.
-  * @retval None
-  */
-__weak void HAL_RTCEx_Tamper1EventCallback(RTC_HandleTypeDef *hrtc)
-{
-  /* Prevent unused argument(s) compilation warning */
-  UNUSED(hrtc);
-
-  /* NOTE: This function should not be modified, when the callback is
needed,
-           the HAL_RTCEx_Tamper1EventCallback could be implemented in
the user file
-   */
-}
-
-#if defined(RTC_TAMPER2_SUPPORT)
-/**
-  * @brief  Tamper 2 callback.
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
-  *                the configuration information for RTC.
-  * @retval None
```

```
-  */
-__weak void HAL_RTCEx_Tamper2EventCallback(RTC_HandleTypeDef *hrtc)
-{
-  /* Prevent unused argument(s) compilation warning */
-  UNUSED(hrtc);
-
-  /* NOTE: This function should not be modified, when the callback is
needed,
-           the HAL_RTCEx_Tamper2EventCallback could be implemented in
the user file
-   */
-}
-#endif /* RTC_TAMPER2_SUPPORT */
-
-/**
-  * @brief  Handles Timestamp polling request.
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
-  *                the configuration information for RTC.
-  * @param  Timeout Timeout duration
-  * @retval HAL status
-  */
-HAL_StatusTypeDef HAL_RTCEx_PollForTimeStampEvent(RTC_HandleTypeDef
*hrtc, uint32_t Timeout)
-{
-  uint32_t tickstart = 0U;
-
-  /* Get tick */
-  tickstart = HAL_GetTick();
-
-  while (__HAL_RTC_TIMESTAMP_GET_FLAG(hrtc, RTC_FLAG_TSF) == 0U)
-  {
-    if (Timeout != HAL_MAX_DELAY)
-    {
-      if ((Timeout == 0U) || ((HAL_GetTick() - tickstart) > Timeout))
-      {
-        hrtc->State = HAL_RTC_STATE_TIMEOUT;
-        return HAL_TIMEOUT;
-      }
-    }
-
-    if (__HAL_RTC_TIMESTAMP_GET_FLAG(hrtc, RTC_FLAG_TSOVF) != 0U)
-    {
-      /* Clear the Timestamp Overrun Flag */
-      __HAL_RTC_TIMESTAMP_CLEAR_FLAG(hrtc, RTC_FLAG_TSOVF);
-
-      /* Change Timestamp state */
-      hrtc->State = HAL_RTC_STATE_ERROR;
-
-      return HAL_ERROR;
-    }
-  }
-
-  /* Change RTC state */
-  hrtc->State = HAL_RTC_STATE_READY;
-
-  return HAL_OK;
-}
-
```

```
-/**
-  * @brief  Handles Tamper 1 Polling.
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
-  *                the configuration information for RTC.
-  * @param  Timeout Timeout duration
-  * @retval HAL status
-  */
-HAL_StatusTypeDef HAL_RTCEx_PollForTamper1Event(RTC_HandleTypeDef *hrtc,
uint32_t Timeout)
-{
-  uint32_t tickstart = 0U;
-
-  /* Get tick */
-  tickstart = HAL_GetTick();
-
-  /* Get the status of the Interrupt */
-  while (__HAL_RTC_TAMPER_GET_FLAG(hrtc, RTC_FLAG_TAMP1F) == 0U)
-  {
-    if (Timeout != HAL_MAX_DELAY)
-    {
-      if ((Timeout == 0U) || ((HAL_GetTick() - tickstart) > Timeout))
-      {
-        hrtc->State = HAL_RTC_STATE_TIMEOUT;
-        return HAL_TIMEOUT;
-      }
-    }
-  }
-
-  /* Clear the Tamper Flag */
-  __HAL_RTC_TAMPER_CLEAR_FLAG(hrtc, RTC_FLAG_TAMP1F);
-
-  /* Change RTC state */
-  hrtc->State = HAL_RTC_STATE_READY;
-
-  return HAL_OK;
-}
-
-#if defined(RTC_TAMPER2_SUPPORT)
-/**
-  * @brief  Handles Tamper 2 Polling.
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
-  *                the configuration information for RTC.
-  * @param  Timeout Timeout duration
-  * @retval HAL status
-  */
-HAL_StatusTypeDef HAL_RTCEx_PollForTamper2Event(RTC_HandleTypeDef *hrtc,
uint32_t Timeout)
-{
-  uint32_t tickstart = 0U;
-
-  /* Get tick */
-  tickstart = HAL_GetTick();
-
-  /* Get the status of the Interrupt */
-  while (__HAL_RTC_TAMPER_GET_FLAG(hrtc, RTC_FLAG_TAMP2F) == 0U)
-  {
-    if (Timeout != HAL_MAX_DELAY)
-    {
```

```
-      if ((Timeout == 0U) || ((HAL_GetTick() - tickstart) > Timeout))
-      {
-        hrtc->State = HAL_RTC_STATE_TIMEOUT;
-        return HAL_TIMEOUT;
-      }
-    }
-  }
-
-  /* Clear the Tamper Flag */
-  __HAL_RTC_TAMPER_CLEAR_FLAG(hrtc, RTC_FLAG_TAMP2F);
-
-  /* Change RTC state */
-  hrtc->State = HAL_RTC_STATE_READY;
-
-  return HAL_OK;
-}
-#endif /* RTC_TAMPER2_SUPPORT */
-
-/**
-  * @}
-  */
-
-/** @defgroup RTCEx_Exported_Functions_Group2 RTC Wakeup functions
-  * @brief    RTC Wakeup functions
-  *
-@verbatim
-
==============================================================================
------
-                         ##### RTC Wakeup functions #####
-
==============================================================================
------
-
- [..] This section provides functions allowing to configure Wakeup
feature
-
-@endverbatim
-  * @{
-  */
-
-/**
-  * @brief  Sets wakeup timer.
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
-  *                the configuration information for RTC.
-  * @param  WakeUpCounter Wakeup counter
-  * @param  WakeUpClock Wakeup clock
-  * @retval HAL status
-  */
-HAL_StatusTypeDef HAL_RTCEx_SetWakeUpTimer(RTC_HandleTypeDef *hrtc,
uint32_t WakeUpCounter, uint32_t WakeUpClock)
-{
-  uint32_t tickstart = 0U;
-
-  /* Check the parameters */
-  assert_param(IS_RTC_WAKEUP_CLOCK(WakeUpClock));
-  assert_param(IS_RTC_WAKEUP_COUNTER(WakeUpCounter));
-
```

```
-  /* Process Locked */
-  __HAL_LOCK(hrtc);
-
-  hrtc->State = HAL_RTC_STATE_BUSY;
-
-  /* Disable the write protection for RTC registers */
-  __HAL_RTC_WRITEPROTECTION_DISABLE(hrtc);
-
-  /* Check RTC WUTWF flag is reset only when wakeup timer enabled*/
-  if ((hrtc->Instance->CR & RTC_CR_WUTE) != 0U)
-  {
-    tickstart = HAL_GetTick();
-
-    /* Wait till RTC WUTWF flag is reset and if timeout is reached exit
*/
-    while (__HAL_RTC_WAKEUPTIMER_GET_FLAG(hrtc, RTC_FLAG_WUTWF) != 0U)
-    {
-      if ((HAL_GetTick() - tickstart) > RTC_TIMEOUT_VALUE)
-      {
-        /* Enable the write protection for RTC registers */
-        __HAL_RTC_WRITEPROTECTION_ENABLE(hrtc);
-
-        hrtc->State = HAL_RTC_STATE_TIMEOUT;
-
-        /* Process Unlocked */
-        __HAL_UNLOCK(hrtc);
-
-        return HAL_TIMEOUT;
-      }
-    }
-  }
-
-  /* Disable the Wakeup timer */
-  __HAL_RTC_WAKEUPTIMER_DISABLE(hrtc);
-
-  /* Clear the Wakeup flag */
-  __HAL_RTC_WAKEUPTIMER_CLEAR_FLAG(hrtc, RTC_FLAG_WUTF);
-
-  /* Get tick */
-  tickstart = HAL_GetTick();
-
-  /* Wait till RTC WUTWF flag is set and if timeout is reached exit */
-  while (__HAL_RTC_WAKEUPTIMER_GET_FLAG(hrtc, RTC_FLAG_WUTWF) == 0U)
-  {
-    if ((HAL_GetTick() - tickstart) > RTC_TIMEOUT_VALUE)
-    {
-      /* Enable the write protection for RTC registers */
-      __HAL_RTC_WRITEPROTECTION_ENABLE(hrtc);
-
-      hrtc->State = HAL_RTC_STATE_TIMEOUT;
-
-      /* Process Unlocked */
-      __HAL_UNLOCK(hrtc);
-
-      return HAL_TIMEOUT;
-    }
-  }
-
```

```
-  /* Clear the Wakeup Timer clock source bits in CR register */
-  hrtc->Instance->CR &= (uint32_t)~RTC_CR_WUCKSEL;
-
-  /* Configure the clock source */
-  hrtc->Instance->CR |= (uint32_t)WakeUpClock;
-
-  /* Configure the Wakeup Timer counter */
-  hrtc->Instance->WUTR = (uint32_t)WakeUpCounter;
-
-  /* Enable the Wakeup Timer */
-  __HAL_RTC_WAKEUPTIMER_ENABLE(hrtc);
-
-  /* Enable the write protection for RTC registers */
-  __HAL_RTC_WRITEPROTECTION_ENABLE(hrtc);
-
-  hrtc->State = HAL_RTC_STATE_READY;
-
-  /* Process Unlocked */
-  __HAL_UNLOCK(hrtc);
-
-  return HAL_OK;
-}
-
-/**
-  * @brief  Sets wakeup timer with interrupt.
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
-  *                the configuration information for RTC.
-  * @param  WakeUpCounter Wakeup counter
-  * @param  WakeUpClock Wakeup clock
-  * @retval HAL status
-  */
-HAL_StatusTypeDef HAL_RTCEx_SetWakeUpTimer_IT(RTC_HandleTypeDef *hrtc,
uint32_t WakeUpCounter, uint32_t WakeUpClock)
-{
-  __IO uint32_t count  = RTC_TIMEOUT_VALUE * (SystemCoreClock / 32U /
1000U);
-
-  /* Check the parameters */
-  assert_param(IS_RTC_WAKEUP_CLOCK(WakeUpClock));
-  assert_param(IS_RTC_WAKEUP_COUNTER(WakeUpCounter));
-
-  /* Process Locked */
-  __HAL_LOCK(hrtc);
-
-  hrtc->State = HAL_RTC_STATE_BUSY;
-
-  /* Disable the write protection for RTC registers */
-  __HAL_RTC_WRITEPROTECTION_DISABLE(hrtc);
-
-  /* Check RTC WUTWF flag is reset only when wakeup timer enabled */
-  if ((hrtc->Instance->CR & RTC_CR_WUTE) != 0U)
-  {
-    /* Wait till RTC WUTWF flag is reset and if timeout is reached exit
*/
-    do
-    {
-      count = count - 1U;
-      if (count == 0U)
```

```
-      {
-        /* Enable the write protection for RTC registers */
-        __HAL_RTC_WRITEPROTECTION_ENABLE(hrtc);
-
-        hrtc->State = HAL_RTC_STATE_TIMEOUT;
-
-        /* Process Unlocked */
-        __HAL_UNLOCK(hrtc);
-
-        return HAL_TIMEOUT;
-      }
-    } while (__HAL_RTC_WAKEUPTIMER_GET_FLAG(hrtc, RTC_FLAG_WUTWF) !=
0U);
-  }
-
-  /* Disable the Wakeup timer */
-  __HAL_RTC_WAKEUPTIMER_DISABLE(hrtc);
-
-  /* Clear the Wakeup flag */
-  __HAL_RTC_WAKEUPTIMER_CLEAR_FLAG(hrtc, RTC_FLAG_WUTF);
-
-  /* Reload the counter */
-  count = RTC_TIMEOUT_VALUE * (SystemCoreClock / 32U / 1000U);
-
-  /* Wait till RTC WUTWF flag is set and if timeout is reached exit */
-  do
-  {
-    count = count - 1U;
-    if (count == 0U)
-    {
-      /* Enable the write protection for RTC registers */
-      __HAL_RTC_WRITEPROTECTION_ENABLE(hrtc);
-
-      hrtc->State = HAL_RTC_STATE_TIMEOUT;
-
-      /* Process Unlocked */
-      __HAL_UNLOCK(hrtc);
-
-      return HAL_TIMEOUT;
-    }
-  } while (__HAL_RTC_WAKEUPTIMER_GET_FLAG(hrtc, RTC_FLAG_WUTWF) == 0U);
-
-  /* Clear the Wakeup Timer clock source bits in CR register */
-  hrtc->Instance->CR &= (uint32_t)~RTC_CR_WUCKSEL;
-
-  /* Configure the clock source */
-  hrtc->Instance->CR |= (uint32_t)WakeUpClock;
-
-  /* Configure the Wakeup Timer counter */
-  hrtc->Instance->WUTR = (uint32_t)WakeUpCounter;
-
-  /* Enable and configure the EXTI line associated to the RTC Wakeup
Timer interrupt */
-  __HAL_RTC_WAKEUPTIMER_EXTI_ENABLE_IT();
-  __HAL_RTC_WAKEUPTIMER_EXTI_ENABLE_RISING_EDGE();
-
-  /* Configure the interrupt in the RTC_CR register */
-  __HAL_RTC_WAKEUPTIMER_ENABLE_IT(hrtc, RTC_IT_WUT);
```

```
-
-  /* Enable the Wakeup Timer */
-  __HAL_RTC_WAKEUPTIMER_ENABLE(hrtc);
-
-  /* Enable the write protection for RTC registers */
-  __HAL_RTC_WRITEPROTECTION_ENABLE(hrtc);
-
-  hrtc->State = HAL_RTC_STATE_READY;
-
-  /* Process Unlocked */
-  __HAL_UNLOCK(hrtc);
-
-  return HAL_OK;
-}
-
-/**
-  * @brief  Deactivates wakeup timer counter.
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
-  *                the configuration information for RTC.
-  * @retval HAL status
-  */
-HAL_StatusTypeDef HAL_RTCEx_DeactivateWakeUpTimer(RTC_HandleTypeDef
*hrtc)
-{
-  uint32_t tickstart = 0U;
-
-  /* Process Locked */
-  __HAL_LOCK(hrtc);
-
-  hrtc->State = HAL_RTC_STATE_BUSY;
-
-  /* Disable the write protection for RTC registers */
-  __HAL_RTC_WRITEPROTECTION_DISABLE(hrtc);
-
-  /* Disable the Wakeup Timer */
-  __HAL_RTC_WAKEUPTIMER_DISABLE(hrtc);
-
-  /* In case interrupt mode is used, the interrupt source must disabled
*/
-  __HAL_RTC_WAKEUPTIMER_DISABLE_IT(hrtc, RTC_IT_WUT);
-
-  /* Get tick */
-  tickstart = HAL_GetTick();
-
-  /* Wait till RTC WUTWF flag is set and if timeout is reached exit */
-  while (__HAL_RTC_WAKEUPTIMER_GET_FLAG(hrtc, RTC_FLAG_WUTWF) == 0U)
-  {
-    if ((HAL_GetTick() - tickstart) > RTC_TIMEOUT_VALUE)
-    {
-      /* Enable the write protection for RTC registers */
-      __HAL_RTC_WRITEPROTECTION_ENABLE(hrtc);
-
-      hrtc->State = HAL_RTC_STATE_TIMEOUT;
-
-      /* Process Unlocked */
-      __HAL_UNLOCK(hrtc);
-
-      return HAL_TIMEOUT;
```

```
-    }
-  }
-
-  /* Enable the write protection for RTC registers */
-  __HAL_RTC_WRITEPROTECTION_ENABLE(hrtc);
-
-  hrtc->State = HAL_RTC_STATE_READY;
-
-  /* Process Unlocked */
-  __HAL_UNLOCK(hrtc);
-
-  return HAL_OK;
-}
-
-/**
-  * @brief  Gets wakeup timer counter.
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
-  *                the configuration information for RTC.
-  * @retval Counter value
-  */
-uint32_t HAL_RTCEx_GetWakeUpTimer(RTC_HandleTypeDef *hrtc)
-{
-  /* Get the counter value */
-  return ((uint32_t)(hrtc->Instance->WUTR & RTC_WUTR_WUT));
-}
-
-/**
-  * @brief  Handles Wakeup Timer interrupt request.
-  * @note   Unlike alarm interrupt line (shared by Alarms A and B) or
tamper
-  *         interrupt line (shared by timestamp and tampers) wakeup
timer
-  *         interrupt line is exclusive to the wakeup timer.
-  *         There is no need in this case to check on the interrupt
enable
-  *         status via __HAL_RTC_WAKEUPTIMER_GET_IT_SOURCE().
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
-  *                the configuration information for RTC.
-  * @retval None
-  */
-void HAL_RTCEx_WakeUpTimerIRQHandler(RTC_HandleTypeDef *hrtc)
-{
-  /* Clear the EXTI flag associated to the RTC Wakeup Timer interrupt */
-  __HAL_RTC_WAKEUPTIMER_EXTI_CLEAR_FLAG();
-
-  /* Get the pending status of the Wakeup timer Interrupt */
-  if (__HAL_RTC_WAKEUPTIMER_GET_FLAG(hrtc, RTC_FLAG_WUTF) != 0U)
-  {
-    /* Clear the Wakeup timer interrupt pending bit */
-    __HAL_RTC_WAKEUPTIMER_CLEAR_FLAG(hrtc, RTC_FLAG_WUTF);
-
-    /* Wakeup timer callback */
-#if (USE_HAL_RTC_REGISTER_CALLBACKS == 1)
-    hrtc->WakeUpTimerEventCallback(hrtc);
-#else
-    HAL_RTCEx_WakeUpTimerEventCallback(hrtc);
-#endif /* USE_HAL_RTC_REGISTER_CALLBACKS */
-  }
```

```
-
-  /* Change RTC state */
-  hrtc->State = HAL_RTC_STATE_READY;
-}
-
-/**
-  * @brief  Wakeup Timer callback.
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
-  *                the configuration information for RTC.
-  * @retval None
-  */
-__weak void HAL_RTCEx_WakeUpTimerEventCallback(RTC_HandleTypeDef *hrtc)
-{
-  /* Prevent unused argument(s) compilation warning */
-  UNUSED(hrtc);
-
-  /* NOTE: This function should not be modified, when the callback is
needed,
-           the HAL_RTCEx_WakeUpTimerEventCallback could be implemented
in the user file
-   */
-}
-
-/**
-  * @brief  Handles Wakeup Timer Polling.
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
-  *                the configuration information for RTC.
-  * @param  Timeout Timeout duration
-  * @retval HAL status
-  */
-HAL_StatusTypeDef HAL_RTCEx_PollForWakeUpTimerEvent(RTC_HandleTypeDef
*hrtc, uint32_t Timeout)
-{
-  uint32_t tickstart = 0U;
-
-  /* Get tick */
-  tickstart = HAL_GetTick();
-
-  while (__HAL_RTC_WAKEUPTIMER_GET_FLAG(hrtc, RTC_FLAG_WUTF) == 0U)
-  {
-    if (Timeout != HAL_MAX_DELAY)
-    {
-      if ((Timeout == 0U) || ((HAL_GetTick() - tickstart) > Timeout))
-      {
-        hrtc->State = HAL_RTC_STATE_TIMEOUT;
-        return HAL_TIMEOUT;
-      }
-    }
-  }
-
-  /* Clear the Wakeup timer Flag */
-  __HAL_RTC_WAKEUPTIMER_CLEAR_FLAG(hrtc, RTC_FLAG_WUTF);
-
-  /* Change RTC state */
-  hrtc->State = HAL_RTC_STATE_READY;
-
-  return HAL_OK;
-}
```

```
-
-/**
-  * @}
-  */
-
-/** @defgroup RTCEx_Exported_Functions_Group3 Extended Peripheral
Control functions
-  * @brief    Extended Peripheral Control functions
-  *
-@verbatim
-
==============================================================================
======
-              ##### Extended Peripheral Control functions #####
-
==============================================================================
======
-    [..]
-    This subsection provides functions allowing to
-      (+) Write a data in a specified RTC Backup data register
-      (+) Read a data in a specified RTC Backup data register
-      (+) Set the Coarse calibration parameters.
-      (+) Deactivate the Coarse calibration parameters
-      (+) Set the Smooth calibration parameters.
-      (+) Configure the Synchronization Shift Control Settings.
-      (+) Configure the Calibration Pinout (RTC_CALIB) Selection (1Hz or
512Hz).
-      (+) Deactivate the Calibration Pinout (RTC_CALIB) Selection (1Hz
or 512Hz).
-      (+) Enable the RTC reference clock detection.
-      (+) Disable the RTC reference clock detection.
-      (+) Enable the Bypass Shadow feature.
-      (+) Disable the Bypass Shadow feature.
-
-@endverbatim
-  * @{
-  */
-
-/**
-  * @brief  Writes a data in a specified RTC Backup data register.
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
-  *                the configuration information for RTC.
-  * @param  BackupRegister RTC Backup data Register number.
-  *          This parameter can be: RTC_BKP_DRx (where x can be from 0
to 19)
-  *                                  to specify the register.
-  * @param  Data Data to be written in the specified RTC Backup data
register.
-  * @retval None
-  */
-void HAL_RTCEx_BKUPWrite(RTC_HandleTypeDef *hrtc, uint32_t
BackupRegister, uint32_t Data)
-{
-  uint32_t tmp = 0U;
-
-  /* Check the parameters */
-  assert_param(IS_RTC_BKP(BackupRegister));
-
```

```
-  tmp = (uint32_t) &(hrtc->Instance->BKP0R);
-  tmp += (BackupRegister * 4U);
-
-  /* Write the specified register */
-  *(__IO uint32_t *)tmp = (uint32_t)Data;
-}
-
-/**
-  * @brief  Reads data from the specified RTC Backup data Register.
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
-  *                the configuration information for RTC.
-  * @param  BackupRegister RTC Backup data Register number.
-  *          This parameter can be: RTC_BKP_DRx (where x can be from 0
to 19)
-  *                                  to specify the register.
-  * @retval Read value
-  */
-uint32_t HAL_RTCEx_BKUPRead(RTC_HandleTypeDef *hrtc, uint32_t
BackupRegister)
-{
-  uint32_t tmp = 0U;
-
-  /* Check the parameters */
-  assert_param(IS_RTC_BKP(BackupRegister));
-
-  tmp = (uint32_t) &(hrtc->Instance->BKP0R);
-  tmp += (BackupRegister * 4U);
-
-  /* Read the specified register */
-  return (*(__IO uint32_t *)tmp);
-}
-
-/**
-  * @brief  Sets the Coarse calibration parameters.
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
-  *                the configuration information for RTC.
-  * @param  CalibSign Specifies the sign of the coarse calibration
value.
-  *          This parameter can be  one of the following values:
-  *             @arg RTC_CALIBSIGN_POSITIVE: The value sign is positive
-  *             @arg RTC_CALIBSIGN_NEGATIVE: The value sign is negative
-  * @param  Value value of coarse calibration expressed in ppm (coded on
5 bits).
-  *
-  * @note   This Calibration value should be between 0 and 63 when using
negative
-  *         sign with a 2-ppm step.
-  *
-  * @note   This Calibration value should be between 0 and 126 when
using positive
-  *         sign with a 4-ppm step.
-  * @retval HAL status
-  */
-HAL_StatusTypeDef HAL_RTCEx_SetCoarseCalib(RTC_HandleTypeDef *hrtc,
uint32_t CalibSign, uint32_t Value)
-{
-  HAL_StatusTypeDef status;
-
```

```
-  /* Check the parameters */
-  assert_param(IS_RTC_CALIB_SIGN(CalibSign));
-  assert_param(IS_RTC_CALIB_VALUE(Value));
-
-  /* Process Locked */
-  __HAL_LOCK(hrtc);
-
-  hrtc->State = HAL_RTC_STATE_BUSY;
-
-  /* Disable the write protection for RTC registers */
-  __HAL_RTC_WRITEPROTECTION_DISABLE(hrtc);
-
-  /* Enter Initialization mode */
-  status = RTC_EnterInitMode(hrtc);
-
-  if (status == HAL_OK)
-  {
-    /* Enable the Coarse Calibration */
-    __HAL_RTC_COARSE_CALIB_ENABLE(hrtc);
-
-    /* Set the coarse calibration value */
-    hrtc->Instance->CALIBR = (uint32_t)(CalibSign | Value);
-
-    /* Exit Initialization mode */
-    status = RTC_ExitInitMode(hrtc);
-  }
-
-  if (status == HAL_OK)
-  {
-    hrtc->State = HAL_RTC_STATE_READY;
-  }
-
-  /* Enable the write protection for RTC registers */
-  __HAL_RTC_WRITEPROTECTION_ENABLE(hrtc);
-
-  /* Process Unlocked */
-  __HAL_UNLOCK(hrtc);
-
-  return status;
-}
-
-/**
-  * @brief  Deactivates the Coarse calibration parameters.
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
-  *                the configuration information for RTC.
-  * @retval HAL status
-  */
-HAL_StatusTypeDef HAL_RTCEx_DeactivateCoarseCalib(RTC_HandleTypeDef
*hrtc)
-{
-  HAL_StatusTypeDef status;
-
-  /* Process Locked */
-  __HAL_LOCK(hrtc);
-
-  hrtc->State = HAL_RTC_STATE_BUSY;
-
-  /* Disable the write protection for RTC registers */
```

```
-  __HAL_RTC_WRITEPROTECTION_DISABLE(hrtc);
-
-  /* Enter Initialization mode */
-  status = RTC_EnterInitMode(hrtc);
-
-  if (status == HAL_OK)
-  {
-    /* Disable the Coarse Calibration */
-    __HAL_RTC_COARSE_CALIB_DISABLE(hrtc);
-
-    /* Exit Initialization mode */
-    status = RTC_ExitInitMode(hrtc);
-  }
-
-  if (status == HAL_OK)
-  {
-    hrtc->State = HAL_RTC_STATE_READY;
-  }
-
-  /* Enable the write protection for RTC registers */
-  __HAL_RTC_WRITEPROTECTION_ENABLE(hrtc);
-
-  /* Process Unlocked */
-  __HAL_UNLOCK(hrtc);
-
-  return status;
-}
-
-/**
-  * @brief  Sets the Smooth calibration parameters.
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
-  *                the configuration information for RTC.
-  * @param  SmoothCalibPeriod Select the Smooth Calibration Period.
-  *          This parameter can be can be one of the following values:
-  *             @arg RTC_SMOOTHCALIB_PERIOD_32SEC: The smooth
calibration period is 32s.
-  *             @arg RTC_SMOOTHCALIB_PERIOD_16SEC: The smooth
calibration period is 16s.
-  *             @arg RTC_SMOOTHCALIB_PERIOD_8SEC: The smooth calibration
period is 8s.
-  * @param  SmoothCalibPlusPulses Select to Set or reset the CALP bit.
-  *          This parameter can be one of the following values:
-  *             @arg RTC_SMOOTHCALIB_PLUSPULSES_SET: Add one RTCCLK
pulse every 2*11 pulses.
-  *             @arg RTC_SMOOTHCALIB_PLUSPULSES_RESET: No RTCCLK pulses
are added.
-  * @param  SmoothCalibMinusPulsesValue Select the value of CALM[8:0]
bits.
-  *          This parameter can be one any value from 0 to 0x000001FF.
-  * @note   To deactivate the smooth calibration, the field
SmoothCalibPlusPulses
-  *         must be equal to SMOOTHCALIB_PLUSPULSES_RESET and the field
-  *         SmoothCalibMinusPulsesValue must be equal to 0.
-  * @retval HAL status
-  */
-HAL_StatusTypeDef HAL_RTCEx_SetSmoothCalib(RTC_HandleTypeDef *hrtc,
uint32_t SmoothCalibPeriod, uint32_t SmoothCalibPlusPulses, uint32_t
SmoothCalibMinusPulsesValue)
```

```
-{
-  uint32_t tickstart = 0U;
-
-  /* Check the parameters */
-  assert_param(IS_RTC_SMOOTH_CALIB_PERIOD(SmoothCalibPeriod));
-  assert_param(IS_RTC_SMOOTH_CALIB_PLUS(SmoothCalibPlusPulses));
-  assert_param(IS_RTC_SMOOTH_CALIB_MINUS(SmoothCalibMinusPulsesValue));
-
-  /* Process Locked */
-  __HAL_LOCK(hrtc);
-
-  hrtc->State = HAL_RTC_STATE_BUSY;
-
-  /* Disable the write protection for RTC registers */
-  __HAL_RTC_WRITEPROTECTION_DISABLE(hrtc);
-
-  /* check if a calibration is pending*/
-  if ((hrtc->Instance->ISR & RTC_ISR_RECALPF) != 0U)
-  {
-    /* Get tick */
-    tickstart = HAL_GetTick();
-
-    /* check if a calibration is pending*/
-    while ((hrtc->Instance->ISR & RTC_ISR_RECALPF) != 0U)
-    {
-      if ((HAL_GetTick() - tickstart) > RTC_TIMEOUT_VALUE)
-      {
-        /* Enable the write protection for RTC registers */
-        __HAL_RTC_WRITEPROTECTION_ENABLE(hrtc);
-
-        /* Change RTC state */
-        hrtc->State = HAL_RTC_STATE_TIMEOUT;
-
-        /* Process Unlocked */
-        __HAL_UNLOCK(hrtc);
-
-        return HAL_TIMEOUT;
-      }
-    }
-  }
-
-  /* Configure the Smooth calibration settings */
-  hrtc->Instance->CALR = (uint32_t)((uint32_t)SmoothCalibPeriod     | \
-                                    (uint32_t)SmoothCalibPlusPulses | \
-
(uint32_t)SmoothCalibMinusPulsesValue);
-
-  /* Enable the write protection for RTC registers */
-  __HAL_RTC_WRITEPROTECTION_ENABLE(hrtc);
-
-  /* Change RTC state */
-  hrtc->State = HAL_RTC_STATE_READY;
-
-  /* Process Unlocked */
-  __HAL_UNLOCK(hrtc);
-
-  return HAL_OK;
-}
```

```
-
-/**
-  * @brief  Configures the Synchronization Shift Control Settings.
-  * @note   When REFCKON is set, firmware must not write to Shift
control register.
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
-  *                the configuration information for RTC.
-  * @param  ShiftAdd1S Select to add or not 1 second to the time
calendar.
-  *          This parameter can be one of the following values:
-  *             @arg RTC_SHIFTADD1S_SET: Add one second to the clock
calendar.
-  *             @arg RTC_SHIFTADD1S_RESET: No effect.
-  * @param  ShiftSubFS Select the number of Second Fractions to
substitute.
-  *          This parameter can be one any value from 0 to 0x7FFF.
-  * @retval HAL status
-  */
-HAL_StatusTypeDef HAL_RTCEx_SetSynchroShift(RTC_HandleTypeDef *hrtc,
uint32_t ShiftAdd1S, uint32_t ShiftSubFS)
-{
-  uint32_t tickstart = 0U;
-
-  /* Check the parameters */
-  assert_param(IS_RTC_SHIFT_ADD1S(ShiftAdd1S));
-  assert_param(IS_RTC_SHIFT_SUBFS(ShiftSubFS));
-
-  /* Process Locked */
-  __HAL_LOCK(hrtc);
-
-  hrtc->State = HAL_RTC_STATE_BUSY;
-
-  /* Disable the write protection for RTC registers */
-  __HAL_RTC_WRITEPROTECTION_DISABLE(hrtc);
-
-  /* Get tick */
-  tickstart = HAL_GetTick();
-
-  /* Wait until the shift is completed */
-  while ((hrtc->Instance->ISR & RTC_ISR_SHPF) != 0U)
-  {
-    if ((HAL_GetTick() - tickstart) > RTC_TIMEOUT_VALUE)
-    {
-      /* Enable the write protection for RTC registers */
-      __HAL_RTC_WRITEPROTECTION_ENABLE(hrtc);
-
-      hrtc->State = HAL_RTC_STATE_TIMEOUT;
-
-      /* Process Unlocked */
-      __HAL_UNLOCK(hrtc);
-
-      return HAL_TIMEOUT;
-    }
-  }
-
-  /* Check if the reference clock detection is disabled */
-  if ((hrtc->Instance->CR & RTC_CR_REFCKON) == 0U)
-  {
```

```
-    /* Configure the Shift settings */
-    hrtc->Instance->SHIFTR = (uint32_t)(uint32_t)(ShiftSubFS) |
(uint32_t)(ShiftAdd1S);
-
-    /* If  RTC_CR_BYPSHAD bit = 0, wait for synchro else this check is
not needed */
-    if ((hrtc->Instance->CR & RTC_CR_BYPSHAD) == 0U)
-    {
-      if (HAL_RTC_WaitForSynchro(hrtc) != HAL_OK)
-      {
-        /* Enable the write protection for RTC registers */
-        __HAL_RTC_WRITEPROTECTION_ENABLE(hrtc);
-
-        hrtc->State = HAL_RTC_STATE_ERROR;
-
-        /* Process Unlocked */
-        __HAL_UNLOCK(hrtc);
-
-        return HAL_ERROR;
-      }
-    }
-  }
-  else
-  {
-    /* Enable the write protection for RTC registers */
-    __HAL_RTC_WRITEPROTECTION_ENABLE(hrtc);
-
-    /* Change RTC state */
-    hrtc->State = HAL_RTC_STATE_ERROR;
-
-    /* Process Unlocked */
-    __HAL_UNLOCK(hrtc);
-
-    return HAL_ERROR;
-  }
-
-  /* Enable the write protection for RTC registers */
-  __HAL_RTC_WRITEPROTECTION_ENABLE(hrtc);
-
-  /* Change RTC state */
-  hrtc->State = HAL_RTC_STATE_READY;
-
-  /* Process Unlocked */
-  __HAL_UNLOCK(hrtc);
-
-  return HAL_OK;
-}
-
-/**
-  * @brief  Configures the Calibration Pinout (RTC_CALIB) Selection (1Hz
or 512Hz).
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
-  *                the configuration information for RTC.
-  * @param  CalibOutput Select the Calibration output Selection.
-  *          This parameter can be one of the following values:
-  *             @arg RTC_CALIBOUTPUT_512HZ: A signal has a regular
waveform at 512Hz.
```

```
-  *             @arg RTC_CALIBOUTPUT_1HZ: A signal has a regular
waveform at 1Hz.
-  * @retval HAL status
-  */
-HAL_StatusTypeDef HAL_RTCEx_SetCalibrationOutPut(RTC_HandleTypeDef
*hrtc, uint32_t CalibOutput)
-{
-  /* Check the parameters */
-  assert_param(IS_RTC_CALIB_OUTPUT(CalibOutput));
-
-  /* Process Locked */
-  __HAL_LOCK(hrtc);
-
-  hrtc->State = HAL_RTC_STATE_BUSY;
-
-  /* Disable the write protection for RTC registers */
-  __HAL_RTC_WRITEPROTECTION_DISABLE(hrtc);
-
-  /* Clear flags before config */
-  hrtc->Instance->CR &= (uint32_t)~RTC_CR_COSEL;
-
-  /* Configure the RTC_CR register */
-  hrtc->Instance->CR |= (uint32_t)CalibOutput;
-
-  __HAL_RTC_CALIBRATION_OUTPUT_ENABLE(hrtc);
-
-  /* Enable the write protection for RTC registers */
-  __HAL_RTC_WRITEPROTECTION_ENABLE(hrtc);
-
-  /* Change RTC state */
-  hrtc->State = HAL_RTC_STATE_READY;
-
-  /* Process Unlocked */
-  __HAL_UNLOCK(hrtc);
-
-  return HAL_OK;
-}
-
-/**
-  * @brief  Deactivates the Calibration Pinout (RTC_CALIB) Selection
(1Hz or 512Hz).
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
-  *               the configuration information for RTC.
-  * @retval HAL status
-  */
-HAL_StatusTypeDef
HAL_RTCEx_DeactivateCalibrationOutPut(RTC_HandleTypeDef *hrtc)
-{
-  /* Process Locked */
-  __HAL_LOCK(hrtc);
-
-  hrtc->State = HAL_RTC_STATE_BUSY;
-
-  /* Disable the write protection for RTC registers */
-  __HAL_RTC_WRITEPROTECTION_DISABLE(hrtc);
-
-  __HAL_RTC_CALIBRATION_OUTPUT_DISABLE(hrtc);
-
```

```
-  /* Enable the write protection for RTC registers */
-  __HAL_RTC_WRITEPROTECTION_ENABLE(hrtc);
-
-  /* Change RTC state */
-  hrtc->State = HAL_RTC_STATE_READY;
-
-  /* Process Unlocked */
-  __HAL_UNLOCK(hrtc);
-
-  return HAL_OK;
-}
-
-/**
-  * @brief  Enables the RTC reference clock detection.
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
-  *                the configuration information for RTC.
-  * @retval HAL status
-  */
-HAL_StatusTypeDef HAL_RTCEx_SetRefClock(RTC_HandleTypeDef *hrtc)
-{
-  HAL_StatusTypeDef status;
-
-  /* Process Locked */
-  __HAL_LOCK(hrtc);
-
-  hrtc->State = HAL_RTC_STATE_BUSY;
-
-  /* Disable the write protection for RTC registers */
-  __HAL_RTC_WRITEPROTECTION_DISABLE(hrtc);
-
-  /* Enter Initialization mode */
-  status = RTC_EnterInitMode(hrtc);
-
-  if (status == HAL_OK)
-  {
-    /* Enable the reference clock detection */
-    __HAL_RTC_CLOCKREF_DETECTION_ENABLE(hrtc);
-
-    /* Exit Initialization mode */
-    status = RTC_ExitInitMode(hrtc);
-  }
-
-  if (status == HAL_OK)
-  {
-    hrtc->State = HAL_RTC_STATE_READY;
-  }
-
-  /* Enable the write protection for RTC registers */
-  __HAL_RTC_WRITEPROTECTION_ENABLE(hrtc);
-
-  /* Process Unlocked */
-  __HAL_UNLOCK(hrtc);
-
-  return status;
-}
-
-/**
-  * @brief  Disable the RTC reference clock detection.
```

```
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
-  *                the configuration information for RTC.
-  * @retval HAL status
-  */
-HAL_StatusTypeDef HAL_RTCEx_DeactivateRefClock(RTC_HandleTypeDef *hrtc)
-{
-  HAL_StatusTypeDef status;
-
-  /* Process Locked */
-  __HAL_LOCK(hrtc);
-
-  hrtc->State = HAL_RTC_STATE_BUSY;
-
-  /* Disable the write protection for RTC registers */
-  __HAL_RTC_WRITEPROTECTION_DISABLE(hrtc);
-
-  /* Enter Initialization mode */
-  status = RTC_EnterInitMode(hrtc);
-
-  if (status == HAL_OK)
-  {
-    /* Disable the reference clock detection */
-    __HAL_RTC_CLOCKREF_DETECTION_DISABLE(hrtc);
-
-    /* Exit Initialization mode */
-    status = RTC_ExitInitMode(hrtc);
-  }
-
-  if (status == HAL_OK)
-  {
-    hrtc->State = HAL_RTC_STATE_READY;
-  }
-
-  /* Enable the write protection for RTC registers */
-  __HAL_RTC_WRITEPROTECTION_ENABLE(hrtc);
-
-  /* Process Unlocked */
-  __HAL_UNLOCK(hrtc);
-
-  return status;
-}
-
-/**
-  * @brief  Enables the Bypass Shadow feature.
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
-  *                the configuration information for RTC.
-  * @note   When the Bypass Shadow is enabled the calendar value are
taken
-  *         directly from the Calendar counter.
-  * @retval HAL status
-  */
-HAL_StatusTypeDef HAL_RTCEx_EnableBypassShadow(RTC_HandleTypeDef *hrtc)
-{
-  /* Process Locked */
-  __HAL_LOCK(hrtc);
-
-  hrtc->State = HAL_RTC_STATE_BUSY;
-
```

```
-  /* Disable the write protection for RTC registers */
-  __HAL_RTC_WRITEPROTECTION_DISABLE(hrtc);
-
-  /* Set the BYPSHAD bit */
-  hrtc->Instance->CR |= (uint32_t)RTC_CR_BYPSHAD;
-
-  /* Enable the write protection for RTC registers */
-  __HAL_RTC_WRITEPROTECTION_ENABLE(hrtc);
-
-  /* Change RTC state */
-  hrtc->State = HAL_RTC_STATE_READY;
-
-  /* Process Unlocked */
-  __HAL_UNLOCK(hrtc);
-
-  return HAL_OK;
-}
-
-/**
-  * @brief  Disables the Bypass Shadow feature.
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
-  *                the configuration information for RTC.
-  * @note   When the Bypass Shadow is enabled the calendar value are
taken
-  *         directly from the Calendar counter.
-  * @retval HAL status
-  */
-HAL_StatusTypeDef HAL_RTCEx_DisableBypassShadow(RTC_HandleTypeDef *hrtc)
-{
-  /* Process Locked */
-  __HAL_LOCK(hrtc);
-
-  hrtc->State = HAL_RTC_STATE_BUSY;
-
-  /* Disable the write protection for RTC registers */
-  __HAL_RTC_WRITEPROTECTION_DISABLE(hrtc);
-
-  /* Reset the BYPSHAD bit */
-  hrtc->Instance->CR &= (uint32_t)~RTC_CR_BYPSHAD;
-
-  /* Enable the write protection for RTC registers */
-  __HAL_RTC_WRITEPROTECTION_ENABLE(hrtc);
-
-  /* Change RTC state */
-  hrtc->State = HAL_RTC_STATE_READY;
-
-  /* Process Unlocked */
-  __HAL_UNLOCK(hrtc);
-
-  return HAL_OK;
-}
-
-/**
-  * @}
-  */
-
-/** @defgroup RTCEx_Exported_Functions_Group4 Extended features
functions
```

```
-  * @brief    Extended features functions
-  *
-@verbatim
-
==============================================================================
======
-                 ##### Extended features functions #####
-
==============================================================================
======
-    [..]  This section provides functions allowing to:
-      (+) RTC Alarm B callback
-      (+) RTC Poll for Alarm B request
-
-@endverbatim
-  * @{
-  */
-
-/**
-  * @brief  Alarm B callback.
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
-  *                the configuration information for RTC.
-  * @retval None
-  */
-__weak void HAL_RTCEx_AlarmBEventCallback(RTC_HandleTypeDef *hrtc)
-{
-  /* Prevent unused argument(s) compilation warning */
-  UNUSED(hrtc);
-
-  /* NOTE: This function should not be modified, when the callback is
needed,
-           the HAL_RTCEx_AlarmBEventCallback could be implemented in the
user file
-   */
-}
-
-/**
-  * @brief  Handles Alarm B Polling request.
-  * @param  hrtc pointer to a RTC_HandleTypeDef structure that contains
-  *                the configuration information for RTC.
-  * @param  Timeout Timeout duration
-  * @retval HAL status
-  */
-HAL_StatusTypeDef HAL_RTCEx_PollForAlarmBEvent(RTC_HandleTypeDef *hrtc,
uint32_t Timeout)
-{
-  uint32_t tickstart = 0U;
-
-  /* Get tick */
-  tickstart = HAL_GetTick();
-
-  /* Wait till RTC ALRBF flag is set and if timeout is reached exit */
-  while (__HAL_RTC_ALARM_GET_FLAG(hrtc, RTC_FLAG_ALRBF) == 0U)
-  {
-    if (Timeout != HAL_MAX_DELAY)
-    {
-      if ((Timeout == 0U) || ((HAL_GetTick() - tickstart) > Timeout))
-      {
```

```
-        hrtc->State = HAL_RTC_STATE_TIMEOUT;
-        return HAL_TIMEOUT;
-      }
-    }
-  }
-
-  /* Clear the Alarm flag */
-  __HAL_RTC_ALARM_CLEAR_FLAG(hrtc, RTC_FLAG_ALRBF);
-
-  /* Change RTC state */
-  hrtc->State = HAL_RTC_STATE_READY;
-
-  return HAL_OK;
-}
-
-/**
-  * @}
-  */
-
-/**
-  * @}
-  */
-
-#endif /* HAL_RTC_MODULE_ENABLED */
-/**
-  * @}
-  */
-
-/**
-  * @}
-  */
diff --git a/Led1.ioc b/Led1.ioc
index 05b39b7..d8a98af 100644
--- a/Led1.ioc
+++ b/Led1.ioc
@@ -17,41 +17,36 @@ Mcu.Family=STM32F4
 Mcu.IP0=FREERTOS
 Mcu.IP1=NVIC
 Mcu.IP2=RCC
-Mcu.IP3=RTC
-Mcu.IP4=SPI2
-Mcu.IP5=SYS
-Mcu.IP6=TIM1
-Mcu.IP7=TIM2
-Mcu.IP8=TIM3
-Mcu.IP9=USART1
-Mcu.IPNb=10
+Mcu.IP3=SPI2
+Mcu.IP4=SYS
+Mcu.IP5=TIM1
+Mcu.IP6=TIM2
+Mcu.IP7=TIM3
+Mcu.IP8=USART1
+Mcu.IPNb=9
 Mcu.Name=STM32F407V(E-G)Tx
 Mcu.Package=LQFP100
 Mcu.Pin0=PE3
 Mcu.Pin1=PE4
```

```
-Mcu.Pin10=PC4
-Mcu.Pin11=PC5
-Mcu.Pin12=PE9
-Mcu.Pin13=PB10
-Mcu.Pin14=PA9
-Mcu.Pin15=PA10
-Mcu.Pin16=PE0
-Mcu.Pin17=PE1
-Mcu.Pin18=VP_FREERTOS_VS_CMSIS_V2
-Mcu.Pin19=VP_RTC_VS_RTC_Activate
-Mcu.Pin2=PC14-OSC32_IN
-Mcu.Pin20=VP_RTC_VS_RTC_WakeUp_intern
-Mcu.Pin21=VP_SYS_VS_tim6
-Mcu.Pin22=VP_TIM1_VS_ClockSourceINT
-Mcu.Pin23=VP_TIM2_VS_ClockSourceINT
-Mcu.Pin3=PC15-OSC32_OUT
-Mcu.Pin4=PH0-OSC_IN
-Mcu.Pin5=PH1-OSC_OUT
-Mcu.Pin6=PC2
-Mcu.Pin7=PC3
-Mcu.Pin8=PA6
-Mcu.Pin9=PA7
-Mcu.PinsNb=24
+Mcu.Pin10=PE9
+Mcu.Pin11=PB10
+Mcu.Pin12=PA9
+Mcu.Pin13=PA10
+Mcu.Pin14=PE0
+Mcu.Pin15=PE1
+Mcu.Pin16=VP_FREERTOS_VS_CMSIS_V2
+Mcu.Pin17=VP_SYS_VS_tim6
+Mcu.Pin18=VP_TIM1_VS_ClockSourceINT
+Mcu.Pin19=VP_TIM2_VS_ClockSourceINT
+Mcu.Pin2=PH0-OSC_IN
+Mcu.Pin3=PH1-OSC_OUT
+Mcu.Pin4=PC2
+Mcu.Pin5=PC3
+Mcu.Pin6=PA6
+Mcu.Pin7=PA7
+Mcu.Pin8=PC4
+Mcu.Pin9=PC5
+Mcu.PinsNb=20
 Mcu.ThirdPartyNb=0
 Mcu.UserConstants=
 Mcu.UserName=STM32F407VETx
@@ -67,7 +62,6 @@
NVIC.MemoryManagement_IRQn=true\:0\:0\:false\:false\:true\:false\:false\:
false\:

NVIC.NonMaskableInt_IRQn=true\:0\:0\:false\:false\:true\:false\:false\:fa
lse\:false

NVIC.PendSV_IRQn=true\:15\:0\:false\:false\:false\:true\:false\:false\:fa
lse
 NVIC.PriorityGroup=NVIC_PRIORITYGROUP_4
-
NVIC.RTC_WKUP_IRQn=true\:5\:0\:false\:false\:true\:true\:true\:true\:true
```

```
NVIC.SVCall_IRQn=true\:0\:0\:false\:false\:false\:false\:false\:false\:fa
lse
 NVIC.SavedPendsvIrqHandlerGenerated=true
 NVIC.SavedSvcallIrqHandlerGenerated=true
@@ -86,16 +80,6 @@ PA9.Mode=Asynchronous
 PA9.Signal=USART1_TX
 PB10.Mode=Full_Duplex_Master
 PB10.Signal=SPI2_SCK
-PC14-OSC32_IN.GPIOParameters=GPIO_Label
-PC14-OSC32_IN.GPIO_Label=RTC_IN
-PC14-OSC32_IN.Locked=true
-PC14-OSC32_IN.Mode=LSE-External-Oscillator
-PC14-OSC32_IN.Signal=RCC_OSC32_IN
-PC15-OSC32_OUT.GPIOParameters=GPIO_Label
-PC15-OSC32_OUT.GPIO_Label=RTC_OUT
-PC15-OSC32_OUT.Locked=true
-PC15-OSC32_OUT.Mode=LSE-External-Oscillator
-PC15-OSC32_OUT.Signal=RCC_OSC32_OUT
 PC2.Mode=Full_Duplex_Master
 PC2.Signal=SPI2_MISO
 PC3.Mode=Full_Duplex_Master
@@ -187,7 +171,8 @@ RCC.HCLKFreq_Value=168000000
 RCC.HSE_VALUE=8000000
 RCC.HSI_VALUE=16000000
 RCC.I2SClocksFreq_Value=96000000
-
RCC.IPParameters=48MHZClocksFreq_Value,AHBFreq_Value,APB1CLKDivider,APB1F
req_Value,APB1TimFreq_Value,APB2CLKDivider,APB2Freq_Value,APB2TimFreq_Val
ue,CortexFreq_Value,EthernetFreq_Value,FCLKCortexFreq_Value,FamilyName,HC
LKFreq_Value,HSE_VALUE,HSI_VALUE,I2SClocksFreq_Value,LSI_VALUE,MCO2PinFre
q_Value,PLLCLKFreq_Value,PLLM,PLLN,PLLQCLKFreq_Value,PLLSourceVirtual,RTC
Freq_Value,RTCHSEDivFreq_Value,SYSCLKFreq_VALUE,SYSCLKSource,VCOI2SOutput
Freq_Value,VCOInputFreq_Value,VCOOutputFreq_Value,VcooutputI2S
+RCC.IPParameters=48MHZClocksFreq_Value,AHBFreq_Value,APB1CLKDivider,APB1
Freq_Value,APB1TimFreq_Value,APB2CLKDivider,APB2Freq_Value,APB2TimFreq_Va
lue,CortexFreq_Value,EthernetFreq_Value,FCLKCortexFreq_Value,FamilyName,H
CLKFreq_Value,HSE_VALUE,HSI_VALUE,I2SClocksFreq_Value,LSE_VALUE,LSI_VALUE
,MCO2PinFreq_Value,PLLCLKFreq_Value,PLLM,PLLN,PLLQCLKFreq_Value,PLLSource
Virtual,RTCFreq_Value,RTCHSEDivFreq_Value,SYSCLKFreq_VALUE,SYSCLKSource,V
COI2SOutputFreq_Value,VCOInputFreq_Value,VCOOutputFreq_Value,VcooutputI2S
+RCC.LSE_VALUE=32768
 RCC.LSI_VALUE=32000
 RCC.MCO2PinFreq_Value=168000000
 RCC.PLLCLKFreq_Value=168000000
@@ -241,10 +226,6 @@ USART1.IPParameters=VirtualMode
 USART1.VirtualMode=VM_ASYNC
 VP_FREERTOS_VS_CMSIS_V2.Mode=CMSIS_V2
 VP_FREERTOS_VS_CMSIS_V2.Signal=FREERTOS_VS_CMSIS_V2
-VP_RTC_VS_RTC_Activate.Mode=RTC_Enabled
-VP_RTC_VS_RTC_Activate.Signal=RTC_VS_RTC_Activate
-VP_RTC_VS_RTC_WakeUp_intern.Mode=WakeUp
-VP_RTC_VS_RTC_WakeUp_intern.Signal=RTC_VS_RTC_WakeUp_intern
 VP_SYS_VS_tim6.Mode=TIM6
 VP_SYS_VS_tim6.Signal=SYS_VS_tim6
 VP_TIM1_VS_ClockSourceINT.Mode=Internal
```